Cursul

Elemente de Grafica pe Calculator

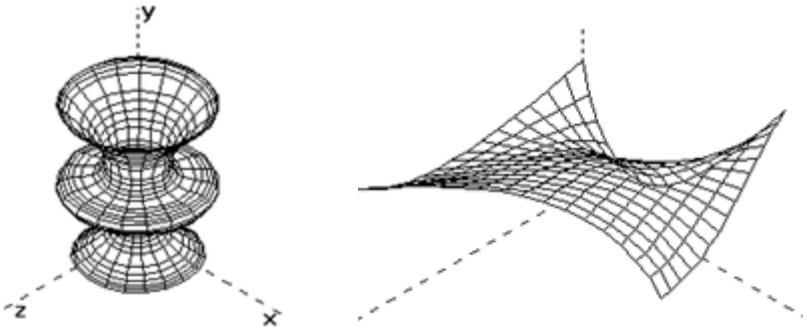
Profesor Florica Moldoveanu

UPB, Automatică şi Calculatoare
2019-2020

Sisteme Grafice: clasificare

- **De sinteza a imaginilor**: crearea de imagini ale unor obiecte / scene reale sau virtuale, statice sau dinamice (animate); domeniul de studiu corespunzator: **Computer Graphics (CG).**
  - Aplicatii în: industria de jocuri si film, crearea de spatii virtuale, arhitectura, design industrial si artistic, etc.
- **De prelucrare si analiza a imaginilor /Sisteme cu viziune**: extragerea de informatii din imagini digitale produse de dispozitive de achizitie (camere video, scanner); domeniul de studiu: **Computer Vision (CV)**.
  - Aplicatii în: analiza imaginilor medicale, recunoasterea obiectelor din mediul încojurator (in robotica, conducerea automata a autovehiculelor, etc), a feţelor, a caracterelor de text, recunoasterea obiectelor din imagini satelitare, etc.
- **De reconstructie si vizualizare a unor obiecte/ fenomene reale pornind de la date volumetrice;** domeniul de studiu: **Volume Graphics (combina algoritmi de CG si CV)**.
  - Aplicatii in: medicina (vizualizarea 3D a organelor interne pornind de la imagini tomografice), vizualizarea 3D a dinamicii moleculare, a datelor meteo, etc.

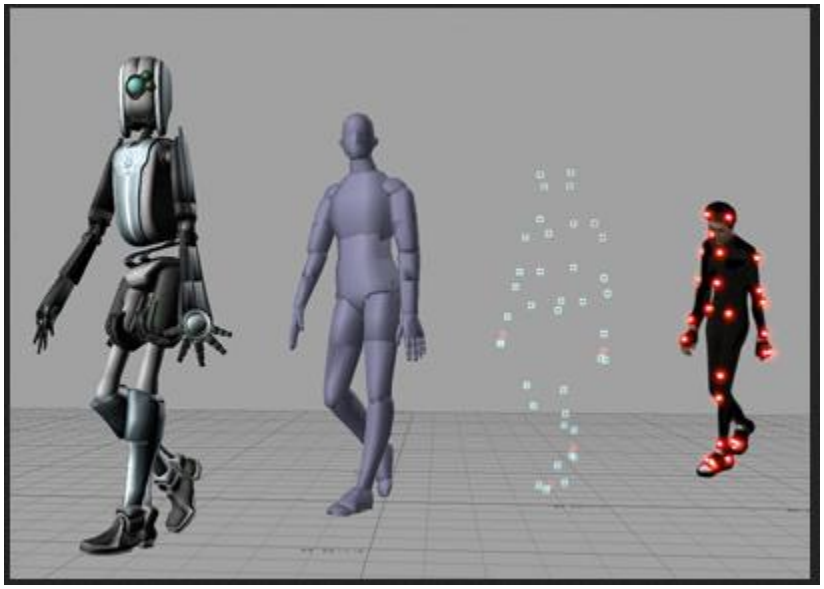
Computer graphics (1)



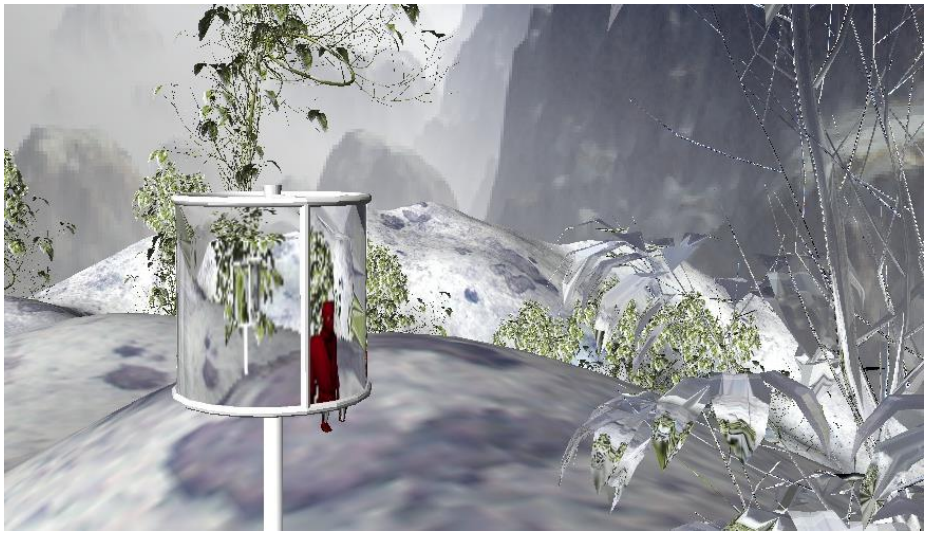
Modelarea suprafetelor 3D

Simularea detaliilor suprafetelor (texturi)

Animatia personajelor prin captura miscarii



Simularea reflexiei si a refractiei luminii



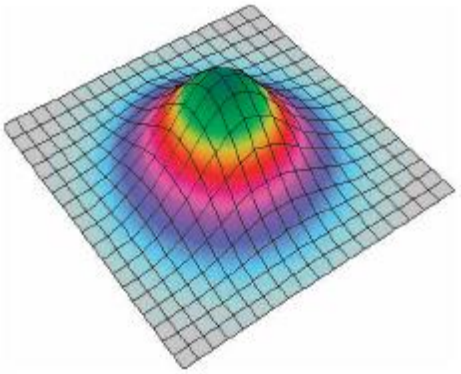
Computer graphics (2)

Imagine sintetizata prin algoritmul Ray-tracing

Computer graphics (3)

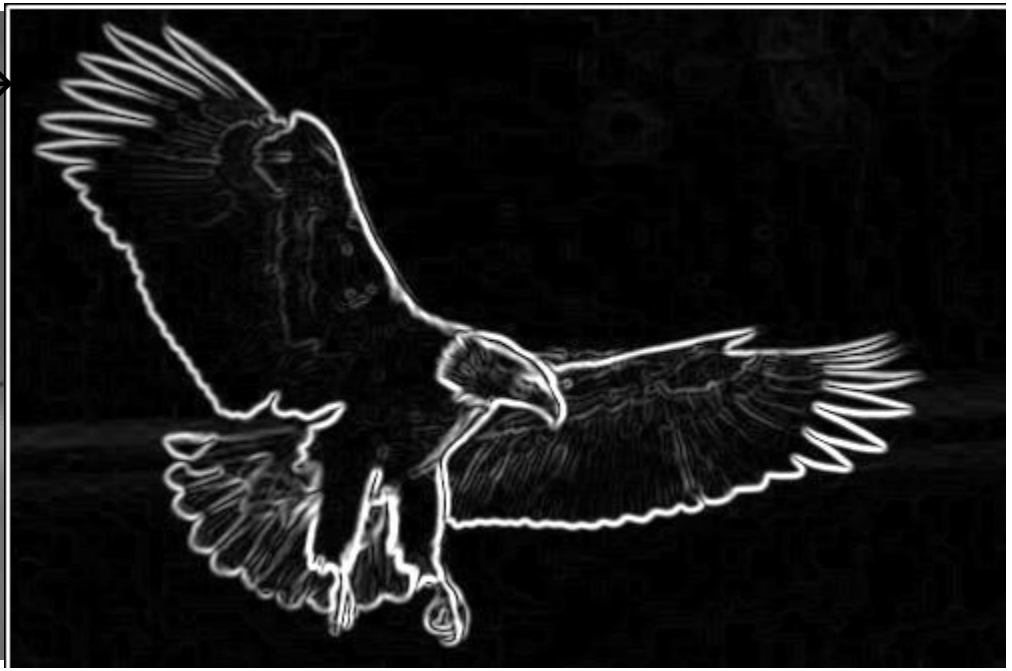
Imagine sintetizata prin Ray-tracing

Prelucrarea si analiza imaginilor (1)



Eliminarea zgomotelor

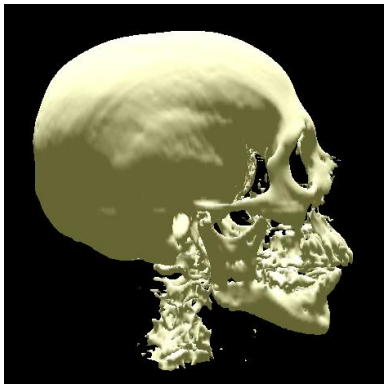
Extragerea frontierelor →



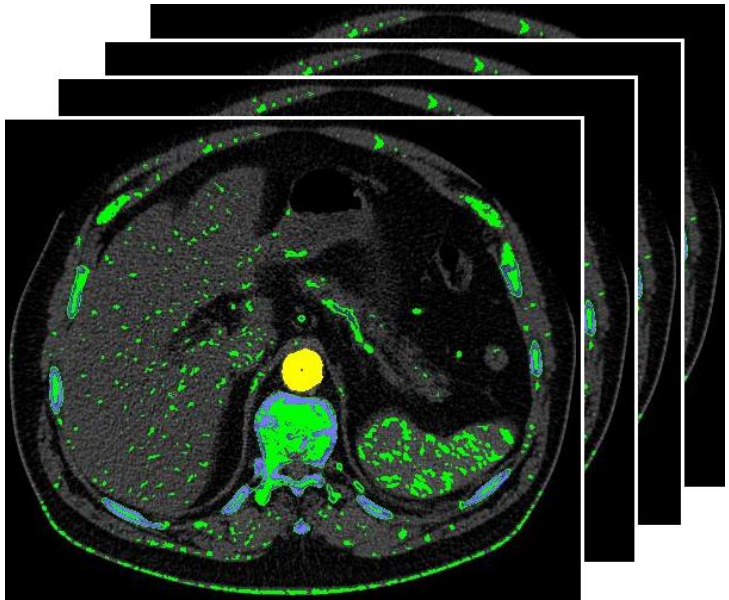
Prelucrarea si analiza imaginilor (2)

Analiza imaginilor medicale
(extragerea formei osului femural si efectuarea de masuratori automate)

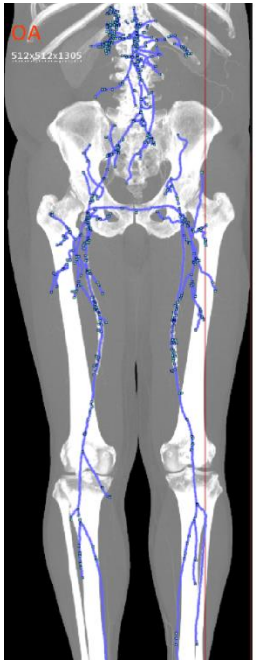
Reconstructia si vizualizarea volumelor



Reconstructia / vizualizarea volumelor pornind de la imagini 3D



Vizualizarea vaselor de sânge



Sisteme grafice de sinteza - aplicaţii

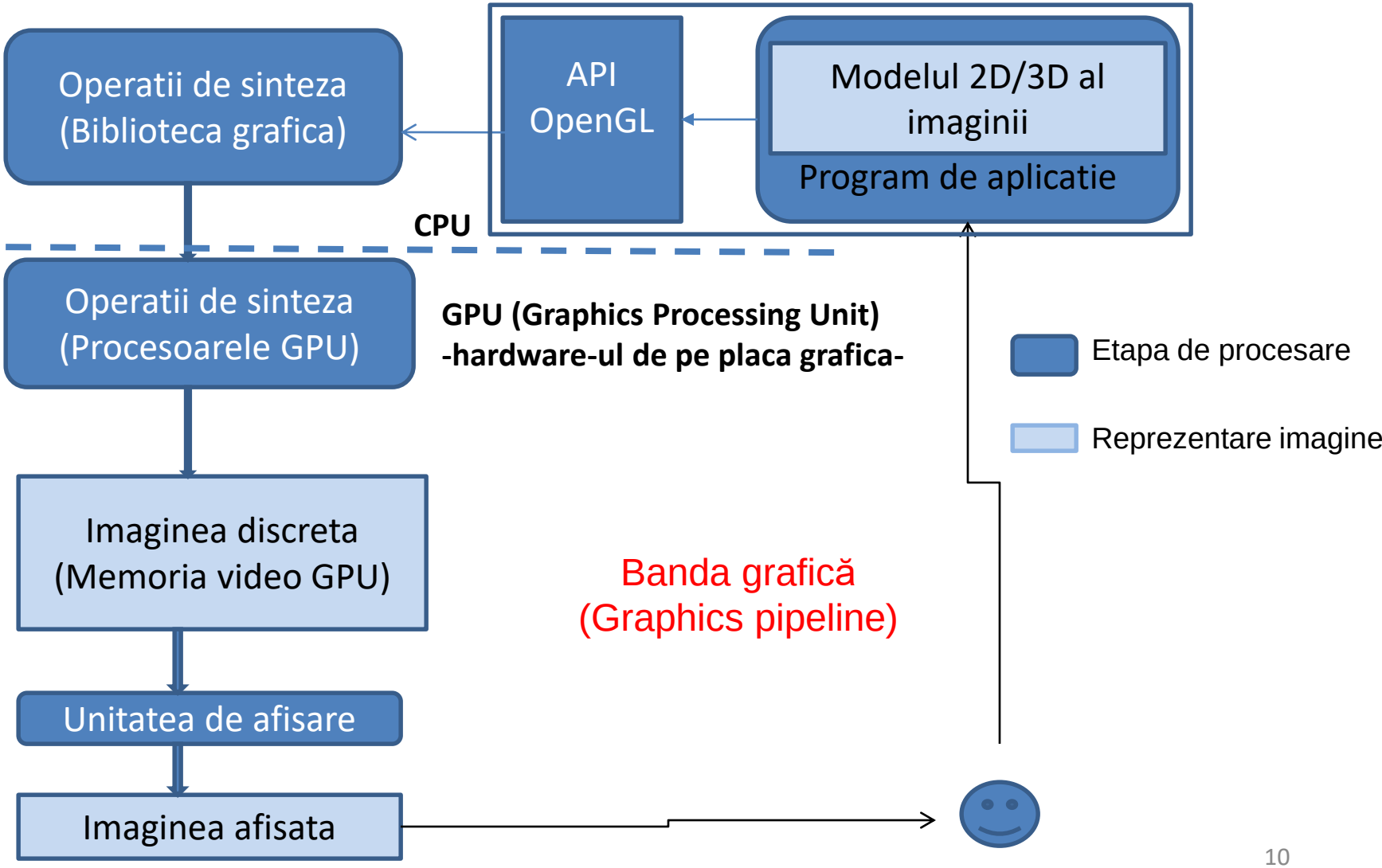
Cursul Elemente de Grafica pe Calculator: curs de **Computer Graphics.**

Obiective:

- însuşirea de către studenţi a noţiunilor fundamentale de sinteză (creare) a imaginilor digitale
- formarea deprinderilor studenţilor de a crea aplicaţii folosind biblioteca grafică OpenGL.

- **Aplicatii ale sistemelor grafice de sinteza**:
  - Proiectarea asistata de calculator: arhitectura, constructii, design industrial, confecţii, etc.
  - Jocuri pe calculator şi pe dispozitive mobile
  - Cartografie
  - Productia de filme
  - Spatii virtuale interactive
  - Etc.

Sisteme grafice de sinteza-model conceptual

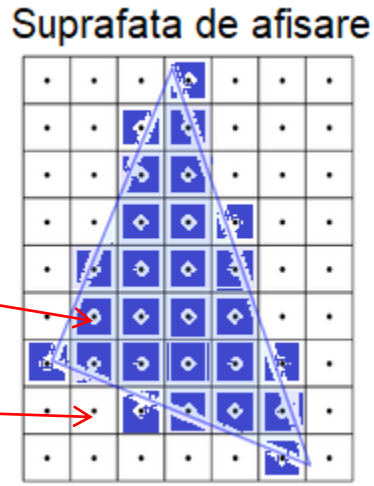


Modelul 2D/3D al imaginii

- Inclus in programul de aplicatie
- Alcatuit din:
  - Primitive grafice 2D, 3D (linii, poligoane) sau obiecte complexe – alcatuiesc **scena 2D/3D redata intr-un cadru imagine**
  - Proprietati ale suprafetelor obiectelor din scena (culoare, textura, proprietati de reflexie/refractie a luminii)
  - Transformari de aplicat obiectelor in scopul compunerii, vizualizarii scenei si animarii obiectelor scenei
  - Surse de lumina (pozitie, caracteristici de culoare/luminozitate) – într-o scena 3D
  - Pozitia observatorului virtual (a camerei virtuale) în scena 3D si directia privirii
- Este modificat de aplicatie conform interactiunii cu utilizatorul aplicatiei
- Orice modificare trebuie sa aiba ca efect modificarea in timp real a imaginii afisate

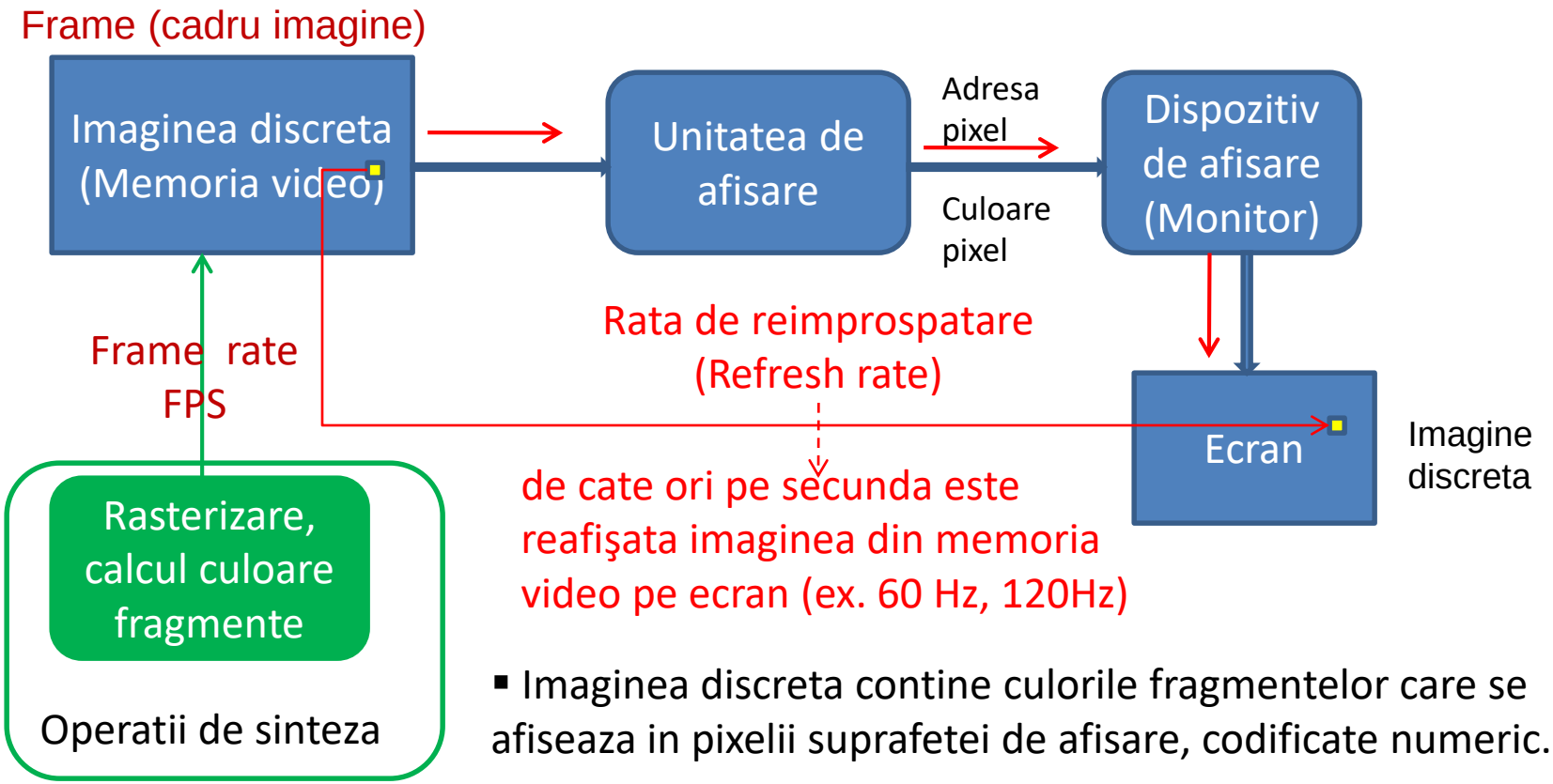
Operatii de sinteza a imaginilor grafice

- Efectuarea transformarilor care se aplică obiectelor pentru modelarea scenei
- Efectuarea transformarilor de proiectie din spatiul 3D in spatiul 2D
- Eliminarea obiectelor nevizibile din imaginea unei scene 3D
- Eliminarea fețelor nevizibile (auto-obturate) ale obiectelor
- Decuparea primitivelor la marginile volumului vizual (field of view)
- Rasterizarea: descompunerea primitivelor grafice in fragmente care se afiseaza in pixeli



- Calculul culorii fiecarui fragment (în fragment shader) folosind: modele de reflexie si refractie a luminii, calcul de umbre, texturi, s. a.
- Operaţii la nivel de pixel (operaţii raster): testul de vizibilitate a fragmentelor în imagine (testul de adancime), combinarea culorii fragmentului cu cea a pixelului in care se afiseaza, ş. a.

Imaginea discreta şi afişarea sa (schema conceptuala)



- Imaginea discreta contine culorile fragmentelor care se afiseaza in pixelii suprafetei de afisare, codificate numeric.
- O culoare se reprezinta prin 3 numere, care definesc o combinatie a celor 3 culori primare Red,Green,Blue (3x 8 biti).

FPS: Frames Per Second
Pentru aplicatii de timp real (imagini dinamice): 30 FPS

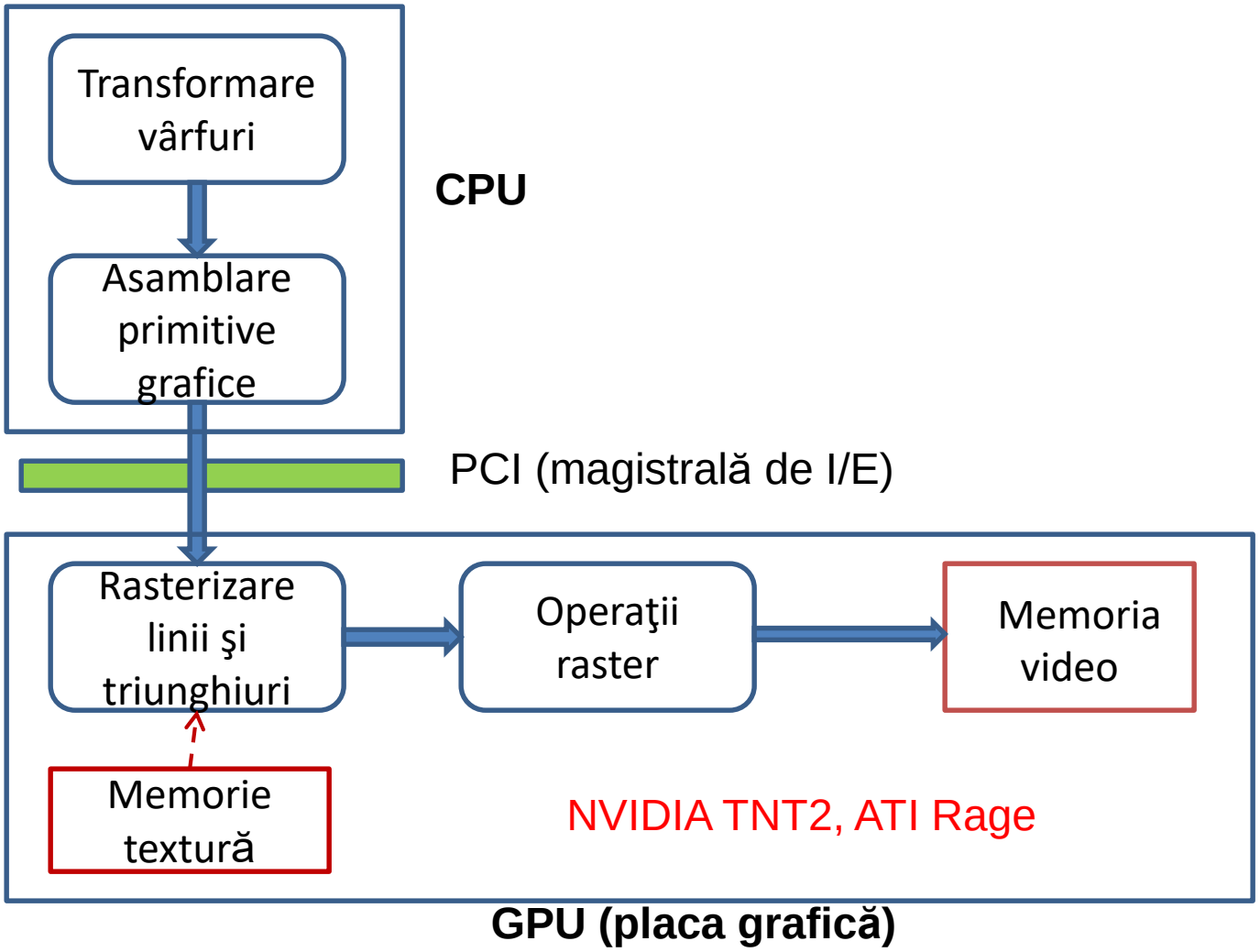
Scurt istoric al hardware-ului grafic de sinteza
Evoluţia benzii grafice

- **Mijlocul anilor ‘90**: cel mai complex hard grafic realiza numai rasterizarea primitivelor.
- **Sfarsitul anilor ‘90**: NVIDIA a introdus termenul GPU, inlocuind termenul VGA – Video Graphics Addapter (introdus de IBM in 1987), devenit neadecvat pentru hardware-ul grafic dintr-un calculator
- **Hardware de accelerare grafica: hardware specializat pentru sinteza imaginilor, dar scump (ex. statiile grafice SiliconGraphics** –SGI, Evans& Sutherland)

❖ **Prima generatie GPU: pana în 1998**

- NVIDIA TNT2, ATI Rage
- **Rasterizare linii si triunghiuri**, cu varfurile transformate pe CPU
- Aplicare 1-2 texturi

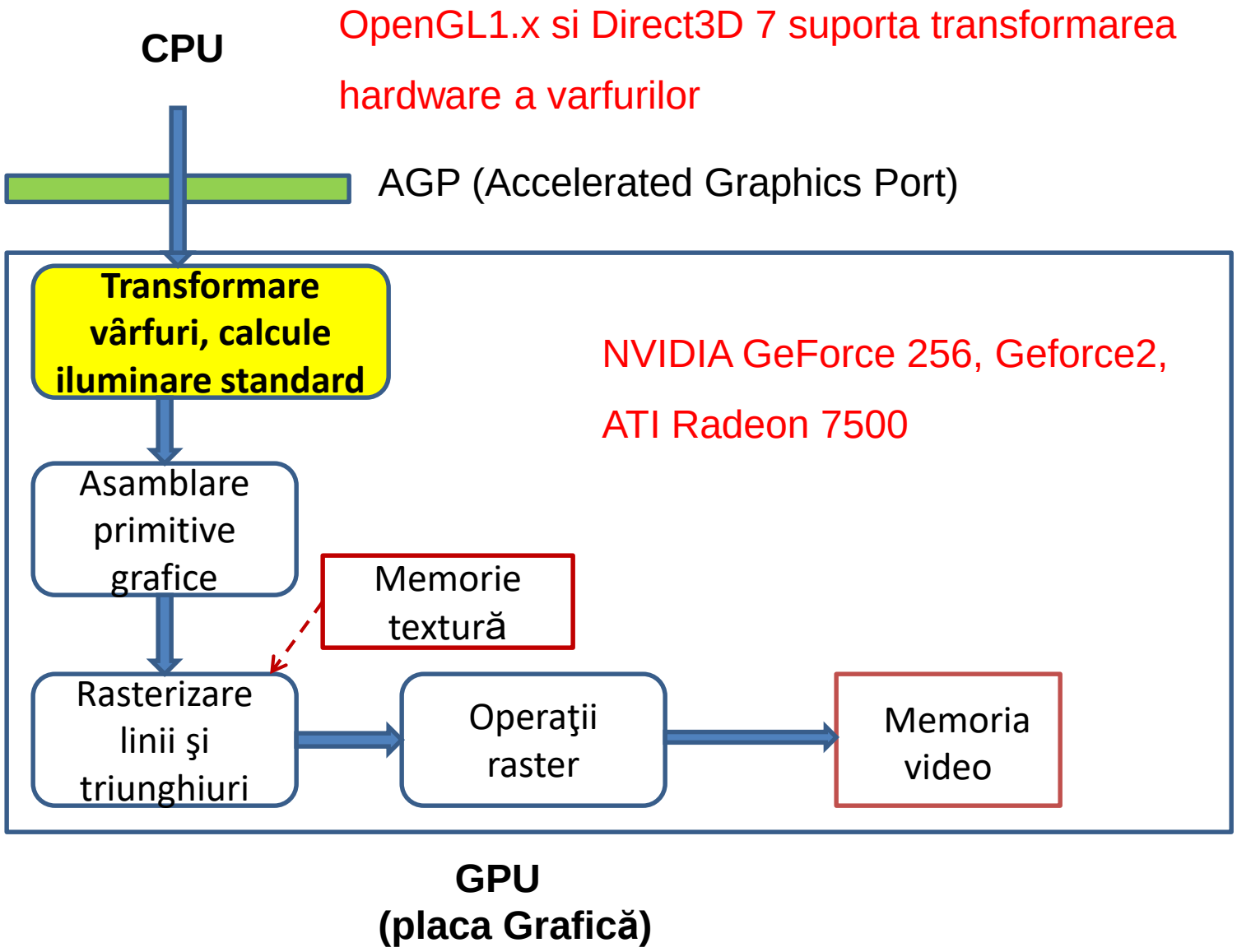
Banda grafică- prima generaţie GPU (până în 1998)



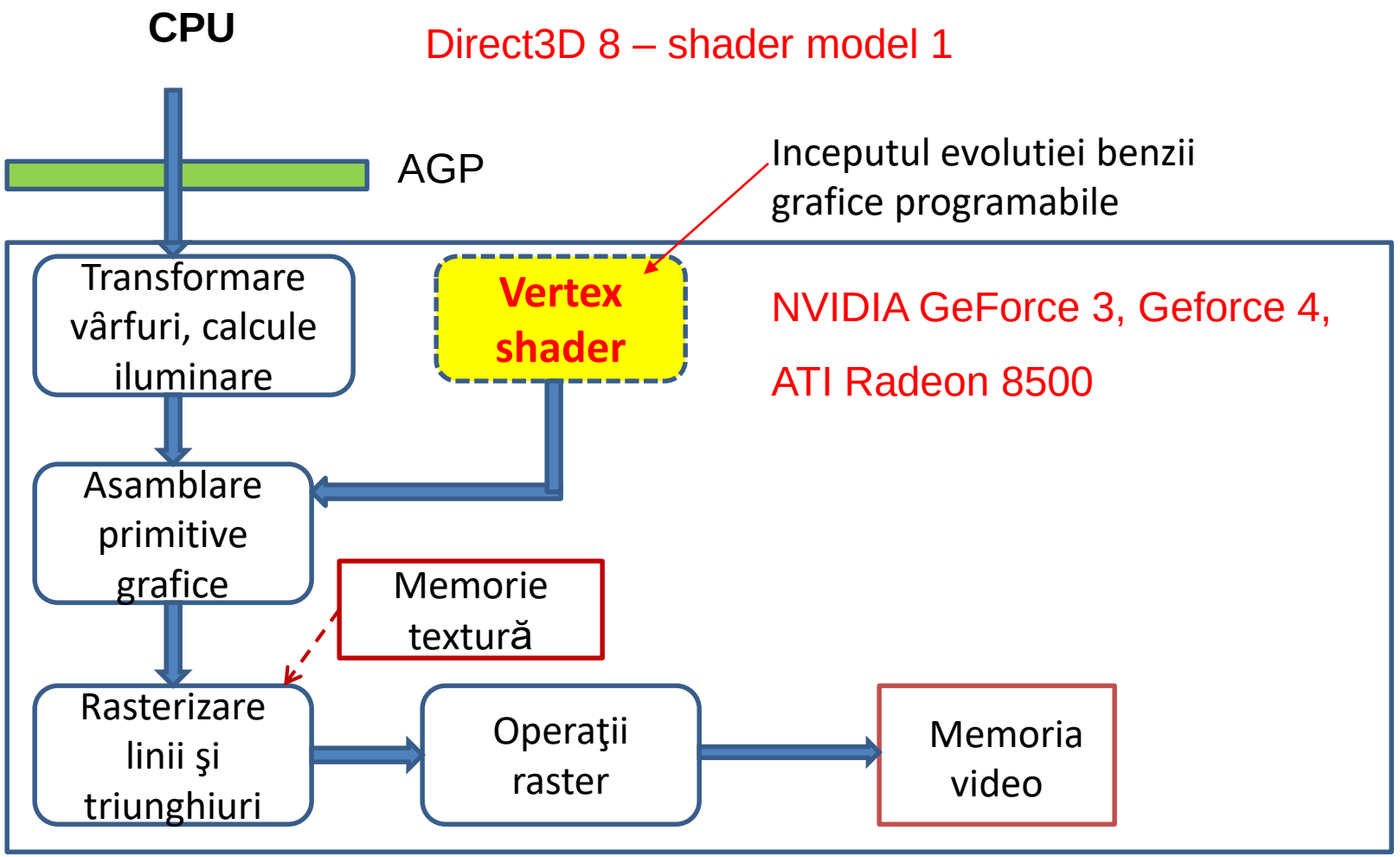
Scurt istoric al hardware-ului grafic de sinteza(2)

- **A 2-a generatie GPU (1999-2000):**
  - NVIDIA GeForce 256, Geforce2, ATI Radeon 7500
  - **Transformare varfuri 3D si calcule de iluminare standard la nivel de varfuri**
  - Bibliotecile OpenGL 1.x si Direct3D 7 suporta transformarea hardware a varfurilor
- **A 3-a generatie GPU (2001-2002)**
  - NVIDIA GeForce 3, Geforce 4, ATI Radeon 8500
  - Transformare varfuri 3D si calcule de iluminare standard la nivel de varfuri
  - **Posibilitatea programarii calculelor de iluminare la nivel de varfuri** → VERTEX SHADER (program scris de programator si transferat la GPU; executat in paralel de un numar mare de procesoare GPU pentru varfuri diferite)
  - Biblioteca Direct3D 8 – introduce “shader model” 1: crearea de programe Vertex Shader folosind un limbaj proprietar compus din instructiuni in lb. de asamblare si in stil C.

Banda grafică- a II-a generaţie GPU (1999-2000)



Banda grafică - a III-a generaţie GPU (2001-2002)

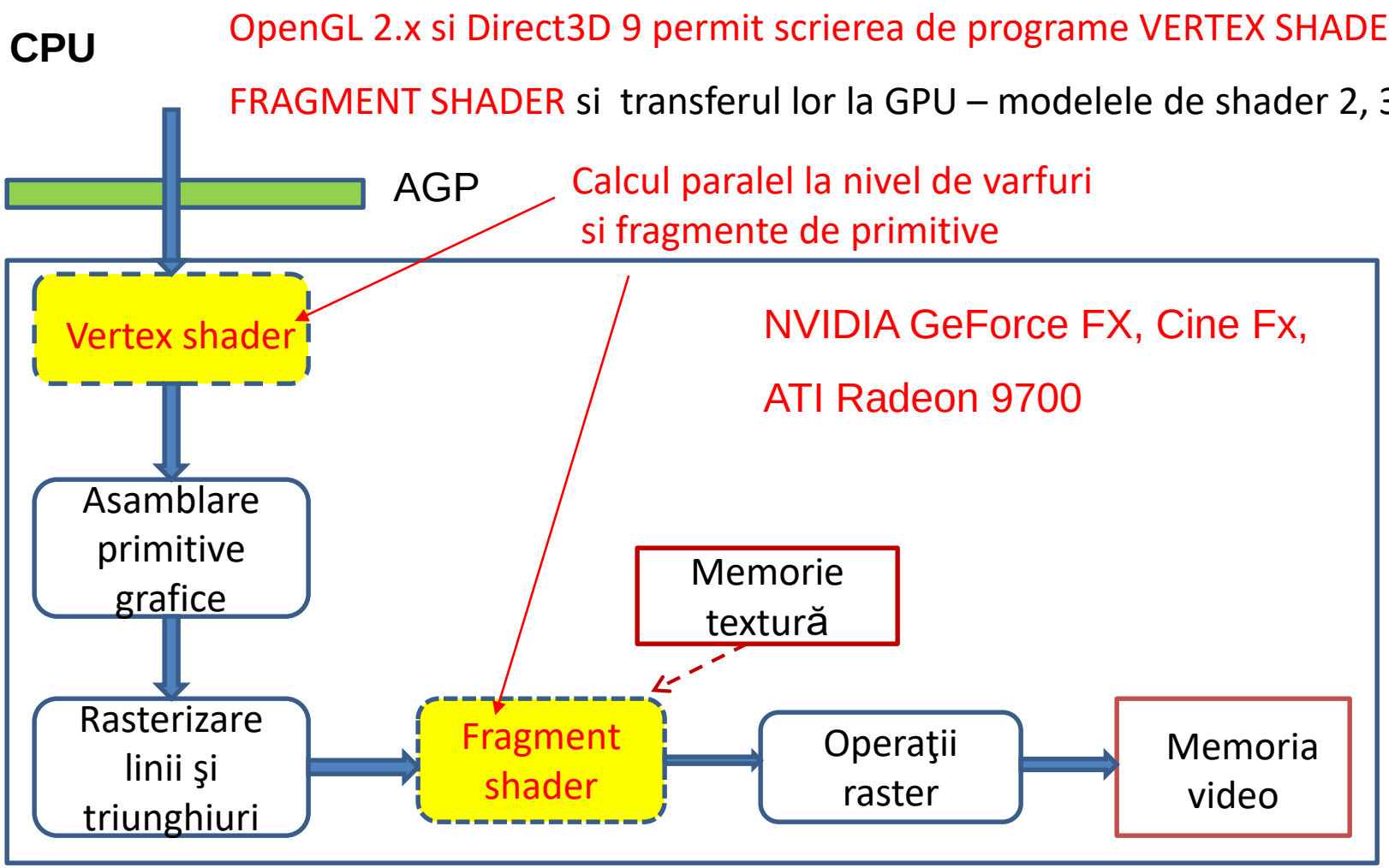


Scurt istoric al hardware-ului grafic de sinteza(3)

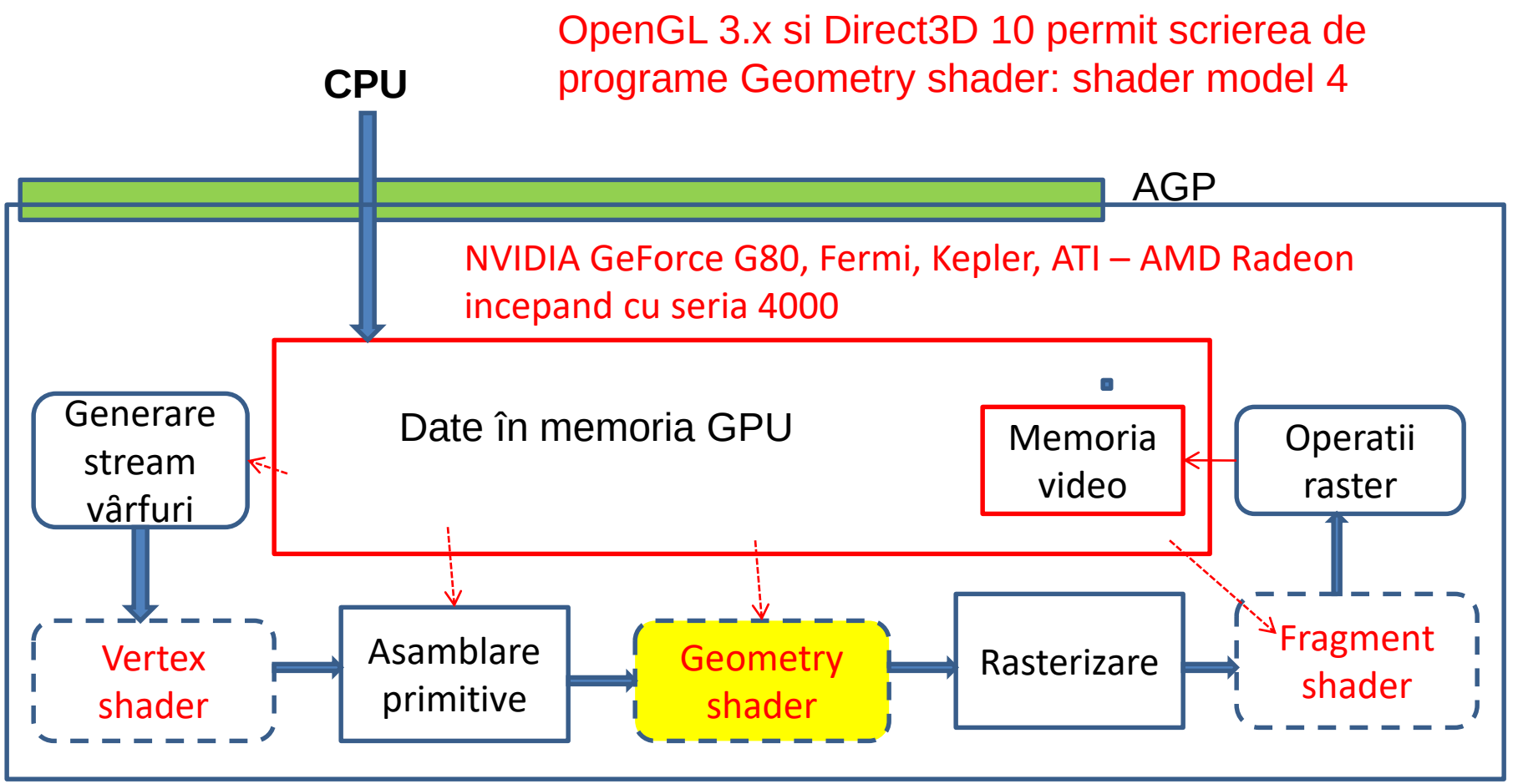
❖ **A 4-a generatie GPU (2003 - 2005):**

- NVIDIA GeForce FX, Cine Fx, ATI Radeon 9700
- Procesoare programabile pentru calcule la nivel de varfuri si de fragment (pixel)
- Procesoare specializate pentru calcule cu vectori si matrici
- Calcul paralel la nivel de varfuri si fragmente de primitive
- OpenGL 2.x si Direct3D 9 permit scrierea de programe VERTEX SHADER si PIXEL SHADER si transferul lor la GPU – shader model 2, 3
- Bibliotecile OpenGL si Direct3D au evolut odata cu hardware-ul grafic
  - OpenGL – independenta de platforma (imlementat pe majoritatea sist. de operare)
  - Direct3D – API proprietar Microsoft, pentru aplicatii Windows
- Limbajele pentru scrierea de shadere: GLSL (OpenGL Shading Language) – parte din OpenGL si HLSL (High-Level Shading Language) – parte din Direct3D

Banda grafică- a IV-a generaţie GPU (2003-2005)



Banda grafică- a V-a generaţie GPU (2006-2010)



GPU (placa grafică)

Scurt istoric al hardware-ului grafic de sinteza(4)

❖**A 5-a generatie GPU (2006 ->): General Purpose GPUs**

- ➢NVIDIA GeForce G80, Fermi, Kepler, ATI – AMD Radeon incepand cu seria 4000
- ➢Adauga etapa programabila GEOMETRY SHADER – in plus fata de generatia a 4-a
- ➢OpenGL 3.x si Direct3D 10 permit scrierea de programe GS → shader model 4

❖**A 6-a generatie GPU (2011 ->):**

- ➢Procesoare programabile pentru teselare (marirea rezolutiei geometrice) – TESSALLATION SHADER
- ➢OpenGL4.x si Direct3D 11 – Shader model 5

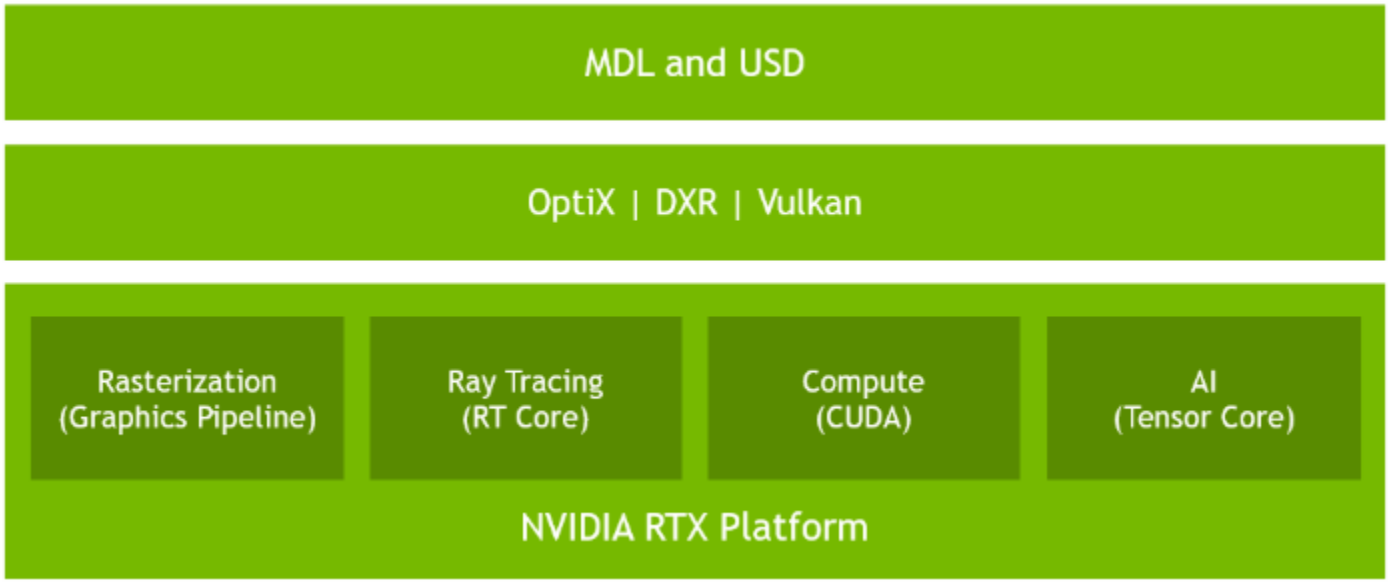
GPGPU (General Purpose Graphics Processing Unit)

- **Performanța placii grafice (arhitectura multiprocessor, grad inalt de parallelism, procesoare dedicate, stream processing)** → programare de aplicații ne-grafice pentru executie pe placa grafică: **GPGPU (General Purpose Graphics Processing Unit)**
- Limbaje de programare paralelă pentru executie pe placa grafica: CUDA, OpenCL
- ComputeShader (scris in GLSL) – program in care se poate implementa orice calcul paralel – nu se executa in banda grafica

Scurt istoric al hardware-ului grafic de sinteza(5)

NVIDIA RTX – cea mai recenta si mai avansata arhitectura GPU

Platforma NVIDIA RTX permite implementarea algoritmului Ray-tracing folosind API-uri si kituri software de dezvoltare dedicate: OptiX, Microsoft si DXR Vulkan.



Evaluare la disciplina EGC

- **Laborator:**
  - 1.5 p – activitate la laborator
  - 4.5 p – teme (3 teme : 1.5p x3) prezentate la laborator
- **Examen**:
  - 4 puncte
- Se acorda bonusuri pentru:
  - Teme cu realizare deosebita (peste cerinţe) si activitate la laborator: max 0.5 + 0.5 puncte
  - Prezenţa la curs: max 1 punct

- **Conditii de promovare:**
  - **Minim 50% punctaj pentru activitatea pe parcurs**
  - **Minim 50% punctaj in examen**

Transformari Geometrice 2D

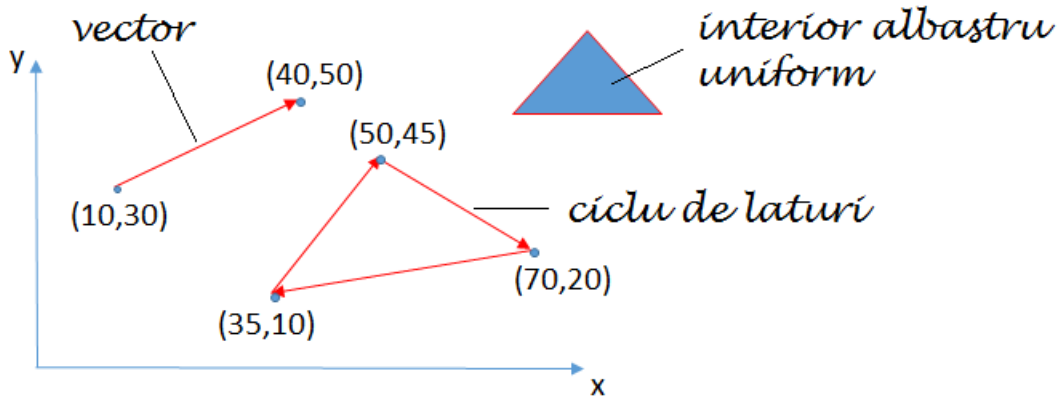
Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Transformari geometrice(1)

Obiectele 2D/3D sunt reprezentate prin:

- **Date geometrice**: coordonatele varfurilor, raportate la un sistem de coordonate carteziene 2D sau 3D;
- **Atribute topologice**, care definesc modul in care sunt conectate varfurile (laturi, ciclul de laturi al unei feţe, ş.a.);
- **Atribute de aspect**: culoarea, tipul de interior pentru suprafete 2D, atribute de material pentru suprafete 3D (ex: reflexia/refractia luminii de catre suprafata), s.a.



Transformarile geometrice se aplica coordonatelor varfurilor obiectului si nu afecteaza atributele sale!

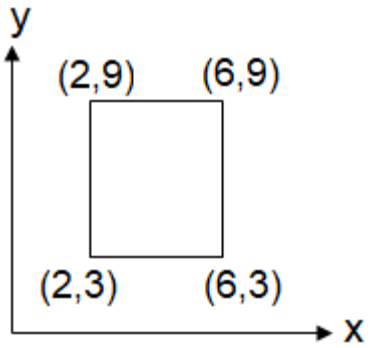
Transformari geometrice (2)

- Sunt operatii fundamentale in sinteza imaginilor
- Folosite pentru:
  - Redarea desenelor/obiectelor 2D/3D la diferite marimi
  - Compunerea desenelor/scenelor 3D din mai multe obiecte
  - Realizarea animatiei
  - Transformarea obiectelor dintr-un sistem de coordinate in alt sistem de coordonate
  - Etc.

Transformari geometrice 2D (1)

Sunt transformari ale obiectelor definite in sistemul de coordonate carteziene XOY, numite şi obiecte 2D. Fiecare vârf al unui obiect 2D este definit printr-o pereche de coordonate (x,y).

Transformarile geometrice 2D elementare



- Translaţia
- Scalarea faţă de origine
- Rotaţia faţă de origine
- Forfecarea faţă de origine
- Oglindirile faţă de axele principale şi faţă de origine

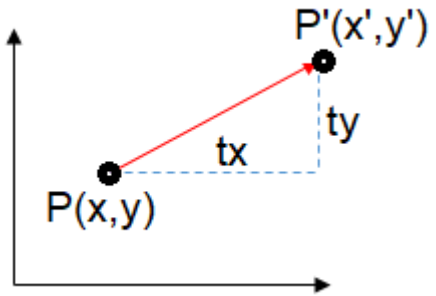
Orice alta transformare se obtine prin compunerea a 2 sau mai multe transformari elementare

- Scalarea /rotaţia compusa cu translatie
- Forfecare față de un punct oarecare din plan, etc

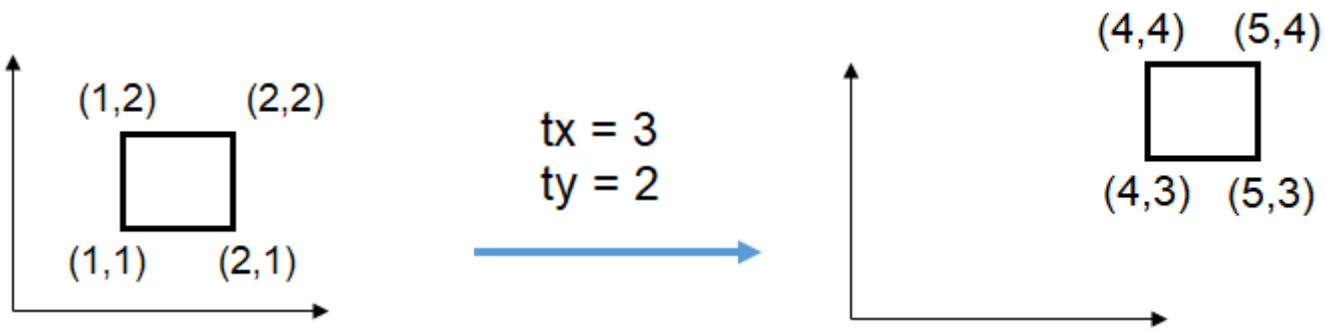
Transformari geometrice 2D elementare (1)

- Translatia (1)

  Este definita printr-un vector, T[tx,ty].

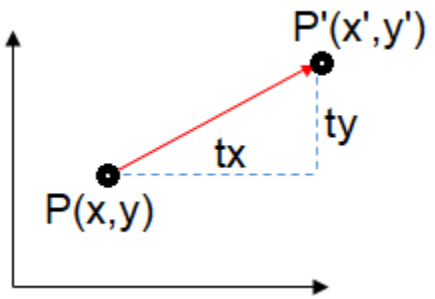


$$x' = x + tx$$
$$y' = y + ty$$



Transformari geometrice 2D elementare (2)

- Translatia (2)



$$x' = x + tx$$
$$y' = y + ty$$

- Se doreste o reprezentare matriciala a transformarii, pentru compunerea sa cu alte transformari, de ex. translatie şi scalare: compunerea transformarilor se realizeaza prin înmultirea matricilor transformarilor elementare.
- Un punct din plan, P(x,y), se reprezintă în coordonate carteziene printr-un vector [x, y] sau $\begin{bmatrix} x \\ y \end{bmatrix}$

→ Nu exista o matrice M, de 2x2, astfel încat sa exprimam translatia prin:

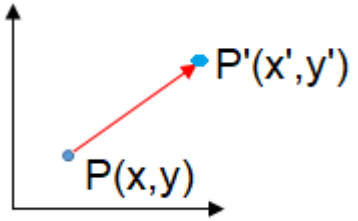
[x', y'] = [x, y] * M sau $\begin{bmatrix} x' \\ y' \end{bmatrix} = M * \begin{bmatrix} x \\ y \end{bmatrix}$

Transformari geometrice 2D elementare (3)

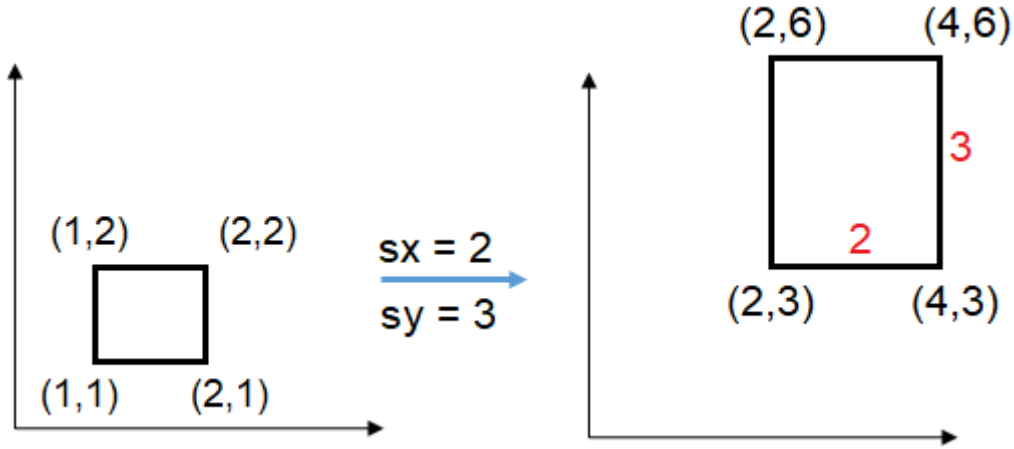
Scalarea fata de origine

Este definita prin 2 numere reale, de regula pozitive:

- sx - scalarea pe axa OX
- sy - scalarea pe axa OY



x' = x * sx
y' = y * sy



Reprezentarea matriciala

$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

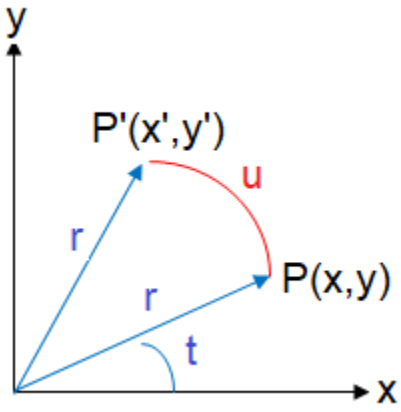
Efecte:

- Daca sx sau sy sau ambii sunt >1: ….
- Daca sx sau sy sau ambii sunt <1:…….

➢ sx = sy , scalare uniforma

Transformari geometrice 2D elementare (4)

Rotaţia fata de origine



x = r*cos(t)
y = r*sin(t)

Relatia dintre coordonatele carteziene si coordonatele polare ale unui punct P(x,y)

x' = r*cos(t+u) = r*(cos(t)*cos(u) – sin(t)*sin(u)) = x*cos(u) – y*sin(u)
y' = r*sin(t+u) = r*(cos(t)*sin(u) + sin(t)*cos(u)) = x*sin(u) – y*cos(u)

Rotatia fata de origine

x' = x*cos(u) - y*sin(u)
y' = x*sin(u) + y*cos(u)

$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) \\ \sin(u) & \cos(u) \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformari geometrice 2D fata de un punct oarecare din plan (1)

Scalarea fata de un punct oarecare din plan

- Punctul fix al transformării este un punct oarecare F(xf,yf) – coordonatele sale nu se modifica prin aplicarea transformării.
- P(x,y) este punctul transformat
- **Scalarea se aplica vectorului FP [x-xf, y-yf]. Prin scalare se transforma în FP' [x'-xf, y'-yf]:**

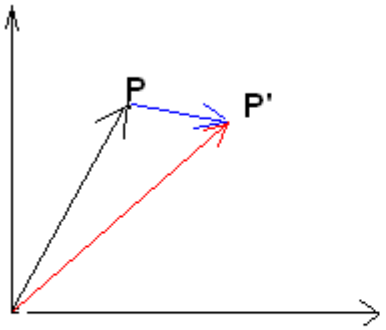
  x' – xf = sx*(x-xf)

  y' – yf = sy*(y – yf)

  Rezultă:

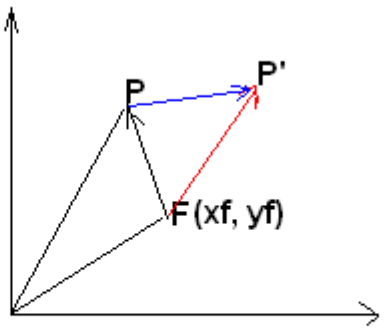
  x' = x*sx + xf –xf*sx

  y' = y*sy + yf –yf*sy



Scalarea fata de origine

x' - 0 = sx (x - 0)
y' - 0 = sy (y - 0)



Scalarea fata de un punct F din plan

x' - xf = sx (x - xf)
y' - yf = sy (y - yf)

Nu poate fi exprimata în coordonate carteziene printr-o matrice de 2x2!

Transformari geometrice 2D fata de un punct oarecare din plan(2)

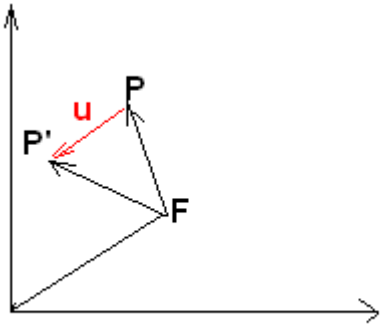
Rotatia fata de un punct oarecare din plan

- Punctul fix al transformarii este F(xf,yf)
- P(x,y) este punctul transformat
- **Rotatia se aplica vectorului FP [x-xf, y-yf], in jurul lui F.**

  Rezulta vectorul FP'[x' – xf, y' – yf]

  x' – xf = (x-xf)* cos(u) - (y – yf) * sin(u)

  y' – yf = (x-xf)* sin(u) + (y – yf) * cos(u)



Rotatia fata de un punct F din plan

Rezultă:

x' = x*cos(u) – y*sin(u) + xf – xf*cos(u) + yf*sin(u)

y' = x*sin(u) + y*cos(u) + yf – xf*sin(u) – yf*cos(u)

Nu poate fi exprimata in coordonate carteziene printr-o matrice de 2x2!

Compunerea transformarilor geometrice

De ce este necesară?

- Pentru a putea aplica o singura transformare care înglobeaza o secvenţă de transformari elementare, în locul aplicării fiecarei transformări din secventa; de ex., se aplică tuturor vârfurilor o transformare care înglobează scalare (S), rotatie (R) si translatie (T) în loc să se aplice fiecarui varf secventa de transformari elementare.

  Pt = M * P sau Pt = P*M^T , unde M^T este transpusa matricei M

  în loc de P' = S * P; P" = R * P'; Pt = T * P" unde M = T * R * S

- Matricea unei transformari compuse se obtine prin înmultirea matricilor transformarilor elementare. Exemplu:

SR = S*R =
$$\begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

RS = S*R =
$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

Atentie: RS # SR

- **Translatia nu poate fi reprezentata în coordonate carteziene printr-o matrice!**
- **Aceasta impune reprezentarea transformarilor în coordonate omogene.**

Reprezentarea transformarilor geometrice 2D în coordonate omogene(1)

Coordonate omogene:

Un punct din plan, P(x,y), se reprezinta in coordonate omogene printr-un vector

[xw, yw, w] sau $\begin{bmatrix} xw \\ yw \\ w \end{bmatrix}$ xw = x * w; yw = y * w; w – orice numar real

Exemplu: P(2, 0.5) → [2, 0.5, 1], [4, 1, 2], [20, 5, 10]

Transformarea din coordonate omogene in coordonate carteziene:

[xw yw w] → P(x, y), unde:

- pentru w #0, x = xw/w, y = yw/w
- pentru w =0, P este un punct la infinit

Exemple: [1, 0, 0] este un punct la infinit pe directia axei OX,
[a, a, 0] este un punct la infinit pe directia dreptei [0,0] → [a,a]

Reprezentarea transformarilor elementare 2D în coordonate omogene

Translația

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

In OpenGL se folosesc vectori coloană pentru a reprezenta puncte din plan sau din spaţiu.

Scalarea față de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotația fața de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} \cos(u) & \sin(u) & 0 \\ -\sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) & 0 \\ \sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformarile inverse ale transformarilor elementare

T(tx, ty): matricea translatiei, in coordonate omogene

S(0, 0, sx, sy): matricea scalarii fata de origine, in coordonate omogene

R(0,0,u): matricea rotatiei fata de origine, in coordonate omogene

Transformarile inverse sunt:

$T(tx, ty)^{-1} = T(-tx, -ty)$

$S(0, 0, sx, sy)^{-1} = S(0, 0, 1/sx, 1/sy)$

$R(0,0, u)^{-1} = R(0, 0, -u)$

Transformari geometrice 2D compuse(1)

Exemple de transformari compuse: scalarea/rotatia fata de un punct oarecare din plan.

- Matricea transformarii compuse se obtine prin înmulțirea matricilor urmatoarelor transformari elementare:
  - Translaţia prin care punctul fix al transformării ajunge în origine: T(-xf, -yf);
  - Scalarea / rotaţia faţă de origine: S(0,0,sx,sy)/R(0,0,u);
  - Translaţia inversă celei de la punctul 1: T(xf, yf).

[x' y' 1] = [x y 1] * T(-xf, -yf) * S(0,0,sx,sy)/R(0,0,u) * T(xf, yf)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = T(xf, yf) * S(0,0,sx,sy)/R(0,0,u) * T(-xf, -yf) * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformari geometrice 2D compuse(2)

Scalarea faţă de punctul F(xf,yf)

$$[x'\ y'\ 1] = [x\ y\ 1] \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_f & -y_f & 1 \end{bmatrix}}_{T(-xf,\ -yf)} \underbrace{\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{S(0,\ 0,\ sx,\ sy)} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_f & y_f & 1 \end{bmatrix}}_{T(xf,\ yf)}$$

Matricea transformarii compuse:

M = T(-xf, -yf) * S(0, 0, sx, sy) * T(xf, yf)

Aplicarea transformarii compuse:

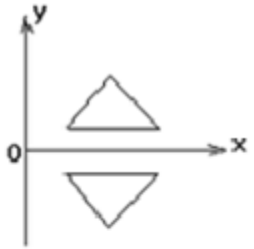
[x' y' 1] = [x y 1] * M

Alte transformari geometrice 2D(1)
Simetria (Oglindirea)

Față de axa OX:

$x' = x$

$y' = -y$

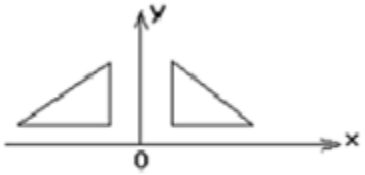


$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de axa OY:

$x' = -x$

$y' = y$

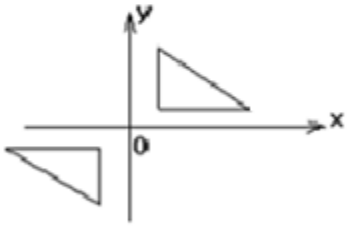


$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de origine:

$x' = -x$

$y' = -y$

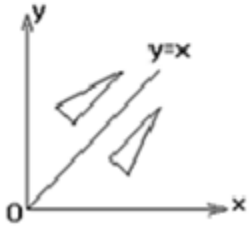


$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de dreapta y=x:

$x' = y$

$y' = x$



$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Alte transformari geometrice elementare 2D(2)
Oglindirea faţă de o dreaptă oarecare

Se exprimă ca transformare compusa prin înmulţirea matricilor urmatoarelor transformari:

1. O translaţie, astfel încât dreapta sa treaca prin origine: T(tx,ty)

2. O rotaţie faţă de origine, a.î. dreapta să se suprapună peste una dintre axele principale: R(0,0,u)

3. Oglindirea faţă de axa principală peste care a fost suprapusă dreapta: O

4. Rotaţia inversă celei de la punctul 2: R^{-1} = R(0,0,-u)

5. Translaţia inversă celei de la punctul 1: T^{-1} = T(-tx,ty)

În notaţie matricială: **M = T * R* O* R^{-1} *T^{-1}** (folosind vectori linie) **sau**

M = T^{-1} * R^{-1} *O *R *T (folosind vectori coloana)

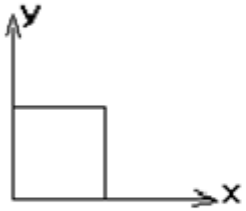
Deduceti elementele matricilor T, R, O, atunci cand dreapta este data printr-un punct, (xd, yd) şi o directie, D[a, b].

Alte transformari geometrice 2D (3)
Forfecarea

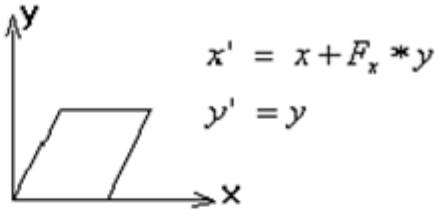
Forfecarea față de origine – transformare elementara- este definita prin 2 numere reale:

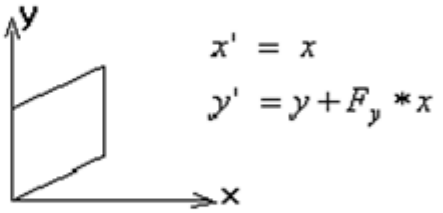
Fx: factorul de forfecare pe axa OX

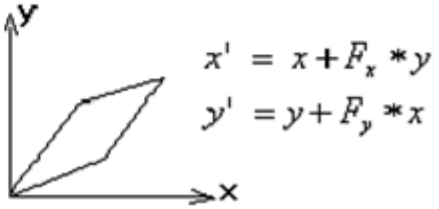
Fy: factorul de forfecare pe axa OY



Forfecarea fata de origine







Deduceti formele matriciale ale transformarilor de forfecare față de origine

Forfecarea fata de un punct oarecare din plan, (xf,yf) , exprimata ca transformare compusa:

1. Translatie prin care punctul (xf, yf) ajunge in origine
2. Forfecarea fata de origine
3. Translatia inversa celei de la pasul 1

Implementare transformari 2D
- exemple -

Translatie patrat (1)

```
//TranslatiePatrat.h
class TranslatiePatrat : public SimpleScene
{ // suprascrie functii din clasele de baza ale ierarhiei, apelate de sistemul Windows
public:
TranslatiePatrat();
~TranslatiePatrat();
void Init() override; //apelata la inceputul executiei aplicatiei
private:
void FrameStart() override;
void Update(float deltaTimeSeconds) override;//apelata pentru crearea frame-ului curent
void FrameEnd() override;
void OnInputUpdate(float deltaTime, int mods) override;
void OnKeyPress(int key, int mods) override; // apelata la apasara unei taste
void OnKeyRelease(int key, int mods) override;
```

Translatie patrat (2)

```
//TranslatiePatrat.h
void OnMouseMove(int mouseX, int mouseY, int deltaX, int deltaY) override;
void OnMouseBtnPress(int mouseX, int mouseY, int button, int mods) override;
void OnMouseBtnRelease(int mouseX, int mouseY, int button, int mods) override;
void OnMouseScroll(int mouseX, int mouseY, int offsetX, int offsetY) override;
void OnWindowResize(int width, int height) override;//apelata la
//redimensionarea ferestrei aplicatiei

protected:
glm::mat3 modelMatrix; //matrice de transformare
int latura; //latura patratului
int xc, yc; //centrul patratului
Mesh* square;//desenul patratului
int nr_figuri;
};
```

Translatie patrat (3)

```
//TranslatiePatrat.cpp
void TranslatiePatrat::Init() //functie apelata o singura data, la inceputul executiei aplicatiei
{   glm::ivec2 resolution = window->GetResolution();
  auto camera = GetSceneCamera();
  camera->SetOrthographic(0, (float)resolution.x, 0, (float)resolution.y, 0.01f, 400);
  camera->SetPosition(glm::vec3(0, 0, 50));
  camera->SetRotation(glm::vec3(0, 0, 0));
  camera->Update();
  GetCameraInput()->SetActive(false)
  latura = 100; //latura patratului
  xc = 100; yc = 100; //centrul patratului
  glm::vec3 corner = glm::vec3(xc - latura / 2, yc - latura / 2, 0); //coltul stanga-jos
  square = Object2D::CreateSquare("square", corner, latura, glm::vec3(1, 0, 0));
 AddMeshToList(square); //adauga la lista figurilor de redat
 nr_figuri = 1;
}
```

Translatie patrat (4)

```
void TranslatiePatrat::Update(float deltaTimeSeconds) // apelata pentru fiecare frame
{ // deltaTimeSeconds – durata de procesare a frame-ului anterior
  modelMatrix = Transform2D::Translate(latura*nr_figuri, latura*nr_figuri);
//afiseaza in cadrul current patratul transformat cu matricea modelMatrix
RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
}
void TranslatiePatrat::OnKeyPress(int key, int mods)
{  nr_figuri +=1 ;
   if (nr_figuri >10)
              nr_figuri = 1;
}
//Clasa Transform2D implementeaza transformarile 2D
//Transform2D.h
inline glm::mat3 Translate(float translateX, float translateY)
{
 // se creaza matricea de translatie apeland functia mat3 din biblioteca glm
return glm::transpose( glm::mat3(1, 0 , translateX, 0, 1 , translateY, 0, 0, 1));
}
```

Translatie patrat (5)

```
void TranslatiePatrat::OnWindowResize(int width, int height)
{ //apleata la redimensionarea ferestrei aplicatiei
//trebuie sa refaca continutul ferestrei
auto camera = GetSceneCamera();
camera->SetOrthographic(0, (float)width, 0, height, 0.01f, 400);
camera->SetPosition(glm::vec3(0, 0, 50));
camera->SetRotation(glm::vec3(0, 0, 0));
camera->Update();

xc = 50; yc = 50;
glm::vec3 corner = glm::vec3(xc + latura / 2, yc - latura / 2, 0);
square = Object2D::CreateSquare("square", corner, latura, glm::vec3(1, 0, 0));
AddMeshToList(square);
}
```

Scalare față de origine

```
//ScalareOrigine.cpp
// Init() la fel ca in TranslatiePatrat
void ScalareOrigine::Update(float deltaTimeSeconds)
{   modelMatrix = Transform2D::Scale(2* nr_figuri, 2* nr_figuri);
    RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
}
void TranslatiePatrat::OnKeyPress(int key, int mods)
{  nr_figuri +=1 ;
   if (nr_figuri >10) nr_figuri = 1;
}
//Transform2D.h
inline glm::mat3 Scale(float scaleX, float scaleY)
{
  return glm::mat3 (scaleX, 0, 0, 0, scaleY, 0, 0, 0, 1);
}
```

Rotatie față de origine

```
//RotatieOrigine.cpp

// Init() la fel ca in TranslatiePatrat

void RotatieOrigine::Update(float deltaTimeSeconds)
{
	if(nr_figuri==1) modelMatrix = glm::mat3(1);
		else modelMatrix = Transform2D::Rotate(0.05* nr_figuri);
	RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
 }

//definita in Transform2D.h
inline glm::mat3 Rotate(float radians)
	{
		float cosu = glm::cos(radians);
		float sinu = glm::sin(radians);
		return glm::transpose( glm::mat3(cosu, -sinu, 0, sinu, cosu, 0, 0, 0, 1));
	}
```

Scalare față de centrul patratului (1)

```
//ScalareCentru.cpp
void ScalareCentru::Init()
{
   glm::ivec2 resolution = window->GetResolution();
   xc = resolution.x / 2; yc = resolution.y / 2; //centrul ferestrei aplicatiei
// la redimensionarea ferestrei (ScalareCentru::OnWindowResize)
     // se recalculeaza xc, yc
   latura = 50;
// patrat centrat in centrul ferestrei aplicatiei
   glm::vec3 corner = glm::vec3(xc - latura / 2, yc - latura / 2,0);
   square = Object2D::CreateSquare("square", corner, latura, glm::vec3(1, 0, 0));
   AddMeshToList(square);
   nr_figuri = 1;
}
```

Scalare față de centrul patratului (2)

```
void ScalareCentru::Update(float deltaTimeSeconds)
{
   if(nr_figuri ==1) modelMatrix = glm::mat3(1);//matricea identica
   else
     {  modelMatrix = Transform2D::Translate(xc, yc) *
                      Transform2D::Scale(2 * nr_figuri, 2 * nr_figuri) *
                      Transform2D::Translate(-xc, -yc);
     }
   RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
}
void ScalareCentru::OnKeyPress(int key, int mods)
{  nr_figuri +=1 ;
  if (nr_figuri >10) nr_figuri = 1;
}
```

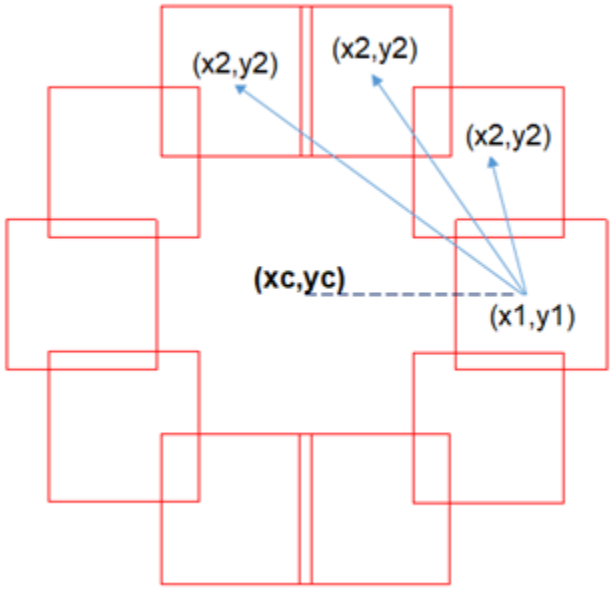
Rotatia față de centrul patratului

```
//RotatieCentru.cpp
// Init() si OnKeyPress() la fel ca in ScalareCentru.cpp

void RotatieCentru::Update(float deltaTimeSeconds)
{
   if(nr_figuri ==1) modelMatrix = glm::mat3(1);
    else
     {      modelMatrix = Transform2D::Translate(xc, yc) *
                                  Transform2D::Rotate(0.05 * nr_figuri) *
                                  Transform2D::Translate(-xc, -yc);
       }
    RenderMesh2D(square, shaders["VertexColor"],     modelMatrix);
}
```

Translatie patrat pe circumferinta unui cerc (1)

```
//AnimatiePatrat1.cpp

void AnimatieTranslatiePatrat1::Init()
{
 ……..
  glm::ivec2 resolution = window->GetResolution();
  xc = resolution.x / 2; yc = resolution.y / 2;
  latura = 100;//latura patrat
  raza = 150; //raza cercului
  angle = 0; //pasul unghiular de rotatie
  glm::vec3 corner = glm::vec3(xc + raza - latura / 2, yc - latura / 2, 0);
  square = Object2D::CreateSquare("square", corner, latura, glm::vec3(1, 0, 0));
   AddMeshToList(square);
}
```

Translatie patrat pe circumferinta unui cerc (2)



Translatie la apasarea unei taste

```
void AnimatieTranslatiePatrat1::OnKeyPress(int key, int mods)
{       angle += 0.5;
}
```

void AnimatieTranslatiePatrat1::Update(float deltaTimeSeconds)

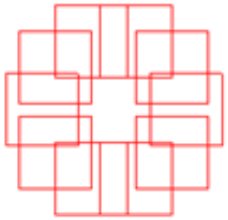
```
{     float x1 = xc + raza, y1 = yc, x2, y2;
      //centrul patratului translatat pe cerc
       x2 = xc + raza * cos(angle); y2 = yc + raza * sin(angle);
       modelMatrix = Transform2D::Translate(x2 - x1, y2 - y1);
       RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
}
```

Animatie patrat pe circumferinta unui cerc

```
//AnimatiePatrat2.cpp
void AnimatieTranslatiePatrat2::Update(float deltaTimeSeconds)
{
        float x1 = xc + raza, y1 = yc, x2, y2;

        angle += deltaTimeSeconds;
        x2 = xc + raza * cos(angle); y2 = yc + raza * sin(angle);
        modelMatrix = Transform2D::Translate(x2 - x1, y2 - y1);
        RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
}
```

Desen translatie patrat pe circumferinta unui cerc (1)

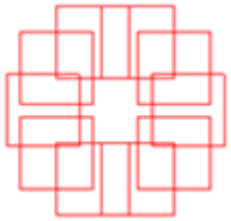
Se creaza desenul



```
//DesenTranslatiePatrat1.cpp
// Init() la fel ca in AnimatiePatrat1.cpp
void DesenTranslatiePatrat1::Update(float deltaTimeSeconds)
{
   float x1 = xc + raza, y1 = yc, x2, y2;
    for (int i = 0; i < 10; i++)
      {       x2 = xc + raza * cos(0.628f*i); y2 = yc + raza * sin(0.628f*i);
              modelMatrix = Transform2D::Translate(x2 - x1, y2 - y1);
              RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
      }
}
```

Desen translatie patrat pe circumferinta unui cerc (2)

Se creaza desenul iterativ, la fiecare apasare de tasta

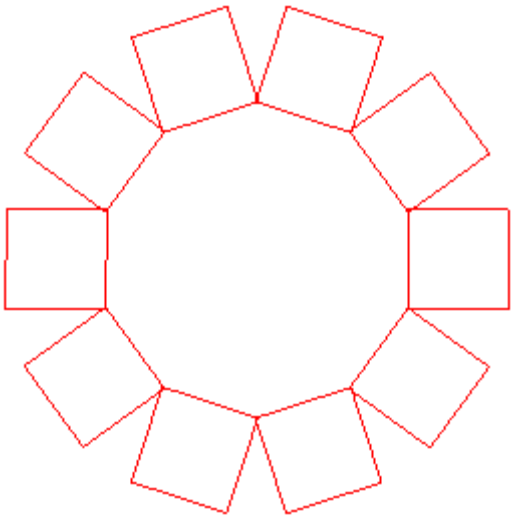


```
void DesenTranslatiePatrat2::Update(float deltaTimeSeconds)
{
    float x1 = xc + raza, y1 = yc, x2, y2;
    for (int i = 0; i < nr_figuri; i++)
            {
              x2 = xc + raza * cos(0.628f*i); y2 = yc + raza * sin(0.628f*i);
              modelMatrix = Transform2D::Translate(x2 - x1, y2 - y1);
              RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
            }
}
void DesenTranslatiePatrat2::OnKeyPress(int key, int mods)
{           nr_figuri +=1 ;
            if (nr_figuri > 10) nr_figuri = 1;
}
```

Rotatia unui patrat pe circumferinta unui cerc (1)

```
// AnimatieRotatiePatrat.cpp
// Init() la fel ca in AnimatiePatrat

void AnimatieRotatiePatrat::Update(float deltaTimeSeconds)
{
        angle += deltaTimeSeconds;
        modelMatrix = Transform2D::Translate(xc,yc) *
                Transform2D::Rotate(angle) *
                Transform2D::Translate(-xc, -yc);
        RenderMesh2D(meshes["square"], shaders["VertexColor"], modelMatrix);

}
```

Desen rotatie patrat pe circumferinta unui cerc



```
void DesenRotatiePatrat::Init()
{
        ……….
        latura = 100;
        raza = 200;
        ……..
}
void DesenRotatiePatrat::Update(float deltaTimeSeconds)
{
        for (int i = 0; i < 10; i++)
        {
            modelMatrix = Transform2D::Translate(xc, yc) *
                          Transform2D::Rotate(0.628f * i) *
                          Transform2D::Translate(-xc, -yc);
           RenderMesh2D(square, shaders["VertexColor"], modelMatrix);
        }
}
```

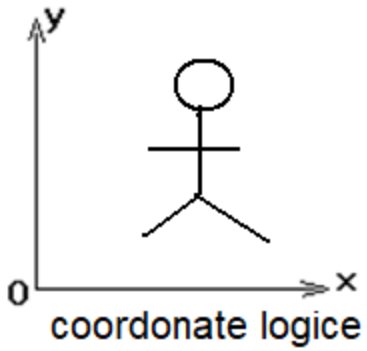
Transformarea de vizualizare 2D

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

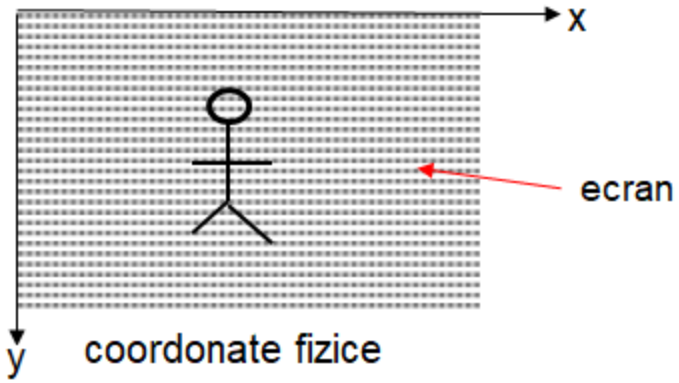
Coordonate logice şi coordonate fizice

- Desenele reprezentate într-un program de aplicatie grafica 2D sunt, de regula, raportate la un sistem de coordonate diferit de cel al suprafetei de afisare.
- Exemple: planul unei case, un desen tehnic, graficul unei functii, etc.
- Editorul grafic trebuie sa permita definirea desenului intr-un sistem de coordonate logice.



(metri, milimetri, viteza, timp, etc.)

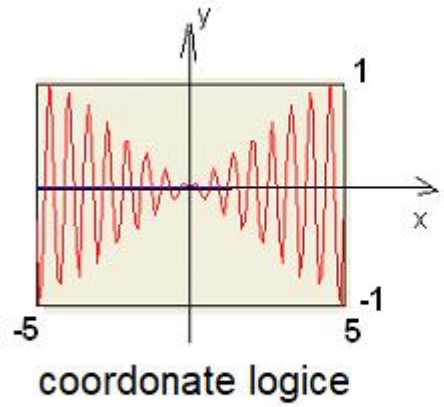
Sistemul de coordonate utilizator

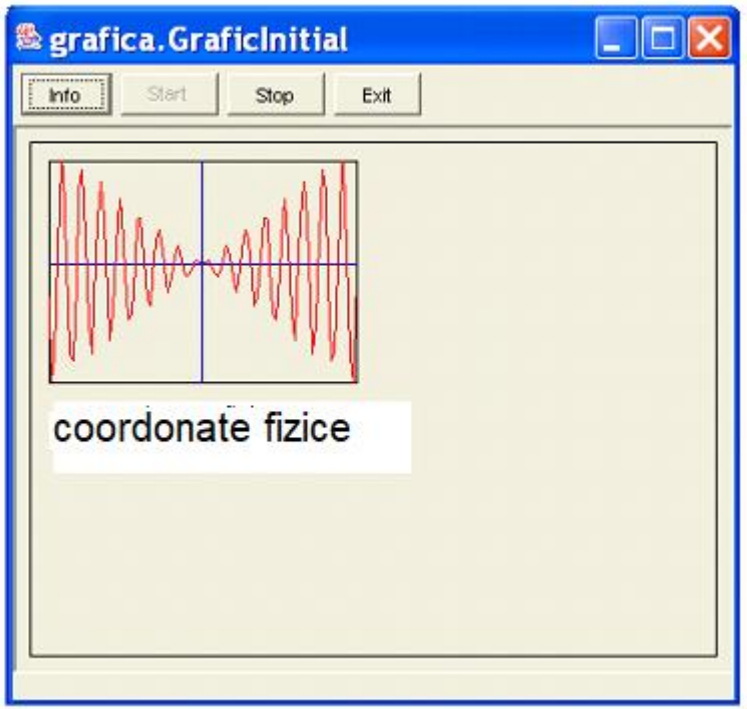


(pixeli, milimetri, s.a.)

Sistemul de coordonate dispozitiv

Transformarea coordonate logice - coordonate fizice



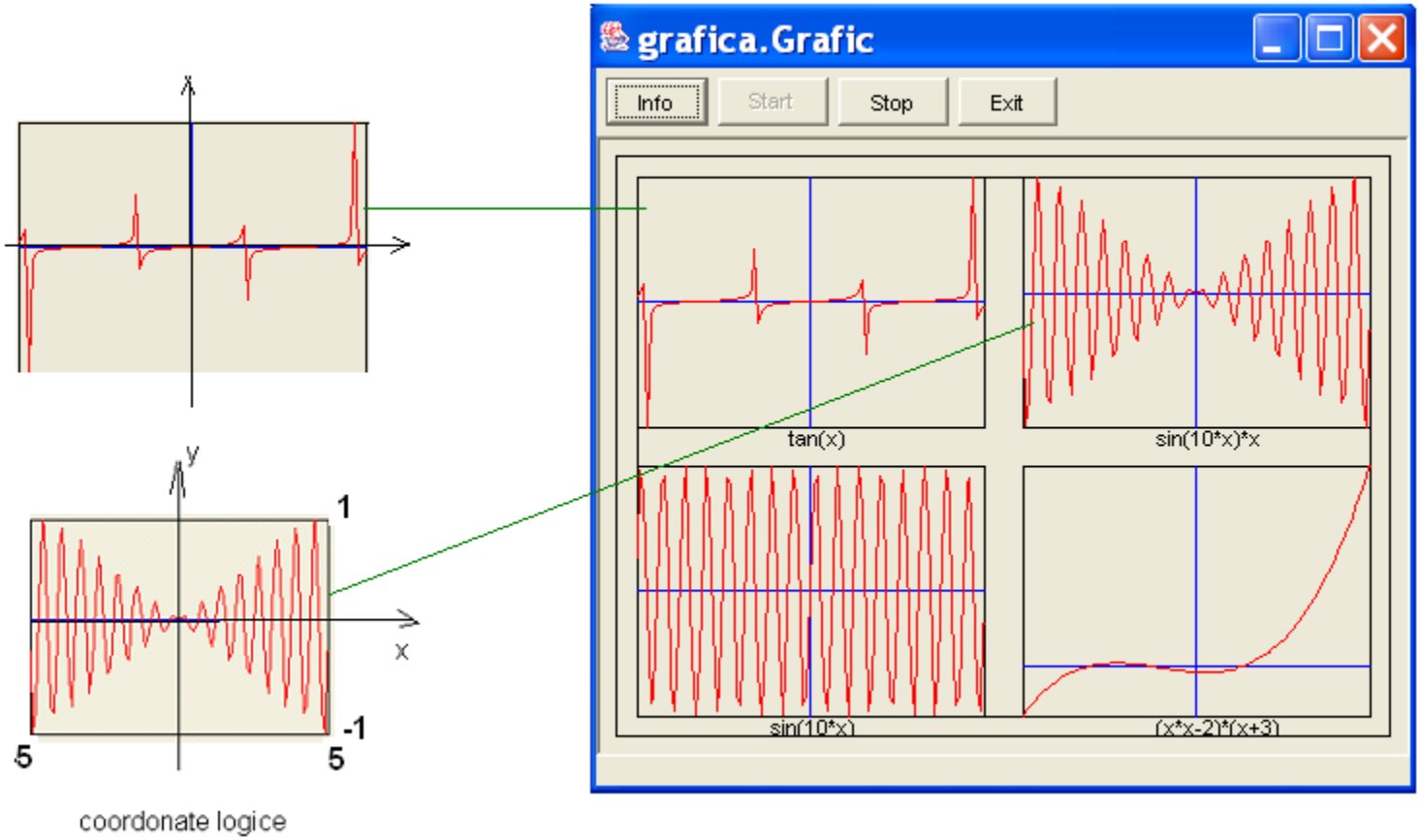


Aplicatia defineste graficul functiei in sistemul de coordonate utilizator (logice).

Coordonatele punctelor de pe graficul afisat pe ecran sunt adrese de pixeli.

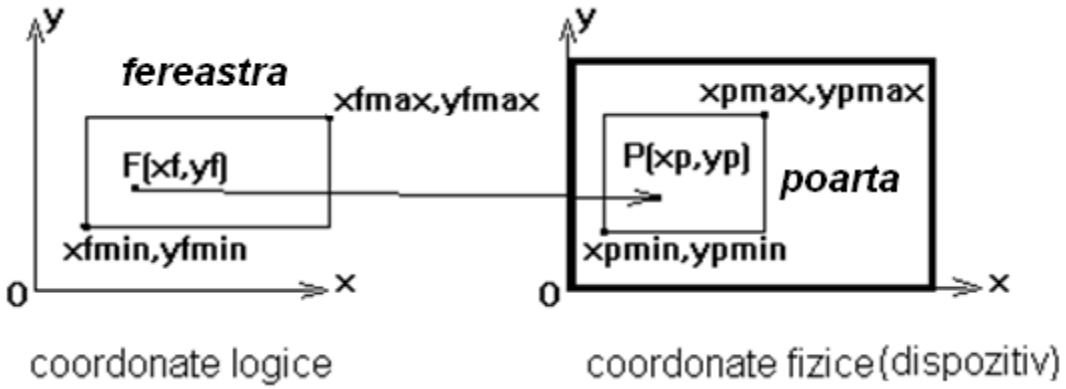
Coordonatele logice trebuie transformate in coordonate fizice.

Transformarea de vizualizare 2D (1)



Este necesara o transformare generala, prin care un desen definit in coordonate logice sa poata fi afisat intr-o zona oarecare a ecranului.

Transformarea de vizualizare 2D (2)
- transformarea "fereastra-poarta"-



- fereastra: zona din spatiul coord. logice al carei continut se va afisa
- poarta: zona din spatiul coord. fizice in care se va afisa continutul ferestrei

▪ Transformarea este definita folosind 2 dreptunghiuri, definite in cele 2 sisteme de coordonate, numite: ***fereastra (de vizualizare) –vizorul*** şi ***poarta (de afisare).***

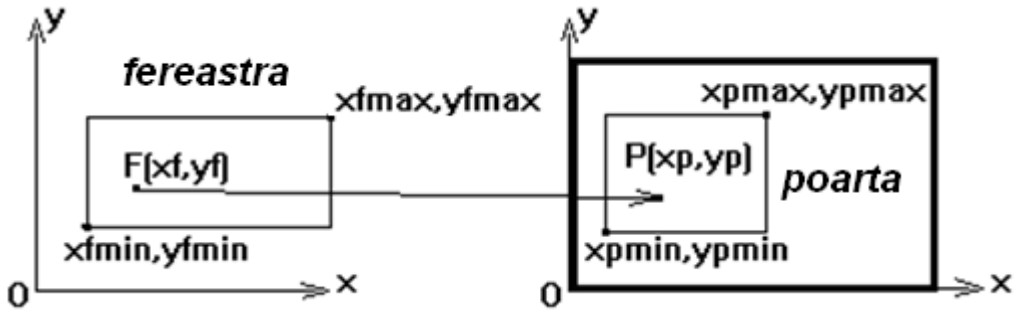
• Transformarea se mai numeste ***transformarea fereastra-poarta (window-viewport)***

F: un punct din fereastra.
P: punctul in care se transforma F prin transformarea fereastra-poarta.

➢Transformarea fereastra-poarta trebuie sa asigure ca:

" pozitia relativa a lui P in poarta de afisare este aceeasi cu pozitia relativa a lui F in fereastra".

Transformarea de vizualizare 2D (3)



$$\frac{xp - xp\min}{xp\max - xp\min} = \frac{xf - xf\min}{xf\max - xf\min}$$

$$\frac{yp - yp\min}{yp\max - yp\min} = \frac{yf - yf\min}{yf\max - yf\min}$$

$$sx = \frac{xp\max - xp\min}{xf\max - xf\min}$$

$$sy = \frac{yp\max - yp\min}{yf\max - yf\min}$$

- sx, sy reprezinta factorii de scalare ai transformarii
- tx, ty depind de pozitiile celor 2 ferestre fata de originea sistemului de coord. in care sunt definite

şi $tx = xpmin - sx*xfmin$ $ty = ypmin - sy*yfmin$

$$xp = xf * sx + tx$$
$$yp = yf * sy + ty$$

← Exprimarea matematica a transformarii fereastra – poarta, considerand o aceeasi orientare a axelor celor 2 sisteme de coordonate.

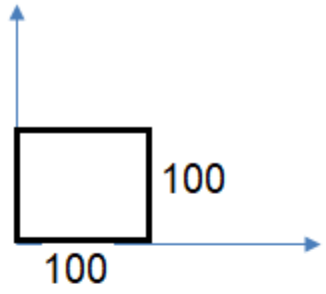
Transformarea de vizualizare 2D (4)

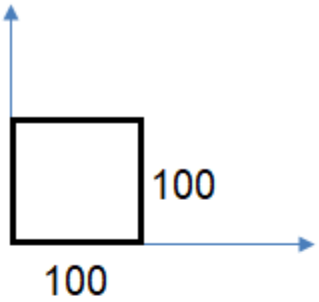
$$sx = \frac{xp\max - xp\min}{xf\max - xf\min}$$

$$sy = \frac{yp\max - yp\min}{yf\max - yf\min}$$

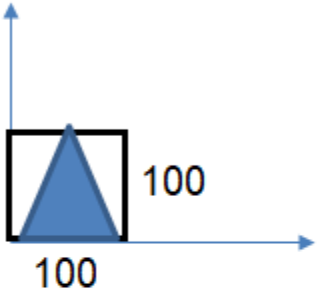
*tx = xpmin- sx*xfmin*

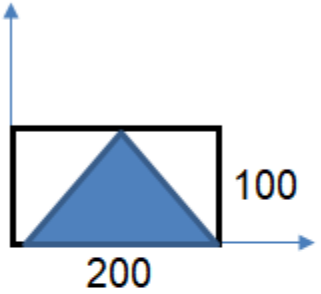
*ty= ypmin- sy*yfmin*



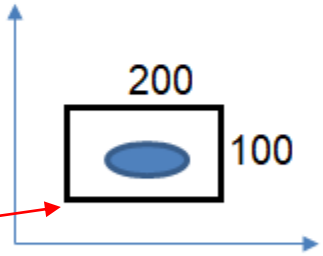


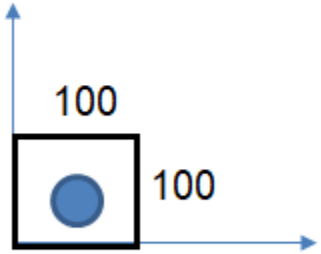
sx=1, sy = 1
tx = 0, ty =0



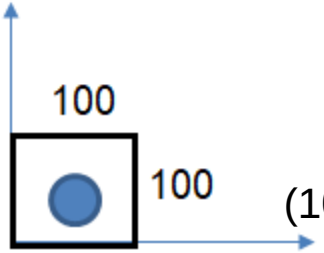


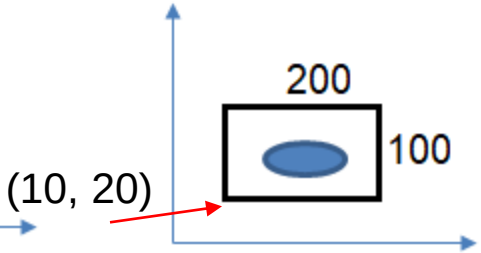
sx=2, sy = 1
tx = 0, ty =0





sx=0.5, sy = 1
tx = 0 - 0.5*10 = - 5
ty = 0 – 1*20 = -20

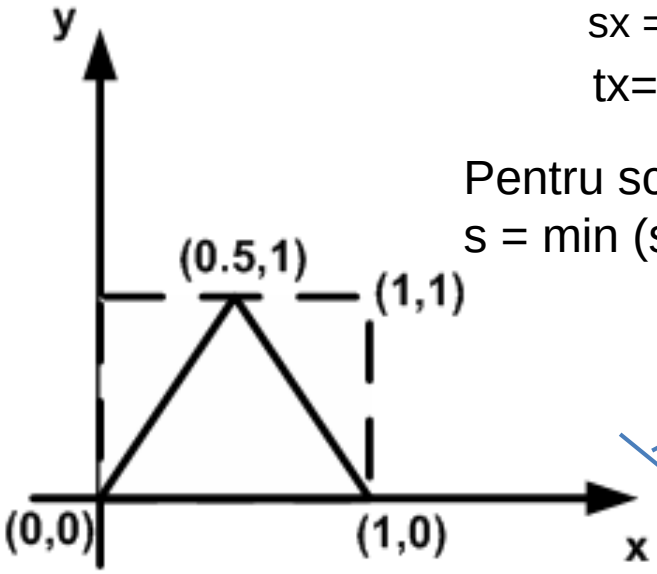




sx=2, sy = 1
tx = 10
ty = 20

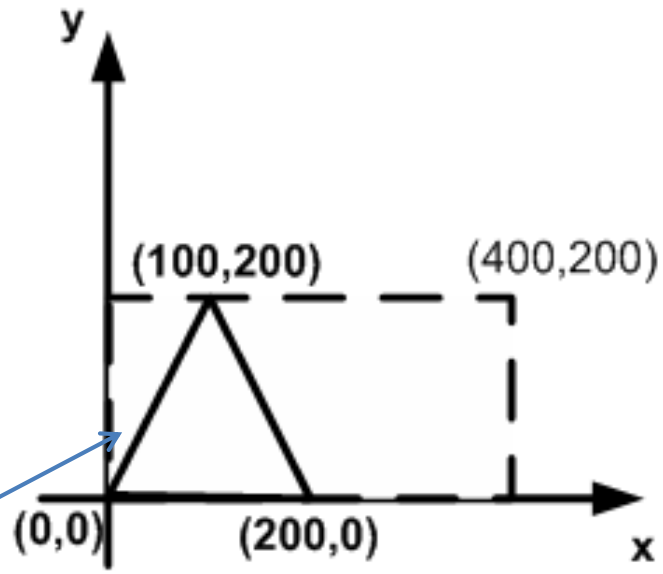
Transformarea de vizualizare 2D (5)

Scalare uniforma
cu afisare centrata in poarta de vizualizare

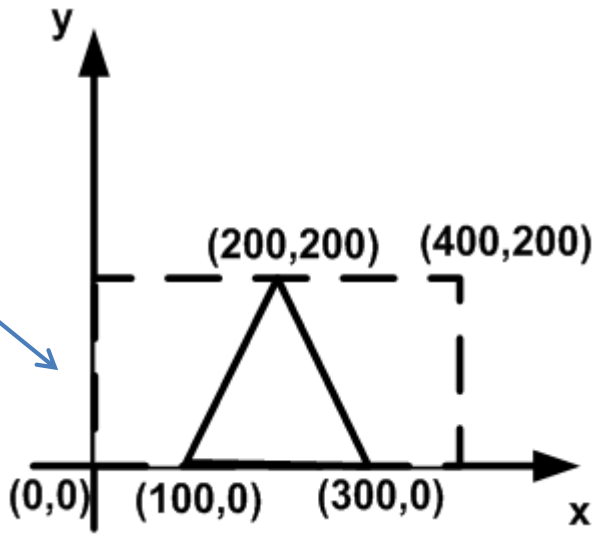


sx = 400, sy = 200
tx=0, ty =0

Pentru scalare uniforma:
s = min (sx, sy)



tsx = 100,
tsy =0



tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2

= (400 – 200*1) /2 = 100

tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2

= (200 – 200*1) /2 = 0

Transformarea de vizualizare 2D (7)

Efectele transformarii

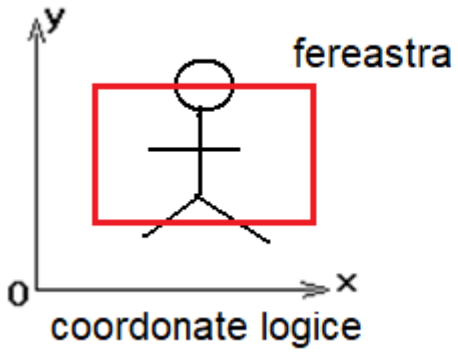
- Marire / micsorare, in functie de dimensiunile ferestrei si ale portii
- Deformare: fereastra si poarta nu sunt dreptunghiuri asemenea
- Pentru scalare uniforma: s = min(sx,sy)
- Afisare centrata in poarta: translatie suplimentara pe axa ox sau pe axa oy:

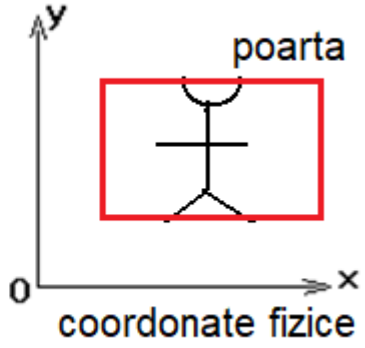
  tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2

  tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2

  Translatia totala: tx+tsx, ty+tsy

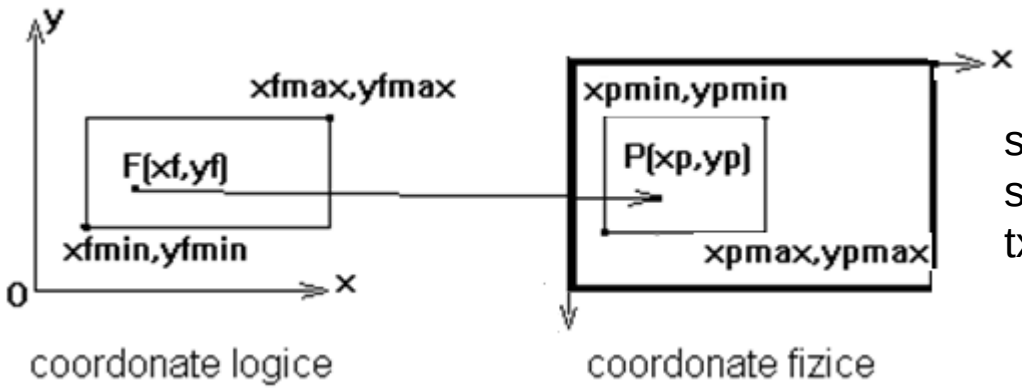
- Decuparea primitivelor aflate in afara ferestrei vizuale

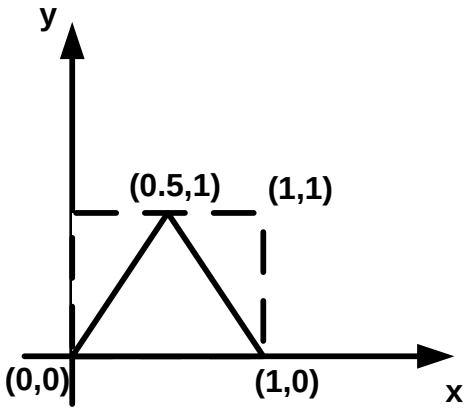




Transformarea de vizualizare 2D (8)

Corectia coordonatei yp atunci cand axa OY a sist. coord. fizice este orientata in jos





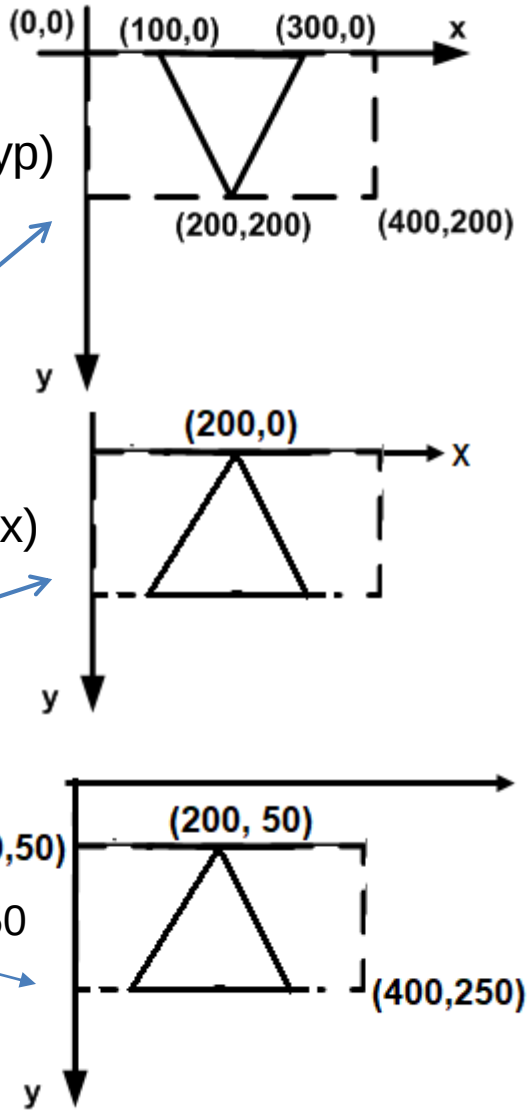
(xp, yp)

sx = 400, sy = 200
s=smin = 200
tx = tsx =100, ty =0

(xp, yp'= -yp+ypmax)

ty = 50

yp' = -yp + ypmax + ypmin



Transformarea de vizualizare 2D (9)

Formulele transformarii fereastra – poarta atunci cand axa OY a sistemului de coordonate fizice este orientata in jos:

xp = xf * sx + tx
yp = – (yf * sy + ty) + ypmin + ypmax = -yf * sy + ypmin + ypmax -ty

Exprimarea matriciala a transformarii fereastra-poarta in cazul corectiei coordonatei y

$$\begin{bmatrix} xp \\ yp \\ 1 \end{bmatrix} = M * \begin{bmatrix} xf \\ yf \\ 1 \end{bmatrix}$$

$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & -sy & yp\min + yp\max - ty \\ 0 & 0 & 1 \end{bmatrix}$$

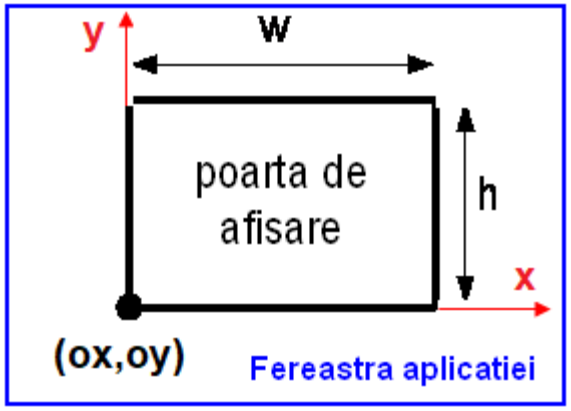
Implementarea transformarii de vizualizare în OpenGL (1)

- Pentru setarea zonei de afisare, în OpenGL se apeleaza

  glViewport(ox, oy, width, height);

unde (ox,oy) sunt coord. coltului stanga jos al viewportului, in coordonatele ferestrei aplicatiei.

- Orientarea axelor viewportului este aceeasi cu a sistemului coordonatelor logice:
  - Nu este necesara corectia coordonatei y → matricea transformarii de vizualizare este:



$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & sy & ty \\ 0 & 0 & 1 \end{bmatrix}$$

- Se poate defini un desen in coordonate logice.
- Pentru obtinerea desenului in coordonate ecran se foloseste ca matrice de transformare a vârfurilor - matricea transformării de vizualizare

Implementarea transformarii de vizualizare in OpenGL (2)

```
class TransformareaFereastraPoartaPatrate : public SimpleScene
{public:
    struct ViewportSpace
    {  ViewportSpace() : x(0), y(0), width(1), height(1) {}
       ViewportSpace(int x, int y, int width, int height)
          : x(x), y(y), width(width), height(height) {}
       int x, int y, int width, int height;
    };
    struct LogicSpace
    {  LogicSpace() : x(0), y(0), width(1), height(1) {}
       LogicSpace(float x, float y, float width, float height)
         : x(x), y(y), width(width), height(height) {}
    float x, float y, float width, float height;
    };
```

Implementarea transformarii de vizualizare in OpenGL (3)

```
public:
TransformareaFereastraPoartaPatrate();
~TransformareaFereastraPoartaPatrate();
void Init() override;
……….
 protected:
   float length;
   ViewportSpace viewSpace; // dreptunghiul “poarta de afisare”
   LogicSpace logicSpace; // dreptunghiul “fereastra”
   glm::mat3 modelMatrix,
         visMatrix; //matricea transformarii de vizualizare
   bool uniform; // transformare uniforma/neuniforma
// figura desenata se obtine prin rotatia unui patrat  in jurul centrului ferestrei
   float raza; // raza cercului pe care se roteste patratul
   float cx, cy; // centrul ferestrei
};
```

Implementarea transformarii de vizualizare in OpenGL (3)

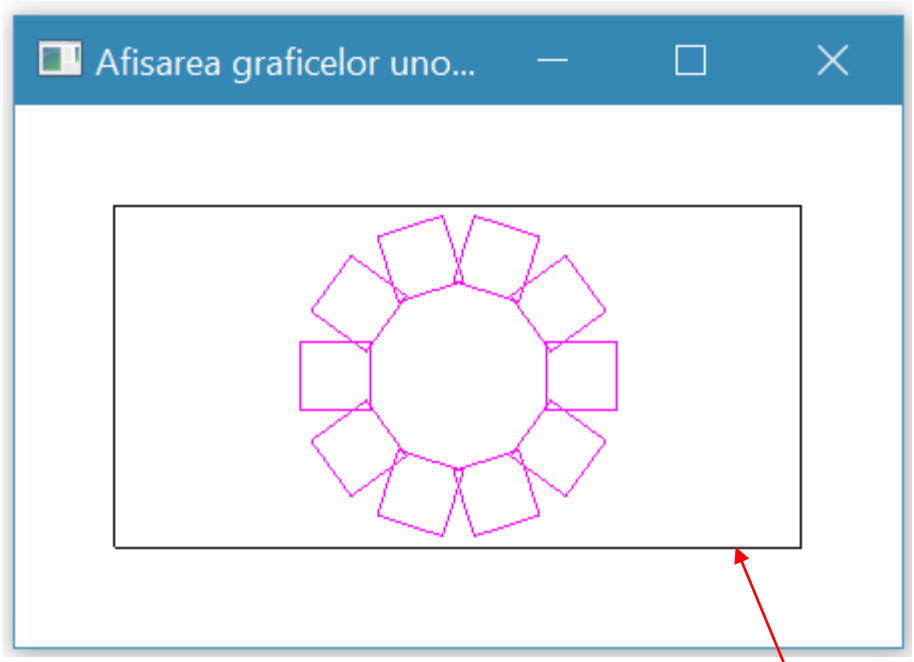
```
void TransformareaFereastraPoartaPatrate::Init()
{ ……..
   logicSpace.x = -5; // logic x
   logicSpace.y = -5; // logic y
   logicSpace.width = 10; // logic width
   logicSpace.height = 10; // logic height
   length = 2.0f; //latura patratului
   raza = 3.6f;

   cx = logicSpace.x + logicSpace.width / 2;
   cy = logicSpace.y +  logicSpace.height / 2;


   glm::vec3 corner = glm::vec3(cx - length / 2 + raza , cy - length / 2, 0);

   Mesh* square = Object2D::CreateSquare("square", corner, length, glm::vec3(1, 0, 0));

   AddMeshToList(square); // patratul care se roteste


   glm::vec3 cornerw = glm::vec3(logicSpace.x, logicSpace.y, 0);
   Mesh* window =
        Object2D::CreateSquare("window", cornerw, logicSpace.width, glm::vec3(0, 0, 0));
   AddMeshToList(window); // cadrul ferestrei afisat in negru

   uniform = true;
   }
```

Implementarea transformarii de vizualizare in OpenGL (4)

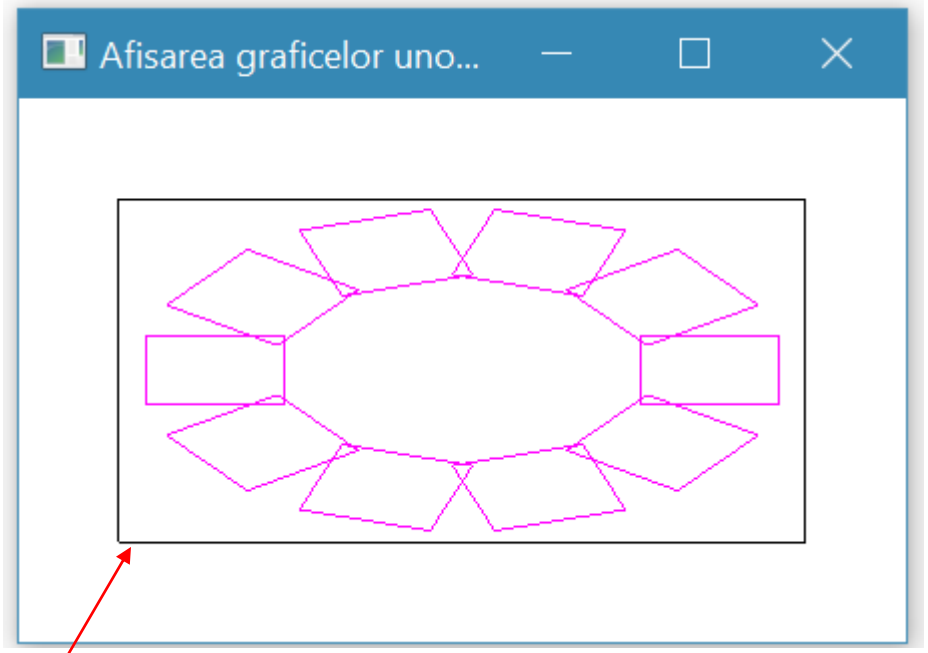
Calculul matricei transformarii de vizualizare

```
glm::mat3 TransformareaFereastraPoartaPatrate::TransfVis2D
            (const LogicSpace & logicSpace, const ViewportSpace & viewSpace)
{   float sx, sy, tx, ty;
    sx = viewSpace.width / logicSpace.width;  sy = viewSpace.height / logicSpace.height;
    tx = viewSpace.x - sx * logicSpace.x;  ty = viewSpace.y - sy * logicSpace.y;
    return glm::transpose(glm::mat3(sx, 0.0f, tx,0.0f, sy, ty,0.0f, 0.0f, 1.0f));
}

glm::mat3 TransformareaFereastraPoartaPatrate:: TransfVis2DUnif(const LogicSpace
                                & logicSpace, const ViewportSpace & viewSpace)
{   float sx, sy, tx, ty, smin;
    sx = viewSpace.width / logicSpace.width; sy = viewSpace.height / logicSpace.height;
    if (sx < sy) smin = sx;  else smin = sy;
    tx = viewSpace.x - smin * logicSpace.x + (viewSpace.width - smin * logicSpace.width) / 2;
    ty = viewSpace.y - smin * logicSpace.y + (viewSpace.height - smin * logicSpace.height) / 2;
    return glm::transpose(glm::mat3(smin, 0.0f, tx,0.0f, smin, ty,.0f, 0.0f, 1.0f));
}
```

Transformare uniformă/neuniformă



Desenul cu transformare uniforma



Desenul cu transformare neuniforma

Cadrul ferestrei mapat pe poarta de afisare
(neafectat de transformarea uniforma)

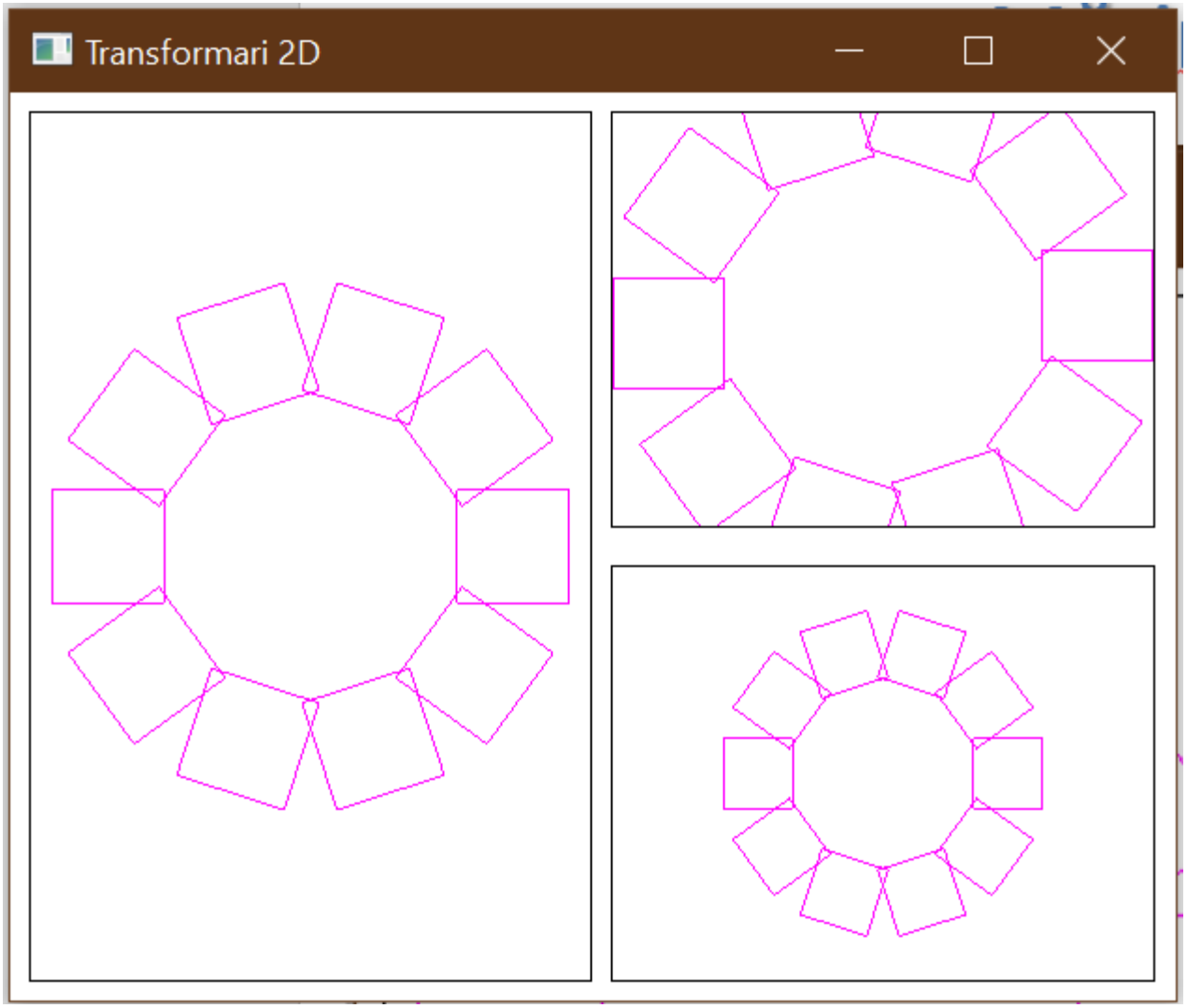
Transformare uniformă/neuniformă - implementare

```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{   glm::ivec2 resolution = window->GetResolution();
//setare poarta de vizualizare
    viewSpace = ViewportSpace(50, 50, resolution.x - 100, resolution.y - 100);
// seteaza zona ecran in care se va desena
    SetViewportArea(viewSpace, glm::vec3(1), true);
    visMatrix = TransfVis2D (logicSpace, viewSpace);
//deseneaza cadrul ferestrei
    RenderMesh2D(meshes["window"], visMatrix, glm::vec3(0, 0, 0));
    if (uniform)   //modificata la apasarea tastei “s”
      visMatrix = TransfVis2DUnif(logicSpace, viewSpace);
    else
      visMatrix = TransfVis2D(logicSpace, viewSpace);
//deseneaza patratele rotite in culoarea rosu
   DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
}
```

Transformare uniformă/neuniformă - implementare

```
void TransformareaFereastraPoartaPatrate::DrawRotatedSquares
            (glm::mat3 visMatrix, glm::vec3 color, int iterations)
{
    float delta = 2 * 3.14159 / iterations;

   cx = logicSpace.x + logicSpace.width / 2;
   cy = logicSpace.y + logicSpace.height / 2;

    for (int i = 0; i < iterations; i++)
   {
      modelMatrix = visMatrix *
                        Transform2D::Translate(cx,cy ) *
                        Transform2D::Rotate(delta * i) *
                        Transform2D::Translate(-cx,-cy);
      RenderMesh2D(meshes["square"], modelMatrix, color);
   }
}
```

Mărirea/Micşorarea ferestrei



Fereastra micşorată. Efectul de mărire a desenului şi decupare la marginile ferestrei

Fereastra mărită. Efectul de micsorare a desenului

Mărirea/Micşorarea ferestrei – implementare (1)

```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{
	glm::ivec2 resolution = window->GetResolution();
	viewSpace = ViewportSpace(10, 10, resolution.x / 2 -10, resolution.y-20);
	SetViewportArea(viewSpace, glm::vec3(1), true);
	//deseneaza cadrul ferestrei initiale cu negru
	visMatrix = TransfVis2D(logicSpace, viewSpace);
	RenderMesh2D(meshes["window"], visMatrix, glm::vec3(0, 0, 0));
	if (uniform) visMatrix = TransfVis2DUnif(logicSpace, viewSpace);
	else visMatrix = TransfVis2D(logicSpace, viewSpace);
	//deseneaza patratele rotite cu rosu
	DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
```

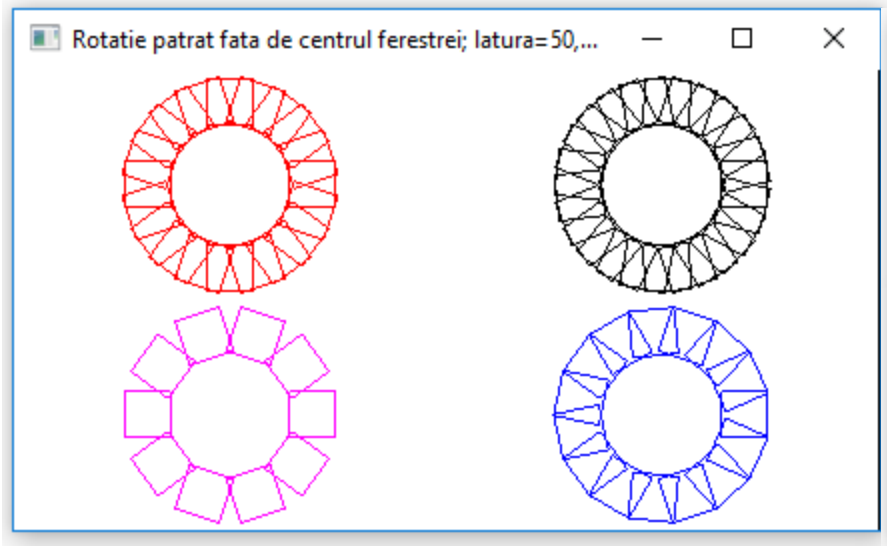
Mărirea/Micşorarea ferestrei – implementare (2)

```
//Micsorare fereastra
logicSpace.x = -4; logicSpace.y = -4; logicSpace.width = 7.5; logicSpace.height = 7.5;
// Creaza patrat contur fereastra in culoarea negru
glm::vec3 cornerw1 = glm::vec3(logicSpace.x +0.002, logicSpace.y+0.002, 0);
Mesh* window1 = Object2D::CreateSquare("window1", cornerw1, logicSpace.width-0.002,
glm::vec3(0, 0, 0));
AddMeshToList(window1);

viewSpace = ViewportSpace(resolution.x/2+10, resolution.y / 2+10, resolution.x/2-20,
                resolution.y/2-20);
SetViewportArea(viewSpace, glm::vec3(1), true);
visMatrix = TransfVis2D(logicSpace, viewSpace);
RenderMesh2D(meshes["window1"], visMatrix, glm::vec3(0, 0, 0));
if (uniform) visMatrix = TransfVis2DUnif(logicSpace, viewSpace);
  else
visMatrix = TransfVis2D(logicSpace, viewSpace);
DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
```

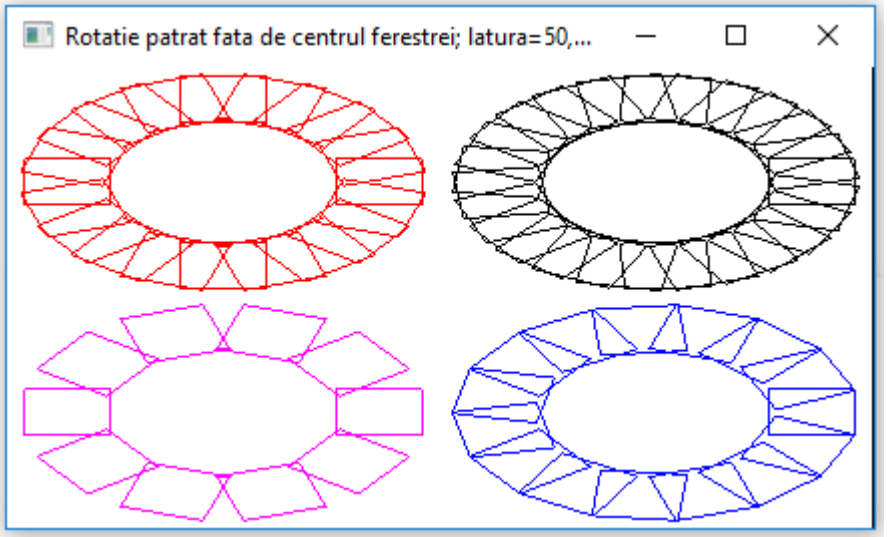
Mărirea/Micşorarea ferestrei – implementare (3)

```
//marire fereastra
logicSpace.x = -6; logicSpace.y = -6;// logic x/y -1
logicSpace.width = 12; logicSpace.height = 12;// logic width/height +2
// Creaza patrat contur fereastra
glm::vec3 cornerw2 = glm::vec3(logicSpace.x + 0.002, logicSpace.y + 0.002, 0);
Mesh* window2 = Object2D::CreateSquare("window2", cornerw2, logicSpace.width - 0.002,
                        glm::vec3(0, 0, 0));
AddMeshToList(window2);
viewSpace = ViewportSpace(resolution.x / 2 +10, 10, resolution.x / 2 -20, resolution.y / 2 -20);
SetViewportArea(viewSpace, glm::vec3(1), true);
visMatrix = TransfVis2D(logicSpace, viewSpace);
RenderMesh2D(meshes["window2"], visMatrix, glm::vec3(0, 0, 0));
if (uniform) visMatrix = TransfVis2DUnif(logicSpace, viewSpace);
            else visMatrix = TransfVis2D(logicSpace, viewSpace);
DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
//se redefineste fereastra initiala (logicSpace)
……..……
}
```

Desenare în 4 porţi de afişare



Transformare uniforma

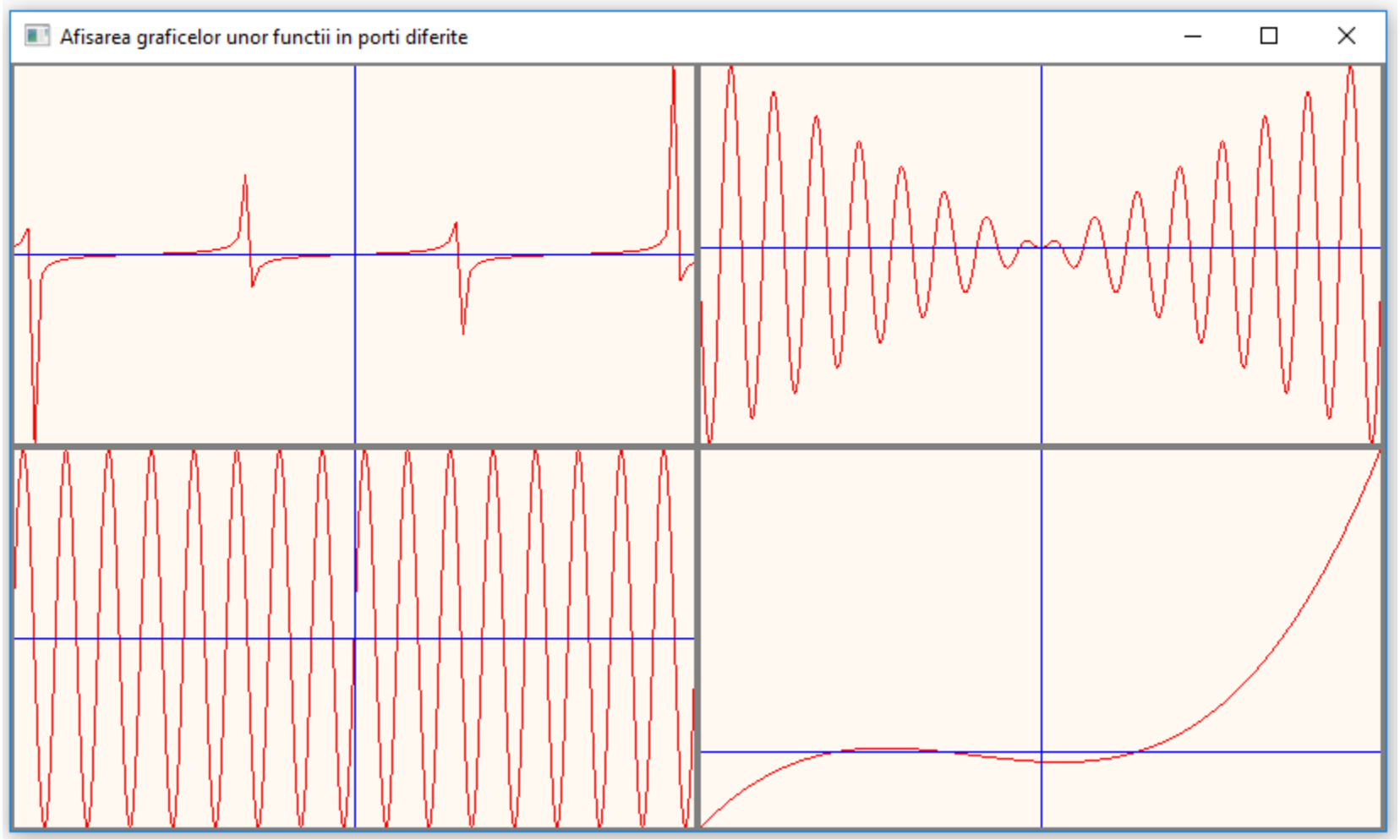


Transformare neuniforma

Desenare în 4 porţi de afişare - implementare

```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{
   glm::ivec2 resolution = window->GetResolution(); //fereastra aplicatiei

   // desenare in partea stanga-jos a ferestrei aplicatiei
   viewSpace = ViewportSpace(0, 0, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1), true);
    if (uniform) visMatrix = TransfVis2DUnif(logicSpace, viewSpace);
     else  visMatrix = TransfVis2D(logicSpace, viewSpace);
   DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
    // desenare in partea dreapta-jos a ferestrei aplicatiei
   viewSpace = ViewportSpace(resolution.x / 2, 0, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1), true);
   if (uniform) visMatrix = TransVis2DUnif(logicSpace, viewSpace);
   else visMatrix = TransfVis2D(logicSpace, viewSpace);
   DrawRotatedSquares(visMatrix, glm::vec3(0, 0, 1), 15);
// analog pentru desenare in partea stanga-sus si dreapta-sus a ferestrei
}
```

Afişarea graficelor unor funcţii în porţi diferite



Afişarea graficelor unor funcţii în porţi diferite – implementare (1)

```
Mesh* Object2DGrafic::CreateFunctionPlot(std::string name,
        float &x1, float &x2, float &y1, float &y2, float pas, float(*f)(double f))
{   // calculeaza punctele de pe graficul functiei matematice y = f(x) in intervalul x1 – x2,
    //in coordonate logice
   // întoarce în (x1,y1), (x2,y2) încadrarea graficului incluzand si axele sistemului de
coordonate
    float x, y, xmin=x1, xmax=x2;

   // calculeaza incadrarea graficului pe axa OY( [y1-y2]) in intervalul xmin - xmax
      y1 = y2 = f(xmin);
      for (x = xmin + pas; x<xmax; x += pas)
            {
                  if ((y = f(x))<y1) y1 = y;
                  if (y>y2)y2 = y;
            }
      if ((y = f(xmax))<y1) y1 = y;
      if (y>y2) y2 = y;
```

Afisarea graficelor unor functii in porti diferite – implementare (2)

```
// Include axele OX, OY în încadrarea graficului
   if (0<xmin)x1 = 0; else x1 = xmin;
   if (0<y1)y1 = 0;
   if (0>xmax)x2 = 0; else x2 = xmax;
   if (0>y2)y2 = 0;

Mesh* plot = new Mesh(name);
plot->SetDrawMode(GL_LINES); // se vor uni câte 2 varfuri succesive
std::vector<VertexFormat> vertices;
// memoreaza varfurile axei OX – se asociaza varfurilor culoarea albastru
    vertices.push_back(VertexFormat(glm::vec3(x1, 0, 0), glm::vec3(0, 0, 1)));
    vertices.push_back(VertexFormat(glm::vec3(x2, 0, 0), glm::vec3(0, 0, 1)));
// memoreaza varfurile axei OY – se asociaza varfurilor culoarea albastru
   vertices.push_back(VertexFormat(glm::vec3(0, y1, 0), glm::vec3(0, 0, 1)));
   vertices.push_back(VertexFormat(glm::vec3(0, y2, 0), glm::vec3(0, 0, 1)));
   std::vector<unsigned short> indices;
   indices.push_back(0); indices.push_back(1); indices.push_back(2);indices.push_back(3);
```

Afisarea graficelor unor functii in porti diferite – implementare (3)

```
// se adauga varfurile care aproximeaza graficul
// se asociaza varfurilor culoarea roşu
   int contor = 4;
   for (x = xmin; x<xmax - pas; x += pas)
    { vertices.push_back(VertexFormat(glm::vec3(x, f(x), 0), glm::vec3(1, 0, 0)));
      indices.push_back(contor); indices.push_back(contor+1);
      contor++;
     }
// se adauga ultimul varf
    vertices.push_back(VertexFormat(glm::vec3(xmax, f(xmax), 0), glm::vec3(1, 0, 0)));

    plot->InitFromData(vertices, indices);
   return plot;

//CreateFunctionPlot
}
```

Afisarea graficelor unor functii in porti diferite – implementare (4)

```
float f1(double x)
{
return (float)(sin(10 * x)*x);
}

float f2(double x)
{
return (float)(tan(x));
}

float f3(double x)
{
return (float)(sin(10 * x));
}

float f4(double x)
{
return (float)((x * x - 2) * (x + 3));
}
```

Afisarea graficelor unor functii in porti diferite – implementare (5)

```
void TransformareaFereastraPoartaGrafic::Init()
{
//setare camera
……….

// se deseneaza graficele celor 4 functii in intervalul xmin=-5, xmax=5
        float y1, y2, x1=-5.0f, x2 = 5.0f;
// se creaza un mesh cu graficul functiei f1; se seteaza spatiul logic pentru graficul functiei f1
        Mesh* mesh_f1 = Object2DGrafic::CreateFunctionPlot("mesh_f1", x1, x2, y1, y2, 0.01f, f1);
        AddMeshToList(mesh_f1);
        logicSpace1.x = x1;
        logicSpace1.width = x2 - x1;
        logicSpace1.y = y1;
        logicSpace1.height = y2 - y1;
//se creaza, in mod analog, mesh_f2, mesh_f3, mesh_f4 şi logicSpace2, logicSpace3, logicSpace4
……..
}
```

Afisarea graficelor unor functii in porti diferite – implementare (6)

```
void TransformareaFereastraPoartaGrafic::Update(float deltaTimeSeconds)
{
    glm::ivec2 resolution = window->GetResolution(); // fereastra aplicatiei

// seteaza zona de desenare – partea stanga-sus
   viewSpace = ViewportSpace(0, resolution.y / 2, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1, 0.98, 0.95), true);
   visMatrix = TransfVis2D(logicSpace2, viewSpace);
   DrawFunctionPlot(visMatrix, 2);//deseneaza gaficul functiei f2
// partea dreapta-sus a ferestrei aplicatiei
   viewSpace = ViewportSpace(resolution.x / 2, resolution.y / 2, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1,0.98,0.95), true); //culoarea fondului pt viewport: bej
   visMatrix = TransfVis2D(logicSpace1, viewSpace);
   DrawFunctionPlot(visMatrix, 1); //deseneaza graficul functiei f1
// analog pentru desenarea celorlalte functii
}
```

Afisarea graficelor unor functii in porti diferite – implementare (7)

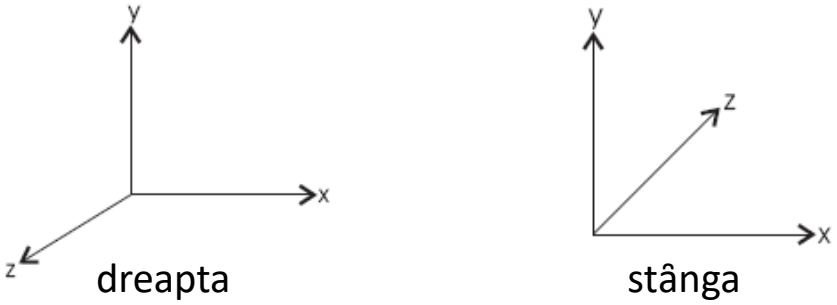
```
void TransformareaFereastraPoartaGrafic::DrawFunctionPlot(glm::mat3 visMatrix, int nr_functie)
{
    modelMatrix = visMatrix;
    switch (nr_functie)
    {
     case 1:
        RenderMesh2D(meshes["mesh_f1"],shaders["ShaderGrafic"],modelMatrix);
        break;
     case 2:
        RenderMesh2D(meshes["mesh_f2"], shaders["ShaderGrafic"], modelMatrix);
        break;
     case 3:
        RenderMesh2D(meshes["mesh_f3"], shaders["ShaderGrafic"], modelMatrix);
        break;
     case 4:
        RenderMesh2D(meshes["mesh_f4"], shaders["ShaderGrafic"], modelMatrix);
        break;
     default:
        break;
    }
}
```

Transformari geometrice 3D

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Sisteme de coordonate carteziene 3D



Sisteme de coordonate carteziene 3D

i, j, k: versorii directiilor axelor sistemului de coordonate

dreapta: $k = (i \times j)/|i \times j|$

stanga: $k = (j \times i)/|j \times i|$

Sistemul de coordonate în care este descrisa scena într-o aplicatie OpenGL: sistem de coordonate carteziene 3D dreapta.

Transformările geometrice 3D elementare

- Translaţia
- Scalarea faţă de origine
- Rotaţiile în jurul axelor principale ale sistemului de coordonate
- Oglindirea faţă de un plan principal al sistemului de coordonate
- Forfecarea faţă de originea sistemului de coordonate

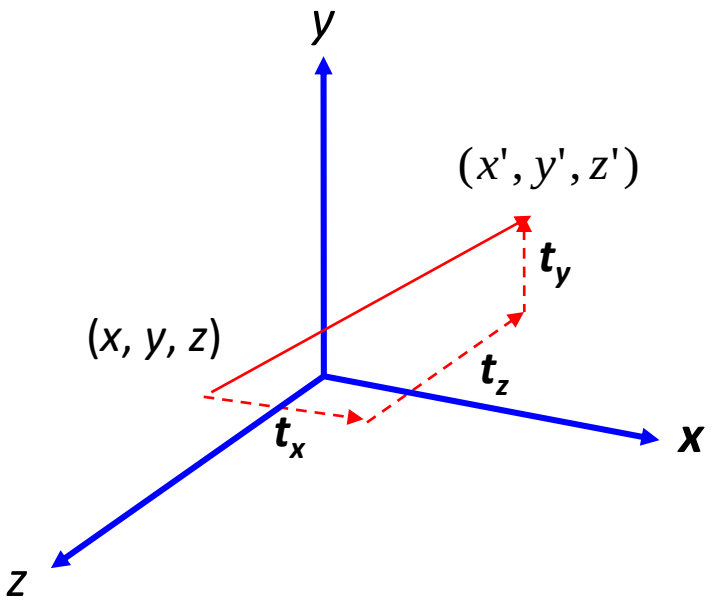
Considerăm punctele din spaţiu reprezentate prin vectori coloană, pentru a respecta conventia din OpenGL:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

M este matricea de transformare a punctului (x, y, z), în coordonate omogene

Translaţia

Definita printr-un vector T[tx,ty,tz]



$$x' = x + t_x$$

$$y' = y + t_y$$

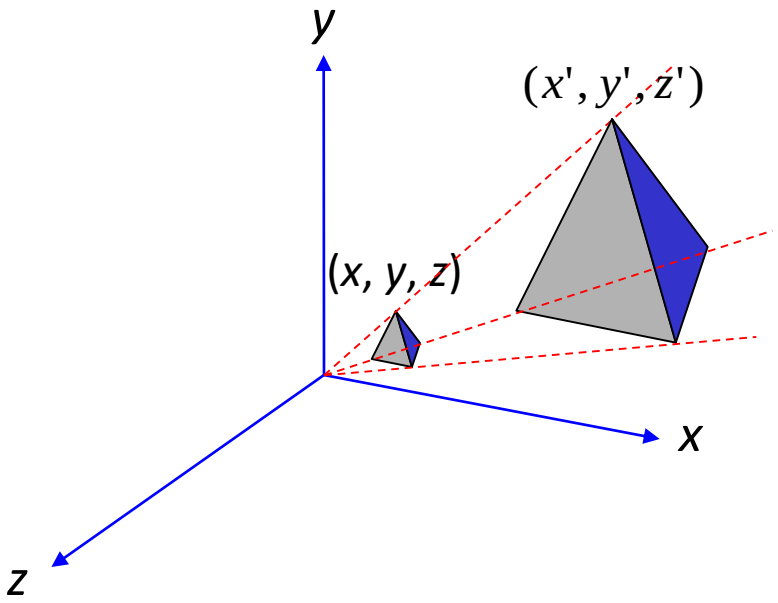
$$z' = z + t_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$T(t_x, t_y, t_z)$

Scalarea faţă de origine

Definita prin 3 numere reale: Sx,Sy, Sz – factorii de scalare pe cele 3 axe



$S_x = S_y = S_z \quad \Rightarrow \quad$ scalare uniforma

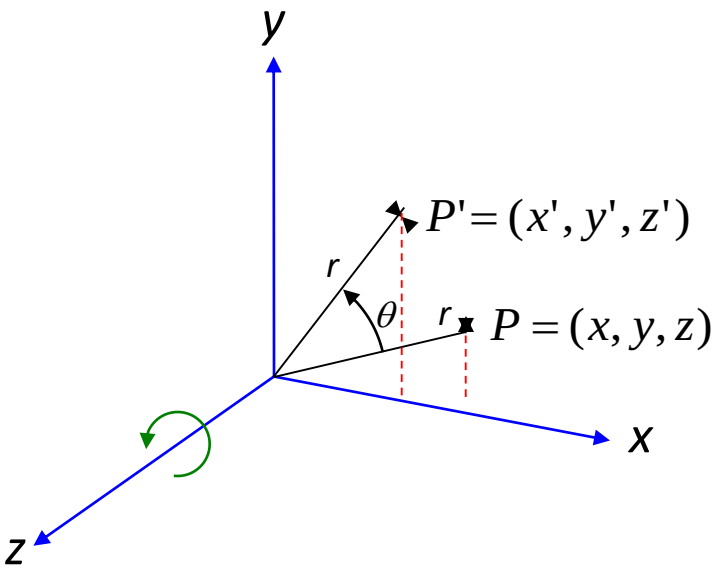
altfel, scalare neuniforma

$$x' = x \cdot s_x$$
$$y' = y \cdot s_y$$
$$z' = z \cdot s_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$S(s_x, s_y, s_z)$

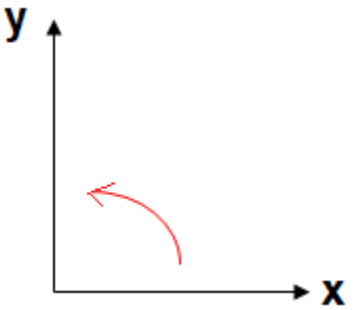
Rotaţia pozitivă (trigonometrica) în jurul axei oz



Este o rotaţie într-un plan de z constant (prin rotaţie nu se modifică coordonata z).

Rotatia in planul XOY:

$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$
$$z' = z = 0$$



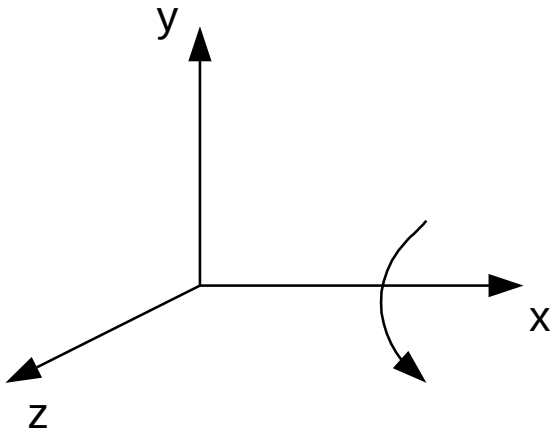
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$R_z(\theta)$

Rotatia punctului P(x,y,z) in jurul axei Oz cu unghiul θ:

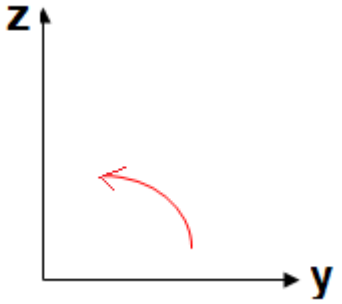
$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$
$$z' = z$$

Rotaţia pozitivă în jurul axei ox



Rotatia in planul Y0Z

$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$
$$x' = x = 0$$



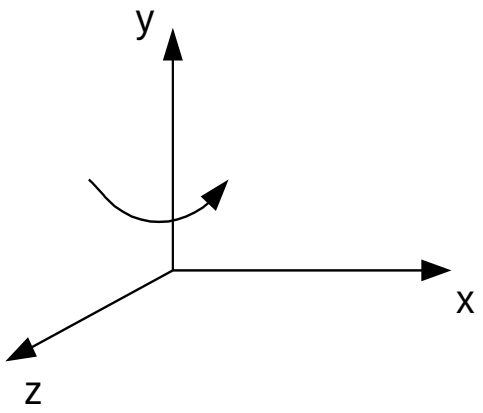
Rotatia punctului P(x,y,z) in jurul axei Ox cu unghiul θ:

$$x' = x$$
$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

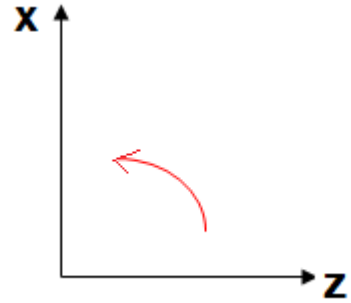
$R_x(\theta)$

Rotaţia pozitivă în jurul axei oy



Rotatia in planul ZOX

$$z' = z\cos\theta - x\sin\theta$$
$$x' = x\cos\theta + z\sin\theta$$
$$y' = y = 0$$



Rotatia punctului P(x,y,z) in jurul axei Oy cu unghiul θ:

$$x' = x\cos\theta + z\sin\theta$$
$$y' = y$$
$$z' = -x\sin\theta + z\cos\theta$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_y(\theta)$

Alte transformari geometrice 3D elementare

Oglindirea faţă de un plan principal al sistemului de coordonate

$$O_{xoy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Forfecarea faţă de origine

- **De-a lungul axei OZ: modifica x si y proportional cu z;** Fx, Fy - factorii de forfecare

  x’ = x + Fx*z

  y’ = y + Fy*z

  z’ = z

- **Analog pentru forfecarea de-a lungul axei OX si a axei OY**

- **Cazul general (forfecarea pe toate cele 3 axe):**

  x’ = x + y* d + z*g

  y’ = x*b + y + z*i

  z’ = x*c + y*f + z

$$[x' \; y' \; z' \; 1] = [x \; y \; z \; 1] * \begin{vmatrix} 1 & b & c & 0 \\ d & 1 & f & 0 \\ g & i & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Transformari geometrice 3D compuse

Exemple:

1. **Scalarea faţă de un punct oarecare, F(xf, yf, zf):**

   T(xf,yf,xf)*S(sx,sy,sz)*T(-xf,-yf,-zf)

2. **Rotatia cu unghiul u in jurul unei axe paralele cu o axa a sistemului de coordonate**:

   1. Translatia obiectului astfel incat axa de rotatie sa se suprapuna peste o axa a sistemului de coordonate.
   2. Rotatia obiectului in jurul axei sistemului de coordonate - R(u): $R_x(u)/R_y(u)/ R_z(u)$
   3. Translatia inversa celei din pasul 1.

   Rezulta:

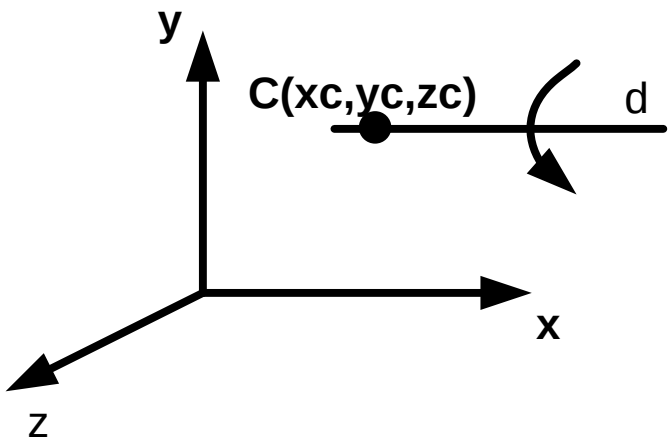
   M = T(xd, yd, zd) * R(u) * T(-xd, -yd, -zd)

   u : unghiul de rotatie

   xd, yd, zd : un punct de pe axa de rotatie

$$\begin{bmatrix} \mathbf{x'} \\ \mathbf{y'} \\ \mathbf{z'} \\ \mathbf{1} \end{bmatrix} = M * \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \\ \mathbf{1} \end{bmatrix}$$

Rotaţia în jurul unei drepte paralele cu axa OX



$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = T(xc, yc, zc)\, R_X(u)\, T(-xc, -yc, -zc) \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Rotatia cu în jurul unei drepte oarecare (1)

Se considera dreapta data printr-un punct (xd, yd, zd) si directia sa, D[a, b, c].

1. Translatie prin care dreapta va trece prin origine: T(-xd, -yd, -zd)
2. Alinierea dreptei cu una dintre axele principale, de ex. cu axa OZ:

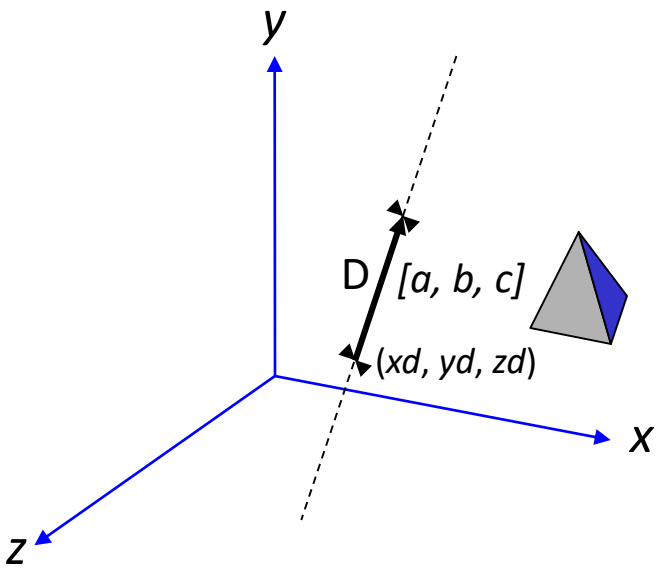
   2.1. Rotatie in jurul axei OX, cu un unghi ux, prin care dreapta ajunge in planul XOZ: Rx(ux)

   2.2. Rotatie in jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: Ry(uy)
3. Rotatia cu unghiul dat, u, in jurul axei pe care s-a aliniat dreapta: rotatie in jurul axei OZ : Rz(u)
4. Transformarea inversa celei din pasul 2:

   4.1. Rotatie in jurul axei OY, cu unghiul –uy: Ry(-uy)

   4.2. Rotatie in jurul axei OX, cu unghiul –ux: Rx(-ux)
5. Transformarea inversa celei de la pasul 1: T(xd, yd, zd)

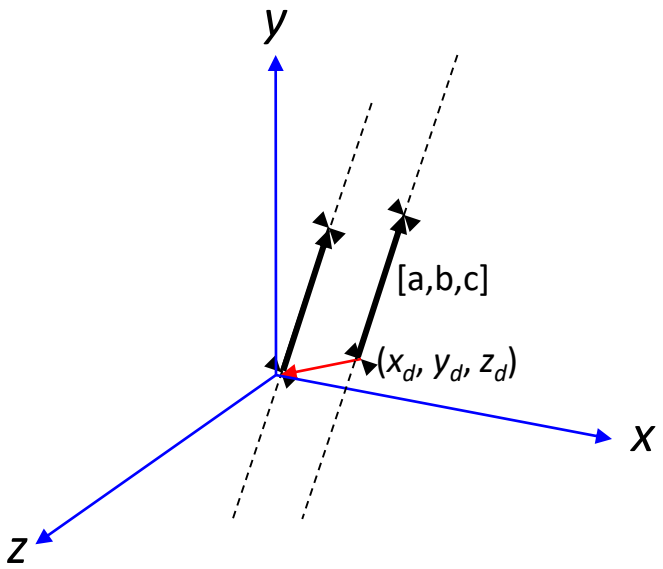
Rotatia in jurul unei drepte oarecare (2)



1. Translatie astfel incat dreapta sa treaca prin origine
2. Aliniere dreapta cu una din axe
3. Rotatia propriu-zisa.
4. Transformarea inversa de la punctul 2
5. Transformarea inversa de la punctul 1

Rotatia in jurul unei drepte oarecare (3)

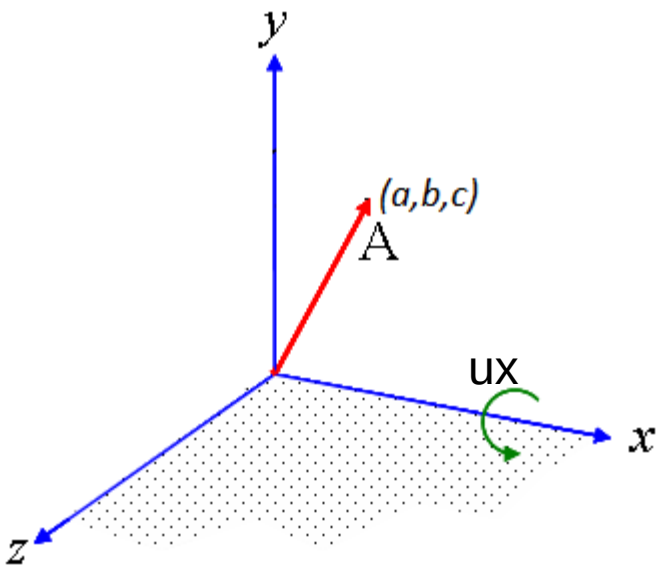
1. Translaţie

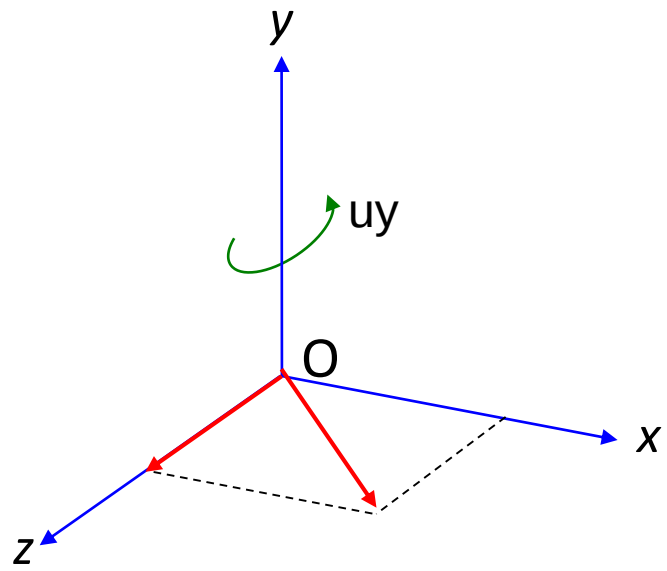


$$T = \begin{bmatrix} 1 & 0 & 0 & -xd \\ 0 & 1 & 0 & -yd \\ 0 & 0 & 1 & -zd \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotatia in jurul unei drepte oarecare (4)

2. Aliniere *D* cu axa OZ



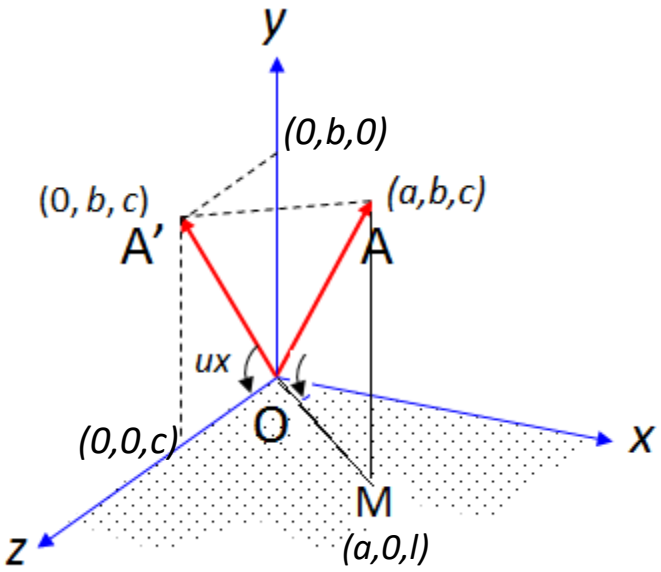


Rotatie in jurul axei OX cu un unghi ux, prin care dreapta ajunge in planul XOZ: Rx(ux)

Rotatie in jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: Ry(uy)

Rotatia in jurul unei drepte oarecare (5)

2. 1. Rotatia in jurul axei OX



A' – proiectia lui A in planul YOZ

Lungime (OA) = L; lungime (OA') = l;

$$L = \sqrt{a^2 + b^2 + c^2}$$

$$l = \sqrt{b^2 + c^2}$$

b = l*sin(ux)
c = l*cos(ux)

$$\sin(ux) = \frac{b}{l}$$

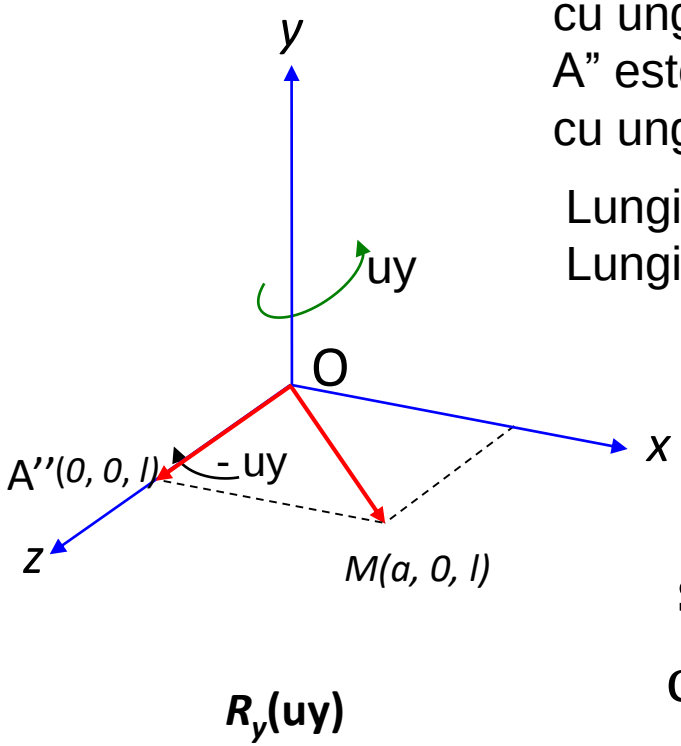
$$\cos(ux) = \frac{c}{l}$$

$R_x(ux)$

Prin rotatia lui OA in jurul axei Ox cu unghiul ux, acesta va ajunge in planul XOZ

Rotatia in jurul unei drepte oarecare (6)

2. 2. Rotatia in jurul axei OY



R_y(uy)

M este punctul obtinut prin rotatia lui A in jurul axei Ox, cu unghiul ux.
A" este punctul obtinut prin rotatia lui A' in jurul axei Ox, cu unghiul ux

Lungime (OM) = Lungime (OA) = L
Lungime (OA") = Lungime (OA') = l

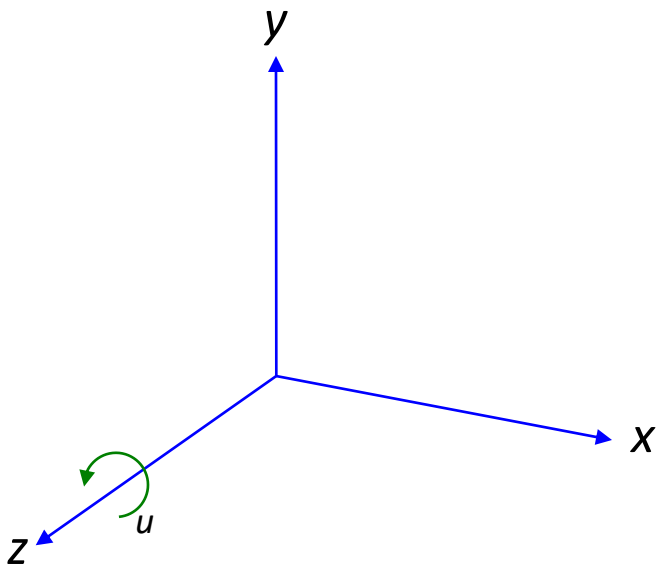
a = L*sin(-uy)

l = L*cos(-uy)

$$\sin(-uy) = a/L -> \sin(uy) = -a/L$$
$$\cos(-uy) = \cos(uy) = l/L$$

Rotatia in jurul unei drepte oarecare (7)

3. Rotatia in jurul axei OZ cu unghiul dat

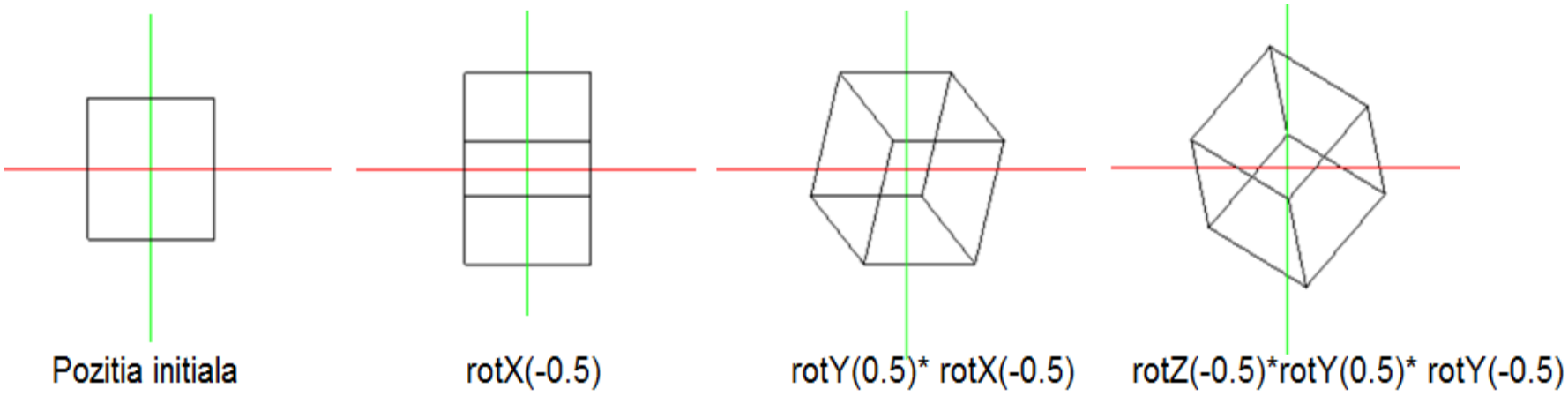


$$R_z(u) = \begin{bmatrix} \cos(u) & -\sin(u) & 0 & 0 \\ \sin(u) & \cos(u) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Exemplu: rotatia unui cub (1)

Cub centrat in originea sistemului de coordonate XYZ

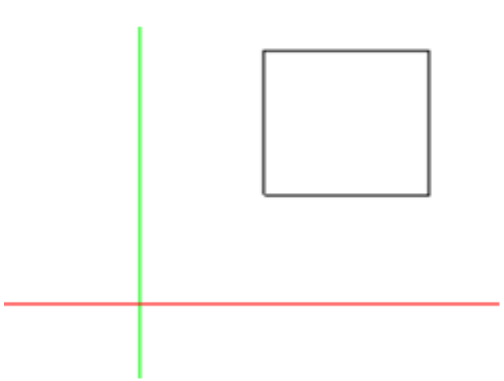
Proiectie ortografica in XOY: (x,y,z) → (x' = x; y' = y)



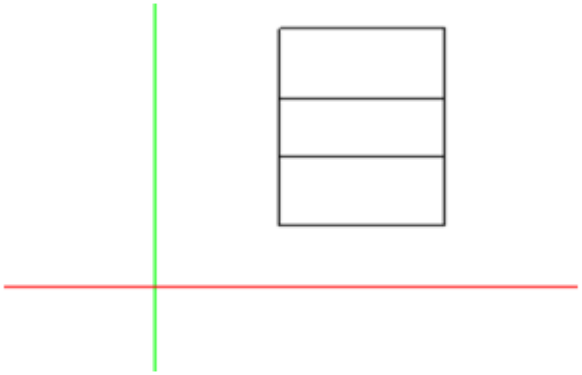
Exemplu: rotatia unui cub (2)

Cub cu centrul in (100, 100, 0)

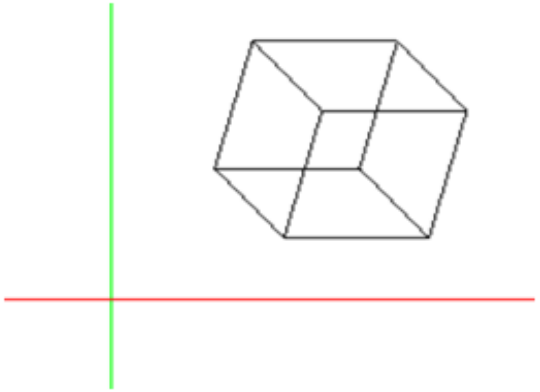
Proiectie ortografica in XOY



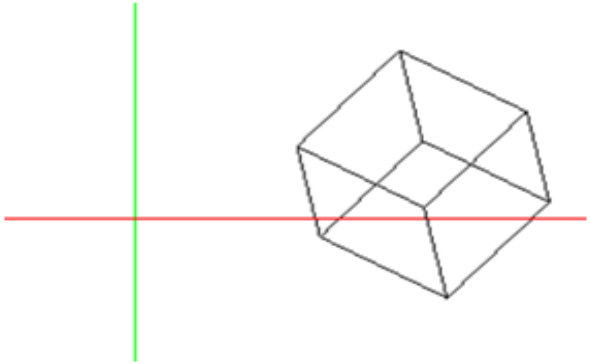
Pozitia initiala



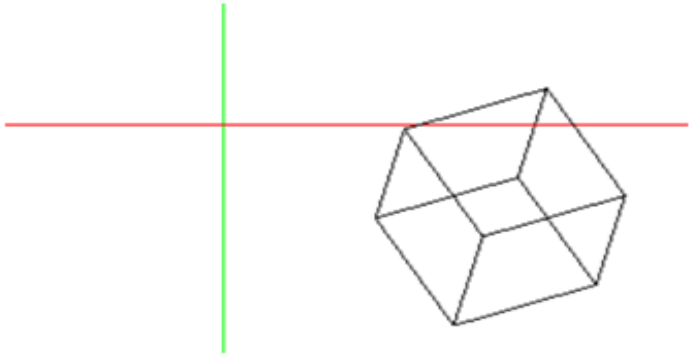
rotX(-0.5)



rotY(0.5)*rotX(-0.5)



rotZ(-0.5)*rotY(0.5)*rotX(-0.5)



rotZ(-0.5)*rotZ(-0.5)*rotY(0.5)*rotX(-0.5)

Implementarea transformarilor 3D – exemplu (1)

```
void AnimatieTransformari3D::Init()
{ polygonMode = GL_FILL;
 Mesh* mesh = new Mesh("box");
 mesh->LoadMesh(RESOURCE_PATH::MODELS + "Primitives", "box.obj");
 meshes[mesh->GetMeshID()] = mesh;
 auto camera = GetSceneCamera();
 camera->SetPositionAndRotation(glm::vec3(0, 4, 10), glm::quat(glm::vec3(-20 *
            TO_RADIANS, 0, 0)));
  camera->Update();
// initializare tx, ty and tz (pasii de translatie)
translateX = 0; translateY = 0; translateZ = 0;
// initialize sx, sy and sz (factorii de scalare)
scaleX = 1; scaleY = 1; scaleZ = 1;
// initializare pasi de rotatie
angularStepOX = 0; angularStepOY = 0; angularStepOZ = 0;
}
```

Implementarea transformarilor 3D – exemplu (2)

```
void AnimatieTransformari3D::FrameStart()
{    // clears the color buffer and depth buffer
    glClearColor(0, 0, 0, 1); glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glm::ivec2 resolution = window->GetResolution();
   // sets the screen area where to draw
   glViewport(0, 0, resolution.x, resolution.y);
}
void AnimatieTransformari3D::Update(float deltaTimeSeconds)
{   modelMatrix = glm::mat4(1);
   modelMatrix *= Transform3D::RotateOX(angularStepOX);
   modelMatrix *= Transform3D::RotateOY(angularStepOY);
   modelMatrix *= Transform3D::RotateOZ(angularStepOZ);
   modelMatrix *= Transform3D::Scale(scaleX, scaleY, scaleZ);
   modelMatrix *= Transform3D::Translate(translateX, translateY, translateZ);
  RenderMesh(meshes["box"], shaders["VertexNormal"], modelMatrix);
}
void AnimatieTransformari3D::FrameEnd()
{ DrawCoordinatSystem(); }
```

Implementarea transformarilor 3D – exemplu (3)

```
void AnimatieTransformari3D::OnInputUpdate(float deltaTime, int mods)
{//apelata la fiecare deltaTime secunde
    if (window->KeyHold(GLFW_KEY_1)) angularStepOX += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_2)) angularStepOX -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_3)) angularStepOY += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_4)) angularStepOY -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_5)) angularStepOZ += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_6)) angularStepOZ -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_7)) scaleX += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_8)) scaleX -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_9)) scaleY += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_0)) scaleY -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_MINUS)) scaleZ -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_EQUAL)) scaleZ += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_UP)) translateY += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_DOWN)) translateY -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_LEFT)) translateX -= step * deltaTime;
    if (window->KeyHold(GLFW_KEY_RIGHT)) translateX += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_PAGE_UP)) translateZ += step * deltaTime;
    if (window->KeyHold(GLFW_KEY_PAGE_DOWN)) translateZ -= step * deltaTime;
}
```

Implementarea transformarilor 3D – exemplu (4)

```
void AnimatieTransformari3D::OnKeyPress(int key, int mods)
{
  if (key == GLFW_KEY_SPACE)
   { //initializare
            angularStepOX = 0; angularStepOY = 0; angularStepOZ = 0;
            scaleX = 1; scaleY = 1; scaleZ = 1;
            translateX = 0; translateY = 0; translateZ = 0;
  }
}
```

Proiectii

Prof. univ. dr. ing. Florica Moldoveanu

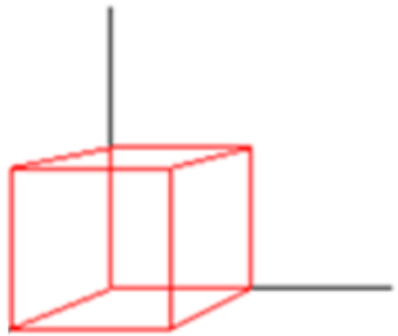
Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Proiectii

❖ Transformari dintr-un spatiu n-dimensional intr-un spatiu k-dimensional, k<n.

Proiectii $R^3 \rightarrow R^2$

- Transformari din spatiul tri-dimensional într-un spatiu bi-dimensional
- Sse efectueaza într-un plan: planul de proiectie
- Se aplica varfurilor obiectelor
- Nu modifica legaturile dintre varfuri (topologia)

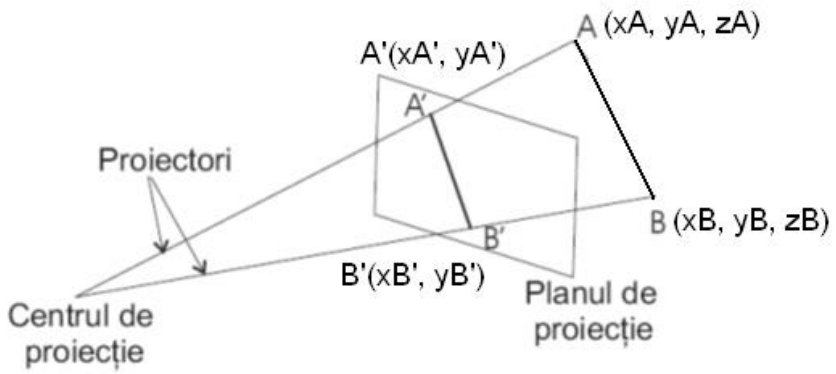


Exemplu:

- Proiectia unui cub in planul XOY
- Legaturile dintre varfurile 2D sunt aceleasi cu legaturile dintre varfurile 3D proiectate

Proiectii $R^3 \rightarrow R^2$

- Proiectia unui varf 3D într-un plan este punctul de intersectie dintre plan si proiectorul care pleaca din ***centrul de proiectie*** si trece prin varf.



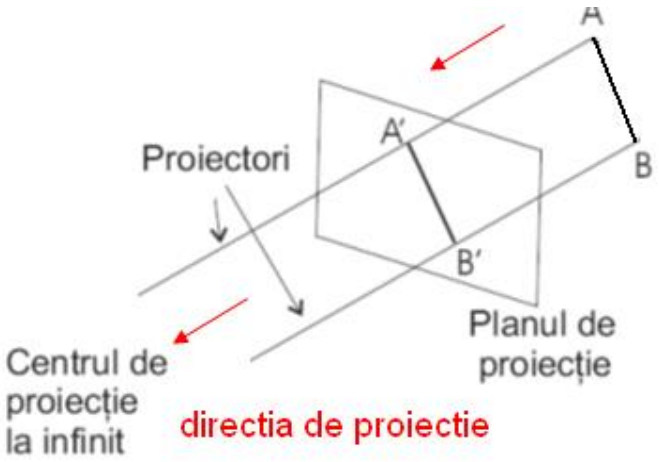
Segmentul A'-B' este proiectia segmentului A-B in planul de proiecţie XOY (z=0).

Exista 2 clase de proiectii:

- Proiectii perspectiva: centrul de proiectie este situat la distanta finita fata de planul de proiectie (ca in figura de mai sus); proiectorii sunt drepte convergente in CP.

Proiectii $R^3 \rightarrow R^2$

- Proiectii paralele - centrul de proiectie este la infinit; proiectorii sunt linii paralele care trec prin varfurile obiectului proiectat si au directia specificata (vectorul ***directie de proiectie***)



Proiectii perspectiva(1)

Produc imagini asemanatoare celor obtinute cu aparatele de fotografiat.

Caracteristici:

1. Efectul de micsorare.

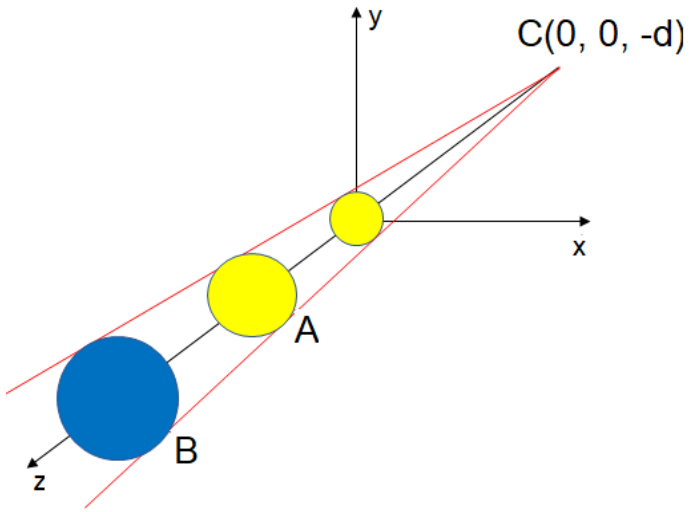
Dimensiunea obiectului in proiectia 2D este invers proportionala cu distanta de la centrul de proiectie la obiect.

Exemplu:

A si B sunt 2 sfere, cu centrele pe axa OZ, care se proiecteaza în acelaşi disc în planul XOY.

raza sferei B = 2* raza sferei A

Dist(centru_B-C) = 2* Dist(centru_A-C)

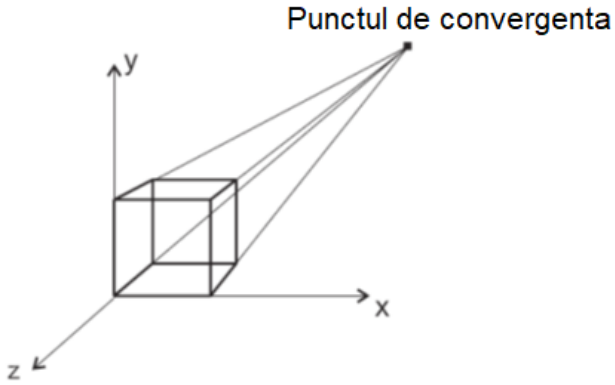


Proiectii perspectiva(2)

2. **Modifica unghiurile dintre dreptele care nu sunt paralele cu planul de proiectie**

Proiectiile liniilor paralele care nu sunt paralele cu planul de proiectie converg catre un punct din plan, numit punct de convergenta

- Punctul catre care converg liniile paralele cu una dintre axele principale: ***punct de convergenta principal***

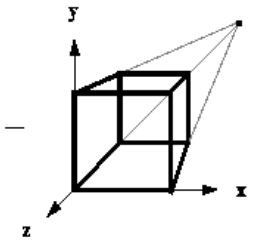


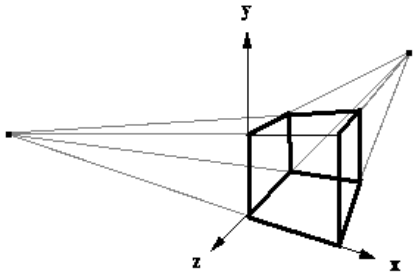
Proiectia perspectiva in planul XOY a unui cub cu feţele paralele cu planele principale

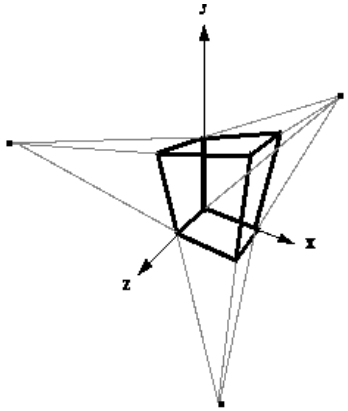
Proiectii perspectiva(3)

Pot fi efectuate proiectii cu unul, doua sau trei *puncte de convergenta principale.*

Numarul de puncte de convergenta principale = numarul de axe principale intersectate de planul de proiectie.

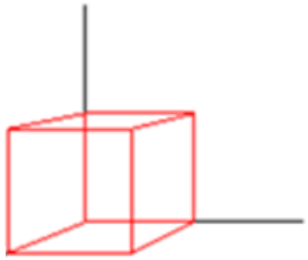




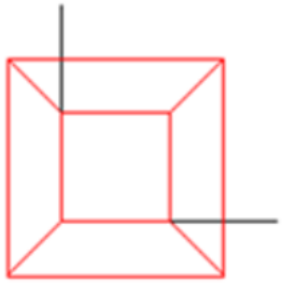


Exemple de proiectii perspectiva

Proiectii ale unui cub cu latura egala cu 1, feţele paralele cu planele principale ale sistemului de coordonate şi coltul de (xmin,ymin,zmin) in originea sistemului de coordonate.

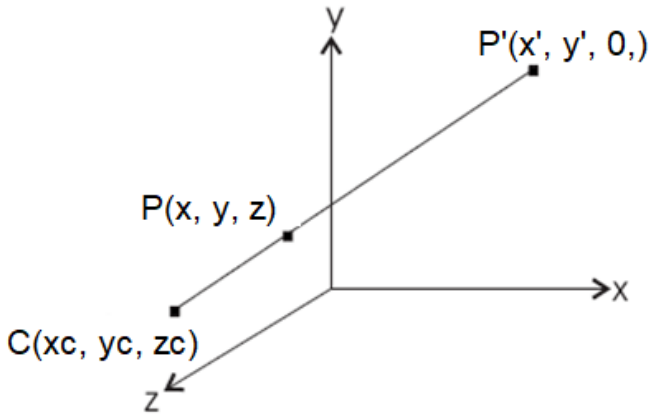


Proiectie perspectiva cu centrul in (5,2,8)



Proiectie perspectiva cu centrul in (0.5,0.5,2)

Proiectia perspectiva in planul XOY (1)



C – centrul de proiectie
P – punctul proiectat
P' – proiectia lui P in XOY

Ecuatiile parametrice ale proiectorului care trece prin P:

x = xc + (x – xc)*t
y = yc + (y – yc)*t 0<= t <= 1, pentru punctele aflate pe segmentul C-P
z = zc + (z – zc) *t

0 = zc + (z –zc) *t' → t' = – zc / (z – zc) → **x' = (xc*z – zc*x)/(z – zc)**

y' = (yc*z – zc*y)/(z – zc)

Proiectia perspectiva in planul XOY (2)

Reprezentarea matricială a transformării de proiectie

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} \frac{-zc}{z-zc} & 0 & \frac{xc}{z-zc} & 0 \\ 0 & \frac{-zc}{z-zc} & \frac{yc}{z-zc} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Matricea proiectiei depinde de coordonata z a punctului proiectat:

→ nu poate fi folosita pentru toate varfurile aceluiaşi obiect

$$\begin{vmatrix} xw \\ yw \\ zw \\ w \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z

yw = -zc*y + yc*z

zw = 0

w = z – zc

Coordonatele carteziene ale punctului P'
- împărţirea perspectivă -

x' = xw/w = (-zc*x + xc*z)/(z-zc)

y' = yw/w = (-zc*y + yc*z)/(z-zc)

z' = zw/w = 0

Proiectia perspectiva in planul XOY (3)

Reprezentarea matricială a transformării de proiectie

$$\begin{vmatrix} xw \\ yw \\ zw \\ w \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} 1 & 0 & \frac{-xc}{zc} & 0 \\ 0 & 1 & \frac{-yc}{zc} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{-1}{zc} & 1 \end{vmatrix}$$

Coordonatele omogene ale punctului P'

xw = x + (-xc/zc)*z

yw = y + (-yc/zc)*z

zw = 0

w = (-1/zc)*z +1 = -(z – zc)/zc

Coordonatele carteziene ale punctului P'

x' = xw/w = -(x + (-xc/zc)*z)*zc/(z – zc)

= (-zc*x + xc*z)/(z-zc)

y' = yw/w = (-zc*y + yc*z)/(z-zc)

z' = zw/w = 0

Coordonatele omogene ale punctului obtinut prin proiectie perspectiva au w ≠ 1.

Proiectia perspectiva in planul XOY(4)
- pastrarea informatiei de adancime -

Pentru conservarea informatiei de adancime (necesara pentru eliminarea din imagine a partilor nevizibile ale scenei 3D) se foloseste matricea :

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z → x' = (-zc*x + xc*z)/(z-zc)

yw = -zc*y + yc*z → y' = (-zc*y + yc*z)/(z-zc)

zw = z → z' = z/(z – zc)

w = z – zc

$$Pperspectiva = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

Proiectia perspectiva standard

Xc = 0, Yc = 0, Zc = - d

xw = x → x' = x*d/(z+d)

yw = y → y'= y*d/(z+d)

zw = z → z' = z/(z+d)

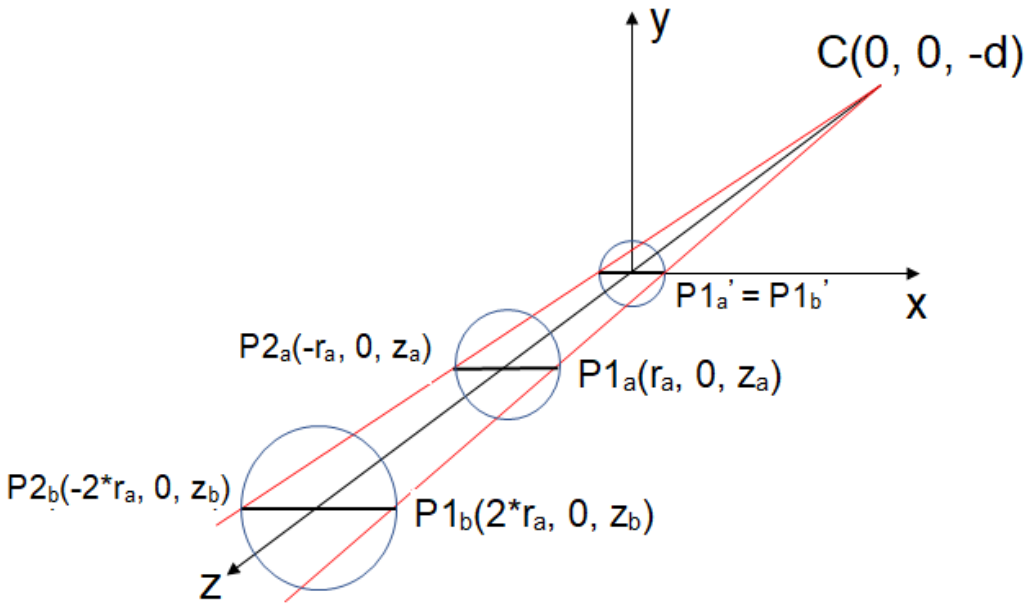
w = z/d + 1 = (z+d)/d

$$Ppers_standard = \begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{vmatrix}$$

Proiectia perspectiva in planul XOY – exemplu - efectul de micsorare -

A şi B sunt 2 sfere cu centrele pe axa OZ, A de rază r_a şi B de rază $2*r_a$.

Distanța de la centrul sferei B la centrul de proiecție C este de 2 ori mai mare decât distanța de la centrul sferei A la C.



Proiecția perspectivă standard

xw = x → x' = x*d/(z+d)

yw = y → y'= y*d/(z+d)

zw = z → z' = z/(z+d)

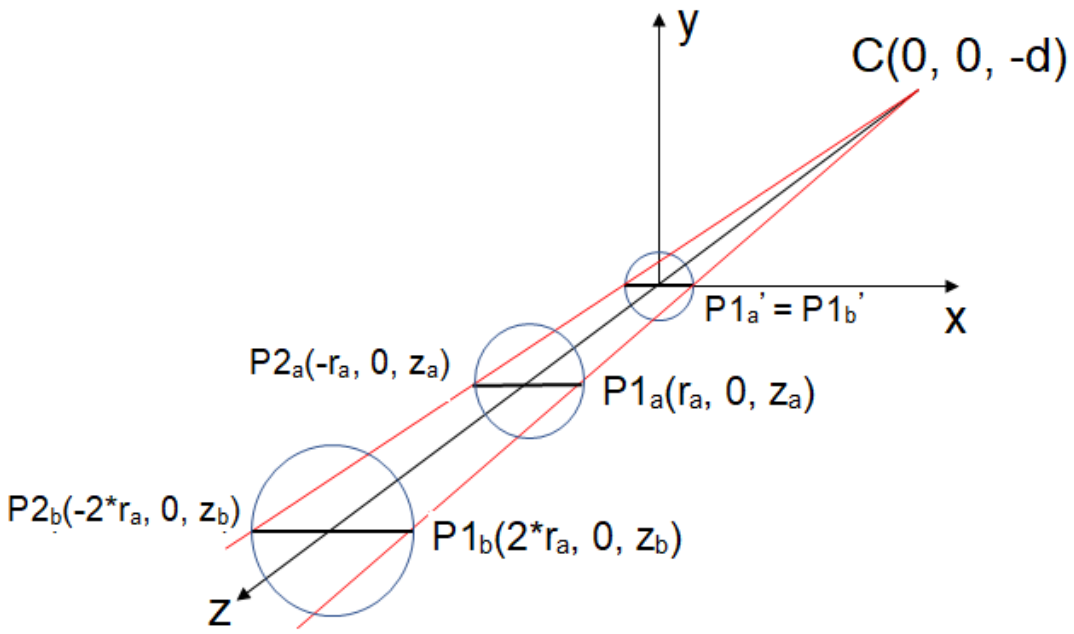
w = z/d + 1 = (z+d)/d

$x_P1_a' = r_a*d/(z_a + d)$; $y_P1_a' = 0 = y_P1_b'$

$x_P1_b' = 2*r_a*d/(z_b + d) = 2*r_a*d/(2*(z_a + d)) = r_a*d/(z_a + d) = x_P1_a'$

Analog se demonstrează că $P2_b' = P2_a'$

Utilizarea coordonatei de adâncime în eliminarea suprafetelor nevizibile din imagini



Proiecția perspectivă standard

zw = z → z' = z/(z+d)

$z_P1_a' = z_a / (z_a + d)$

$z_P1_b' = z_b / (z_b + d)$

Consideram:

d = 50; z_a =100;

$z_b = 2(z_a + d) = 300$

$z_P1_a' = 100/150 = 0.666$;

$z_P1_b' = 300/350 = 0.857$

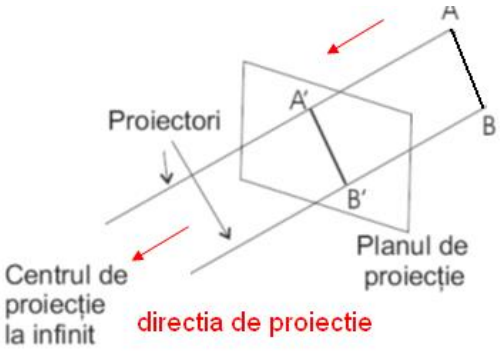
Daca 2 puncte apartinand la 2 suprafete 3D se proiecteaza in acelasi punct, se va reda in imagine culoarea punctului mai apropiat de observator (centrul de proiectie):

$z_P1_a' < z_P1_b'$ → discul in care se proiecteaza cele 2 sfere va fi afisat in culoarea sferei A

Proiectii paralele(1)

Caracteristici

- **Proiectorii sunt drepte paralele de directie data**
- **Conserva paralelismul liniilor: sunt transformari afine**
- **Unghiurile se conserva doar pentru feţele obiectului paralele cu planul de proiectie.**



- Clasificare dupa unghiul dintre proiectori si planul de proiectie:
  - *Proiectii **ortogonale**:* proiectorii sunt perpendiculari pe planul de proiectie.
    - Ortografice: planul de proiectie este II cu un plan principal (⊥ pe o axa principala)
    - Axonometrice: planul de proiectie este oarecare
  - *Proiectii **oblice**:* proiectorii nu sunt perpendiculari pe planul de proiectie – cazuri particulare:
    - proiectii Cavalier (unghiul dintre proiectori si plan = 45 grade)
    - proiectii Cabinet (unghiul dintre proiectori si plan = 63.43 grade)

Proiectii paralele(2) - clasificare

Proiectii paralele

Oblice

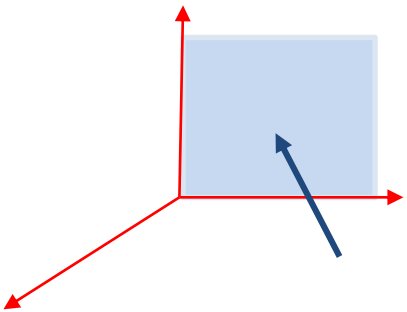
Cavalier Cabinet Generale

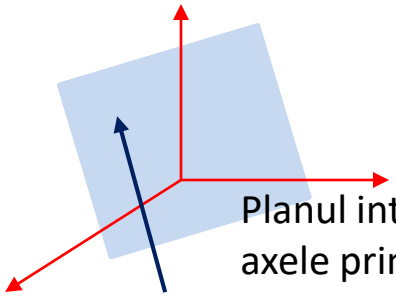
Ortogonale
proiectori ⊥ pe planul de proiectie

Axonometrice

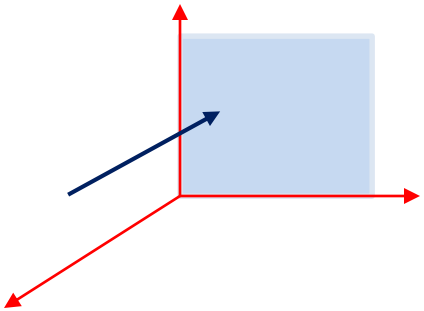
Izometrice Dimetrice Trimetrice

Ortografice(6 vederi)
planul de proiectie ⊥ pe o axa principala



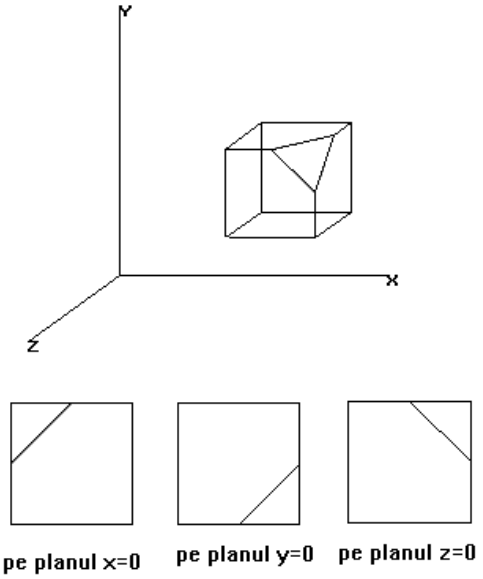


Planul intersecteaza
axele principale



Proiectii paralele(3)

1. Proiectii ortogonale - ortografice



Proiectii ortografice ale unui cub cu un colt taiat

- Proiectii in planele sistemului de coordonate (planul de proiectie ⊥ pe o axa principala) → numite şi **vederi**

- în YOZ: *vederea din stanga, vederea din dreapta,*
- în XOZ : *vederea de sus, vederea de jos,*
- în XOY : *vederea din fata, vederea din spate.*

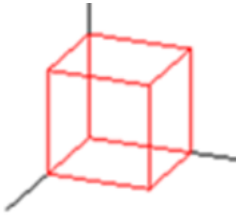
- Conserva lungimile laturilor si unghiurile dintre laturi
- Utile in desenul tehnic

Proiectii paralele(4)

2. Proiectii ortogonale - axonometrice

- Planul de proiectie nu este perpendicular pe nici una din axele sistemului de coordonate.
- Redau mai multe feţe ale obiectului proiectat (ca si cele perspectiva): alegand planul de proiectie se poate controla scalarea laturilor.

Izometrica



Dimetrica: sx=sy

- In functie de unghiurile pe care planul de proiectie le face cu axele sistemului de coordonate:

- proiectii *izometrice*, planul face unghiuri egale cu toate cele trei axe → prin proiectie, laturile sunt scalate cu factori egali pe cele 3 axe: sx=sy=sz

- proiectii *dimetrice*, planul face unghiuri egale cu doua dintre axe → laturile sunt scalate cu factori de scalare egali pe 2 axe: sx=sy sau sx=sz sau sy=sz

- proiectii *trimetrice*, unghiurile dintre cele trei axe si plan sunt diferite → factorii de scalare ai laturilor pe cele 3 axe sunt diferiti: sx # sy # sz

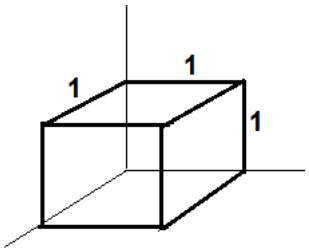
Proiectii paralele(5)

3. Proiectii oblice

- Planul de proiectie este perpendicular pe o axa principala.
- Directia de proiectie nu este perpendiculara pe planul de proiectie.
- Fețele II cu planul de proiectie se proiecteaza fara modificarea unghiurilor si a marimii laturilor
- Alegand directia de proiectie se obtin cazurile particulare:

❑ **Proiectii Cavalier**

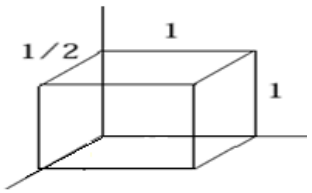
(unghiul proiectorilor cu planul de proiectie = 45 grade)



- Lungimea proiectiei unei laturi perpendiculare pe plan este egala cu lungimea laturii 3D (conserva lungimea laturilor perpendiculare pe planul de proiectie)

❑ **Proiectii Cabinet**

(unghiul proiectorilor cu planul de proiectie = 63.43 grade)

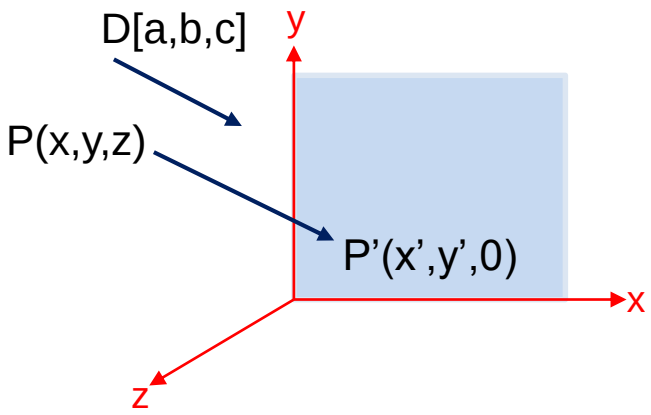


- Lungimea proiectiei unei laturi perpendiculare pe plan este egala cu jumatate din lungimea laturii 3D

Proiectii paralele in planul XOY(6)

Fie punctul P(x,y,z) şi direcţia de proiectie D[a,b,c].

Proiectia lui P in XOY este punctul P' (x',y',0).



Din conditia D || PP', rezulta:

PP' = s*D, unde s este un scalar oarecare

x' – x = s*a

y' – y = s*b

0 – z = s*c → s = z/c → x' = x – (a/c)*z

y' = y – (b/c) * z

Proiectii paralele in planul XOY(7)

Reprezentarea matriciala a proiectiilor paralele:

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pparalela \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix} \qquad Pparalela = \begin{vmatrix} 1 & 0 & \frac{-a}{c} & 0 \\ 0 & 1 & \frac{-b}{c} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Ca si in cazul proiectiilor perspectiva, pentru conservarea informatiei de adancime se foloseste matricea →

$$Pparalela = \begin{vmatrix} 1 & 0 & \frac{-a}{c} & 0 \\ 0 & 1 & \frac{-b}{c} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Cazuri particulare

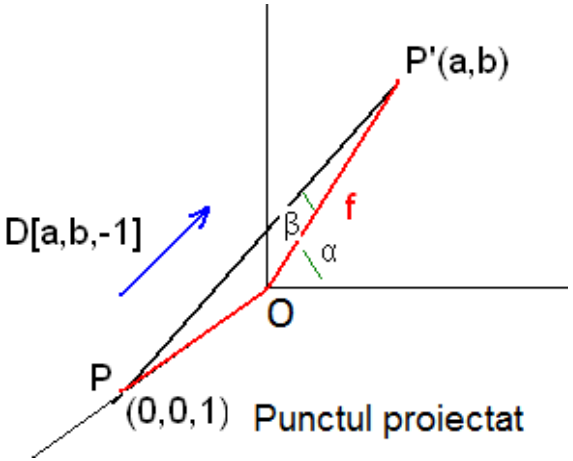
1. Proiectia ortografica in XOY (proiectorii perpendiculari pe XOY):

D[a, b, c]
a= 0, b = 0 →

$$P_{O\text{-}XOY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Proiectii paralele in planul XOY(8)

2. Proiectii oblice in XOY



c = -1 → $P_{OBl} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

$P_{OBl} = \begin{bmatrix} 1 & 0 & f*\cos(\alpha) & 0 \\ 0 & 1 & f*\sin(\alpha) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

OP de lungime 1 se proiecteaza in OP' de lungime f: → f este factorul de scalare al laturilor perpendiculare pe XOY (OP este perpendicular pe XOY).

$tg(\beta) = 1/f$

$f = 0 \rightarrow \beta = 90^0$: proiectie ortografica

$f = 1 \rightarrow \beta = 45^0$: proiectie Cavalier

$f = 0.5 \rightarrow \beta = 63.43^0$: proiectie Cabinet

α este un parametru liber

in mod uzual, $\alpha = 30^0$ sau 45^0

Exemple de proiectii oblice

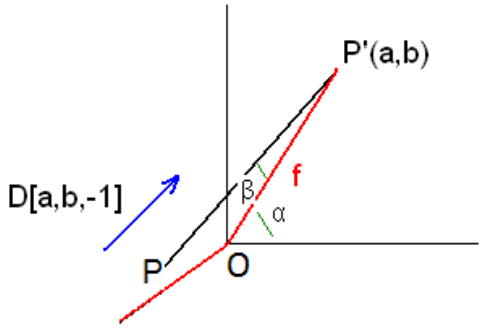
Proiectie Cavalier: f=1, beta=45, alfa=45

Directia de proiectie: [cos(45),sin(45),-1]

Proiectie Cabinet: f=0.5, beta=63.43, alfa=45

Directia de proiectie:

[0.5*cos(45),0.5*sin(45),-1]



Proiectii paralele in planul XOY(9)

3. Proiectii axonometrice(1)

- **proiectii ortogonale într-un plan care intersecteaza 2/3 axe principale**

Sunt 2 posibilități de exprimare matematica:

1. Obiect fix in spatiu, se alege planul astfel incat sa obtinem tipul de proiectie dorit (izometrica, dimetrica, generala):

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pax(N,R) \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

- Pax - proiectie ortogonala intr-un plan oarecare (N,R)

2. Plan de proiectie fix (XOY), se pozitioneaza obiectul față de plan astfel incat sa obtinem tipul de proiectie dorit :

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Po\text{-}xoy * TA \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

- Pax - transformare geometrica a obiectului (TA) urmata de proiectie ortografica in XOY (Po-xoy)– cu efect echivalent

TA consta din rotatii in jurul axelor OX si OY. Particularizand unghiurile de rotatie obtinem cazurile de proiectii axonometrice: izometrice, dimetrice, generale.

Proiectii paralele in planul XOY(10)

Proiectii axonometrice(2)

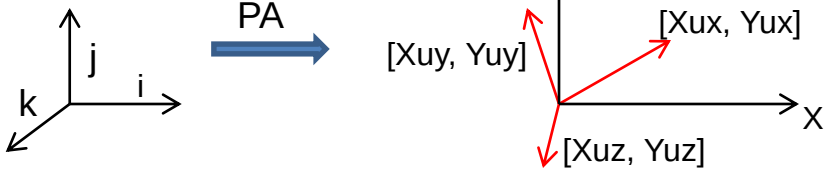
Exemplu:

TA = Rx(ux) * Ry(uy) PA = Po-xoy* TA =

$$\begin{bmatrix} \cos(uy) & 0 & \sin(uy) & 0 \\ \sin(ux)^*\sin(uy) & \cos(ux) & -\cos(uy)^*\sin(ux) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se aplica transformarea PA versorilor axelor principale:

$$\begin{bmatrix} Xux & Xuy & Xuz \\ Yux & Yuy & Yuz \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} = PA \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{matrix} ux \\ uy \\ uz \end{matrix}$$



Factorii de scalare a laturilor pe cele 3 axe sunt lungimile proiectiilor celor trei vectori unitate:

$$sx = (Xux^2 + Yux^2)^{0.5}$$

$$sy = (Xuy^2 + Yuy^2)^{0.5}$$

$$sz = (Xuz^2 + Yuz^2)^{0.5}$$

Proiectii paralele in planul XOY(11)

Proiectii axonometrice(3)

Se deduc unghiurile de rotatie in jurul axelor pentru obtinerea celor 2 cazuri particulare de proiectii axonometrice: izometrice si dimetrice.

1. *Proiectii izometrice:*

- Se impune conditia: sx = sy = sz
- Rezulta: ux = (+/-) 35.26 grade ; uy = (+/-) 45 grade

2. *Proiectii dimetrice:* doi dintre factorii de scalare sunt egali

- Daca se impune conditia: sx = sy, rezulta:

  ux = arcsin((+/-) sz $\sqrt{2}$), uy = arcsin((+/-) sz $\sqrt{(2-sz^2)}$

  sz poate fi ales intre 0 si 1;

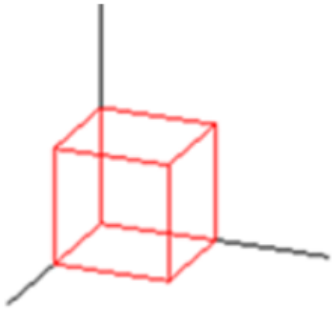
  - pentru fiecare valoare a lui sz exista 4 proiectii dimetrice; de exemplu:
  - pentru sz = 0.5 : ux = +/- 20.705 grade, uy = +/- 22.208 grade

Exemple de proiectii axonometrice

Proiectie izometrica:

P_ort_XOY * R_Ox(35.26)* R_Oy(45)

sx = sy = sz



Proiectie dimetrica:

P_ort_XOY * R_Ox(20.705)* R_Oy(22.208)

sx = sy # sz

Transformarea de vizualizare din spatiul 3D

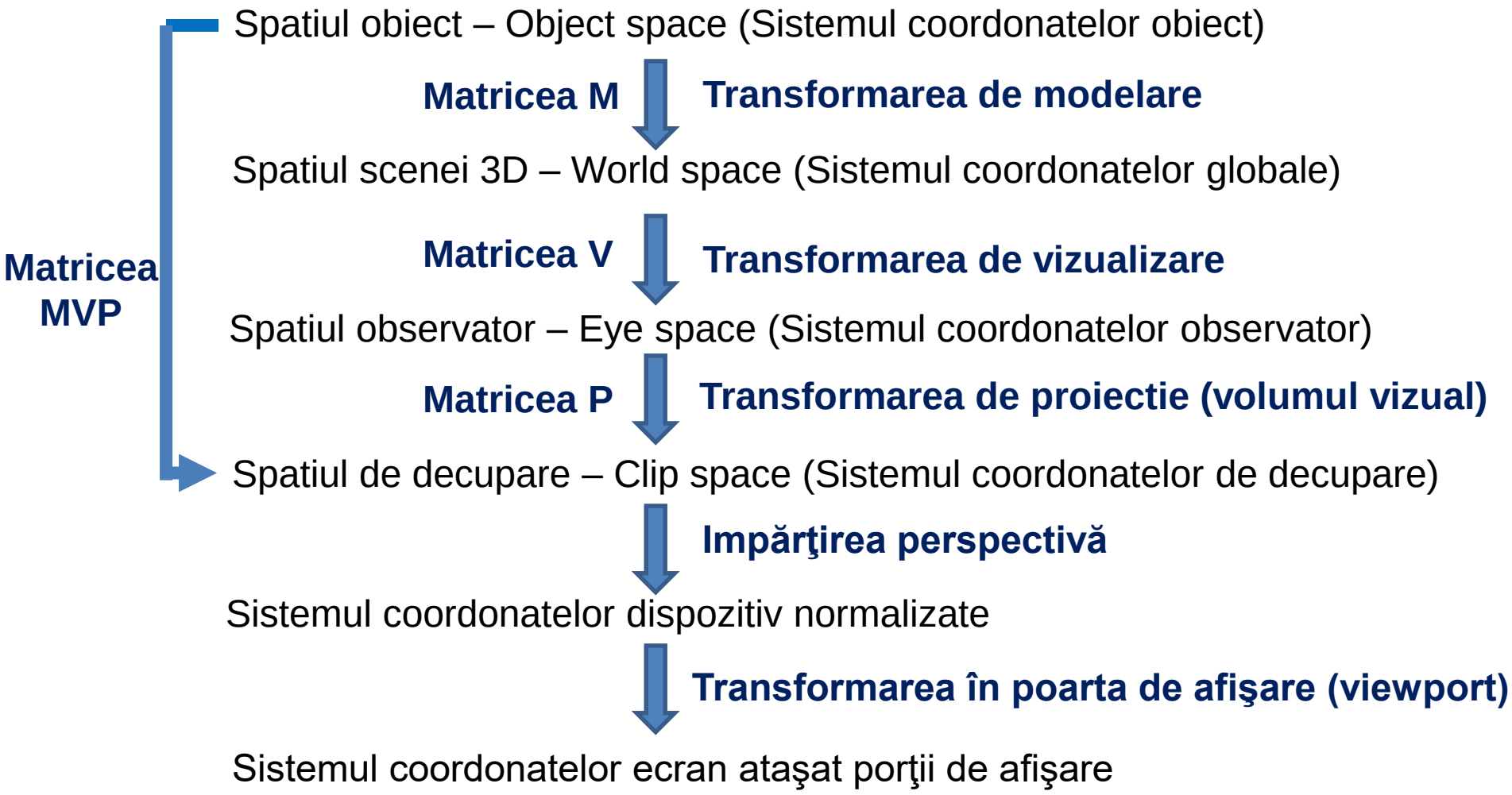
Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Transformarea de vizualizare din spatiul 3D

- Are ca scop obtinerea unei vederi a unei scene 3D pe o suprafaţă de afişare.
- Poate fi comparată cu transformarea efectuată de o cameră foto asupra lumii reale pentru a se obţine imaginea dintr-o fotografie.
  - Imaginea obţinută depinde de:
    - poziţia camerei (observatorului)
    - orientarea camerei
    - ceea ce vede ochiul prin vizorul camerei foto (volumul vizual)
- Este o transformare compusă din mai multe transformări, între care şi o proiectie $R^3 \rightarrow R^2$
- Transformările se efectuează asupra vârfurilor obiectelor din scena 3D.
- Include şi operaţia de decupare (clipping) a primitivelor grafice la marginea volumului vizual (eliminarea părţilor din scena 3D care sunt în afara volumului vizual al vederii curente).

Transformarea varfurilor în OpenGL



Matricile transformarii varfurilor obiectelor 3D

M – matricea de modelare: difera de la obiect la obiect

V – matricea de vizualizare: aceeasi pentru intreaga scena vizualizata

P – matricea de proiectie: aceeasi pentru intreaga scena vizualizata

MV = V· M (matricea “model-view”)

MVP = P · V · M (matricea “model-view-projection”)

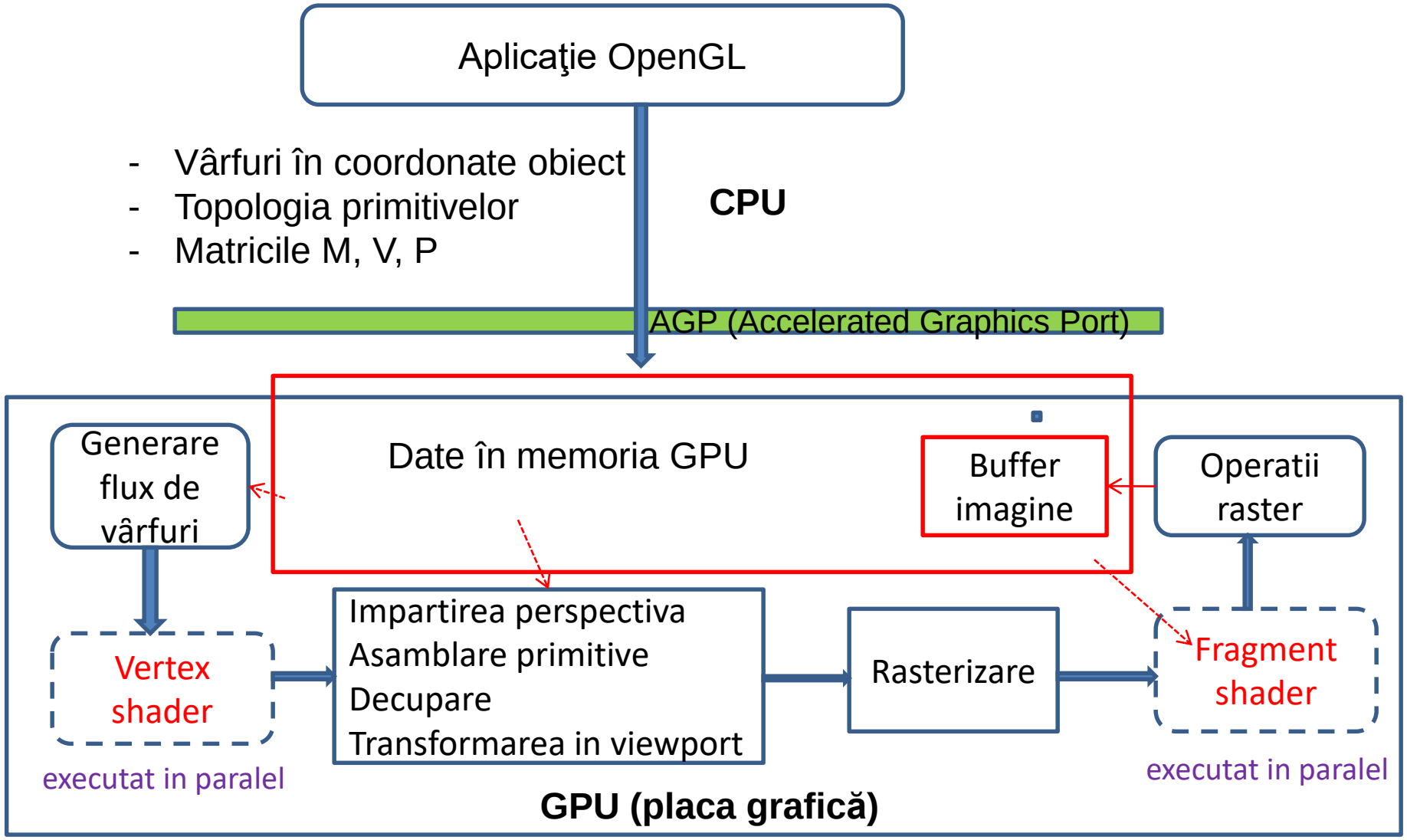
Transformarea vârfurilor din coordonate obiect în coordonate de decupare (clip):

$$[xc, yc, zc]^T = PVM \cdot [xo, yo, zo]^T$$

- Matricile sunt calculate în programul OpenGL folosind functii ale bibliotecii **glm** şi transferate la GPU
- Transformarea vârfurilor este efectuată în programul ***Vertex Shader executat pe GPU,*** folosindu-se matricea MVP.
- Urmează: împarțirea perspectivă prin care se obțin coord. carteziene ale varfurilor; asamblarea primitivelor folosind topologia transmisa la GPU de programul OpenGL; decuparea primitivelor la frontiera volumului vizual canonic; transformarea primitivelor decupate, în viewport.

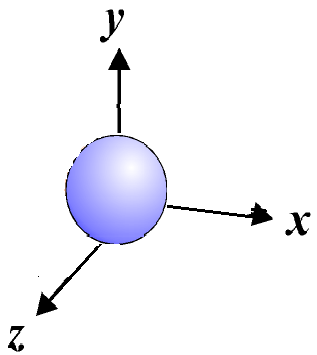
Banda grafică- a V-a generaţie GPU

➢Incepand cu OpenGL 3.0:



Spatiul obiect

Spatiul obiect (Sistemul coordonatelor obiect)



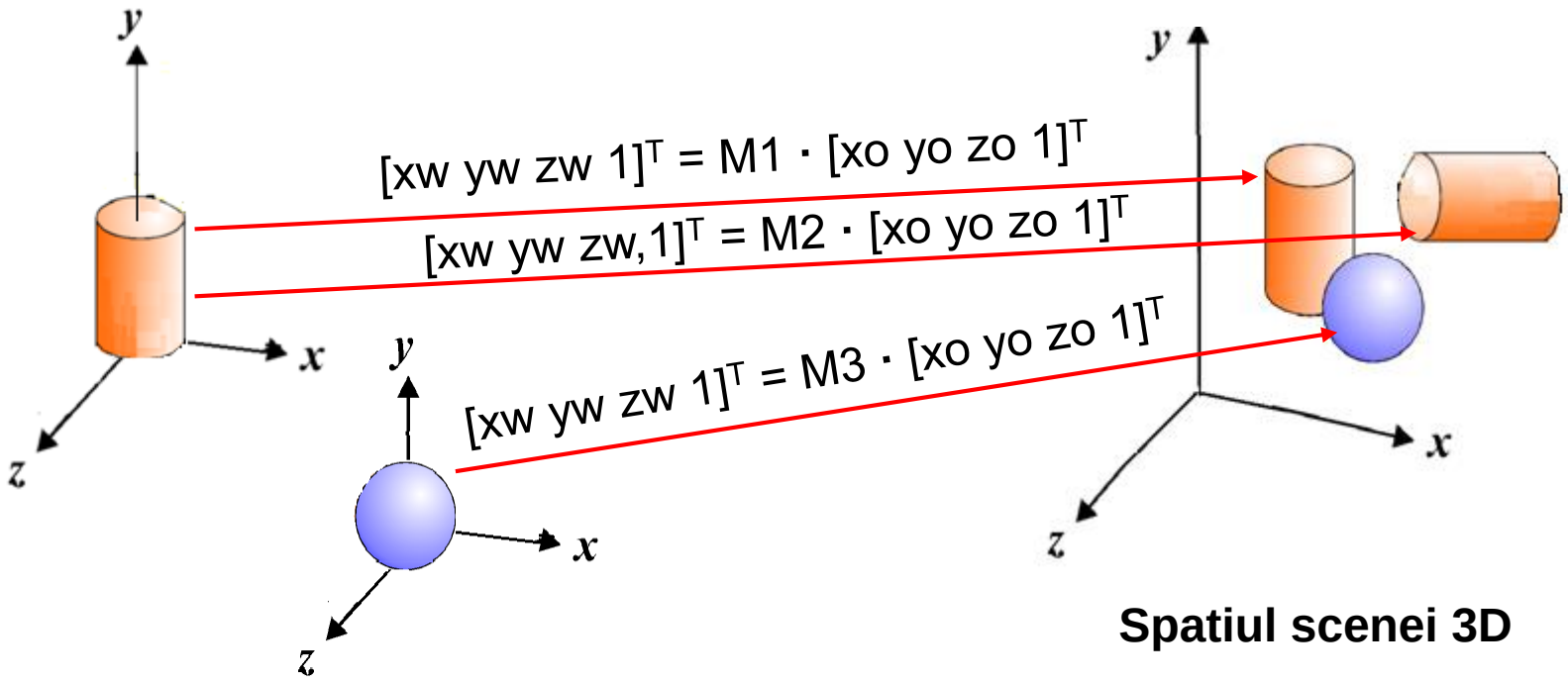
- In mod uzual, fiecare obiect este definit in propriul sau sistem de coordonate (sistemul coordonatelor locale – coordonate obiect)
- Sistem de coordonate carteziene 3D « dreapta »
- Exemple: cubul, sfera, scheletul unui personaj animat, elemente de mobilier, etc.

  De regula, obiectul este definit centrat in originea sistemului de coordonate.
- Avantaje: usurinta modelarii şi reutilizarea modelelor obiectelor 3D

Spatiul scenei 3D -World space (Sistemul coordonatelor globale)

- Spaţiul in care este compusă scena 3D
- Scena - raportată la un sistem de referinţă global – sistem de coordonate carteziene 3D “dreapta”, numit *Sistemul coordonatelor globale*
- Originea sistemului – aleasa de modelatorul scenei

Spatiul scenei 3D
Transformarea de modelare



[xo yo zo 1]: coord omogene varf în sist de coord al unui obiect

[xw yw zw 1]: coord omogene varf în sist de coord globale

Transformarea de modelare:

Transformare geometrica compusa. Exemple:

M1 = T(tx1,ty1,tz1)
M2 = T(tx2,ty2,tz2)* Roz (1.57)

Spatiul scenei 3D

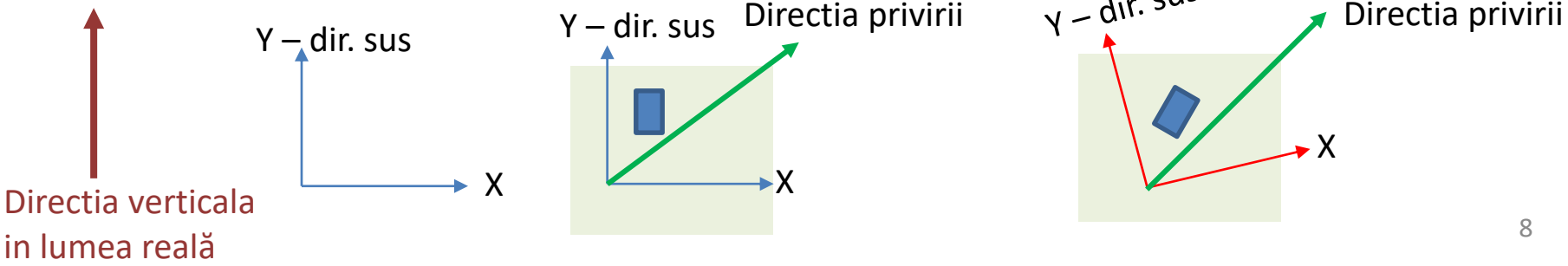
Obiecte definite în sistemul coordonatelor globale

Spatiul camerei (observator)

Analogie cu aparatul de fotografiat (camera reala)

Un aparat foto poate produce o fotografie in care apar o parte din obiectele spatiului real (3D).

- Continutul fotografiei depinde de:
  - pozitia aparatului in spatial real
  - orientarea acestuia: directia in care priveste fotograful
  - rotatia aparatului față de pozitia sa normală
- Fotografia este formata intr-un plan 2D, fiind raportata la un sistem cartezian 2D, XOY:
- **In pozitia normala a aparatului, directia axei OY (axa verticala) este directia "sus" a aparatului**.
- Directia in care priveste fotograful este perpendiculara pe planul imaginii, formand impreuna cu axele OX, OY un sistem de coord carteziene 3D.
- Fotograful poate roti aparatul de fotografiat, astfel incat directia "sus" a aparatului se schimbă.



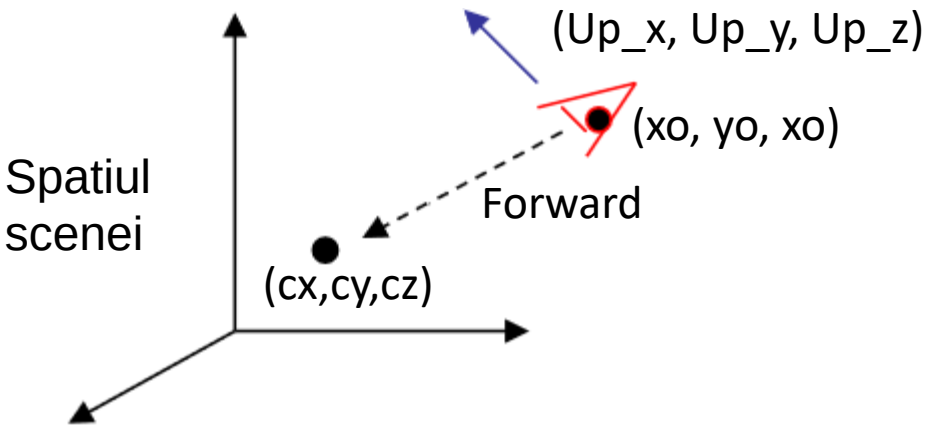
Spatiul camerei (observator)
Sistemul de coordonate observator

- Spaţiul camerei este spaţiul scenei 3D văzut din pozitia camerei virtuale (observatorului) pe direcţia în care priveşte observatorul.
- Spatiul camerei este raportat la un sistem de coordonate atasat camerei – sistemul de coordonate observator.
- **Sistemul de coordonate observator este definit prin 3 parametri**:

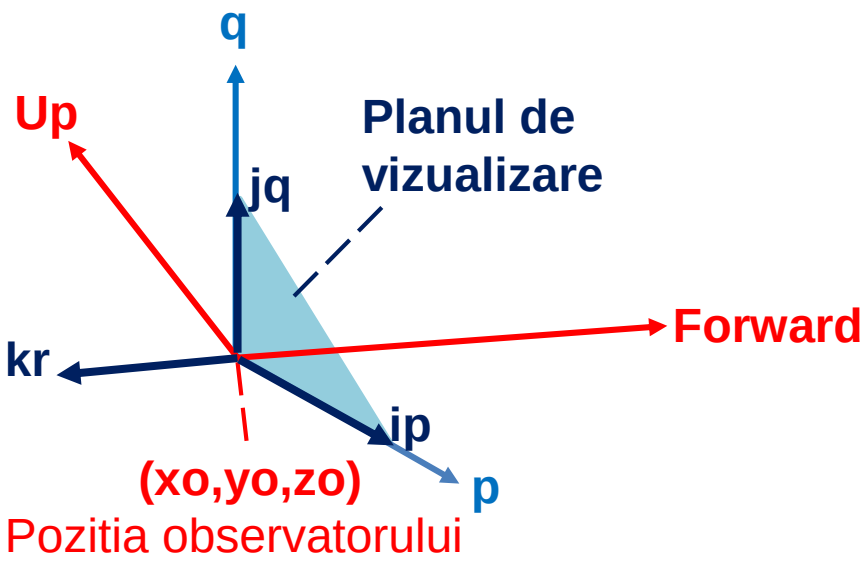
1. Originea sa: **pozitia camerei virtuale** in spatiul scenei **(xo, yo, xo).**

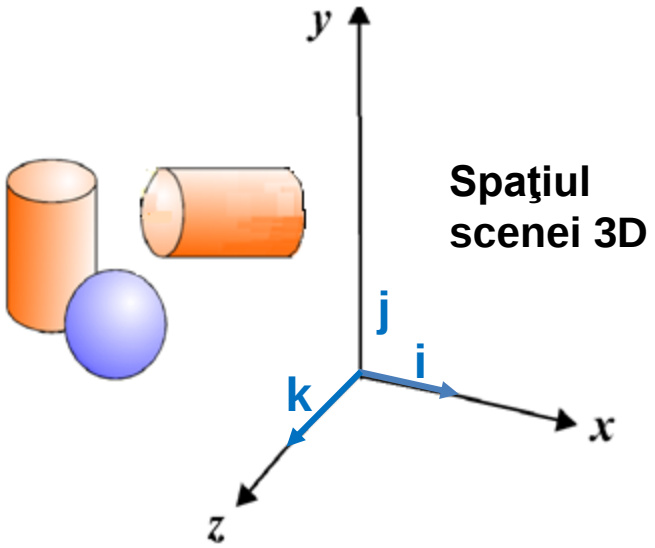
2. Directia in care priveste observatorul (spre un punct din scenă) – **vectorul Forward**

3. **Directia "sus" a camerei** (**vectorul Up**): reprezinta **rotatia camerei**

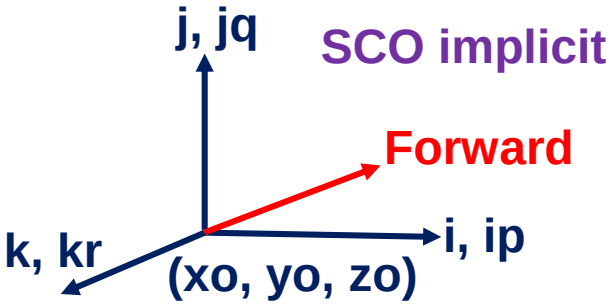


Sistemul de coordonate observator (SCO)



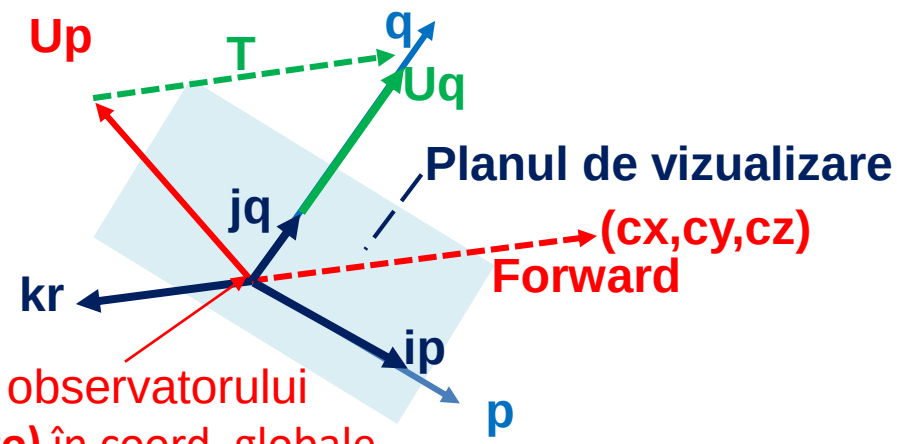


- (xo, yo, zo): originea sistemului de coordonate observator (pozitia camerei)
- (xo, yo, zo) impreuna cu vectorul **Forward** definesc un plan, numit **planul de vizualizare.**
- Forward – normala la planul de vizualizare
- axa q: axa verticală a planului de vizualizare
- axa p: axa orizontală a planului de vizualizare
- (ip,jq,kr) formeaza un sistem de coordonate 3D dreapta



ip, jq, kr: versorii axelor sistemului de coordonate observator coincid cu cei ai sist. coordonatelor globale

Determinarea axelor sistemului de coordonate observator (1)



Pozitia observatorului
(xo,yo,zo) în coord. globale
(0,0,0) în coord observator (camera)

kr = - Forward/|Forward|
Versorul directiei axei z a SCO

- Uq: proiectia ortogonala a vectorului Up in planul de vizualizare: Uq = Up + T
- Uq dă direcţia axei verticale a planului de vizualizare

T || (-kr) → T = s*(-kr) = - s*kr (s un scalar oarecare) → Uq = Up - s*kr

Uq perpendicular pe kr → Uq·kr = 0 → (Up - s*kr)·kr =0 → Up·kr - s*kr·kr = 0 → s = Up·kr

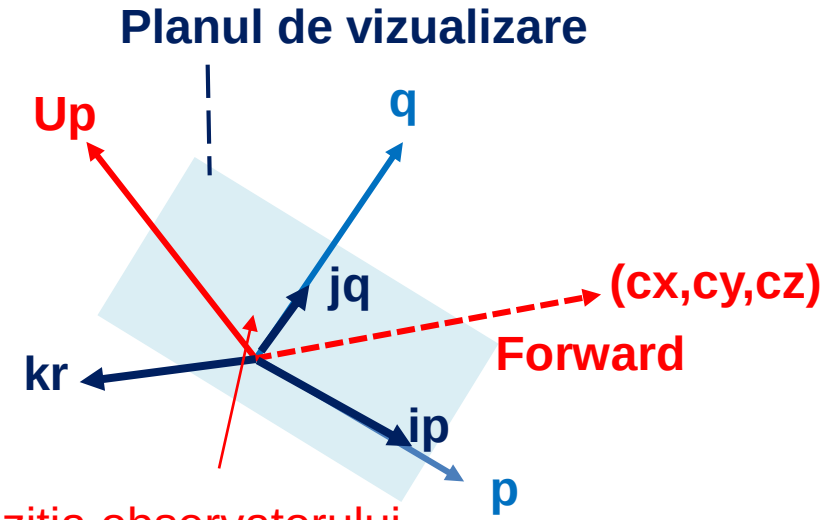
Uq = Up – (Up·kr)·kr

jq = Uq/|Uq| este versorul axei verticale a planului de vizualizare

ip = jq x kr este versorul axei orizontale a planului de vizualizare

(ip, jq, kr) formeaza un sistem de coordonate carteziene 3D dreapta

Determinarea axelor sistemului de coordonate observator (2) - o alta metoda -



Pozitia observatorului
(xo,yo,zo) în coord. globale
(0,0,0) în coord observator (camera)

kr = - Forward/|Forward|
Versorul directiei axei z a SCO

p este un vector perpendicular pe planul format de vectorii Up si kr:

p = Up x kr

ip = p/ |p|

(ip, jq, kr) trebuie sa formeze un sistem de coordonate carteziene 3D dreapta

Deci:

jq = kr x ip este versorul axei verticale a planului de vizualizare

Matricea transformarii de vizualizare (V) -1

Transformarea de vizualizare: **din spatiul scenei → în spatiul camerei**

$[xe\ ye\ ze\ 1]^T = V \bullet [xw\ yw\ zw\ 1]^T$ eye coordinates ← world coordinates

Sistemul de coordonate observator implicit in OpenGL

- poziţia observatorului: **originea sistemului de coordonate globale**
- direcţia în care priveşte observatorul: **direcţia negativa a axei OZ**
- direcţia sus a planului de vizualizare: **direcţia pozitivã a axei OY**

Cu aceste valori implicite, transformarea de vizualizare este transformarea identica.

Pentru calculul matricei transformarii de vizualizare poate fi folosita functia **lookAt:**

```
glm::mat4 V =
        glm::lookAt (glm::vec3 (xo,yo,zo), glm::vec3 (cx,cy,cz), glm::vec3 (Upx, Upy, Upz))
```

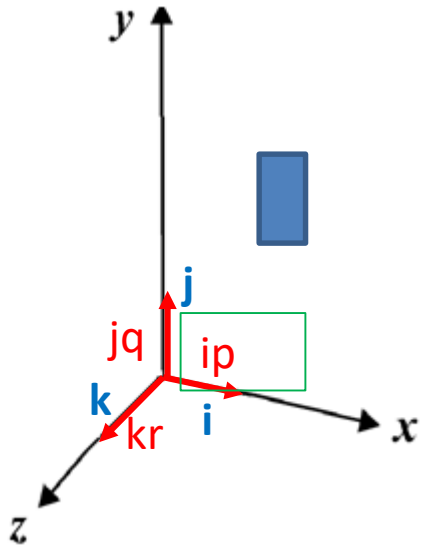
(xo, yo, zo): **poziţia observatorului**

(cx,cy,cz): centrul de interes - punctul din scena 3D catre care priveste observatorul;

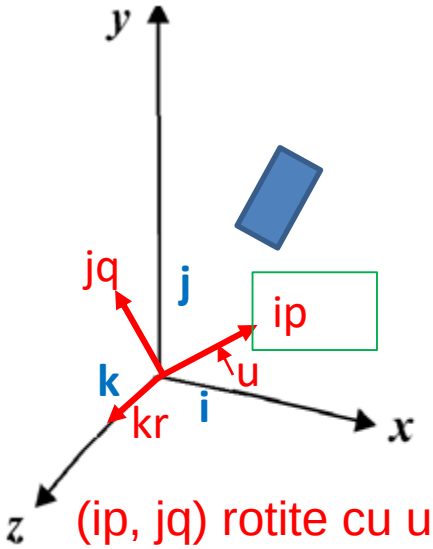
vectorul (xo,yo,zo) → (cx,cy,cz) reprezinta **vectorul Forward**

(Upx, Upy, Upz): v**ectorul « sus » al camerei (Up)**

Matricea transformarii de vizualizare -2



V = I (matricea identica)



(ip, jq) rotite cu unghiul u in jurul axei oz

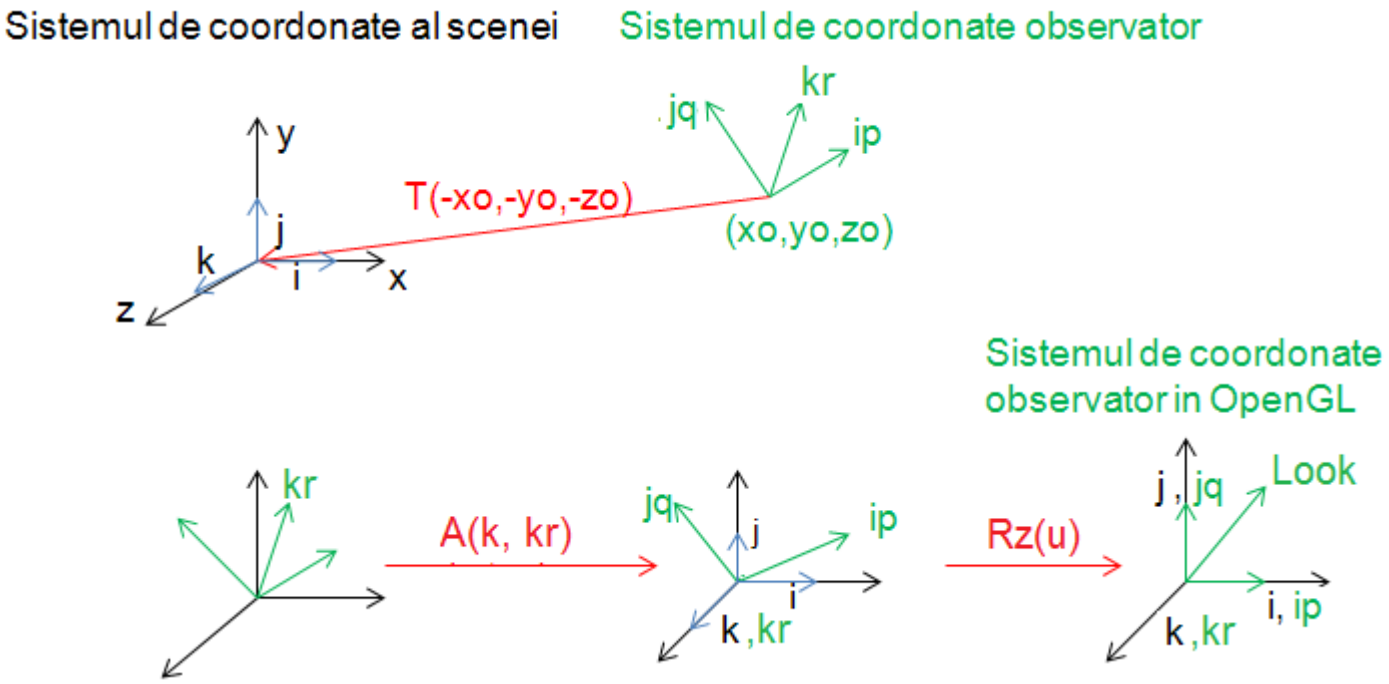
V?

V: matricea care transforma varfurile obiectelor din spatiul scenei in spatiul camerei (observator - eye) → $[xe\ ye\ ze\ 1]^T = R_{OZ}(-u) \bullet [xw\ yw\ zw\ 1]^T$

Matricea transformarii de vizualizare: $V = R_{OZ}(-u)$

- Aplicand V axelor sistemului de coordonate observator, se vor suprapune pe cele ale sistemului de coordonate globale.

Matricea transformarii de vizualizare -3



- A(k,kr) este o transformare prin care se aliniaza axele z ale celor 2 sisteme de coordonate: consta din 2 rotatii – o rotatie in jurul axei X si una in jurul axei Y.
- Prin rotatia in jurul axei z (Rz) se aliniaza celelalte 2 axe.

Matricea transformarii de vizualizare: V = Rz(u) * A(k,kr) * T(-xo,-yo,-zo)

Transformarea de modelare si vizualizare

din sistemul de coordonate obiect → in sistemul de coordonate observator

- Transformare compusa (model-view) : MV = V•M

  M – matricea de modelare folosita pentru transformarea unui obiect in spatiul scenei

  V – matricea de vizualizare a scenei

  Transformarea varfurilor din coordonate obiect in coordonate observator (*eye*):

  $[xe\ ye\ \ ze\ we]^T = V \bullet M \bullet [xo\ yo\ zo\ wo]^T = MV \bullet [xo\ yo\ zo\ wo]^T$

- Pentru calculele de iluminare a obiectelor scenei 3D se folosesc vectori normali atasati varfurilor obiectelor.

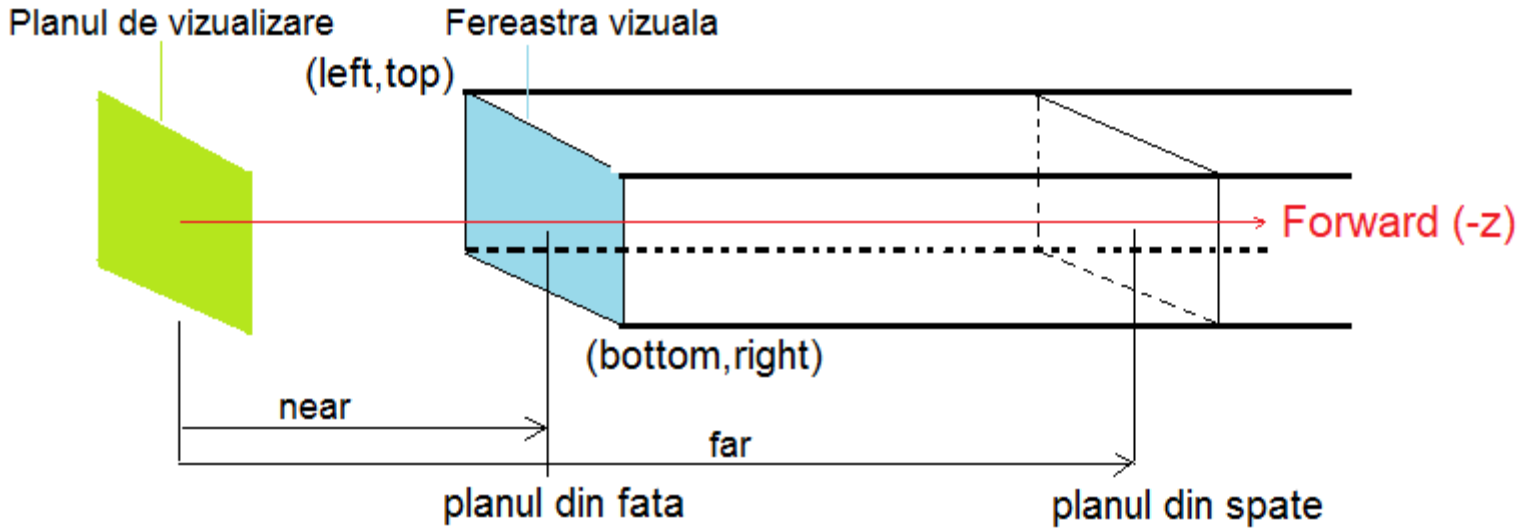
  Transformarea normalelor din coordonate obiect in coordonate observator:

  $[nxe\ nye\ \ nze\ we]^T = (MV^{-1})^T \bullet [nxo\ nyo\ nzo\ nwo]^T$

Transformarea de proiectie (1)

Determinata de **volumul vizual** definit de programator

Volumul vizual al proiectiei ortografice: paralelipiped dreptunghic cu laturile paralele cu axa z a sistemului de coordonate observator (SCO), delimitat in adancime de planul din fata (near) si planul din spate (far) al vederii; near si far sunt distanţe (pozitive) faţă de planul de vizualizare masurate pe axa z negativa a SCO.

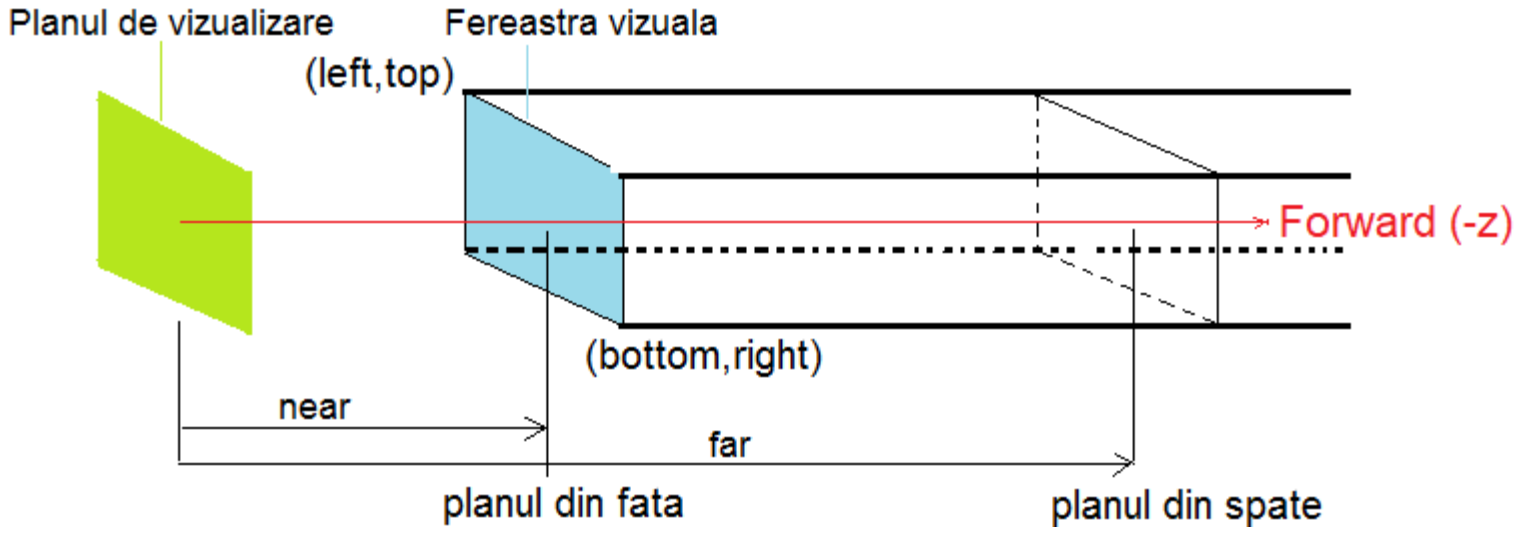


- Coordonatele colturilor ferestrei vizuale sunt raportate la sistemul de coordonate al planului din faţă, care are axele paralele cu cele ale planului de vizualizare.

Transformarea de proiectie (2)

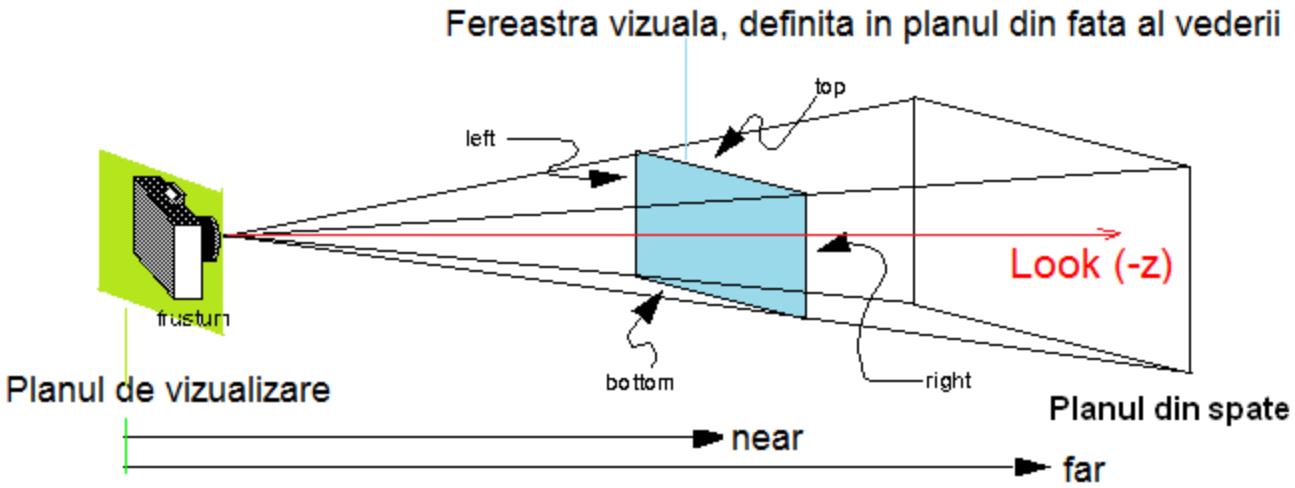
Proiectia ortografica

- Se efectueaza proiectia ortografica în fereastra vizuala a obiectelor cuprinse in volumul vizual. Primitivele grafice situate complet in afara volumului vizual sunt eliminate iar cele care intersecteaza volumul sunt decupate la frontiera volumului creindu-se noi primitive.



Transformarea de proiectie (3)

Volumul vizual al proiectiei perspectiva (View frustum): trunchi de piramida delimitat in adancime de planul din faţă (near) si planul din spate (far) al vederii, cu varful in pozitia observatorului.



Se efectueaza proiectia perspectiva în planul din faţă (in fereastra vizuala), cu centrul de proiecţie în poziţia observatorului, a obiectelor cuprinse in volumul vizual. Obiectele situate in afara volumului vizual sunt eliminate sau decupate la frontiera volumului vizual.

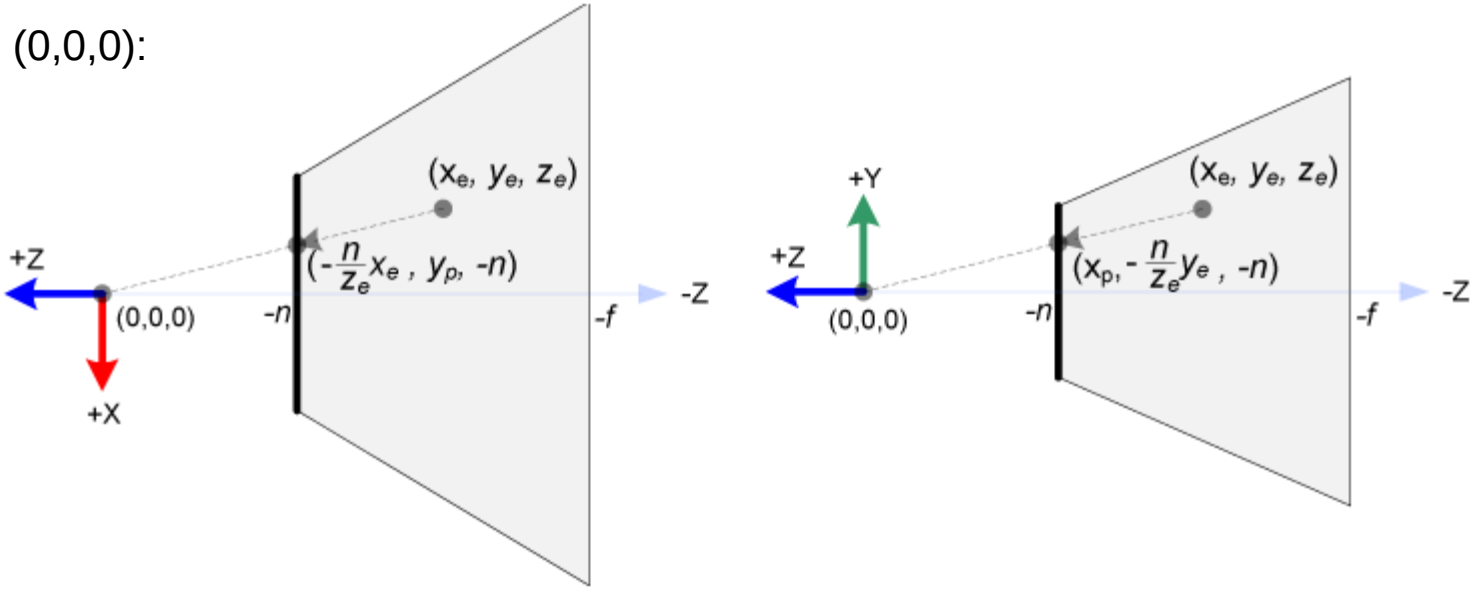
Transformarea de proiectie (4) - proiectia

- **Proiectia se efectueaza in fereastra vizuala din planul din** faţă: planul z =-near (in SCO).
- Varfurile proiectate sunt raportate la sist. coordonatelor observator.

Proiectia ortografica a unui punct din sist. coordonatelor observator, (xe,ye,ze), in fereastra vizuala:

xp = xe, yp = ye, zp = -near.

Proiectia perspectiva a unui punct din SCO, (xe,ye,ze), in planul z = -near, cu centrul de proiectie in (0,0,0):



xp = (-near/ze)*xe, yp =(-near/ze)*ye – se pot obtine aplicand formulele proiectiei perspectiva cu centrul de proiectie in (0,0,0) si planul de proiectie este z=-near.

Transformarea de proiectie (5) – Sistemul coordonatelor de decupare

- Transformarea de proiectie este insotita de operatia de decupare a primitivelor grafice la marginile volumului vizual.
- Pentru simplificarea calculelor de decupare, volumul vizual definit de programator este transformat in volumul vizual canonic: cub cu latura de 2 unitati, centrat in originea sistemului coordonatelor de decupare, cu laturile paralele cu axele.
- Transformarea prin care volumul vizual este transformat in volumul vizual canonic se numeste transformarea de normalizare.

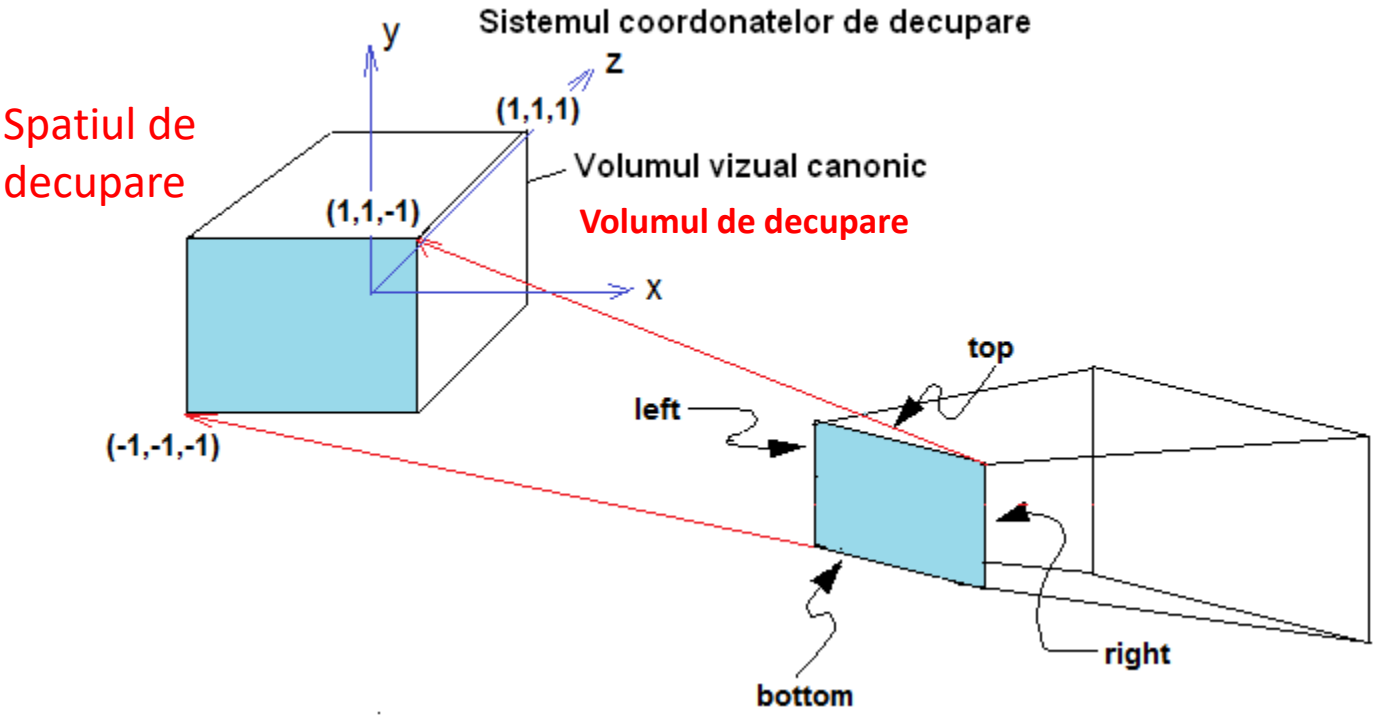
- In OpenGL transformarea de proiectie include: proiectia definita prin volumul de vizualizare compusa cu transformarea de normalizare.

  = transformarea coordonatelor observator in coordonate de decupare.

- Dupa transformarea varfurilor prin transformarea de proiectie, varfurile sunt unite pe baza topologiei definite de programator, formand primitive grafice 3D. Acestea sunt decupate la marginile volumului vizual canonic.

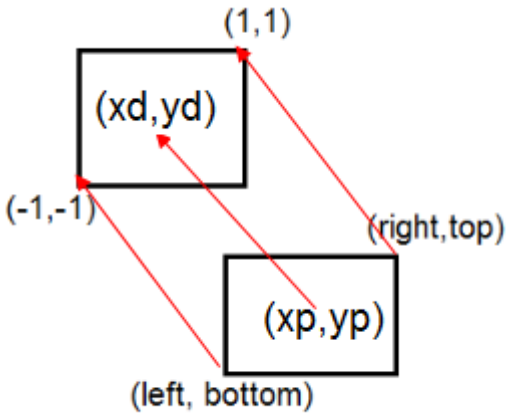
Transformarea de proiectie (6) – normalizarea

- *Sistemul coordonatelor de decupare*: sistem de coordonate carteziene 3D stanga.
- **Prin transformarea de normalizare:**
  - colţurile ferestrei 2D din planul de proiecţie, (left, bottom,-near) şi (right, top,-near) sunt mapate pe colţurile stânga-jos şi dreapta-sus ale ferestrei 2D din sistemul coordonatelor de decupare, adică (-1,-1,-1) şi (1,1,-1);
  - planul z= -near se mapeaza pe z= -1, iar z= -far se mapeaza pe z= 1



Transformarea de proiectie (7) calculul coordonatelor de decupare

- Impunand conditia (left,bottom) → (-1,-1) si (right,top) → (1,1), rezulta transformarea punctelor proiectate pe planul din faţă, (xp,yp), in sistemul coordonatelor de decupare – este o transformare freastra-poarta: xp = (-near/ze)*xe



xd = 2*xp/(right – left) – (right + left)/(right – left)

yd = 2*yp/(top – bottom) – (top + bottom)/(top-bottom)

Pentru proiectia perspectiva:

Inlocuind xp si yp prin expresiile lor in functie de (xe,ye, near) rezulta:

xd = (xe*2*near/(right – left) + ze*(right + left)/(right – left))/(-ze)

yd = (ye*2*near/(top – bottom) + ze*(top + bottom)/(top-bottom))/(-ze)

Pentru transformarea coordonatelor z se impun conditiile: -near → -1, -far → 1; rezulta:

zd = (-ze*(far+near)/(far-near) – 2*far*near/(far-near)) / (-ze)

(xd,yd,zd) sunt coordonate carteziene in sistemul coordonatelor de decupare (clip coordinates);

Matricile transformarii de proiectie -1

Pentru proiectia perspectiva:

$$Per = \begin{bmatrix} \frac{2 * near}{right - left} & 0 & A & 0 \\ 0 & \frac{2 * near}{top - bottom} & B & 0 \\ 0 & 0 & C & D \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$$A = \frac{right + left}{right - left} \qquad B = \frac{top + bottom}{top - bottom}$$

$$C = -\frac{far + near}{far - near} \qquad D = -\frac{2 * far * near}{far - near}$$

$[xc\ yc\ \ zc\ wc]^T = Per \bullet [xe\ ye\ ze\ 1]^T$ $\quad xc = xd*wc,\ yc = yd*wc,\ zc=zd*wc,\ wc = -ze$

Pentru proiectia ortografica:

xp=xe si yp=ye → xd = 2*xe/(right – left) – (right + left)/(right – left)

yd = 2*ye/(top – bottom) – (top + bottom)/(top-bottom)

$$Par = \begin{bmatrix} \frac{2}{right - left} & 0 & 0 & -A \\ 0 & \frac{2}{top - bottom} & 0 & -B \\ 0 & 0 & \frac{-2}{far - near} & C \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$[xc\ yc\ \ zc\ 1]^T = Par \bullet [xe\ ye\ ze\ 1]^T$

Matricile transformarii de proiectie -2

Matricile de transformarii de proiectie pot fi obtinute apeland urmatoarele functii din biblioteca glm:

glm::ortho(float left, float right, float bottom, float top, float zNear, float zFar);

glm::frustum (float left, float right, float bottom, float top, float zNear, float zFar);

glm::perspective (float fov, float aspect, float znear, float zfar);

Creaza o matrice de proiectie pentru un volum perspectiva simetric:

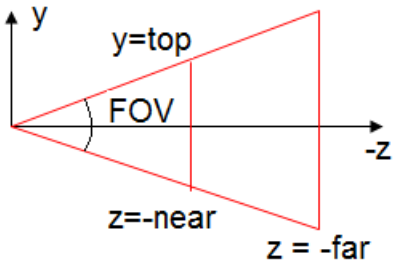
- fov: Field of View

  tan(FOV/2) = top/near

- aspect: aspect ratio

  aspect ratio = (right-left)/(top-bottom)

  = raportul dintre latimea si inaltimea ferestrei vizuale



Impartirea perspectiva

Transformarea de proiectie: coordonate observator → coordonate de decupare

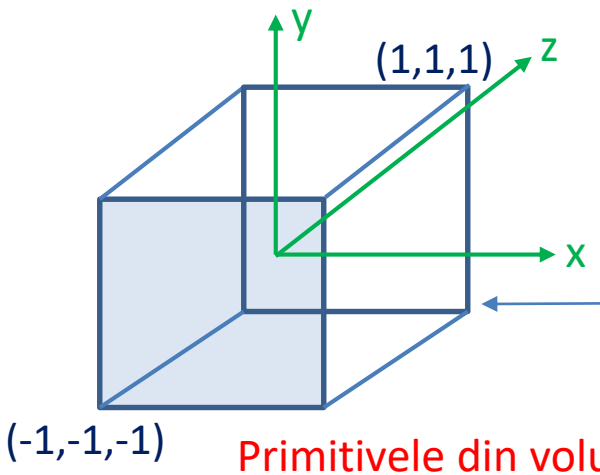
$[xc\ yc\ zc\ wc]^T$ = Par/Per • $[xe\ ye\ ze\ 1]^T$ (clip coordinates) ← eye coordinates)

Impartirea perspectiva: coordonate de decupare → coordonate dispozitiv normalizate

xd = xc/wc , yd = yc/wc , zd= zc/wc

- **Varfurile rezultate din împartirea perspectiva sunt asamblate în primitive grafice care sunt decupate la frontiera volumului vizual canonic.**

Sistemul coordonatelor de decupare



După transformarea de proiectie, observatorul este la infinit pe axa OZ negativă a sistemului coord. de decupare.

Distanța față de observator se masoara pe axa OZ pozitivă.

Volumul de decupare

După decupare: -1 <= xd, yd,zd <=1

Primitivele din volumul de decupare sunt proiectate ortografic pe fața "din față" a volumului: (-1,-1,-1) – (1,1,-1).

Transformarea în poarta de afisare
coordonate dispozitiv normalizate → coordonate ecran

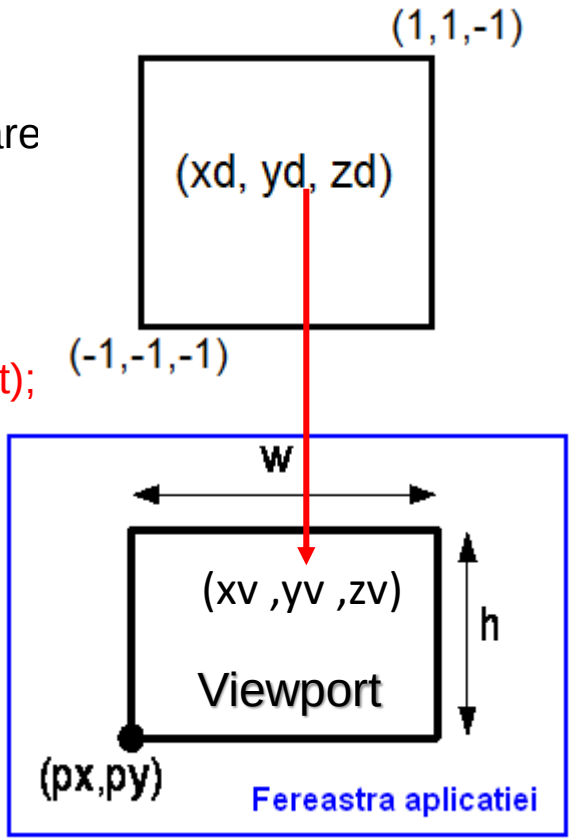
- Este o transformare fereasta-poarta, in care:
- Fereastra este fereastra 2D din sistemul coordonatelor de decupare

  având colţurile în (-1, -1, -1) - (1, 1, -1).
- Poarta este definita de programator folosind functia:

  void **glViewport**(GLint px, GLint Py, GLsizei width, GLsizei height);

  (*px, py*) reprezintă coordonate relative la coltul stanga-jos al ferestrei aplicatiei (în pixeli). Valorile implicite : (0,0).

  width, height reprezintă lăţimea/ înălţimea porţii de afişare. Valorile implicite sunt: lăţimea şi înălţimea ferestrei aplicatiei.



- **Transformarea în poarta de afisare este definita astfel:**

  xv = (width/2)xd + ox , ox = px+ width/2

  yv = (height/2)yd + oy, oy = py + height/2

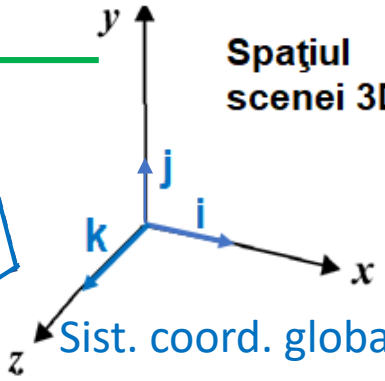
  zv = ((f-n)/2)zd + (n+f)/2 // conserva coordonata z pentru eliminarea partilor nevizibile

  n (near) şi f (far) au valorile implicite 0.0 respectiv 1.0, pentru care 0<=zv<=1

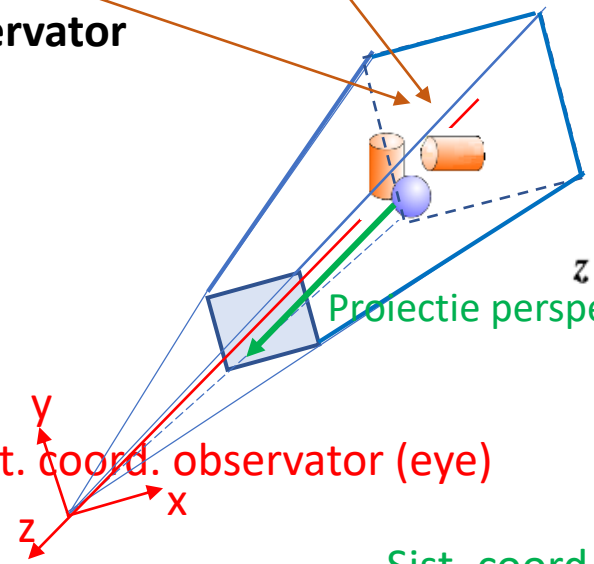
  n si f pot fi modificate la valori cuprinse în intervalul [0,1] folosind funcţia ***DepthRange()***

[xe ye ze 1] = V [xw yw zw 1]

Spatiul observator



Sist. coord. globale (world)



Proiectie perspectiva în planul din faţă

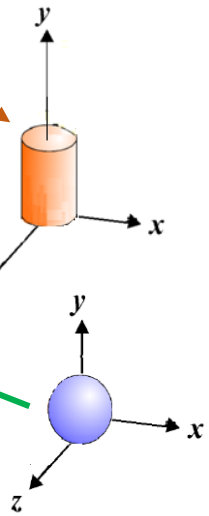
Sist. coord. observator (eye)

Spaţiul scenei 3D

(xo,yo,zo)

M = M1

M = M2



[xc yc zc wc] = P•V•M [xo yo zo 1]

[xc yc zc wc] = P [xe ye ze 1]

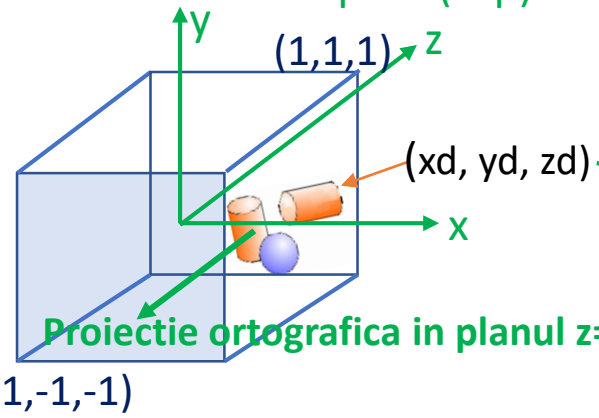
wc = -ze

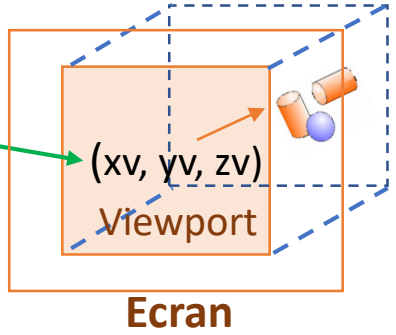
Spatiul de decupare

xd = xc/wc yd = yc/wc zd = zc/wc

Coord.dispozitiv normalizate

Sist. coord. de decupare (clip)





Observatorul la infinit pe axa OZ negativă

→îndepartarea de observator se măsoara prin coord. z pozitive

Eliminarea partilor nevizibile ale scenelor 3D din imagini - 1

Prof. univ. dr. ing. Florica Moldoveanu

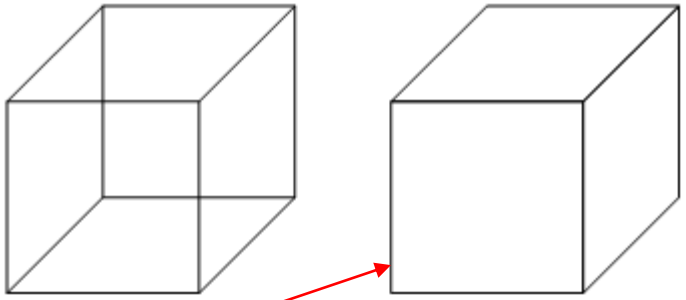
Eliminarea partilor nevizibile ale scenelor 3D, din imagini (1)

Vizibilitatea obiectelor dintr-o scena 3D, într-o imagine, depinde de:

– Pozitia observatorului si directia in care priveste

– Pozitionarea obiectelor din scena unul față de celalalt

– Volumul vizual

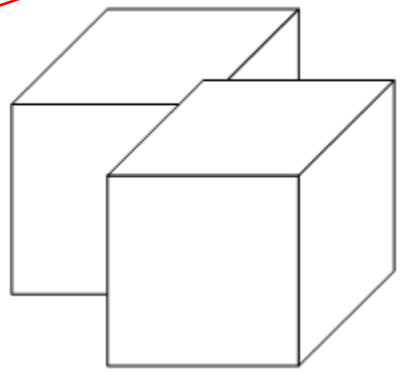


Partile unei scene 3D nevizibile într-o imagine sunt:

– Părți aflate total sau parțial în afara volumului vizual

– Fețe auto-obturate: fețele unui obiect obturate de celelalte parti ale aceluiasi obiect

– Părți obturate (ascunse) de obiecte ale scenei, aflate în fața lor în raport cu observatorul



Eliminarea partilor nevizibile ale scenelor 3D, din imagini (2)

- Eliminarea părților dintr-o scenă 3D care nu sunt nevizibile într-o imagine este efectuata la mai multe niveluri:

1. **Eliminare obiecte sau grupuri de obiecte: grupuri de primitive** - **Object culling**:
   - **Frustum Culling**: eliminarea din banda grafica a grupurilor de primitive care sunt complet în afara volumului visual.
   - **Occlusion Culling**: – eliminarea din banda grafica a grupurilor de primitive care sunt complet obturate de alte grupuri de primitive (obiecte).
   - nu este efectuata de OpenGL, nici de GPU
   - poate fi efectuata prin algoritmi implementati într-o biblioteca folosita de aplicatia grafica 3D (motorul grafic).
2. **Eliminarea fețelor auto-obturate ale obiectelor** - ***Back face culling:***
   - este efectuata de GPU*.*

Eliminarea partilor nevizibile ale scenelor 3D, din imagini (3)

3. **Eliminarea părților nevizibile ale primitivelor grafice** (poligoane, linii):

   - Eliminarea primitivelor/partilor de primitive aflate în afara volumului vizual, prin operatia de **decupare (clipping)** efectuata de GPU

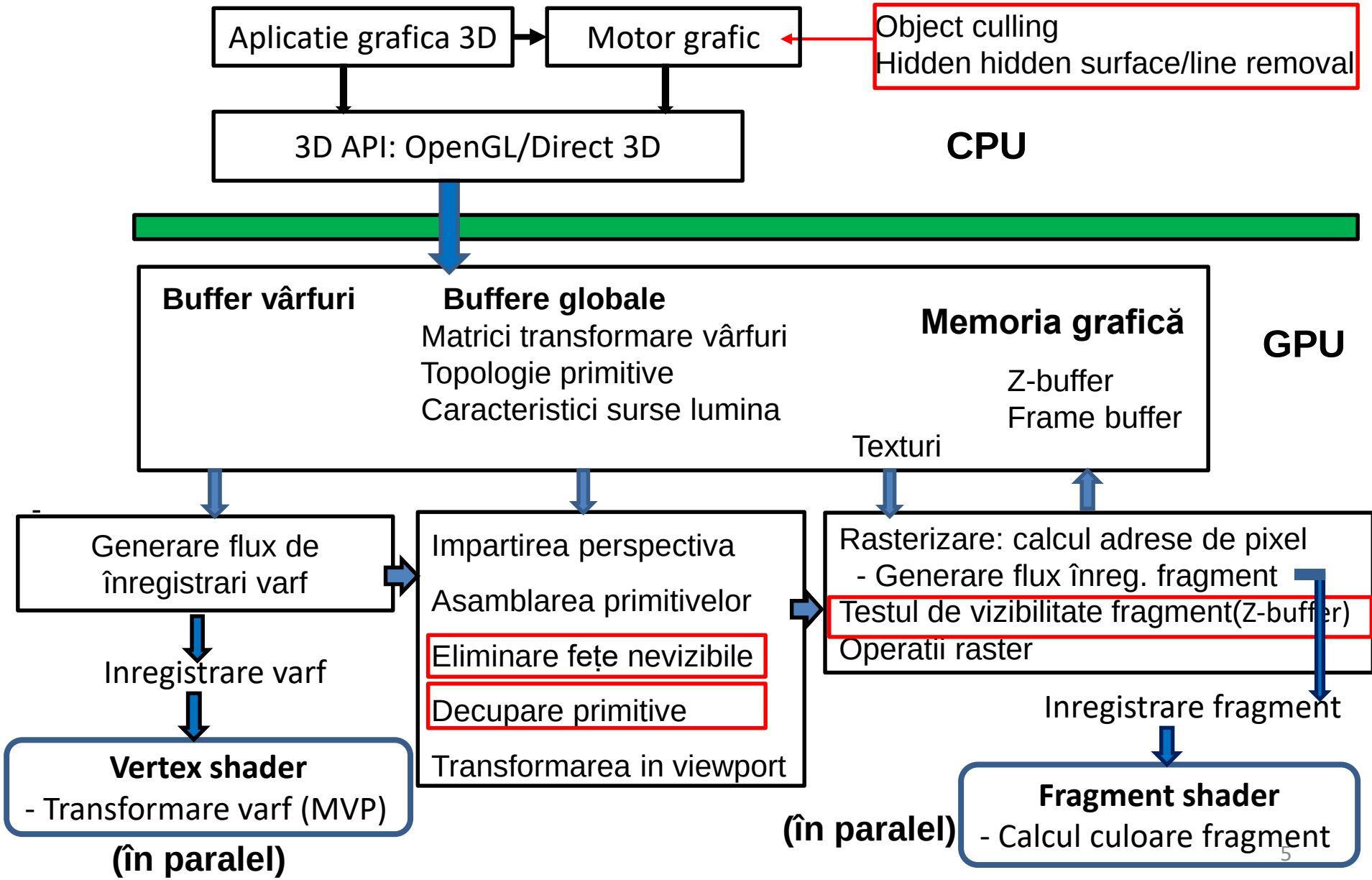
   - Eliminarea părților obturate (ascunse) de alte primitive grafice: ***hidden surface/ hidden line removal***

   - poate fi efectuata prin algoritmi implementati în motorul grafic folosit de aplicație: algoritmul BSP, algoritmul Pictorului, ş.a.

4. **Eliminarea fragmentelor nevizibile**

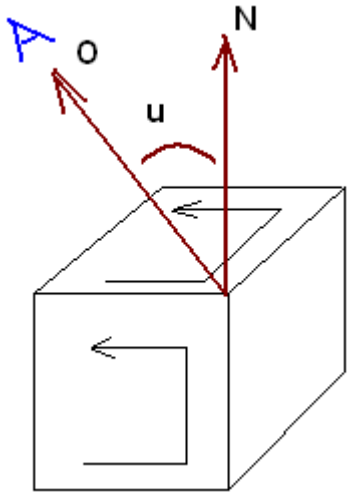
   - este efectuata de GPU: fragmentele de primitive obturate de alte fragmente (aflate mai aproape de observator) sunt suprascrise la momentul rasterizarii (algoritmul Z-buffer)

Eliminarea partilor nevizibile in banda grafica



Eliminarea fețelor auto-obturate pentru poliedre convexe (Back face culling)(1)

- Poate reduce substantial numarul de poligoane rasterizate.
- Clasificarea fetelor unui obiect in “fete din fata (vizibile)” si “fete din spate” se face pe baza pozitiei obiectului fata de observator



Algoritmul se bazeaza pe urmatoarele presupuneri:

1. Observatorul si obiectul sunt raportati la acelasi sistem de coordonate.

2. Conturul fiecarei fețe a obiectului este orientat in sens trigonometric atunci cand poliedrul este vazut din exterior; conditia asigura ca normala fiecarei fete vazuta din exterior este orientata catre spatial exterior obiectului.

3. Observatorul este situat in afara obiectului.

O față este vizibila daca : $0 <= u < 90^o$ sau $0 < \cos(u) <= 1$

u: unghiul dintre normala la față si vectorul orientat catre observator

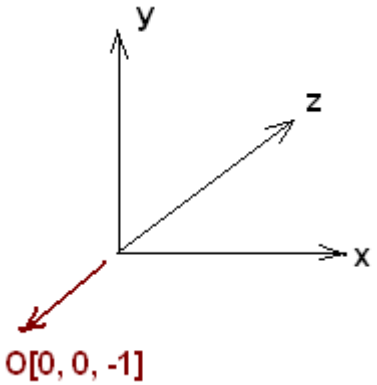
Produsul scalar: $N \bullet O = |N| * |O| * \cos(u) \rightarrow \cos(u) = N \bullet O / (|N| * |O|) = Nu \bullet Ou$

Conditia de vizibilitate: Nu • Ou > 0 (produsul scalar al versorilor)

Determinarea fetelor auto-obturate pentru poliedre convexe(2)

Determinarea feţelor auto-obturate poate fi efectuata:

1. In sistemul coordonatelor globale (in care este definit si observatorul – camera virtuala)
2. In sistemul de coordonate observator
3. In sistemul coordonatelor de decupare, transformand pozitia observatorului cu matricea V•P. In acest caz:



Sistemul coordonatelor de decupare

Nu • Ou = [nx, ny, nz] • [0, 0, -1] = - nz

Conditia de vizibilitate devine: nz <0
(N si O au aceeasi orientare)

Dupa transformarea de proiectie pozitia observatorului este la infinit, pe axa z negativa.

Vectorul orientat catre observator este acelasi pentru orice punct al unui obiect.

Determinarea fetelor auto-obturate pentru poliedre convexe(3)

Calculul normalei la o faţă a poliedrului

Fie V1(x1, y1, z1), V2(x2, y2, z2), V3(x3, y3, z3), 3 varfuri succesive ale conturului feţei.
Normala la faţă se poate obtine calculand produsul vectorial: (V1-V2)x(V2-V3)

$$\mathbf{N} = (V1-V2)x(V2-V3) = \begin{vmatrix} i & j & k \\ x2-x1 & y2-y1 & z2-z1 \\ x3-x2 & y3-y2 & z3-z2 \end{vmatrix} =$$

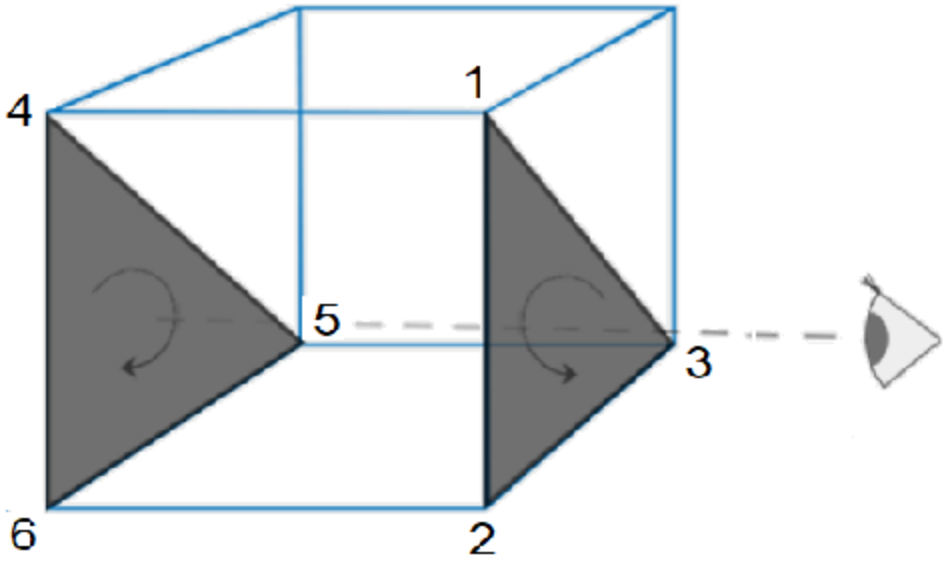
$$[(z3-z2)*(y2-y1) - (z2-z1)*(y3-y2)]*\mathbf{i} - [(x2-x1)*(z3-z2) - (x3-x2)*(z2-z1)*\mathbf{j} + [(x2-x1)*(y3-y2) - (x3-x2)*(y2-y1)]*\mathbf{k}$$

unde **i**, **j**, **k** sunt versorii directiilor axelor sistemului de coordonate carteziene 3D

Determinarea fetelor nevizibile pe GPU (1)

- Presupunem ca pentru toate fetele unui cub s-a definit in program orientarea trigonometrica a contururilor lor, atunci and sunt vazute din exteriorul cubului: 1,2,3; 4,5,6, etc.
- Dupa transformarea de proiectie, in raport cu pozitia observatorului, ordinea varfurilor fețelor proiectate este:

- trigonometrica pentru fețele “din fata” (vizibile); ex. 1,2,3;
- anti-trigonometrica pentru fetele “din spate”; ex. ordinea 4,5,6 din pozitia observatorului este anti-trigonometrica – corespunde feței vazute din interior



Determinarea fetelor nevizibile pe GPU (2)

OpenGL da posibilitatea programatorului sa specifice care este orientarea fețelor “din față” şi care fațe trebuie sa fie eliminate (sunt nevizibile).

1. **Pentru activarea operatiei de eliminare a fetelor nevizibile (“face culling”) se apeleaza:**

   glEnable(GL_CULL_FACE); //implicit, operatia este dezactivata

2. **Pentru a specifica orientarea feţelor “din faţă”**: trigonometrica (GL_CCW)/sensul acelor de ceas (GL_CW):

   glFrontFace(GL_CCW/GL_CW);

3. **Pentru a specifica fețele care se doreste a fi eliminate:** din faţă / din spate/toate

   glCullFace(GL_FRONT/ GL_BACK/GL_FRONT_AND_BACK);

   In cazul **GL_FRONT_AND_BACK** nu vor fi afisate fețe, ci numai alte primitive: puncte, linii.

Exemplu: daca toate feţele unui obiect sunt orientate trigonometric, apeland glCullFace (GL_BACK) şi glFrontFace(GL_CCW) sau glCullFace (GL_FRONT) şi glFrontFace(GL_CW) se elimina feţele ale caror proiectii nu sunt orientate trigonometric.

Algoritmul z-buffer(1)

- Algoritm de eliminare a fragmentelor nevizibile ale primitivelor grafice.

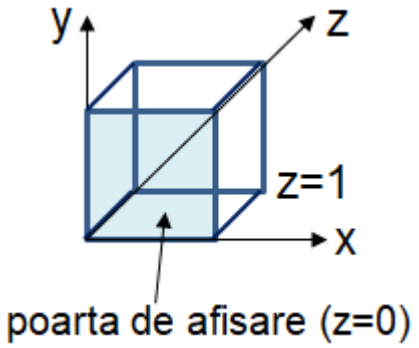
- Executat de GPU, fiind integrat in procesul de rasterizare a primitivelor

❖ Primitivele rasterizate au coordonatele raportate la spatiul ecran

❖ Observatorul este la infinit, pe axa z negativa

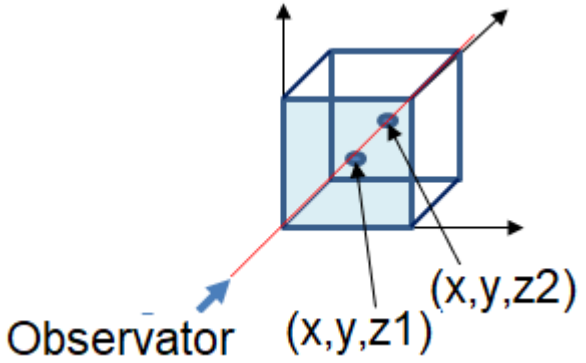
❖ Asupra primitivelor se efectueaza proiectie ortografica

in planul XOY (poarta de afisare)



Daca doua fragmente aflate pe acelasi proiector, rezultate din rasterizarea a doua primitive diferite, au coordonata z diferita:

➢ Fragmentul avand coordonata z mai mica va fi afisat în pixelul (x,y).



Algoritmul z-bufer(2)

- Primitivele grafice (triunghiuri, linii) in care a fost descompusa scena 3D sunt rasterizate in ordinea in care au fost transmise din programul de aplicatie (nu sunt sortate).

- Buffer-ul imagine (frame buffer) este actualizat pe masura rasterizarii primitivelor, astfel:

daca (fragmentul curent se proiecteaza in pixelul (x,y) si
z-fragment < z-fragment afisat in pixelul (x,y))
atunci
actualizeaza culoarea pixelului (x,y) in bufer-ul imagine, la culoarea fragmentului curent

Rezulta: este necesar sa se memoreze coordonatele z ale fragmentelor afisate in pixelii imaginii

Z-buffer: - tablou bidimensional cu numar de elemente egal cu numarul de pixeli ai suprafetei de afisare

- în memoria grafica

Algoritmul z-bufer(3)

Algoritmul z-buffer (integrat in procesul de rasterizare):

*Initializeaza buffer-ul imagine (frame buffer) la culoarea de fond
*Initializeaza Z-buffer la coordonata z maxima (z=1)

Pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive

* calculeaza culoarea fragmentului (in fragment shader)

//testul de vizibilitate (adancime)

daca z< Z-buffer[y][x] atunci

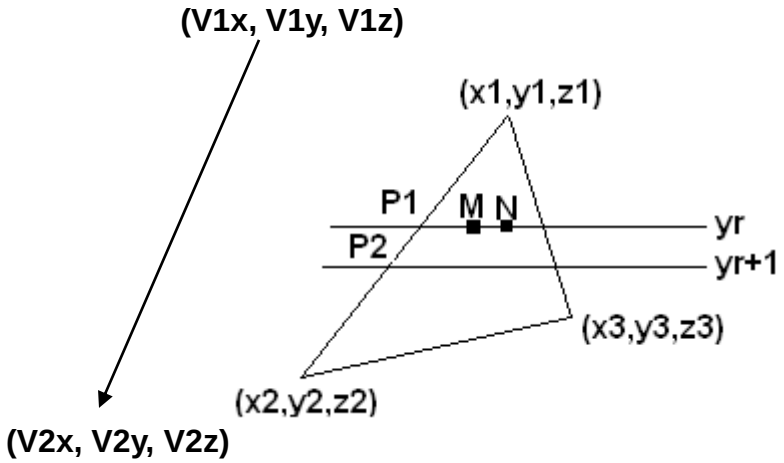
{ Z-buffer[y][x] = z

*actualizeaza culoarea pixelului (x,y) in buffer-ul imagine la culoarea fragmentului f

}

Algoritmul z-bufer(4)

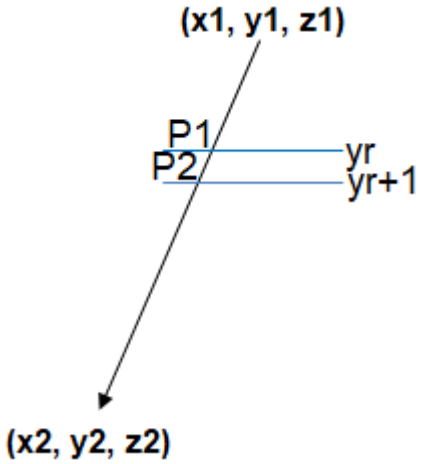
Calculul coordonatei z a unui fragment



Coordonatele z ale fragmentelor rezultate la rasterizarea vectorilor si a suprafetelor triunghiulare se obtin prin calcul incremental, in algoritmul de rasterizare.

Algoritmul z-bufer(5)

Calculul coordonatei z la rasterizarea vectorilor



x = x1 + t(x2-x1)
y = y1 + t(y2-y1) Ecuatiile parametrice ale vectorului
z = z1 + t(z2-z1)

yr, yr+1: ordonatele a 2 linii raster (ecran) care intersecteaza vectorul

y(P1) = yr = y1 + t'(y2-y1) → t' = (yr-y1)/(y2-y1)

z(P1) = z1 + (yr-y1)(z2-z1)/(y2-y1) = z1 + (yr-y1)*K,
K este o constanta la rasterizarea vectorului

z(P2) = z1 + (yr+1-y1)*K = z1 + (yr-y1)*K + K

Z(P2) = z(P1) + K → coordonatele z ale fragmentelor de pe vector se obtin prin calcul incremental.

Algoritmul z-bufer(6)

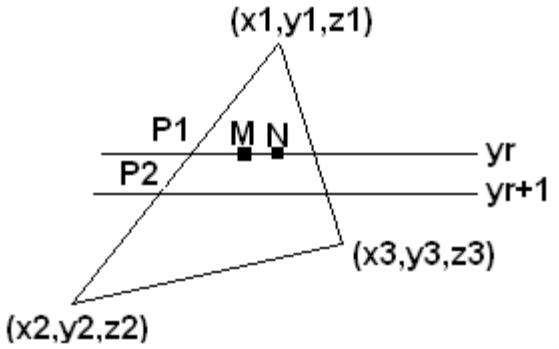
Calculul coordonatei z la rasterizarea poligoanelor

P1(xP1, yr, zP1), P2(xP2, yr+1, zP2)

m- panta laturii (x1,y1,z1) – (x2,y2,z2)

xP2 = xP1 + 1/m (vezi curs - rasterizare)

A*x + B*y + C*z + D = 0 ec planului poligonului



zP1 = (-A*xP1 - B*yr –D)/C

zP2 = (-A*xP2 - B*(yr +1) – D)/C = (-A(xP1 + 1/m) - B*yr –B – D)/C = zP1 + (-A/m –B) → **zP2 = zP1 + K1**

M(xM, yr, zM), N(xM + 1, yr, zN)

zN = (-A*(xM + 1) – B*yr – D)/ C = zM + (-A/C) → **zN = zM + k2**

Observatie: atunci cand un vector se suprapune pe suprafata unui poligon pot aparea defecte datorita modului diferit in care se calculeaza coordonatele z ale fragmentelor care se afiseaza in aceiasi pixeli!

Algoritmul z-bufer(7)

Aprecieri asupra algoritmului z-Buffer

1) Numarul de comparatii de valori z pentru un pixel = numarul de fragmente care se proiecteaza in acel pixel.

   Timpul de calcul tinde sa devina independent de numarul de poligoane: in medie, numarul de pixeli acoperiti de un poligon este invers proportional cu numarul de poligoane.

2) Bufferul Z este un buffer de numere intregi: pentru o precizie buna, este necesara o reprezentare pe 32 biţi a valorilor din z-buffer.

3) "Depth fighting": doua fragmente care se afiseaza in acelasi pixel, cu z diferit in spatiul observator, au acelasi z in Z-buffer → culoarea pixelului depinde de ordinea rasterizarii primitivelor din care fac parte cele 2 fragmente

4) "Depth complexity": numarul de suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine (presupunand calcul culoare fragment, utilizare texturi)

- poate fi redusa prin "deffered rendering": se calculeaza culoarea numai pentru fragmentele vizibile in pixelii imaginii.

Eliminarea partilor nevizibile ale scenelor 3D din imagini -2

Prof. univ. dr. ing. Florica Moldoveanu

Algoritmul BSP pentru eliminarea partilor nevizibile ale primitivelor

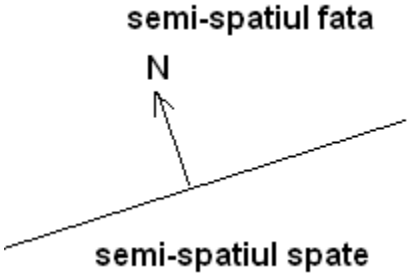
- Scena 3D este reprezentata printr-un arbore binar, denumit sugestiv arbore BSP (**Binary Space Partitioning)**
- Initial, arborele BSP a fost folosit pentru operatia de *object culling*.
- Ulterior a fost folosit şi ca algoritm pentru eliminarea părților nevizibile ale primitivelor grafice (hidden surface removal), fiind folosit în acest scop în jocuri ca *Doom* şi *Quake*, pentru a creste viteza de redare a scenelor 3D.
- Algoritmul este eficient pentru eliminarea partilor nevizibile ale poligoanelor. Eliminarea partilor nevizibile ale liniilor se rezolva eficient la nivel de fragment.
- **Arborele BSP al unei scene este independent de pozitia observatorului - reprezinta scena în sistemul coordonatelor globale.**

Intrarea algoritmului de construire a arborelui: lista poligoanelor care compun scena 3D, în orice ordine; nu are importanta din care obiect face parte fiecare poligon.

Construirea arborelui BSP (1)

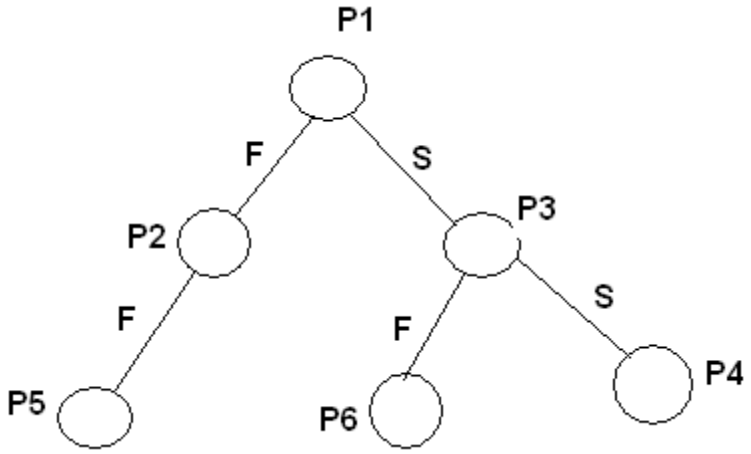
Construirea arborelui BSP al unei scene

- Fiecare nod al arborelui corespunde unui plan de partitionare a spatiului 3D (de regula, planul unui poligon al scenei)



- Fiecare plan de partitionare împarte spatiul în 2 semi-spatii:
  - cel din faţa planului (de aceeasi parte cu normala la plan)
  - cel din spatele planului

1. Se începe cu un poligon oarecare din lista (de regula prim pentru care se creaza nodul radacina al arborelui



- Poligoanele aflate in semi-spatiul “faţă” formeaza “lista-faţă”, care va genera subarborele “faţă” al nodului
- Primitivele din semi-spatiul “spate” formeaza “lista-spate”, care va genera subarborele “spate” al nodului.

2. Se alege un poligon din lista-faţă si se creaza nodul radacina al subarborelui “faţă, etc.

Construirea arborelui BSP (2)

Algoritmul de creare a arborelui BSP:

arbore * creareBSP (Poligon * LP)

{ Poligon P, * ListaFata, * ListaSpate, * ListaNod;

daca (LP este vida) return NULL;

ListaFata = ListaSpate = ListaNod = NULL;

* alege un poligon P din LP;

* elimina P din LP; adauga P la ListaNod;

pentru (fiecare poligon Q din LP) executa

{ **daca** (Q este in semispatiul față al planului lui P) **atunci**

* adauga Q in ListaFata

altfel

Construirea arborelui BSP (3)

daca (Q este in semispatiul spate al planului lui P) **atunci**

* adauga Q in ListaSpate

altfel

daca Q este in acelasi plan cu P **atunci**

* adauga Q in ListaNod

altfel // Q este intersectat de planul lui P

* divizeaza Q cu planul lui P
* adauga fiecare poligon rezultat din intersectie in ListaFata sau ListaSpate, in functie de pozitia sa

} // pentru fiecare poligon

arbore * radacina = combina(creareBSP(ListaFata), creareBSP(ListaSpate));

*ataseaza ListaNod la radacina;

return radacina;

}

Afisarea scenei reprezentata prin arbore BSP

❖ **Tine cont de pozitia observatorului**

- **Afişarea "back-to-front"** - poligoanele sunt trimise in banda grafica in ordinea: de la cel mai indepartat de observator pana la cel mai apropiat de observator

- Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in fiecare nod coborandu-se in subarborele nodului care se afla de partea opusa observatorului fata de planul nodului.

```
void afisareBSP_back_to_front(arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (Observator este in semispatiul fata al planului radacinii arborelui A) atunci
    { afisareBSP(A->spate); *afisare poligoane din nodul radacina; afisareBSP(A->fata);}
  altfel
    { afisareBSP(A->fata); *afisare poligoane din nodul radacina; afisareBSP(A->spate);}
}
```

Afisarea scenei reprezentata prin arbore BSP

- **Afişarea "front-to-back"** - poligoanele sunt trimise in banda grafica in ordinea: de la cel mai apropiat de observator pana la cel mai îndepartat de observator.

- Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in fiecare nod coborandu-se in subarborele nodului care se afla de aceeasi parte cu observatorul fata de planul nodului.

```
void afisareBSP_front_to_back(arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (Observator este in semispatiul fata al planului radacinii arborelui A) atunci
    { afisareBSP(A->fata); *afisare poligoane din nodul radacina; afisareBSP(A->spate);}
  altfel
    { afisareBSP(A->spate); *afisare poligoane din nodul radacina; afisareBSP(A->fata);}
}
```

Algoritmul BSP - aprecieri (1)

- Arborele nu trebuie sa fie reconstruit pentru fiecare cadru imagine, daca scena nu s-a modificat: avantaj pentru scenele statice
- Modificarea pozitiei observatorului → executia functiei de afisare (daca scena nu s-a modificat)
- Pentru obiectele dinamice se pot face diverse adaptari ale algoritmului; de ex:
  - pentru fiecare cadru imagine se insereaza in arborele BSP cu obiectele statice, noduri corespunzand obiectelor dinamice, in functie de pozitia fiecaruia;
  - se construieste arborele BSP numai cu obiectele statice iar obiectele dinamice sunt trimise direct in banda grafica, eliminarea partilor obturate de acestea fiind realizata la rasterizare.
- **In cazul afisarii back-to front**, poligoanele mari (ex. pereti) din spate sunt afisate primele si apoi părți mari din ele sunt obturate de poligoanele mai apropriate de observator, prin suprascrierea pixelilor → timp consumat inutil cu calculul culorilor fragmentelor poligoanelor din spate.

Algoritmul BSP – aprecieri (2)

- **In cazul afisarii front-to back**, poligoanele apropriate de observator sunt afisate primele, apoi cele din spate sunt “decupate” de poligoanele deja afisate, prin testul de adancime (pixelii acoperiti de poligoanele din fata nu sunt suprascrisi). In plus prin “early depth test”, o optiune implementata de placile grafice recente, se evita calculul culorii fragmentelor care nu trec testul de adancime.

➢ Deoarece nu au loc suprascrieri de pixeli, majoritatea motoarelor grafice care folosesc BSP utilizeaza afisarea front-to-back.

➢ **In cazul afisarii back-to front** nu mai este necesar testul de adâncime la afisarea fragmentelor (fragmentele care se proiecteaza in acelasi pixel sunt suprascrise), de aceea se poate dezactiva testul de adâncime:

glEnable(GL_DEPTH_TEST) / glDisable(GL_DEPTH_TEST)

- **Arborele BSP este foarte eficient daca se foloseşte pentru operatia *object culling***
- Construirea si afisarea arborelui BSP sunt functii de management al scenei 3D, implementate de regula intr-un motor grafic 3D.

Algoritmul pictorului(1)

– Algoritm din categoria “Hidden surface removal”.

– Intrare: lista poligoanelor care alcatuiesc scena 3D, transformate in spatiul de afisare

– Ideea: eliminarea partilor nevizibile se rezolva prin afisare “din spate in fata”, la fel ca in pictura.

– Eficient pentru aplicatiile in care primitivele grafice sunt situate in zone disjuncte pe axa de adancime (OZ)

Forma generala a algoritmului

1. Se calculeaza “extensia” fiecarui poligon din listă pe axele OX, OY, OZ (xmin,ymin,zmin – xmax, ymax, zmax): paralelipipedul incadrator al poligonului, cu feţele paralele cu planele principale ale sistemului de coordonate.

2. Se ordoneaza poligoanele crescator dupa coordonata zmin a fiecarui poligon: primul in lista va fi cel mai apropiat de observator.

3. Se descompun poligoanele ale caror extensii pe axa OZ se suprapun, astfel incat extensiile lor pe axa OZ sa fie disjuncte.

4. Se transmit in banda grafica poligoanele incepand cu ultimul din lista (cel mai indepartat de observator).

Algoritmul pictorului(2)

Pasul 3. Se descompun poligoanele ale caror extensii pe axa OZ se suprapun

- Sunt multe aplicatii in care acest pas nu este necesar, poligoanele fiind amplasate in plane de Z-constant: cartografie, generarea straturilor circuitelor imprimate, etc.
- Poate fi optimizat, stiind ca nu intotdeauna atunci cand extensiile pe axa OZ se suprapun este necesara descompunerea poligoanelor; descompunerea nu este necesara daca:

  - extensiile pe axa OX nu se suprapun

  - extensiile pe axa OY nu se suprapun

  - proiectiile poligoanelor nu se suprapun in imagine

- Testele se efectueaza progresiv, in functie de complexitatea calculelor presupuse.

- **Principalul efort de calcul:** sortarea listei de poligoane si descompunerea, daca este necesara.

Tehnici de eliminare a obiectelor nevizibile - Frustum culling -

❖ Scopul: eliminarea din banda grafica a obiectelor sau grupurilor de obiecte care sunt complet in afara volumului vizual

❖ Se bazeaza pe reprezentarea scenei printr-o structura de date spatiala.

Structuri de date spatiale pentru managementul scenei 3D

- Ierarhie de volume incadratoare (BVH)
- Arbore BSP
- Arbore AA-BSP
- Arbore octal
- ş.a.

- Managementul scenei este realizat in cadrul unui motor grafic 3D, implementat peste OpenGL / Direct 3D/Vulkan.

- Structurile de date sunt utile si pentru alte operatii: mai sunt numite "**structuri de date de accelerare**".

1. Scena- Ierarhie de volume incadratoare (1) (Bounding Volume Hierarchy - BVH)

- **Scena este reprezentata printr-un arbore care are in fiecare nod un volum încadrator pentru un obiect sau un grup de obiecte**: sfera, paralelipipedul aliniat cu axele (Axis-Aligned minimum Bounding Box: AABB), poliedrul convex minimal incadrator, cilindrul, elipsoidul.

Construirea arborelui:

1. **Top-down** – radacia arborelui este asociata cu intreaga scena

- Se partitioneaza setul de obiecte in 2 sau mai multe subseturi si se aloca fiecare subset unui volum incadrator – rezulta primul nivel al arborelui. Volumele incadratoare trebuie sa fie disjuncte sau minimal suprapuse.
- Se continua procedura recursiv la nivelul fiecarui grup de pe nivelul anterior pana cand se ajunge la un singur obiect (sau primitiva) pe nod – nod frunza
- Metoda este simpla dar, in general, nu produce arbori optimi.

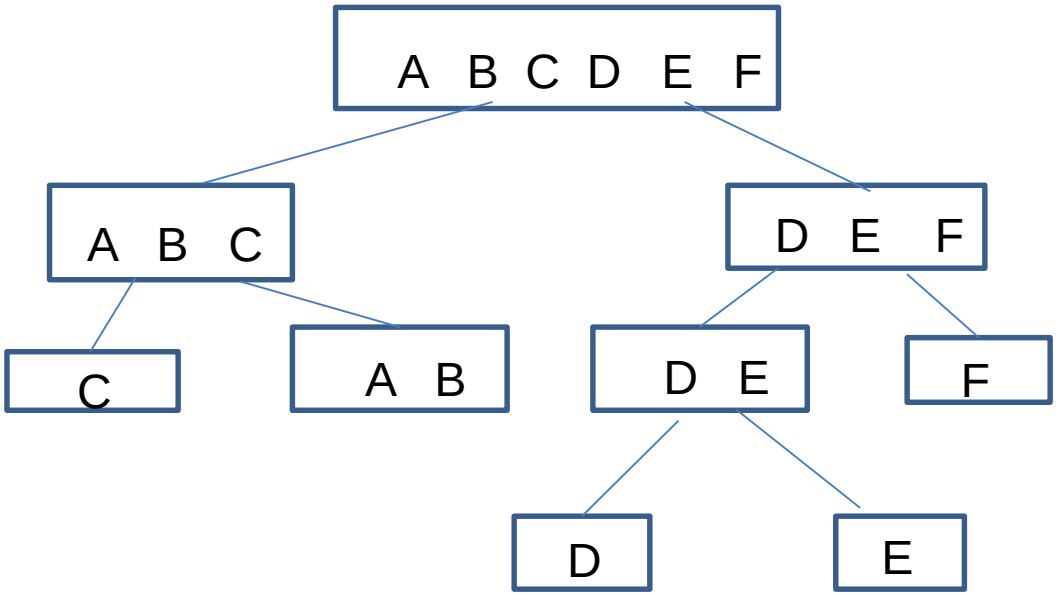
Ierarhie de volume incadratoare (2)

Construirea arborelui – cont.

2. **Bottom-up**

- Se pleaca de la setul de obiecte si se defineste un volum incadrator pentru fiecare obiect sau primitiva – nodurile frunza.
- Se grupeaza 2 sau mai multe volume nod frunza, rezultand noduri de pe nivelul urmator
- Se continua gruparea pana cand totul este grupat intr-un singur volum – nodul radacina

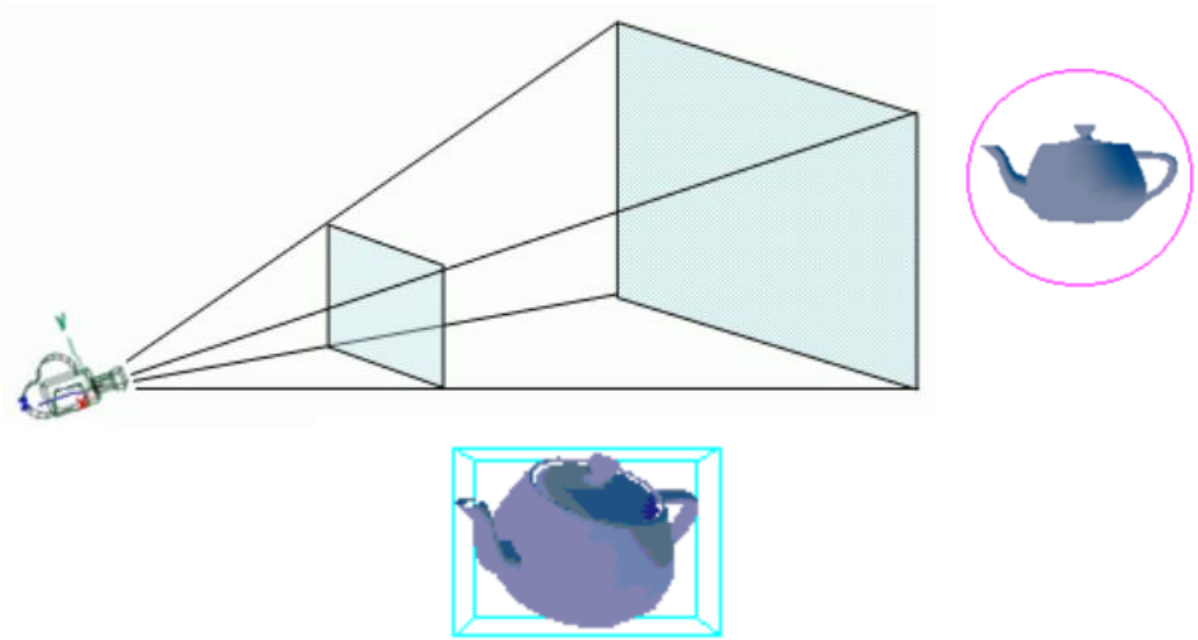
❖Construirea bottom-up produce arbori mai eficienti decat cea top-down.



Ierarhie de volume incadratoare (3)

- **Afisarea scenei 3D folosind arborele BVH**

Daca volumul incadrator al unui nod este in afara volumului vizual atunci obiectele din subarborele său **NU** sunt trimise in banda grafica.



Ierarhie de volume incadratoare (4)

❖ **Afisarea scenei 3D folosind arborele BVH**

- testele de vizibilitate incep cu radacina arborelui;

- **daca** volumul incadrator al nodului curent este complet in afara volumului vizual

atunci se revine pe nivelul anterior al arborelui (intregul grup de obiecte din subarborele nodului curent este exclus din banda grafica)

altfel se coboara in arbore;

- **daca** se ajunge la o frunza al carei volum incadrator nu este in afara volumului vizual,

atunci - grupul de primitive (obiectul) asociat frunzei este trimis in banda grafica;

- se revine pe nivelul anterior al arborelui;

➢ **Pentru simplitate si eficienţă, testele de vizibilitate se pot face in spatiul coordonatelor de decupare, transformand volumele incadratoare prin matricea MVP si testand intersectia cu volumul vizual canonic (cub cu laturile paralele cu axele)**

❖ Structura BVH este frecvent utilizata pentru detectia coliziunilor dintre obiectele unei scene dinamice.

Ierarhie de volume incadratoare (5)

❖ **Eficienta structurii BVH privind eliminarea obiectelor nevizibile din banda grafica**

- depinde mult de felul in care sunt grupate obiectele in volume incadratoare; poate fi eficienta pentru anumite pozitii ale observatorului si ineficienta pentru alte pozitii.

Alegerea tipului de volum incadrator pentru un nod:

- Sa aiba o forma simpla, a.i. testele de intersectie cu volumul vizual sa fie simple si rapide
- Sa incadreze minimal obiectul/obiectele

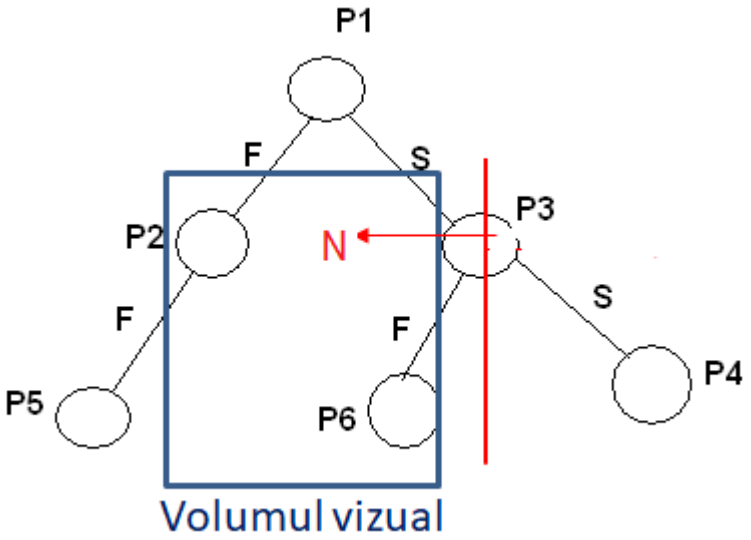
Proprietati dorite ale ierarhiei de volume incadratoare:

- Volumele nodurilor dintr-un subarbore trebuie sa fie apropriate spatial, cu atat mai apropiate cu cat sunt situate mai jos in arbore
- Volumul fiecarui nod sa incadreze minimal obiectul/grupul de obiecte corespunzator
- Volumul incadrator al intregii scene trebuie sa fie minimal
- Volumelor nodurilor vecine sa fie disjuncte sau minimal suprapuse.

2. Scena - Arbore BSP (Polygon Aligned BSP tree) (1)

- **Planele de partitionare sunt planele poligoanelor (Polygon Aligned - PA BSP tree)**
  - Arborele scenei se construieste ca in algoritmul BSP pentru eliminarea suprafetelor nevizibile
- **Eliminarea grupurilor de primitive nevizibile**

Daca planul de divizare atasat unui nod nu intersecteaza volumul vizual, atunci subarborele nodului care este situat de partea opusa a planului fata de volumul vizual este complet nevizibil si poate fi eliminat din procesul de redare. La fel şi primitivele ataşate nodului.



Eliminarea se efectueaza in spatiul coordonatelor globale, folosind volumul vizual definit de aplicatie (trunchi de piramida sau paralelipiped dreptunghic).

3. Scena - Axis Aligned BSP Tree (1)

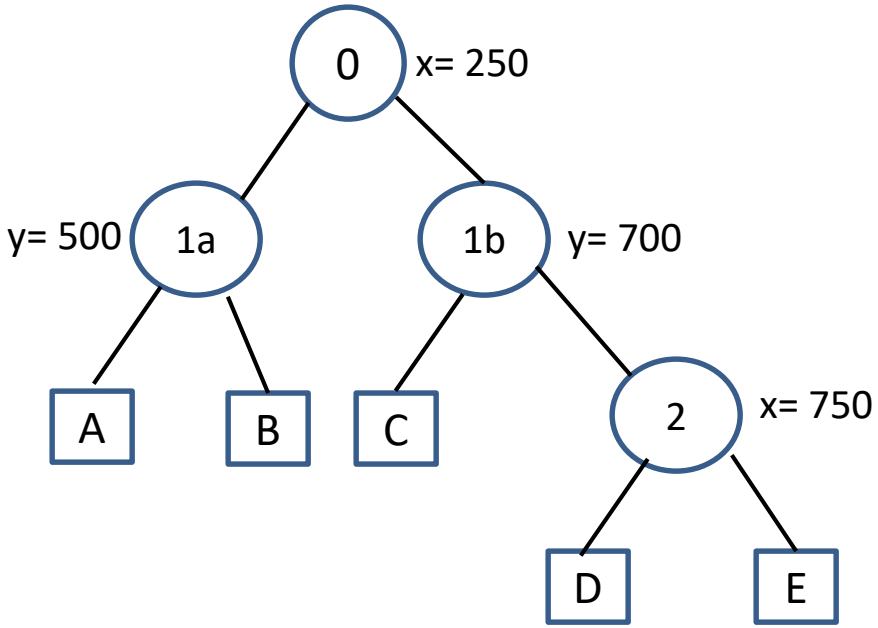
- **Planele de partitionare sunt aliniate cu axele sist de coord 3D -**perpendiculare pe cele 3 axe: Axis-Aligned BSP Tree (AA-BSP)
- AA-BSP tree este un k-d tree (*k-dimensional tree):* arbore binar in care fiecare nod corespunde unei coordonate intr-un spatiu k-dimensional. Pentru k=3, un nod va corespunde uneia dintre coordonatele x, y sau z. De exemplu, daca planul de partitionare al unui nod este perpendicular pe axa OX, atunci nodul este asociat coordonatei x a planului.
- **Construirea arborelui:**

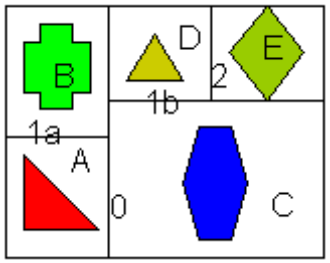
  - Scena este divizata recursiv prin plane paralele cu planele sistemului de coordonate, intr-o ordine fixa, tinand cont si de geometria scenei; de exemplu, in pasul 1 se alege un plan perpendicular pe OX, in pasul 2 un plan perpendicular pe OY, in pasul 3 un plan perpendicular pe OZ, in pasul 4 un plan perpendicular pe OX, s.a.m.d.
- **Alegerea planelor:**

  - numarul de obiecte din fiecare semispatiu sa fie acelasi sau diferit cu 1, pentru ca arborele sa fie echilibrat

  - numarul de intersectii dintre planele de partitionare si obiectele scenei sa fie minimizat.

3. Scena- Axis Aligned BSP Tree (2)





Exemplu de divizare
cu plane aliniate cu axele, in 2D:

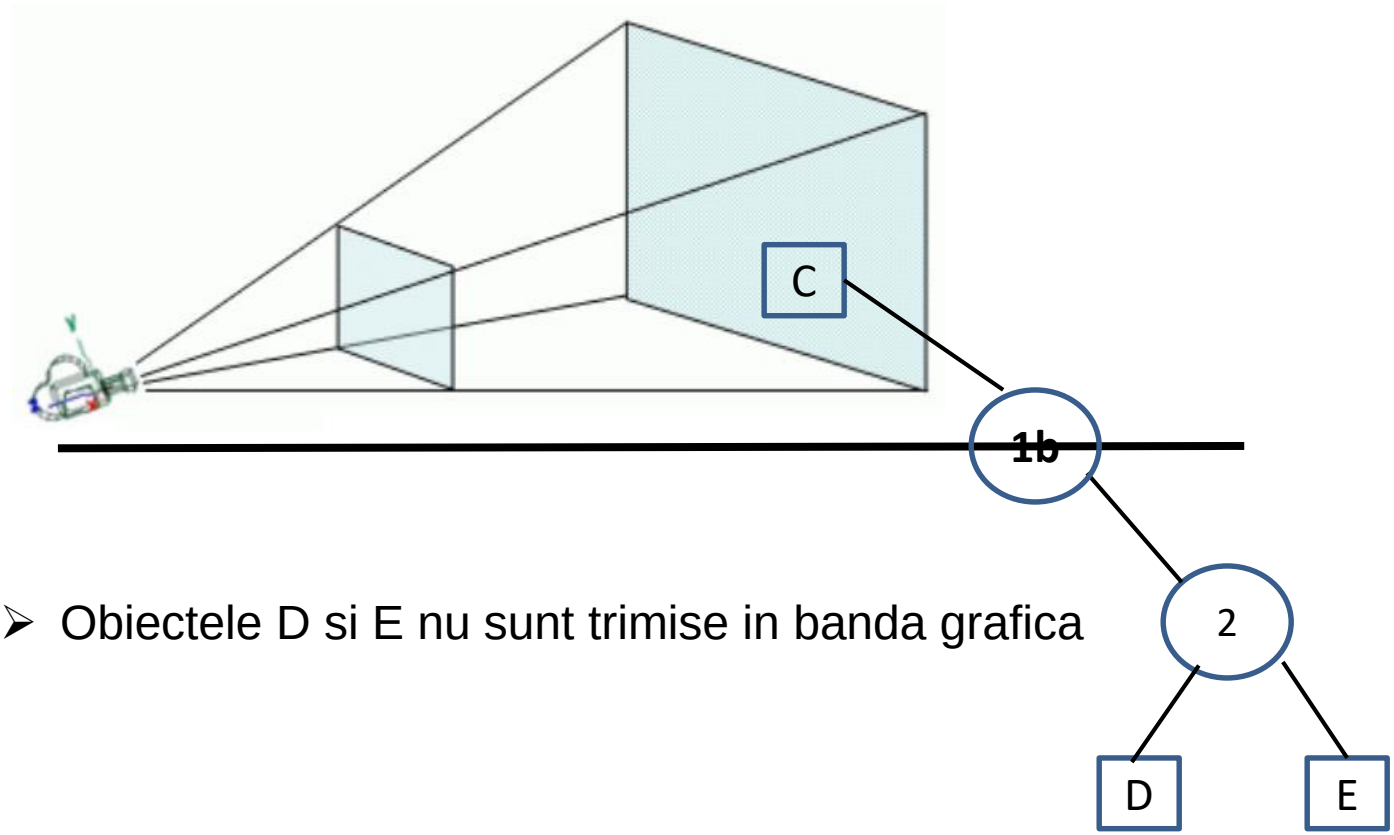
In fiecare pas planul de divizare este ales a.i. sa se evite intersectia sa cu obiectele scenei.

❖ **Eliminarea obiectelor nevizibile: la fel ca in cazul arborelui PA- BSP:** daca planul de partitionare al unui nod nu intersecteaza volumul vizual, intregul grup de obiecte din subarborele aflat de partea opusa volumului este exclus din banda grafica.

3. Scena- Axis Aligned BSP Tree (3)

❖ **Afisarea scenei 3D folosind arborele AABSP**

Daca planul atasat unui nod nu intersecteaza volumul vizual atunci întregul grup de obiecte din subarborele aflat de partea opusa volumului **NU** este trimis în banda grafica.

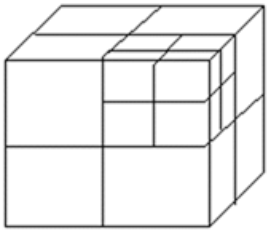


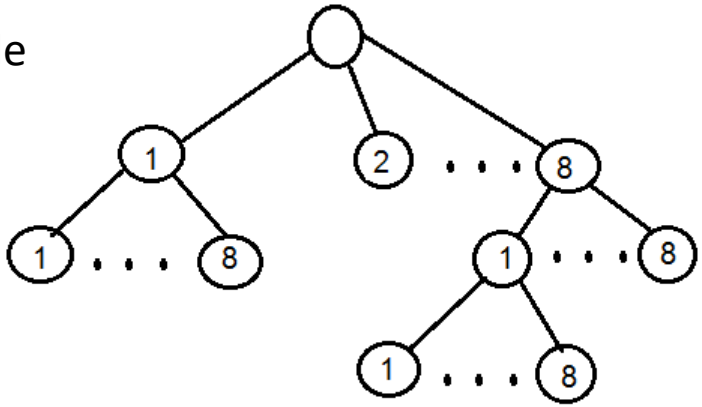
➢ Obiectele D si E nu sunt trimise in banda grafica

4. Scena - Arbore octal (1)

❖ **Construirea arborelui octal al scenei**

- Se pleaca de la volumul incadrator al scenei, aliniat cu axele, care se asociaza nodului radacina al arborelui.
- Se divizeaza volumul incadrator in 8 subvolume egale
- Se impart primitivele in 8 liste, in functie de pozitionarea lor in cei 8 octanti





- Se continua cu divizarea recursiva a subvolumelor si asocierea primitivelor cu subvolumele;

  fiecare primitiva este asociata cu nodul celui mai mic octant care o incadreaza complet.
- Un subvolum nu este divizat daca s-a ajuns la un nivel de divizare prestabilit sau subvolumul contine suficient de putine primitive sau nici o primitiva.
- Rezulta un arbore in care fiecare nod intern are 8 copii.

Scena - Arbore octal (2)

❖ **Fiecare nod al arborelui octal are ataşat un volum incadrator**

➢ **Eliminarea obiectelor nevizibile folosind arborele octal al scenei : la fel ca in cazul BVH**

- Se efectueaza testand intersectiile subvolumelor arborelui cu volumul vizual (cub, in sistemul coordonatelor de decupare), incepand cu nodul radacina.

- Daca volumul asociat nodului curent intersecteaza volumul vizual, se coboara in arbore, testand intersectia dintre fiecare copil al său si volumul vizual.

- Daca subvolumul asociat unui nod este in afara volumului vizual, atunci se revine in arbore pe nivelul anterior (se elimina din banda grafica toate obiectele din subarborele nodului).

Scena - Arbore octal (3)

❖**Eficienta arborelui octal al scenei**

- Subdivizarea stricta a spatiului impreuna cu metoda de asociere a fiecarei primitive cu cel mai mic octant care o incadreaza complet, poate conduce la ineficienţă;

de ex., daca o primitiva este situata in apropierea centrului volumului incadrator al scenei, primitiva va fi asociata cu acest volum, chiar daca este foarte mica in raport cu volumul!

- O alternativa: o primitiva poate fi asociata cu 2 sau mai multi octanti vecini care impreuna o incadreaza complet, formand un volum minimal (creste complexitatea construirii arborelui).

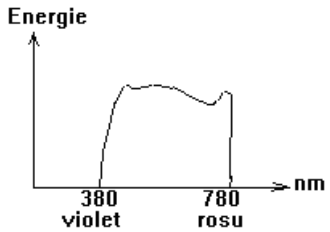
Proprietatile luminii. Modele de culoare

Prof. univ. dr. ing. Florica Moldoveanu

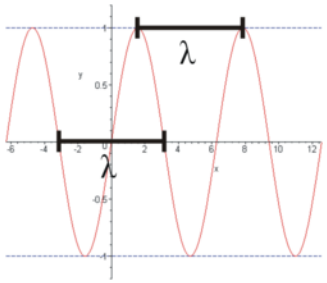
Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Lumina

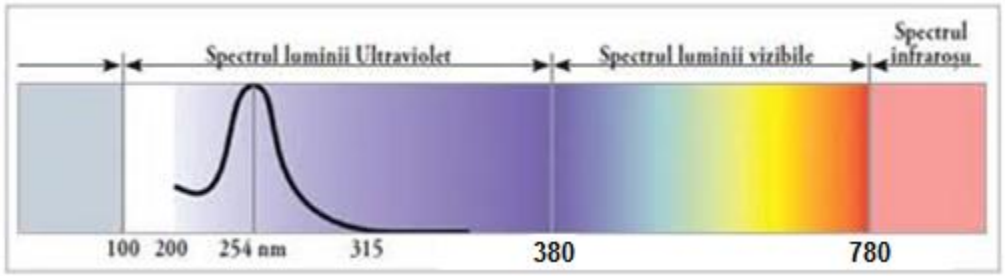
- Lumina este energie electromagnetică: spectrul vizibil
- Spectrul vizibil este alcatuit din undele cu lungimi cuprinse intre 380-780 nanometri.



Spectrul vizibil



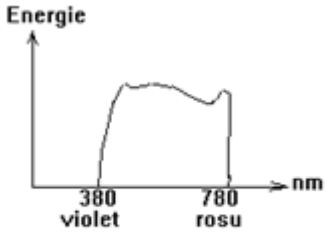
Lungimea de unda: distanța dintre două puncte din spațiu între care unda a parcurs o perioada de repetitie.



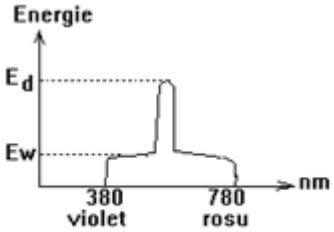
Proprietatile luminii(1)

- Atunci când cade pe suprafaţa unui obiect, lumina poate fi: absorbită, reflectată sau transmisă.

- **Sistemul vizual uman percepe: - lumina provenita direct de la o sursa şi**
  - lumina reflectata sau transmisa de obiectele din mediu.



Lumina acromatica



Lumina cromatica

- Lumina care conţine toate lungimile de undă din spectrul vizibil în proporţii aproximativ egale se numeşte **acromatică**.
- Lumina în care predomina anumite lungimi de undă se numeşte **cromatică.**

- Lumina acromatică provenită de la o sursă este percepută ca albă.
- Lumina acromatică provenită de la un obiect este percepută ca albă, neagră sau ca o nuanţă de gri în funcţie de proprietăţile fizice ale suprafeţei obiectului.
- Obiectele care reflectă acromatic mai mult de 80% din lumina incidentă albă apar ca albe. Cele care reflectă acromatic mai puţin de 3% din lumina incidentă apar ca negre.

Proprietatile luminii(2)

- Culoarea unui obiect, perceputa de ochiul uman, depinde atât de distribuţia lungimilor de undă în lumina care cade pe obiect cât şi de caracteristicile fizice ale suprafetei obiectului: reflecta/absoarbe anumite lungimi de unda.

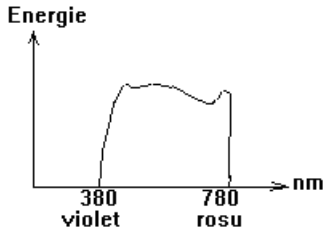
➢ Energia electromagnetică nu are culoare!

➢ **Culoarea este rezultatul unui proces psiho-fiziologic**.

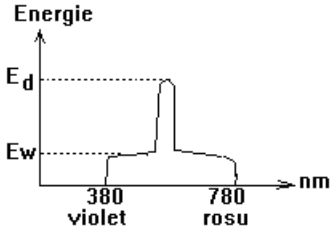
- **Definiţia psiho-fiziologică a unei culori cuprinde: nuanța, luminozitatea (stralucirea), saturatia**
  - **Nuanţa:** roşu, galben, verde, etc. – determinată de lungimea de undă dominantă a distribuţiei spectrale a luminii.
  - **Luminozitatea sau strălucirea** - reprezintă intensitatea luminii.
    - Luminozitatea: caracteristică a unui obiect emiţător de lumină
    - Strălucirea: caracterizează un obiect neemiţător, care reflectă lumina.

Proprietatile luminii(3)

- **Saturaţia sau puritatea -** o măsură a amestecului de alb într-o culoare pură;
  - permite să se facă distincţie între roşu şi roz, între albastru şi bleu, etc;
  - o culoare pură are saturaţia 100%;
  - lumina acromatică are saturaţia 0%.



Lumina acromatica



Lumina cromatica

Culoarea este data de lungimea de unda dominanta.

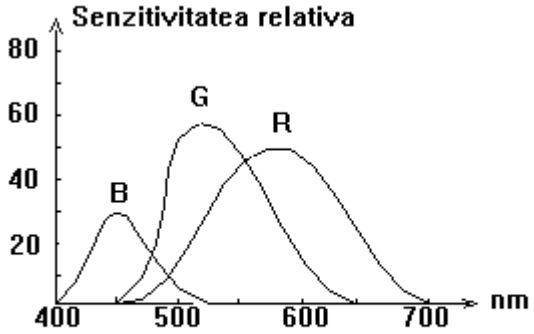
Ew=0 – culoarea are puritatea 100% : monocromatica

(largime de banda de un nanometru)

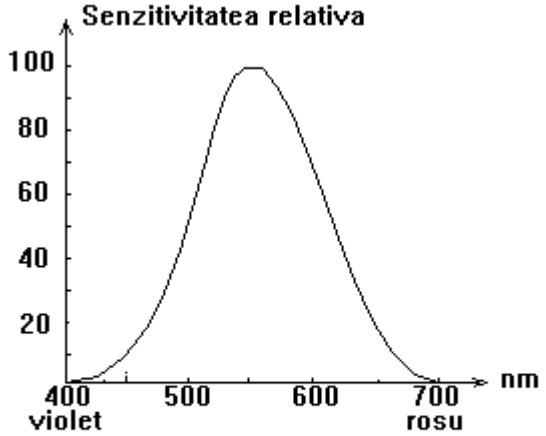
– Cea mai cunoscută dintre teoriile privind formarea culorilor în sistemul ochi-creier este aceea conform căreia în retina ochiului uman există trei tipuri de conuri, fiecare tip fiind sensibil la una dintre culorile roşu, verde şi albastru: teoria tricromaticitatii (Tristimulus theory)

Teoria tricromaticitatii (1)

- Culorile rosu, verde si albastru se numesc culori primare.
- Cf. teoriei tricromaticitatii, orice culoare care poate fi perceputa de ochiul uman se obtine printr-un amestec aditiv al celor 3 culori primare.
- Prin amestecul lor în proporţii egale se obţine alb.



Raspunsul ochiului la cele 3 culori primare



Curba de luminozitate
Suma celor 3 curbe de senzitivitate

Senzitivitatea maximă corespunde luminii cu lungimea de undă în jur de 550 nm, percepută ca galben-verde.

Teoria tricromaticitatii (2)

- **Două culori care prin amestec produc lumină albă se numesc complementare.**

• Culorile complementare culorilor primare sunt : cian (Cyan), magenta (Magenta), galben (Yellow).

- Amestecând două culori primare in proportii egale se obţine culoarea complementară celei de a treia:

R **G** **B**

C M Y

- De exemplu, albastru+verde=cian, roşu+verde=galben, roşu+albastru= magenta.
- Culorile **roşu, verde şi albastru** se numesc şi **"primitive aditive"** deoarece ele permit formarea de culori prin **adunarea** lor în diferite proporţii.
- Acest mod de definire a culorilor corespunde echipamentelor emiţătoare de lumină (dispozitive de vizualizare cu ecran).

Teoria tricromaticitatii (3)

- Culorile **cian, magenta, galben** se numesc **"primitive extractive"**.
- Se obtin prin **extragerea** culorilor primare din lumina alba:

cian = alb-roşu, magenta = alb-verde, galben = alb-albastru

R G B
C M Y

– **Se folosesc pentru a defini culorile reflectate de documentele imprimate**: pigmenţii existenţi în cernelurile tipografice absorb din lumina incidenta componentele complementare acelora ale pigmenţilor.

Ex: pigmentul de culoare magenta absoarbe din lumina incidentă lumina verde, iar cel de culoare galben, lumina albastra.

➢ O suprafaţă care conţine pigmenţi magenta şi galben va reflecta (sau transmite) lumină roşie.

– **C, M, Y permit specificarea de nuanţe prin extragerea culorilor primare în diferite proporţii din alb. Scăzându-le în proporţii egale din alb se obţin diferite nuanţe de gri.**

Modele de culoare folosite în sinteza imaginilor

- Un model de culoare specifică un sistem de coordonate 3D şi un subspaţiu al culorilor în sistemul de coordonate respectiv.
- Fiecare culoare se reprezintă printr-un punct în subspaţiul culorilor.

- Orientate către echipamente
  - se bazează pe culorile primare folosite de echipamente pentru redarea culorilor:

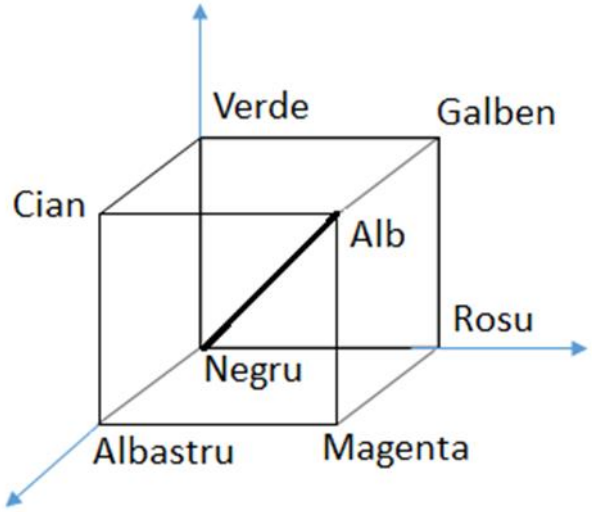
    RGB, CMY
  - YIQ: pentru transmisia imaginilor TV color
- Orientate către utilizator
  - se bazează pe proprietăţile psiho-fiziologice ale culorilor :
  - HSV şi HLS.

Modelul RGB

RGB (Red, Green, Blue)

RGB: definesc culorile la dispozitivele emitatoare de lumina

Culoare emisa: C(x,y,z) = x_Red + y_Green + z_Blue, 0<= x,y,z <=1



Rosu: 1,0,0

Verde: 0,1,0

Albastru: 0,0,1

Alb: 1,1,1; Negru: 0,0,0

Cian: 0, 1, 1 - culoarea complementara culorii Rosu !

Pe diagonala Negru-Alb: nuante de gri - obtinute prin amestec in proportii egale intre R,G,B

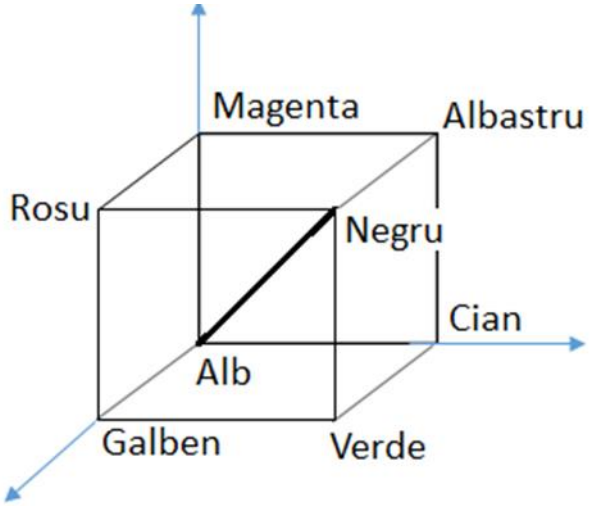
Modelul CMY

CMY (Cyan, Magenta, Yellow)

CMYK (Cyan, Magenta, Yellow, Key) → key = negru

CMY: definesc culorile reflectate de documentele imprimate

Culoare reflectata: C(x,y,z) = (1,1,1) – (x,y,z) = (1-x) + (1-y) + (1-z), 0<= x,y,z <=1



Cian: 1,0,0→ extrage rosu din lumina incidenta

lumina reflectata: (1,1,1) – (1,0,0) = (0,1,1): Cian in RGB

Magenta: 0,1,0 → extrage verde din lumina incidenta

lumina reflectata: (1,1,1) – (0,1,0) = (1,0,1): Magenta in RGB

Alb: 0,0,0 → (1,1,1) – (0,0,0) = (1,1,1) in RGB

Negru: 1,1,1 - (R,G,B) sunt extrase in proportie maxima din lumina alba

Unele dispositive de imprimare includ culoarea negru intre culorile primare C,M,Y: CMYK

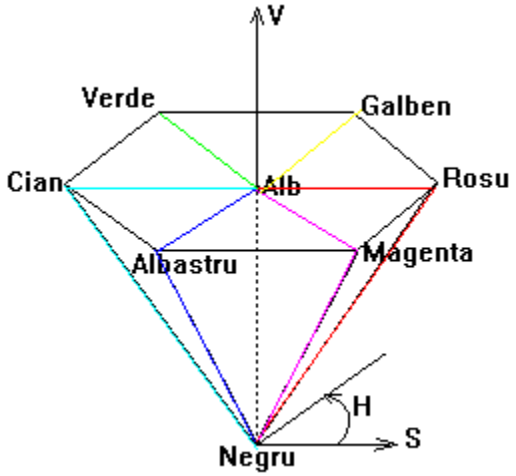
Coversia RGB ←→ CMY

[C M Y] = [1 1 1] - [R G B]
[R G B] = [1 1 1] - [C M Y]

Modelul HSV(1)

HSV (Hue, Saturation, Value)

- Model de culoare orientat catre utilizator
- Artiştii specifică culorile prin tente, nuanţe şi tonuri



0<= V<= 1, 0<= S <= 1, 0<= H <= 360

V=1 – baza hexaconului

S=0 - pe axa hexaconului;

S=1 - pe frontiera bazei hexaconului: culorile pure

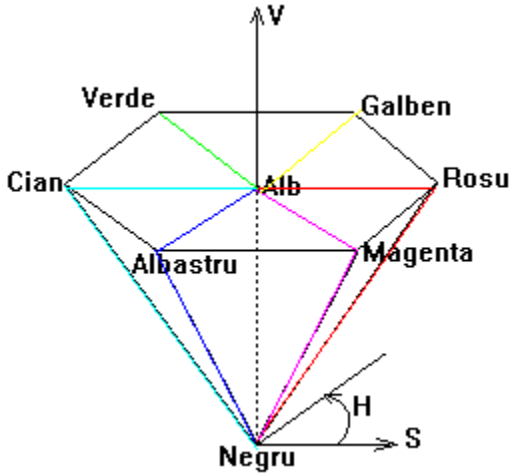
Rosu: (0, 1, 1)

Culorile complementare – la 180 grade pe baza hexaconului

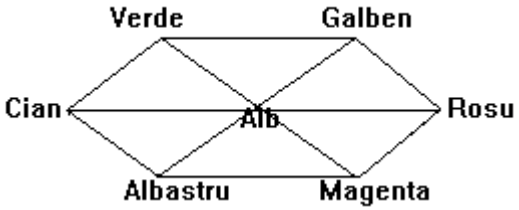
Cian: (180, 1, 1)

- Hexaconul din figura reprezinta subspatiul culorilor in modelul HSV

Modelul HSV(2)



- Saturaţia intr-un punct este definită prin distanţa de la axa hexaconului la punctul respectiv.
- Combinaţia (S=0,V=1) corespunde culorii alb iar prin (S=0, 0<=V<=1) se reprezintă nivelurile de gri.
- Pentru S=0 valoarea lui H este nesemnificativă.
- Culorile pure sunt reprezentate prin V=1 şi S=1.

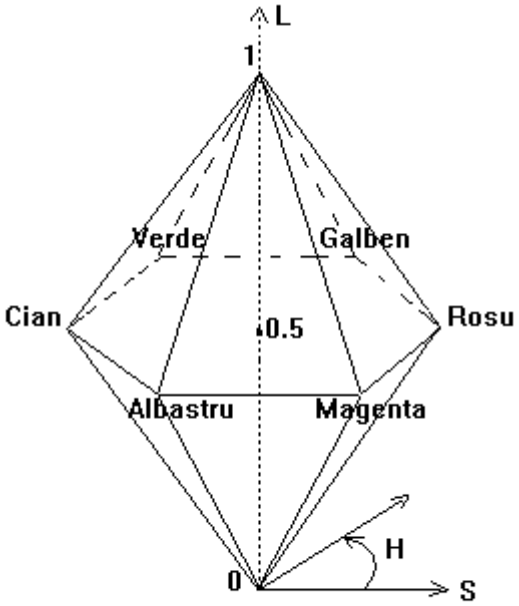


- Baza hexaconului corespunde suprafeţei văzute atunci când se priveşte cubul RGB dinspre vârful (1,1,1) de-a lungul diagonalei principale.

- Fiecare plan de V constant din spaţiul HSV corespunde vederii unui subcub din cubul RGB.
- Diagonala principală a cubului RGB devine axa principală în spaţiul HSV.
- Această interpretare sta la baza algoritmilor de conversie între modelele RGB şi HSV.

Modelul HLS

HLS (Hue, Lightness, Saturation)



Modelul HLS

Culorile primare cu saturaţie maximă şi complementarele lor sunt reprezentate prin S=1, L=0.5.

Conversii RGB → HSV/HLS

max = sup(R, G, B) min = inf(R, G, B)

V = max

S= (max-min)/max daca max ≠ 0

0 daca max =0

H = (G – B) / (max – min) daca R = max

(B – R) / (max – min) + 2 daca G = max

(R – G) / (max – min) + 4 daca B = max

daca H<0 H = H +6

H = H x 60

RGB → HSV

L = (max-min)/2

S = (max – min)/(max + min) daca L<=0.5

= (max – min)/ (2 – max – min) daca L > 0.5

H = (G – B) / (max – min) daca R = max

(B – R) / (max – min) + 2 daca G = max

(R – G) / (max – min) + 4 daca B = max

daca H<0 H = H +6

H = H x 60

RGB → HLS

Interpolarea în spatiul culorilor

- Rezultatul interpolării între două culori depinde de modelul de culoare în care sunt specificate.
- Fie două culori specificate în modelul RGB, C1=(1,0,0) şi C2=(0,1,0). Le interpolăm cu ponderi egale în modelele RGB şi HSV:
- Interpolare in modelul RGB:

  C=t*C2 + (1-t)*C1 unde t=0.5, se obţine

  C=(0.5, 0.5, 0), care convertită în HSV ne dă (60, 1, 0.5)
- Interpolare in modelul HSV:

  C1 se reprezintă în HSV prin (0, 1, 1) iar

  C2 prin (120, 1, 1)

  C= 0.5*(0,1,1) + 0.5*(120,1,1) = (60, 1, 1)

Modele de reflexie -1

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

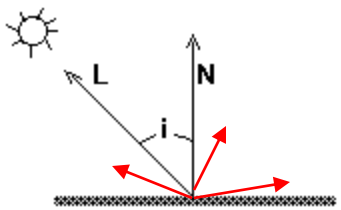
Modele empirice pentru calculul reflexiei luminii

- Caracteristicile luminii reflectate intr-un punct al suprafaţei unui obiect depind de :
  - lungimile de undă conţinute în lumina incidentă,
  - direcţia şi geometria sursei luminii incidente,
  - orientarea suprafeţei in punctul respectiv,
  - proprietăţile materialului din care este construită suprafaţa.
- Expresia care modelează intensitatea luminii reflectate într-un punct al unei suprafeţe este definită pentru o **lumină incidentă monocromatică**, deci caracterizată printr-o lungime de undă, λ.
- In cazul general, **lumina incidentă nu este monocromatică**, de aceea pentru calculul său expresia ar trebui să fie evaluată continuu pe întregul domeniu al spectrului luminii incidente.
- In practică, expresia se evaluează separat pentru cele trei culori primare, R, G, B.

Modelarea reflexiei difuze a luminii provenite de la o sursa

Aproximarea reflexiei difuze intr-un punct al unei suprafete 3D

- Lumina reflectată difuz de o suprafaţă este dispersată regulat în toate direcţiile.
- **Legea lui Lambert** defineşte reflexia luminii provenite de la o **sursă punctiformă**, de către un difuzor perfect:



Reflexia difuza

$Id = Isursa * kd * \cos(i) \qquad 0 <= i <= \pi/2$

N – normala la suprafata in punctul considerat

Isursa – este intensitatea luminii incidente (provenita de la sursa de lumina)

Kd – este coeficientul de difuzie a luminii incidente, dependent de materialul suprafetei: suprafata reflecta/absoarbe/transmite lumina in anumite proportii: $0 <= kd <= 1$

Id este intensitatea luminii reflectate difuz de suprafata

- Dacă i este mai mare ca π/2, suprafaţa nu primeşte lumină de la sursă (sursa de lumină se află în spatele suprafeţei).

Modelarea reflexiei luminii ambiante

Lumina ambianta

- Modeleaza lumina provenita de la celelelte obiecte ale scenei 3D: este o sursa de lumina distribuita uniform in spatiu.
- Lumina ambianta este reflectata la fel in price punct al unei suprafete. Intensitatea sa depinde de intensitatea luminii ambiante si de proprietatile suprafetei.
- Reflexia luminii ambiante este modelata prin:

  Ia*ka, unde

  Ia– este intensitatea luminii ambiante iar

  0<= ka<= 1 este coeficientul de difuzie a luminii ambiante, dependent de materialul suprafetei.

Atunci:

Id – intensitatea luminii reflectate difuz intr-un punct al unei suprafete

Id = reflexia luminii ambiante + reflexia difuza a luminii de la sursa = Ia * ka + Isursa * kd* cos(i)

$0 <= i <= \pi/2$

Modelarea atenuarii luminii de la o sursa

Sursa directionala: sursa aflata la distanta foarte mare (infinit) de scena 3D

- Vectorul L este acelasi in orice punct al suprafetei
- Pentru 2 suprafete paralele, cu aceleasi proprietati de material, rezulta aceeasi Id → daca proiectiile suprafetelor se suprapun in imagine, ele nu se vor distinge.

❖ Intensitatea luminii descreşte proporţional cu inversul pătratului distanţei de la sursa de lumină la suprafata.

Id = Ia* ka + fat * Isursa* kd* cos(i), unde fat = $1/d^2$ este funcţia de atenuare a luminii de la o sursa

d este distanţa de la sursă la punctul de pe suprafaţă considerat.

- Corecţia nu satisface cazurile în care sursa este foarte îndepărtată: atenuare prea mare
- Dacă sursa este la distanţă foarte mică de scenă, atenuarile obţinute pentru două suprafeţe aflate la distante putin diferite de sursa, vor fi mult diferite.
- In practica se foloseste factorul de atenuare: fat = $\min(1/(c1 + c2*d + c3*d^2), 1)$

  c1, c2 şi c3 sunt trei constante care se asociază sursei de lumină.
- Constanta c1 se alege astfel încât numitorul să nu devină prea mic atunci când sursa este foarte apropiată. Valoarea funcţiei este limitată la 1 pentru a se asigura atenuarea.

Culoarea luminii reflectate difuz (1)

- Lumina incidenta poate contine mai multe lungimi de unda: → reflectate in mod diferit de o suprafata

  Csursa = [IsursaR, IsursaG, IsursaB] – culoarea luminii sursei

  Ca = [IaR, IaG, IaB] – culoarea luminii ambientale

- O suprafata reflecta in mod diferit componentele R, G, B ale luminii incidente.

❖ Ex: o suprafata care absoarbe din lumina incidenta componentele de verde (G) si albastru (B):

  – Daca lumina incidenta este alba, suprafata va fi vazuta vazuta ca rosie:
    [1,1,1] – [0,1,1] = [1,0,0]
  – Daca lumina incidenta este rosie, suprafata va fi vazuta tot ca rosie:
    [1,0,0] – [0,1,1] = [1,0,0] (componenetele G si B au valoarea zero – nu pot fi extrase)
  – Daca lumina incidenta este verde, suprafata va fi vazuta ca neagra (componenta verde a fost absorbita): [0,1,0] – [0,1,1] = [0,0,0]

➢ Rezulta: difuzia luminii de catre o suprafata depinde de componentele luminii incidente si de proprietatile suprafetei.

Culoarea luminii reflectate difuz (2)

ka = [kaR, kaG, kaB] – coeficientii de difuzie a componentelor R, G, B din lumina ambientala

kd = [kdR, kdG, kdB] – coeficientii de difuzie a componentelor R, G, B din lumina provenita de la sursa

- Expresia luminii reflectate difuz se evalueaza separat pentru cele 3 componente ale luminii incidente, R,G,B:

  $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \cos(i)$, unde λ: lungimea de unda

sau:

- Cd = [IdR, IdG, IdB] - culoarea luminii reflectate difuz

  Cd = [IaR *kaR + fat*IsursaR *kdR *cos(i), IaG *kaG + fat*IsursaG *kdG *cos(i),

  IaB *kaB + fat*IsursaB *kdB *cos(i)]

- ka (kaR, kaG, kaB) si kd (kdR, kdG, kdB) sunt caracteristici ale materialului suprafetei, numite: culoarea ambiantala si culoarea difuza a materialului.

Modelarea reflexiei difuze

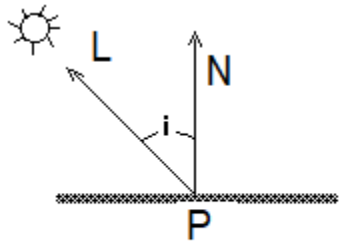
- Stiind că

  $\cos(i)=L\cdot N / (|L|\cdot|N|) = Lu\cdot Nu$

  unde: N este normala la suprafata in P

  L este vectorul din P catre sursa de lumina

  $(L\cdot N)$ – produsul scalar al vectorilor L, N



Reflexia difuza in punctul P

- Rezulta ecuatia care modeleaza lumina reflectata difuz:

❖ $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * (Lu \cdot Nu)$

$Cd = [IdR, IdG, IdB]$ – culoarea luminii reflectate difuz

Pentru a include si cazul in care $i > \pi/2$ (lumina de la sursa nu ajunge in punctul considerat):

$Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \max((Lu \cdot Nu), 0)$

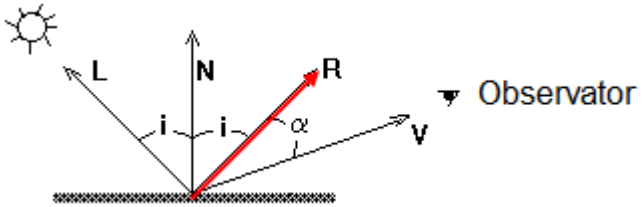
Modele de reflexie -2

Prof. univ. dr. ing. Florica Moldoveanu

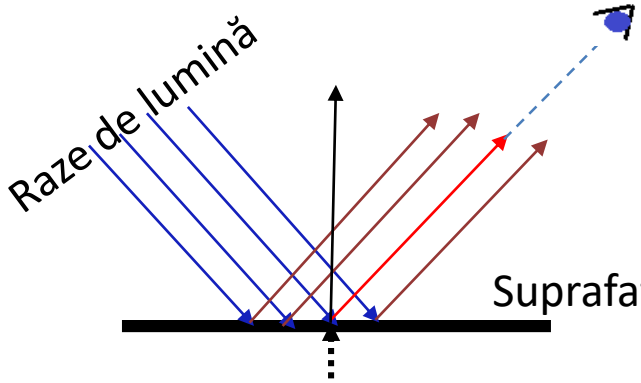
Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Modelarea reflexiei speculare (1)

- Un reflector perfect, de exemplu o oglindă, reflectă lumina numai într-o singură direcţie, R, care este simetrică cu L faţă de normala la suprafaţă → numai un observator situat exact pe direcţia respectivă va percepe raza reflectată specular:

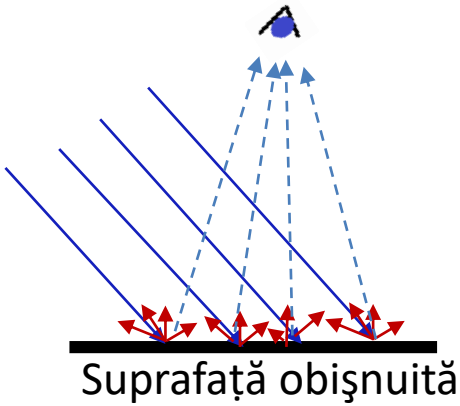


Reflexia speculară



Observatorul vede reflexia speculara a luminii dintr-un singur punct al suprafeței.

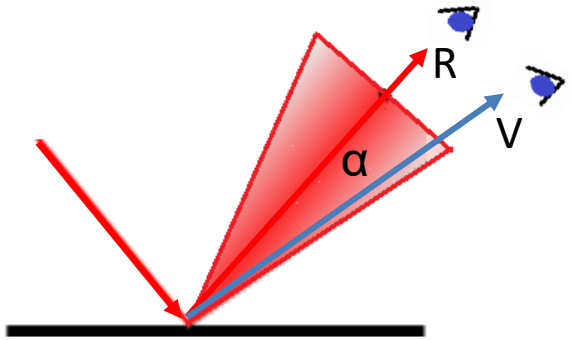
Reflexia difuză



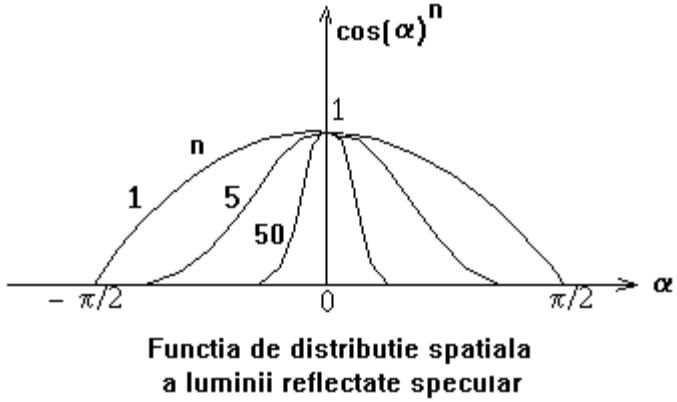
Observatorul vede reflexia difuza a luminii din toate punctele suprafeței.
(neregularitatile microscopice ale suprafetei deviaza directia reflexiei in punctele vecine)

Modelarea reflexiei speculare (2)

• In practica se considera ca reflexia speculara se produce nu ca o raza perfecta ci ca un con de lumina, cu o deschidere care depinde de reflectanta suprafatei.



Observatorul aflat in centrul conului vede reflexia maxima



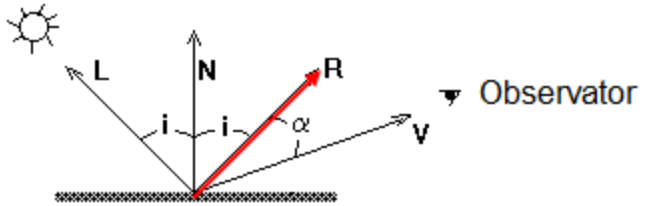
Functia de distributie spatiala a luminii reflectate specular

- Pentru materialele imperfect reflectante cantitatea de lumină care ajunge la observator depinde de distribuţia spaţială a luminii reflectate specular:
  - la suprafeţele netede (ex. metale) distribuţia este dreaptă şi focalizată;
  - la suprafeţele cu rugozităţi (ex. hartia) ea este dispersată.
  - se aproximeaza prin $\cos(\alpha)^n$ (modelul Bui-Tuong Phong) unde n este exponentul de stralucire al materialului.

Modelarea reflexiei speculare (3)

Modelul Phong pentru aproximarea reflexiei speculare intr-un punct al unei suprafete 3D:

$$Is\lambda = Isursa\lambda * w(i,\lambda) * \cos(\alpha)^n$$



w(i, λ) este funcţia de reflectanţă,

i - unghiul de incidenţă iar λ - lungimea de undă a luminii incidente

In practica, w(i, λ) este înlocuită cu o constantă determinată experimental, numită **coeficientul de reflexie speculară al materialului, notat** $ks\lambda$.

- Modelul practic al reflexiei speculare:

$$Is\lambda = Isursa\lambda * ks\lambda * \cos(\alpha)^n$$

$\cos(\alpha) = R \cdot V / (|R| \cdot |V|) = Ru \cdot Vu$

Rezulta, adaugand si factorul de atenuare:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * (Ru \cdot Vu)^n$$

Pentru a include si cazul in care α = 90:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n, 0)$$

Modelarea reflexiei speculare (4)

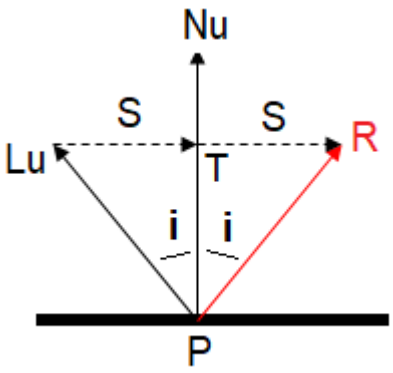
Reflexia speculara nu poate avea loc daca in punctul considerat nu se primeste lumina de la sursa:

$$Is\lambda = lum * Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n, 0)$$

$$lum = 1 \text{ daca } (Lu \cdot Nu) > 0 \quad (0 <= i < \pi/2)$$

$$= 0 \text{ altfel}$$

Calculul directiei reflexiei speculare



• Vectorul R este simetricul vectorului L faţă de N.

• Proiectia scalara a vectorului Lu pe directia normalei este marimea segmentului PT: $|Lu|\cos(i) = \cos(i) = Lu \bullet Nu$

• Proiectia vectoriala a lui Lu pe directia normalei este **vectorul PT**, care are lungimea segmentului PT si directia normalei:

$(Lu \bullet Nu) * Nu / |Nu| = (Lu \bullet Nu) * Nu$

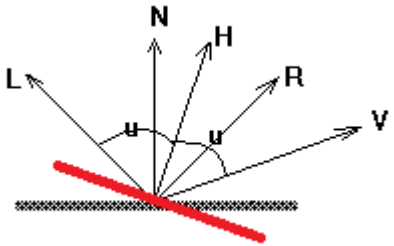
$R = Lu + 2 * S$, $PT = Lu + S \rightarrow S = PT - Lu$

$R = Lu + 2(PT - Lu) = Lu + 2 * (Lu \bullet Nu) * Nu - 2 * Lu = 2 * (Lu \bullet Nu) * Nu - Lu$

$R = 2 * (Lu \bullet Nu) * Nu - Lu$

Modelarea reflexiei speculare (5)

O altă formulare a modelului Phong pentru reflexia speculara foloseste vectorul median, notat cu H în figura. El face unghiuri egale cu L şi cu V.



❖ Justificare: dacă suprafaţa ar fi orientată astfel încât normala să aibă direcţia lui H, atunci observatorul ar percepe lumina specular㠠maximă (V aliniat cu R) → unghiul dintre L si H determina intensitatea reflexiei speculare.

❖ Termenul care exprimă reflexia specular㠠este în acest caz:

$Is\lambda = Isursa\lambda * fat * ks\lambda * (Nu \bullet Hu)^n$, unde Hu=(Lu+Vu) / |(Lu+Vu)|

Atunci când sursa de lumină şi observatorul sunt la infinit (L si V aceleasi in toate punctele unui poligon), Hu este pe suprafata poligonului.

Modelul de iluminare locala(1)
- lumina reflectata intr-un punct al unei suprafete -

Modelul de iluminare locala (numit şi Modelul de reflexie Phong)

Combina culorile datorate:

reflexiei luminii ambientale – culoarea_ambientala,

reflexiei difuze a luminii de la o sursa / surse – culoarea_difuza

reflexiei speculare a luminii de la o sursa/ surse – culoarea_speculara

❖ Reflexia luminii într-un punct al unei suprafeţe determina culoarea sa în acel punct.

$I\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda [kd\lambda * max((Lu \bullet Nu),0) + lum * ks\lambda * max((Ru \bullet Vu)^n ,0)]$

lum = 1 daca (Lu • Nu) > 0 (0<= i < π/2)

= 0 altfel

Modelul de iluminare locala(2)

- Dacă scena 3D este luminată de m surse de lumină:

$$I\lambda = Ia\lambda * ka\lambda + \sum_{1<=i<=m} fat_i * Isursa\lambda_i * [kd\lambda * max((Lu_i \bullet Nu),0) + lum_i * ks\lambda * max((Ru_i \bullet Vu)^n,0)]$$

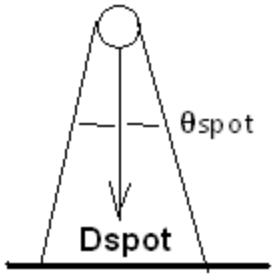
- La cele 3 componente se poate adauga culoarea_emisa de suprafata. Rezulta:

  Culoarea intr-un punct al unei suprafete
  = culoarea_emisa + culoarea_ambiantala + culoarea_difuza + culoarea_speculara

- **Culoarea emisa** :
  - lumina emisa de o suprafata
  - independenta de sursele de lumina
  - aceeasi in orice punct al suprafetei
  - nu lumineaza obiectele din jur
  - culoarea_emisa = Ke, o constanta

Efectul de spot

- In modelul de iluminare locala, sursa de lumina este considerata punctiforma. Lumina produsa de o sursa punctiforma este omnidirectionala.
- Efectul de spot simuleaza o sursa de lumina care produce un con de lumina.
- Numai suprafetele aflate in conul de lumina primesc lumina de la sursa.

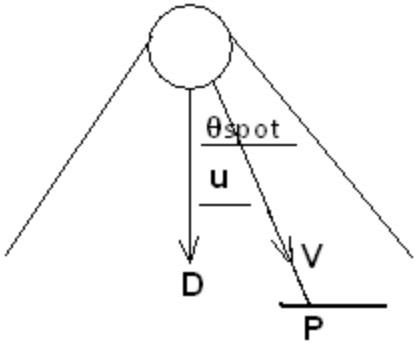


Conul de imprastiere a luminii este definit prin:

-Pozitia spotului (sursei de lumina)

-Directia spotului

-Imprastierea luminii spotului (θspot)



Un punct P al unei suprafete primeste lumina de la spot daca u< θspot:

$$\cos(u) > \cos(\theta spot)$$

sau

$$Vu \bullet Du > \cos(\theta spot)$$

Modele de "shading"

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

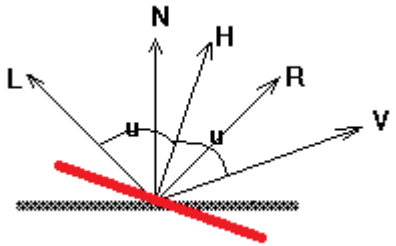
Calculul culorii fragmentelor la redarea suprafetelor 3D (Shading models) (1)

- In vederea vizualizarii, suprafeţele 3D sunt descompuse in faţete poligonale (triunghiulare) conectate. Poligoanele procesate in banda grafica pot fi feţele unui corp poliedral (fețe plate) sau pot proveni din descompunerea unei suprafete oarecare (fețele sunt petice de suprafata curba, aproximate prin fațete plane).
- La momentul rasterizarii nu se cunoaste din ce suprafata face parte un poligon.

Modelul LAMBERT(1)
(Iluminare constantă sau plată)

- Toate fragmentele unui poligon sunt afisate cu aceeasi culoare, calculata folosind modelul de iluminare locala, de regula considerand numai componenta difuza:

$$Id\,\lambda = Ia\,\lambda * ka\lambda + fat * Isursa\,\lambda * [kd\lambda * \max((Nu \bullet Lu),0) + lum * \max((Nu \bullet Hu)^n,0)]$$



Modelul LAMBERT(2)

Modelul se bazează pe următoarele presupuneri:

1. Poligonul face parte din suprafaţa de vizualizat şi nu este o aproximare a unui petic de suprafaţă curbă: vectorul Nu este acelasi in orice punct al poligonului.

2. Sursa de lumină este la infinit: produsul scalar (Nu • Lu) este atunci constant pe întreaga suprafaţă a poligonului.

3. Observatorul este la infinit: (Nu • Hu) este constant pe suprafaţa poligonului;

- Dacă ultimele două cerinţe nu sunt satisfăcute, se poate adopta o convenţie de calcul al vectorilor L şi V pentru un întreg poligon. De exemplu, cei doi vectori pot fi calculaţi în centrul poligonului.
- Dacă prima cerinţă nu este îndeplinită, aproximarea unei suprafeţe curbe prin reţeaua de faţete poligonale este vizibila deoarece intensităţile calculate pentru faţete adiacente cu orientare diferită sunt diferite.

Efectul de banda Mach

- Efectul de banda Mach (descoperit de Mach în 1865) este o iluzie optica datorata filtrarii componentei de luminozitate a imaginii formate pe retina, de catre sistemul vizual uman.
- Efectul măreşte contrastul dintre zone adiacente cu intensitate usor diferita, la frontiera dintre zone, declanşând detecţia frontierei in sistemul vizual uman.
- Atunci cand o suprafaţa curba (de exemplu o sfera) este descompusa in faţete poligonale, este de dorit ca laturile faţetelor sa nu fie vizibile in imaginea suprafetei.
- In cazul folosirii modelului Lambert, culorile faţetelor adiacente cu orientari usor diferite vor fi usor diferite, iar efectul de banda Mach va mări percepţia schimbării de culoare pe laturile faţetelor adiacente.

➢ In concluzie, modelul Lambert este adecvat vizualizarii obiectelor poliedrale.

Modelul LAMBERT(3)

Indicatii de implementare

//vertex shader

```
layout(location = 0) in vec3 v_position;
layout(location = 1) in vec3 v_normal;
uniform vec3 light; // directia sursei de lumina
uniform vec3 eye;//directia observatorului
flat out vec3 color; // culoarea varfului “provocator” va fi folosita la afisarea
    // fragmentelor; in mod implicit ultimul varf al primitivei (ex.: al 3-lea varf pt un triunghi)
void main() {
 //calculeaza light_dir, eye_dir şi culoarea, în pozitia varfului, aplicand modelul de iluminare
……
 color = ambient + diffuse + specular;
}
```

//fragment shader

```
flat in vec3 color; // culoarea varfului “provocator”
void main() {
gl_FragColor = color;
}
```

Modelul Gouraud (1)

- La momentul rasterizarii unui poligon, fiecarui vârf al poligonului îi este asociata o culoare.
- Procesorul de rasterizare calculeaza culoarea fiecarui fragment al poligonului astfel:

  - prin interpolare intre culorile varfurilor, pentru fragmentele de pe laturi

  - prin interpolare intre culorile din capetele segmentului interior pe care se afla fragmentul.

Implementare

1. In Vertex shader se calculeaza culoarea varfului, aplicand modelul de iluminare locala în pozitia varfului (cu N, L, V in cordonate globale, primite din programul de aplicaţie)

   Culoarea varfului este ieşire din vertex shader: out vec3 color

2. Fragment shader primeste ca intrare culoarea fragmentului calculata de procesorul de rasterizare (in vec3 color).

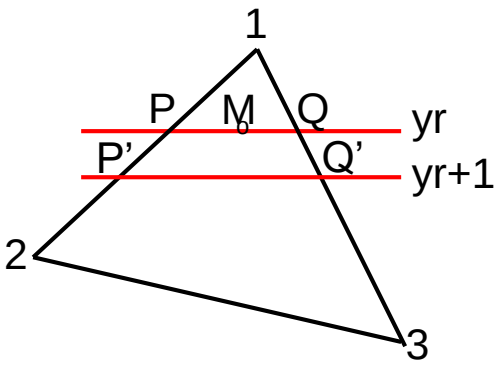
- Normala unui vârf, necesara în vertex shader:

  - poate fi calculata exact, folosind ecuatia analitica a suprafetei pe care se afla varful

  - poate fi aproximata ca medie aritmetica a normalelor la feţele adiacente în varf

Modelul Gouraud (2)

Interpolarea liniara a culorilor



$y = y1 + t\,(y2 - y1) \rightarrow tP = (yr - y1)/(y2 - y1)$

$IP = I1 + tP\,(I2 - I1) = I1 + (I2 - I1)\,(yr-y1) / (y2-y1) = I1 + (yr-y1) * \mathbf{C_{1-2}}$

$C_{1-2} = (I2 - I1) / (y2 - y1)$ - constanta pentru latura 1-2

$IQ = I1 + (I3 - I1)\,(yr-y1) / (y3-y1) = I1 + (yr - y1) * \mathbf{C_{1-3}}$

$C_{1-3} = (I3 - I1) / (y3 - y1)$ - constanta pentru latura 1-3

$x = xP + t(xQ - xP) \rightarrow tM = (xM - xP)/(xQ - xP)$

$IM = IP + tM\,(IQ - IP) = IP + (IQ - IP)\,(xM - xP) / (xQ - xP) = IP + (xM-xP)* \mathbf{C_{P-Q}}$

$C_{P-Q} = (IQ - IP) / (xQ - xP)$ - constanta pentru segmentul P-Q

Calculul incremental al culorilor

$IP' = I1 + (I2 - I1)\,(yr+1 - y1) / (y2 - y1) = I1 + (yr+1 - y1) * \mathbf{C_{1-2}} = IP + \mathbf{C_{1-2}}$

$IP' = IP + C_{1-2}$ - analog, **$IQ' = IQ + C_{1-3}$**

M' (xM + 1, ys)

$IM' = IP + (xM + 1 - xP) * \mathbf{C_{P-Q}} \rightarrow$ **$IM' = IM + CP-Q$**

Modelul Gouraud (3)

Aprecieri asupra modelului Gouraud

- Calculul culorilor fragmentelor este integrat în algoritmul de rasterizare a poligoanelor
- Calcule simple pentru culoarea unui fragment (calcul incremental) → rapid
- Nu permite calculul luminii reflectate specular pentru fragmentele interioare unui poligon (intensitatile fragmentelor interioare nu pot fi mai mari decat cele din varfuri)
- Reduce efectul de banda Mach, dar nu il elimina:
  - Culorile pe latura de adiacenta a 2 poligoane sunt aceleasi
  - Nu tine cont de tangentele la fetele adiacente pe o latura (culorile de pe laturi se calculeaza folosind numai culorile din varfuri)
  - Efectul de banda Mach poate fi observat in vecinatatea siluetei unei suprafeţe 3D si a zonelor de curbura mare.

Modelul Phong (1)

- La momentul rasterizarii unui poligon, fiecarui vârf îi este asociata normala vârfului.
- Procesorul de rasterizare calculeaza normala fiecarui fragment al poligonului astfel:

- prin interpolare intre normalele varfurilor, pentru fragmentele de pe laturi;
- prin interpolare între normalele capetelor segmentului interior pe care se afla fragmentul (la fel ca in modelul Gouraud, pentru culori).

Implementare

- Normala vârfului este transmisa din programul de aplicatie, ca si pentru modelul Gouraud.
- Normala si pozitia vârfului, in coordonate globale, sunt ieşiri din vertex shader.
- Procesorul de rasterizare interpoleaza liniar pozitiile şi normalele varfurilor, transmitand la fragment shader pozitia si normala fragmentului.

- In fragment shader se calculeaza culoarea fragmentului aplicand modelul de iluminare locala în pozitia fragmentului.

Modelul Phong (2)

Dezavantaje fata de modelul Gouraud

❑ Componentele Nx, Ny, Nz ale normalei unui fragment se obţin (in algoritmul de rasterizare) printr-un calcul incremental analog cu cel folosit pentru calculul culorilor in modelul Gouraud dar, pentru folosirea în calculul culorii, normala trebuie să fie normalizată în fragment shader:

$$Nu = N/|N|, \text{ unde } |N| = (Nx^2 + Ny^2 + Nz^2)^{0.5}$$ (se foloseste o functie GLSL)

❑ De asemenea, vectorii L (catre sursa de lumina) si V (catre observator), folositi in modelul de iluminare locala la nivel de fragment, sunt calculati si normalizati in programul fragment shader pentru a fi folositi in calculul culorii fragmentului.

➢ Calcule mai complexe la nivel de fragment decât in modelul Gouraud

Avantaje

❑ Permite redarea reflexiei speculare in orice fragment al unui poligon

❑ Reduce mult efectul de banda Mach.

Interpolarea atributelor varfurilor primitivelor

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Tipul interpolării

- **Atributele ataşate vârfurilor unei primitive**: culoarea, normala, coordonatele textură, s.a.
- Interpolarea este efectuata de procesorul de rasterizare, fiind integrată în algoritmul de rasterizare.
- Tipul de interpolare dorit poate fi specificat, GLSL, printr-un calificativ ataşat atributului, ca :
  - variabilă de ieşire din vertex shader
  - variabilă de intrare în fragment shader

Calificativul de interpolare :

- *flat* : atributul nu este interpolat
- *nonperspective* : valoarea atributului la nivel de fragment (transmisă la fragment shader) se obţine prin interpolarea liniară a valorilor atributului, ataşate vârfurilor primitivei
- *smooth* : valoarea atributului la nivel de fragment se obţine prin interpolarea perspectivă a valorilor atributului, ataşate vârfurilor primitivei.

Calificativul *flat (1)*

Exemple:

Vertex shader	Fragment shader
flat out vec3 color	flat in vec3 color
nonperspective out vec3 color	nonperspective in vec3 color
smooth out vec3 color	smooth in vec3 color
smooth out vec2 text_coord	smooth in vec2 text_coord

In versiunile actuale GLSL, valoarea implicita a atributului de interpolare este *smooth*

→ procesorul de rasterizare efectuează interpolarea perspectivă a atributului.

1. Calificativul *flat*

In programul de aplicație se poate declara pentru fiecare primitivă un vârf “provocator”, prin indicele vârfului în lista varfurilor primitivei. In mod implicit, este ultimul vârf.

Selectia varfului provocator se face cf. conventiei (primul/ultimul varf), in functie de tipul primitivei(https://www.khronos.org/opengl/wiki/Primitive#Provoking_vertex)

void glProvokingVertex(GLenum *provokeMode*);
provokeMode poate fi: GL_FIRST_VERTEX_CONVENTION sau GL_LAST_VERTEX_CONVENTION (implicit)

Calificativul *flat (2)*

Un exemplu de utilizare este in implementarea modelului Lambert:

Vertex shader

flat out vec3 color//culoarea vârfului “provocator”

void main() {

…calculeaza color

}

Fragment shader

flat in vec3 color//culoarea vârfului “provocator”

void main() {

gl_FragColor = color;//afisare fragment in culoarea vârfului “provocator”

}

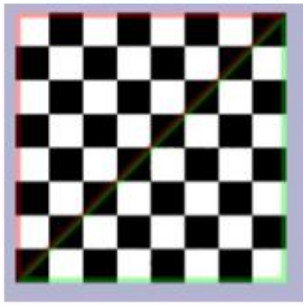
Interpolarea perspectivă (1)

- Este necesara dacă asupra scenei se efectuează o proiectie perspectivă :
  - Proiectia perspectiva produce o scalare a primitivelor dependenta de distanta varfurilor fata de centrul de proiectie (pozitia observatorului), atunci cand primitiva nu este paralela cu planul de proiectie
  - Efectul perspectiva trebuie sa se aplice si atributelor.

Exemplu :

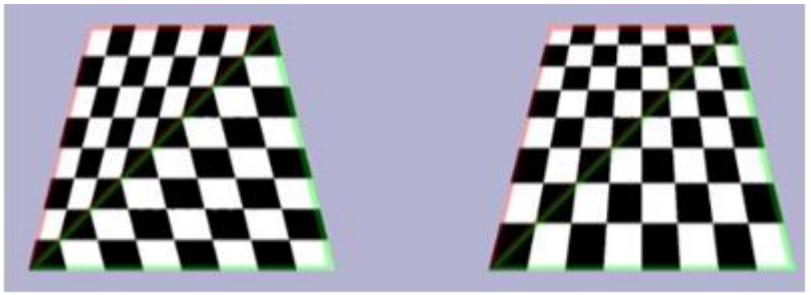
Textura aplicata pe suprafata unui poligon trebuie sa fie scalata la fel ca suprafata poligonului:

Proiectie ortografica



Maparea afina a texturii (interpolarea liniara a coordonatelor textura)

Proiectie perspectiva

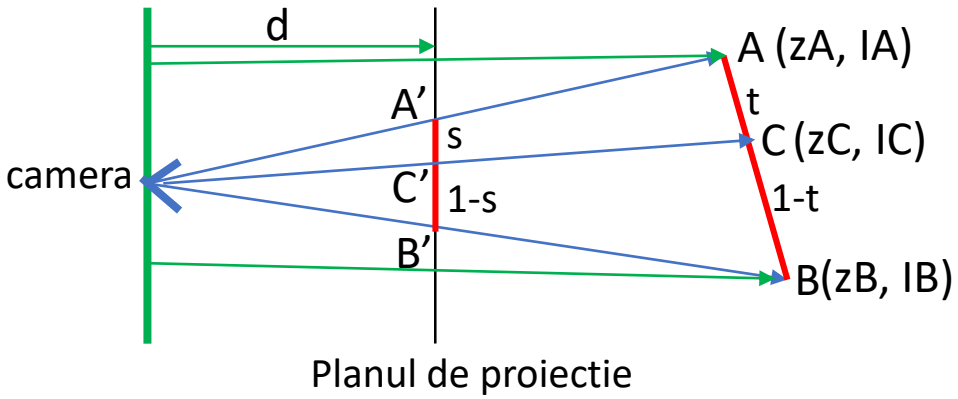


Maparea afina

Maparea perspectiva (interpolarea perspectiva a coordonatelor textura)

Interpolarea perspectivă (2)

A-B, primitiva in spatiul camerei
A'-B', proiectia primitivei
d - distanta la planul de proiectie
I – atributul interpolat



Interpolare liniară a atributului I:
In punctul C: IC = IA + t(IB-IA)
In punctul C': IC' = IA + s(IB – IA)

t/(1-t) ≠ s/(1-s) → IC' nu poate fi obţinut prin interpolare liniara intre IA si IB

Interpolarea perspectiva (ne-liniara):

$$t = \frac{s*ZA}{s*ZA+(1-s)*ZB} \rightarrow IC' = IA + \frac{s*ZA}{s*ZA+(1-s)*ZB}(IB - IA)$$

- Algoritmul de interpolare perspectiva executat pe GPU foloseste un calcul incremental al coordonatelor textura, efectul perspectiva fiind obtinut pe baza coordonatelor w_clip ale varfurilor primitivelor, in locul coordonatelor z.

INTRODUCEREA CETII IN IMAGINI

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

MODELAREA CEȚII(1)

Ceaţa adaugă realism imaginii şi ajută în eliminarea defectelor produse de decuparea la nivelul planului din spate al volumului vizual:

- Fără ceaţă, pe măsură ce observatorul se îndepărtează de un obiect, obiectul se aproprie de planul din spate şi este decupat atunci când planul din spate îl intersectează.
- Cu ceaţă, dacă intensitatea ceţii creşte proporţional cu distanţa de la poziţia observatorului, se crează efectul de distanţă pentru obiectele îndepărtate, iar obiectele decupate la nivelul planului din spate nu mai sunt vizibile.

Cfog – culoarea de ceaţă

Clocal – culoarea calculată într-un punct folosind modelul de iluminare locala

$0 <= \Phi <= 1$ – factorul de ceaţă, proporţional cu distanţa de la observator la punct

($\Phi=1$: ceata este opacă)

Culoarea punctului, afectata de ceata: **$Cloc_fog = (1-\Phi)*Clocal + \Phi*Cfog$**

MODELAREA CETII(2)

Fie dist distanta de la observator la punctul in care se calculeaza culoarea afectata de ceata.

Factorul de ceaţă liniar

- Actioneaza intre 2 distante definite in sistemul coordonatelor observator, z0 si z1, masurate pe directia in care priveste observatorul (axa z).

$\Phi = 0$ pentru dist < z0

$\Phi = (dist - z0) / (z1 - z0)$ pentru z0 <= dist <= z1

$\Phi = 1$ pentru dist > z1

Factorii de ceaţă exponentiali

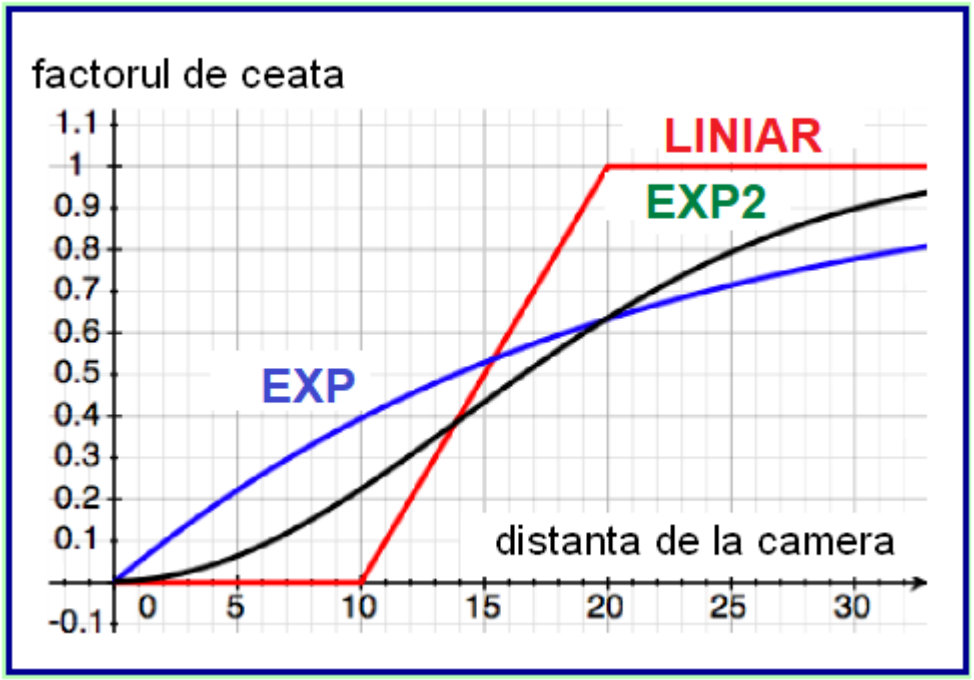
$\Phi = 1 - e^f$

EXP: f = (- fog_density*dist)

EXP2: $f = (- fog_density * dist)^2$

- fog_density: densitatea cetii (un numar real, de ex. 0.05) - modeleaza atenuarea luminii datorata cetii.

MODELAREA CETII(3)



z0 = 10; z1 = 20; fog_density = 0.3

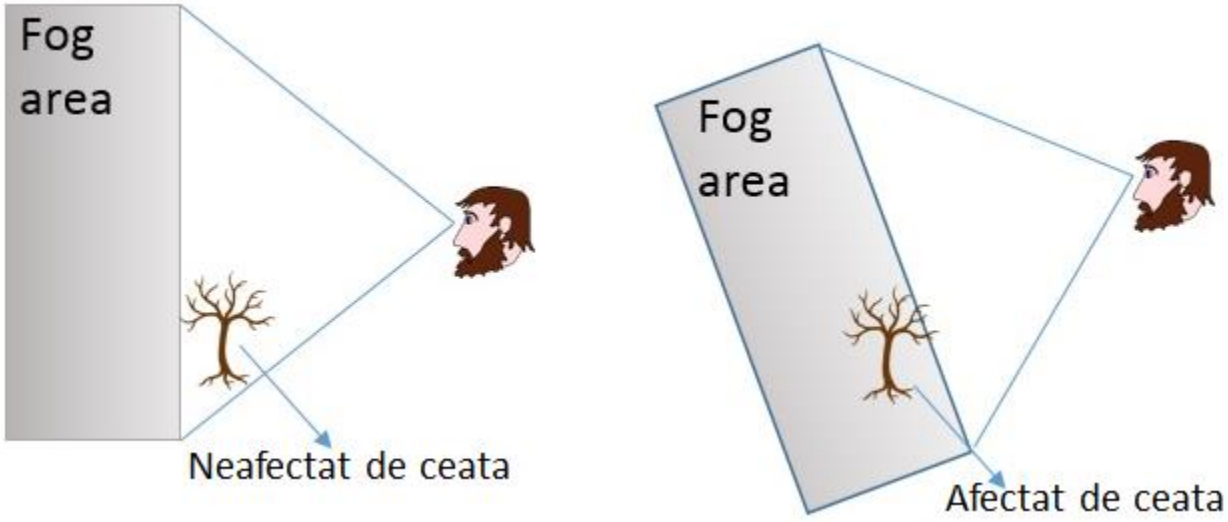
DISTANTA - dist

Distanta fata de observator a punctului in care se calculeaza culoarea:

- Punctul este in coordonate observator

1. **dist** este coordonata z a punctului, la fel ca distantele z0 si z1

Daca se roteste camera, fara a-i modifica pozitia, coord. z a punctului se modifica → un obiect aflat la o distanta d de observator poate sa intre/iasa din zona de ceata:



2. **dist** este lungimea vectorului de la observator (0,0,0) la punct.

Varianta 1 este mai ieftina computational si preferata atunci cand nu este rotita camera

INTRODUCEREA CETII IN IMAGINI (1)

Includerea cetii in calculul culorii se poate efectua:

1. In vertex shader (modelul Gouraud):

- se calculeaza **Φ** folosind distanta (dist) de la observator la pozitia varfului in coord observator

- se calculeaza culoarea varfului afectata de ceata: **Cloc_fog = (1-Φ)*Cvrf + Φ*Cfog**

- **Cloc_fog** este iesire din vertex shader

- culorile afectate de ceată ale varfurilor sunt interpolate de GPU la nivel de fragmente

- culoarea de ceață interpolata, **Cloc_fog,** este intrare pentru fragment shader

2. In fragment shader(modelul Phong):

Varianta (1): fragment shader primeşte pozitia fragmentului în coordonate observator

- se transmite de la vertex shader pozitia varfului în coordonate observator

- pozitiile varfurilor in coordonate observator sunt interpolate de GPU la nivel de fragmente

- în fragment shader se calculeaza **Φ** folosind distanta (dist) de la observator la pozitia fragmentului în coordonate observator

INTRODUCEREA CETII IN IMAGINI (2)

```
// Calculul culorii de ceata in fragment shader- Varianta (1)

//Vertex shader
layout(location = 0) in vec3 in_position;
layout(location = 1) in vec3 in_normal;
uniform mat4 model_matrix, view_matrix, projection_matrix;

out vec3 world_pos;// pozitia varfului in coord globale
out vec3 world_normal;//normala varfului transformata in coord globale
out vec4 view_pos; //pozitia varfului in coord observator

void main(){
  world_pos = (model_matrix * vec4(in_position,1)).xyz;
  world_normal = normalize(mat3(model_matrix) * in_normal);
  view_pos = view_matrix * model_matrix * vec4(in_position,1);
  gl_Position = projection_matrix * view_pos;
 }
```

```
//fragment shader
uniform vec3 light_position;
uniform vec3 eye_position;
uniform int fogSelector; //0 linear; 1 exponential; 2 exponential square
uniform vec3 Cfog = vec3(0.5, 0.5,0.5);//culoarea cetii
……..
in vec3 world_pos;// pozitia interpolata a fragmentului, in coord globale
in vec3 world_normal;// normala interpolata a fragmentului, in coord globale
in vec4 view_pos;// pozitia interpolata a fragmentului, in coord observator
void main(){
//Calculeaza Cfragm aplicand modelul de iluminare locala in pozitia world_pos a fragmentului
   Cfragm = ….
//Calculeaza distanta de la pozitia view_pos a fragmentului la observator
dist = length(view_pos); //lungimea vectorului (0,0,0) → view_pos
if(fogSelector == 0)//linear fog
{  fogFactor = (dist - 20)/(80 - 20); // z0 =20; z1=80
  fogFactor = clamp( fogFactor, 0.0, 1.0 );//restrange fogFactor la intervalul 0-1
 gl_FragColor = (1-fogFactor)*Cfragm + fogFactor*Cfog
} ……..}
```

INTRODUCEREA CETII IN IMAGINI (3)

```
// Calculul culorii de ceata in fragment shader- Varianta (2)
// pozitia fragmentului în coordonate observator se calculeaza în fragment shader
//Vertex shader
out vec3 world_pos;// pozitia varfului in coord globale
out vec3 world_normal;//normala transformata in coord globale
//fragment shader
uniform vec3 light_position;
uniform vec3 eye_position;
uniform vec3 Cfog = vec3(0.5, 0.5,0.5);//culoarea cetii
in vec3 world_pos;// pozitia interpolata a fragmentului in coord globale
in vec3 world_normal;// normala interpolata a fragmentului in coord globale
void main(){
//Calculeaza Cfragment  aplicand modelul de iluminare locala in pozitia world_pos
   Cfragment = ….
// Calculeaza pozitia fragmentului in coordonate observator → frg_eye_pos
//Calculeaza distanta de la frg_eye_pos, la observator
   dist = length(frg_eye_pos);
//Calculeaza fogFactor folosind dist
……
```

INTRODUCEREA CETII IN IMAGINI (4)

Calculeaza pozitia fragmentului in coordonate observator: frg_pos → frg_eye_pos

//transformarea in viewport

gl.FragCoord.x = (width/2)frg_xd + ox , ox = px+ width/2

gl.FragCoord.y = (height/2)frg_yd + oy, oy = py + height/2

gl.FragCoord.z = ((f-n)/2)frg_zd + (n+f)/2 = frg_zd/2 + 0.5 // implicit: n=0, f=1

Coord. disp normalizate→

frg_xd = (gl.FragCoord.x – ox)/(width/2)

frg_yd = (gl.FragCoord.y – oy)/(height/2)

frg_zd = (gl.FragCoord.z – 0.5)*2

//transformarea de normalizare

frg_ xd = frg_ xc/frg_wc; frg_ yd = frg_ yc/frg_wc; frg_ zd = frg_ zc/frg_wc; frg_ wc = -eye

//pozitia fragmentului in coordonate de decupare: frg_xc = frg_xd*frg_wc, frg_yc=…..

frg_clip_pos = $[frg_xc, frg_yc, frg_zc, fgr_wc]^T$

//transformarea de proiectie

frg_clip_pos = projection_matrix * frg_eye_pos

//pozitia fragmentului in coordonate observator

frg_eye_pos = $(projection_matrix)^{-1}$ *frg_clip_pos

Transparenta
în modelul de iluminare locala

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

MODELAREA TRANSPARENTEI (1)

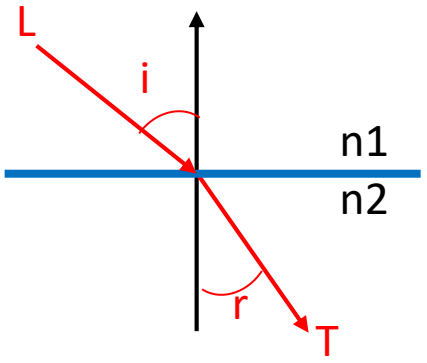
Unele obiecte ale scenei sintetizate pot fi construite din materiale transparente sau translucide.

Transmisia luminii prin obiectele transparente este specular, în timp ce prin cele translucide este difuză.

- Atunci când lumina trece dintr-un mediu într-altul (de exemplu, din aer în apă), direcţia sa se modifică datorită refracţiei.
- Relaţia dintre unghiul razei incidente, i, şi cel al razei refractate, r, este dată de legea lui Snell:

sin(r)/sin(i) = n1/n2

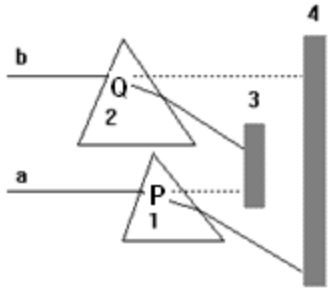
n1 şi n2 sunt indicii de refracţie ai celor două medii (materiale) traversate de lumină.



MODELAREA TRANSPARENTEI (2)

Indicele de refracţie al unui material este dependent de lungimea de undă a luminii incidente şi chiar de temperatură, dar în modelele de sinteza a imaginilor el este considerat constant.

Culoarea vizibila intr-un punct al unei suprafete transparente provine de la suprafata aflata pe directia razei transmise.



3 si 4 sunt poligoane opace

1 si 2 sunt poligoane transparente, aflate in fata poligoanelor 3 si 4

a si b sunt raze de lumina provenind de la surse, incidente in P si Q

Tinand cont de refractie:

- Culoarea in P este data de poligonul 4
- Culoarea in Q este data de poligonul 3

Neglijand refractia (T are directia razei incidente)

- Culoarea in P este data de poligonul 3
- Culoarea in Q este data de poligonul 4

MODELAREA TRANSPARENTEI (3)

- Refracţia produce, de asemenea, o distorsionare a obiectelor, asemănătoare cu aceea produsă de o proiecţie perspectivă→ dacă se doreşte obţinerea de imagini realiste, trebuie să se ţină cont de refracţie.

- Multe metode practice de modelare a transparenţei ignoră refracţia, astfel încât obiectele vizibile printr-o suprafaţă transparentă sunt cele aflate pe direcţia razei incidente.

Motivul ignorării:

- reducerea volumului de calcule;
- obţinerea realismului fotografic în totalitate (fara deformare)

Atunci când suprafaţa vizibilă într-un pixel este transparentă, culoarea în care va afişat pixelul se poate obţine combinând culoarea suprafeţei vizibile cu aceea a suprafeţei aflată imediat în spatele său (sau a fondului), folosind următoarea formulă de interpolare:

MODELAREA TRANSPARENTEI (4)

$I_\lambda = (1-kt_1)*I_{\lambda 1} + kt_1*I_{\lambda 2}$

$I_{\lambda 1}$ este culoarea suprafetei vizibile în pixel

$I_{\lambda 2}$ este culoarea suprafetei din spatele celei vizibile, care se proiecteaza in acelasi pixel

Coeficientul de transmisie, kt_1, este o măsură a transparenţei suprafeţei vizibile în pixel,

$0 <= kt_1 <= 1$

$kt_1 = 0$ → suprafaţa vizibilă este opacă şi deci pixelul va fi afişat în culoarea sa, $I_{\lambda 1}$

$k_{t1} = 1$ → suprafaţa vizibilă este perfect transparentă şi nu contribuie la culoarea pixelului.

Pixelul va fi afişat în culoarea suprafeţei din spate, $I_{\lambda 2}$

Dacă $k_{t1}=1$ şi suprafaţa din spatele celei vizibile este la rândul său transparentă, metoda de calcul se aplică recursiv, până când se întâlneşte o suprafaţă opacă sau fondul.

Aproximarea liniară din model nu dă rezultate bune pentru suprafeţele curbe: în apropierea laturilor siluetei unei suprafeţe curbe (de exemplu, o vază sau o sticlă) grosimea materialului reduce transparenţa.

MODELAREA TRANSPARENTEI (5)

Solutia propusa de Kay: kt se calculeaza in functie de normala la suprafata in punctul considerat:

$k_t = kt_{min} + (kt_{max} - kt_{min})(1-(1-N_z))^m$

unde

kt_{min} şi kt_{max} reprezintă transparenţa minimă şi cea maximă a suprafeţei

N_z este componenta z a normalei normalizate la suprafaţă în punctul pentru care se calculează kt,

m este un exponent ce caracterizează transparenţa. Valorile uzuale pentru m sunt 2 şi 3.

$N_z = 1 \rightarrow k_t = kt_{max}$ $\quad I_\lambda = (1-kt_{max})*I_{\lambda 1} + kt_{max}*I_{\lambda 2}$

$N_z = 0 \rightarrow k_t = kt_{min}$ $\quad I_\lambda = (1-kt_{min})*I_{\lambda 1} + kt_{min}*I_{\lambda 2}$

Majoritatea algoritmilor de eliminare a partilor nevizibile la afişarea scenelor 3D pot fi adaptaţi pentru a îngloba transparenţa.

- **In algoritmii care afişează poligoanele scenei 3D în ordinea "din spate în faţă"** (back to front), de ex. algoritmul BSP şi Pictorului, $I_{\lambda 1}$ corespunde fragmentului curent iar $I_{\lambda 2}$ este culoarea pixelului în care se afiseaza fragmentul.

MODELAREA TRANSPARENTEI (6)

- Adăugarea efectului de transparenţă în algoritmul Z-Buffer este mai dificilă, deoarece poligoanele sunt rasterizate în ordinea în care sunt transmise în banda grafică, neţinându-se cont de distanţa lor faţă de observator.

-Se poate realiza folosind mai multe memorii buffer-imagine si generand imaginea in mai multe etape, imaginea finala obtinandu-se prin combinarea imaginilor poligoanelor opace cu cele ale poligoanelor transparente; nu se foloseste in practica.

In OpenGL, transparenta poate fi simulata prin amestecul dintre culoarea fragmentului curent si cea a pixelului in care se afiseaza, folosind modelul (R, G, B, A);

A – opacitatea culorii: 0 – transparenta; 1 – opaca

Se folosesc functiile:

glEnable(GL_ALPHA_TEST) – activeaza testul alfa

MODELAREA TRANSPARENTEI (7)

glAlphaFunc(func, ref) – specifica functia “alpha test”

func - functia de comparare a valorii A a fragmentului cu o valoare de referinta, 0<=ref<=1;

Exemplu: glAlphaFunc(GL_GREATER, 0.5)

– fragmentul este afisat daca opacitatea sa este > decat 0.5

Daca fragmentul curent

- nu “trece” testul alfa, culoarea pixelului nu se modifica
- “trece” testul alfa, el va modifica culoarea pixelului, dacă:
  - trece testul “stencil”
  - trece testul de adancime

glEnable(GL_BLEND) - activeaza amestecul culorilor

glBlendFunc(..) – specifica modul de amestec al culorilor (daca fragmentul trece testul alfa).

Exemplu:

glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA):

noua_culoare_pixel = Culoare_fragment * A_fragment + Culoare_pixel * (1 – A_fragment)

- Amestecul culorilor este o operatie raster: efectuata de procesorul de rasterizare dupa testul de adancime, inainte de actualizarea valorii pixelului in bufferul imagine (frame buffer).

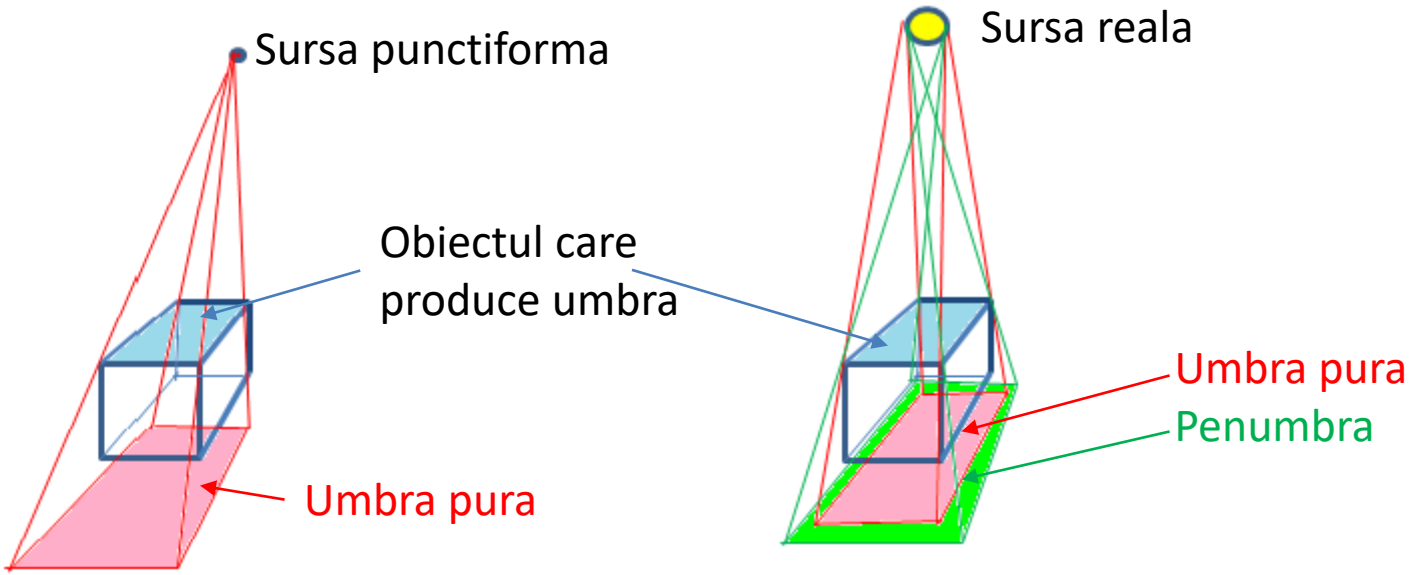
Redarea umbrelor în imagini

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

INTRODUCEREA UMBRELOR IN IMAGINI(1)

- Atunci când un observator priveşte o scenă 3D luminată de o sursă de lumină dintr-o poziţie diferită de aceea a sursei de lumină, va vedea umbrele produse de obiectele scenei.
- Umbrele au o contribuţie însemnată la realismul imaginii, îmbunătăţind percepţia profunzimii.



In general, se considera numai umbra pura: calcule mai simple.

INTRODUCEREA UMBRELOR IN IMAGINI(2)

❑ **Volumul de calcule necesar introducerii umbrelor depinde de poziţia sursei de lumină**:

▪ Dacă sursa este la infinit calculele sunt mai simple: se efectueaza o proiectie paralela din pozitia sursei

▪ In cazul unei surse situate la distanţă finită dar în afara scenei, este necesară o proiecţie perspectivă din poziţia sursei.

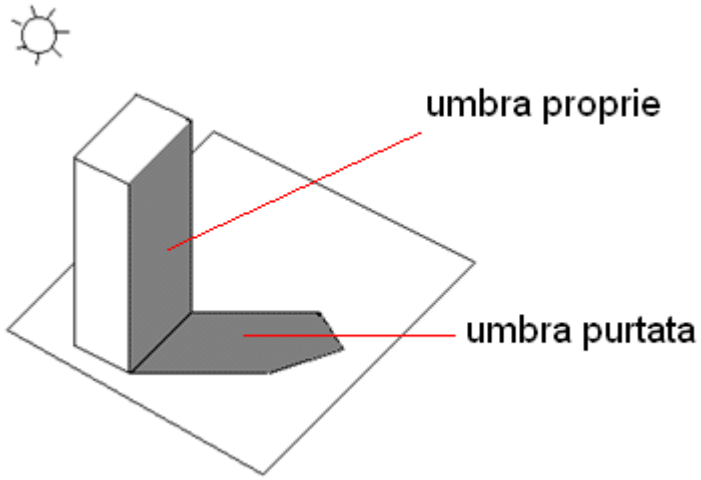
▪ Cazul cel mai dificil este acela în care sursa este situată în scenă.

❑ Problema determinării umbrelor este similară aceleia de determinare a părţilor nevizibile ale unei scene 3D: partile scenei 3D care nu sunt vizibile din poziţia sursei de lumină sunt în umbră.

➢ Crearea unei imagini cu umbre presupune rezolvarea de două ori a problemei suprafeţelor nevizibile: o dată privind scena din poziţia sursei de lumină, a doua oară privind-o din poziţia observatorului.

UMBRE PLANARE (1)

Umbrele produse de obiectele scenei 3D în de două tipuri: **umbre proprii** şi **umbre purtate**.



–**Umbrele proprii** sunt generate de obiectul însuşi, care împiedică lumina să ajungă la unele dintre feţele sale: feţele umbrite de umbra proprie sunt feţele auto-obturate atunci când scena este văzută din poziţia sursei de lumină.

–**Umbrele purtate** sunt umbrele pe care un obiect le produce pe alte obiecte ale scenei, la care lumina nu ajunge din cauza obiectului.

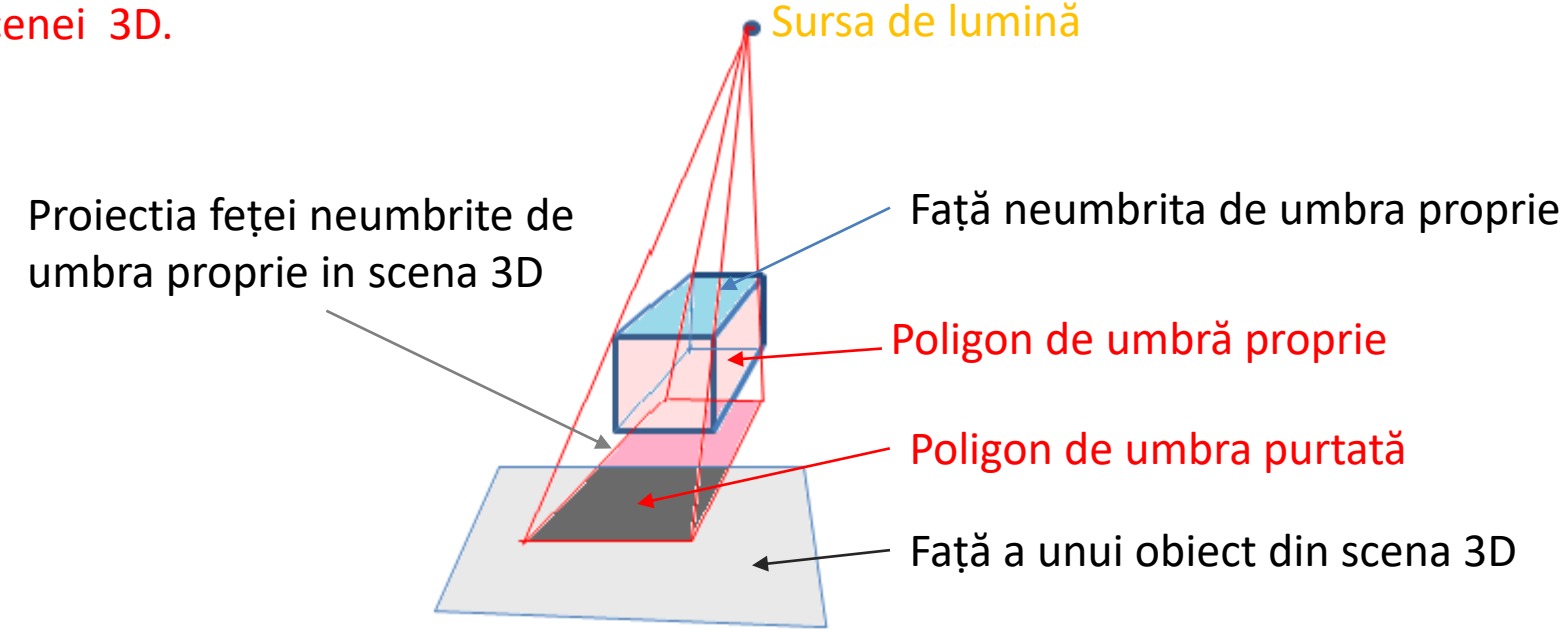
UMBRE PLANARE (2)

❑ Umbrele purtate se pot determina proiectând în scenă, din poziţia sursei, toate poligoanele (feţele) neumbrite de umbra proprie. Proiectiile acestor poligoane pe obiectele scenei 3D se numesc **poligoane de umbră purtată.**

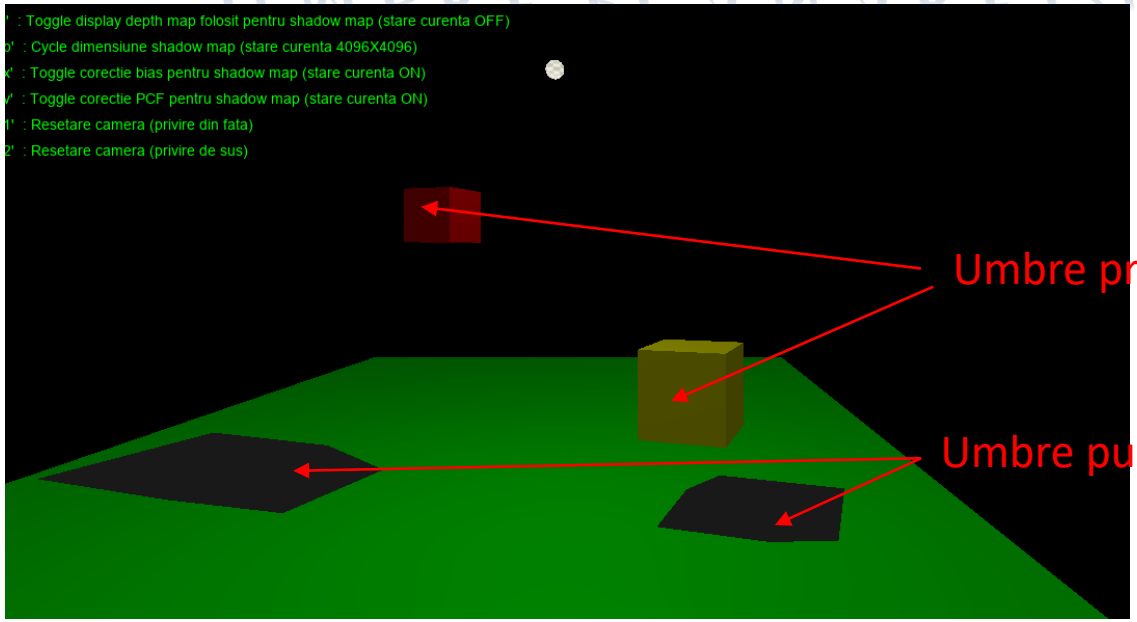
❑ **Un poligon de umbră purtată este coplanar cu fața obiectului care contine umbra purtată.**

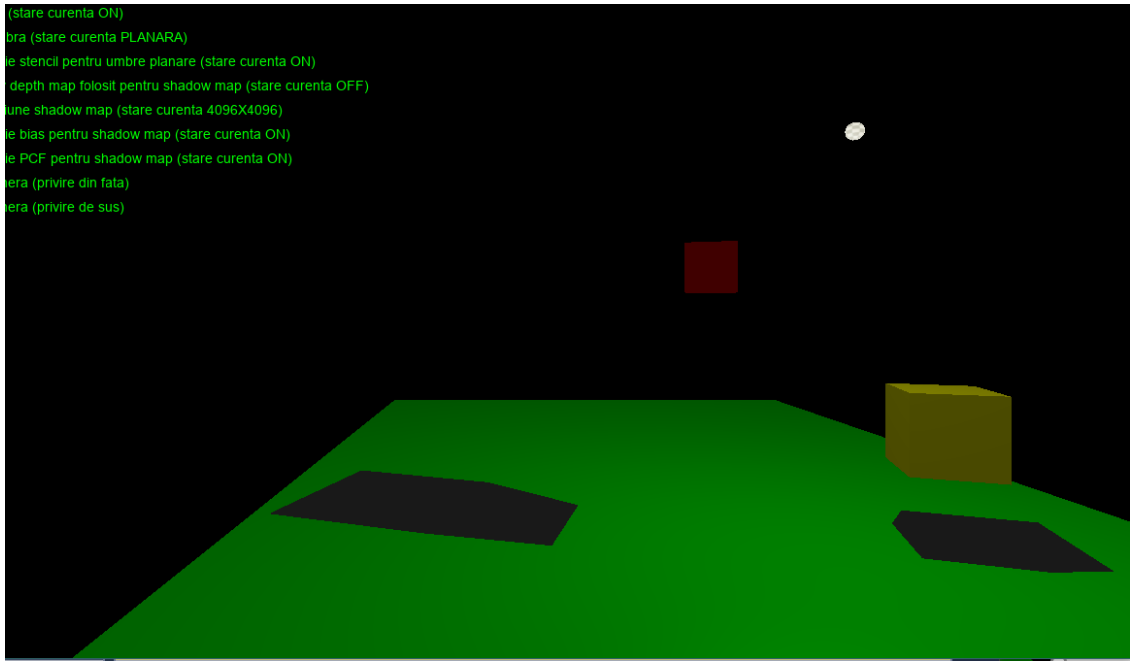
❑ **Un poligon de umbră proprie este un poligon identic cu fața umbrită de umbra proprie.**

❑ Umbrele proprii şi poligoanele de umbra se folosesc ca **poligoane detaliu** la redarea scenei 3D.



UMBRE PLANARE (3)





Scena cu umbre vazuta din altă pozitie a observatorului

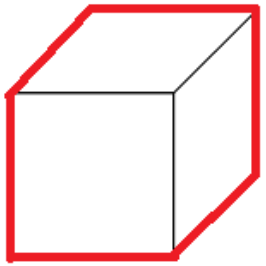
UMBRE PLANARE (4)

Redarea scenei cu umbre

1. Se trimit in banda grafica poligoanele scenei
2. Se dezactivează testul de adâncime (a.i. fragmentele poligoanelor de umbra să suprascrie unele dintre fragmentele poligoanelor scenei)

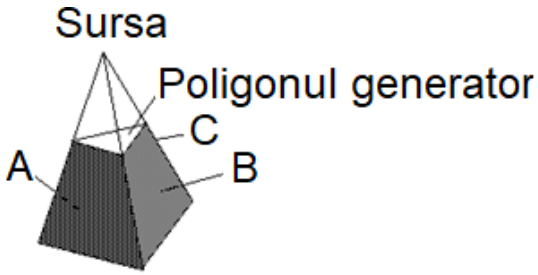
   Se trimit in banda grafica poligoanele de umbră (acestea au o culoare de umbra)
3. Se cere redarea scenei.

- Pot fi generate mai multe vederi ale scenei fără a recalcula umbrele, deoarece umbrele depind numai de poziţia sursei (surselor) de lumină.
- Numărul de poligoane de umbră este mai mic, dacă în loc să se proiecteze fiecare faţă luminată de sursă, se proiectează silueta fiecărui obiect văzută din poziţia sursei de lumină.
- Calculul poligoanelor de umbra este destul de complicat, de aceea in prezent se folosesc alte metode pentru introducerea umbrelor in imagini.



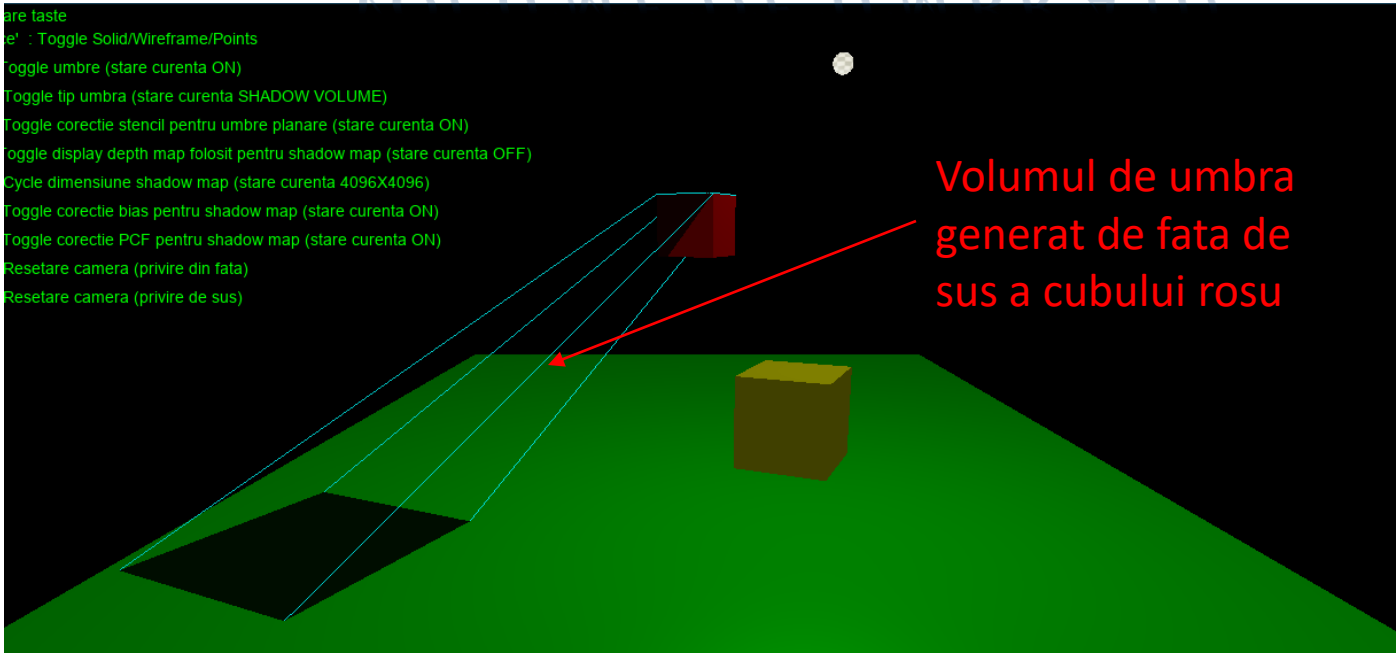
VOLUME DE UMBRA (1)

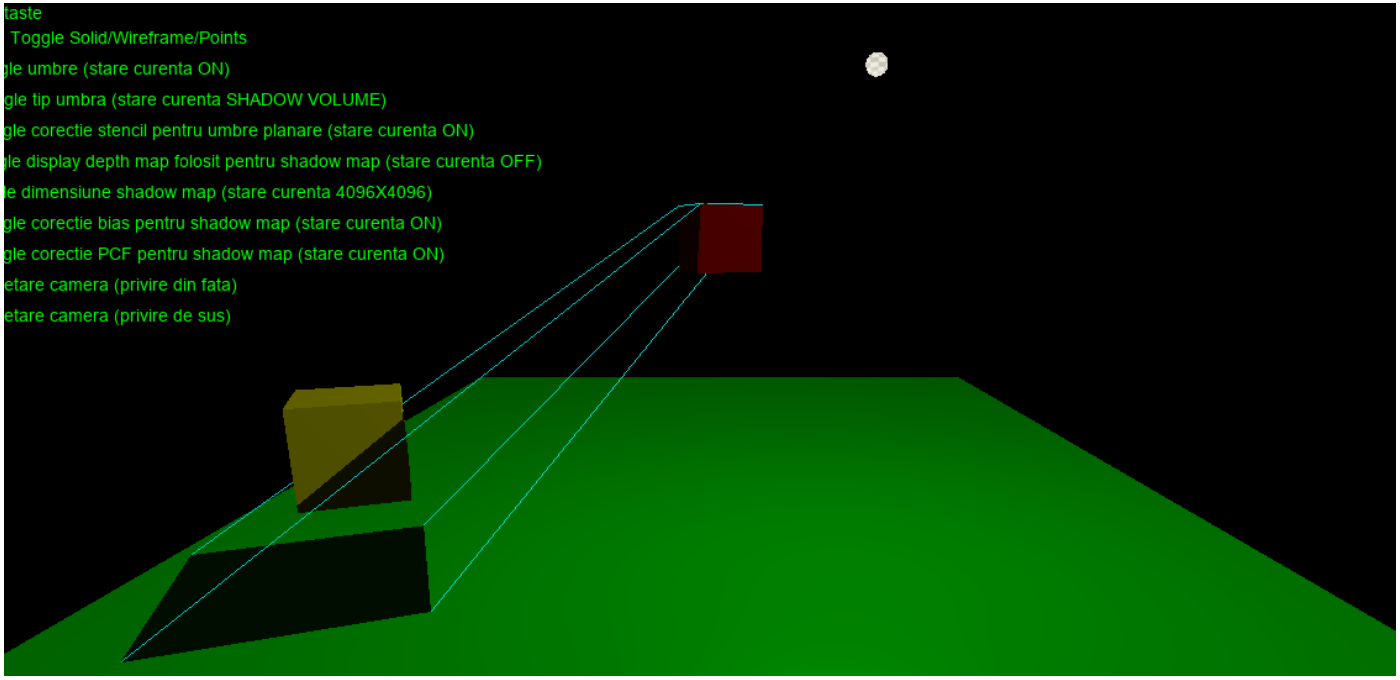
- Sursa de lumină este considerată punctiformă iar scena 3D alcatuita din poligoane.
- Un **volum de umbră** este definit de o sursă de lumină şi un poligon luminat (vizibil din poziţia sursei de lumină), pe care-l vom numi **poligonul generator**.



- Volumul de umbra determinat de o sursă şi un poligon generator este delimitat de o faţă care reprezintă poligonul generator scalat. Această faţă este situată la o distanţă faţă de sursă dincolo de care intensitatea luminii sursei este neglijabilă, deci orice punct aflat dincolo de această limită este umbrit.
- Volumul de umbră poate fi decupat la marginile volumului vizual.

VOLUME DE UMBRA (1)

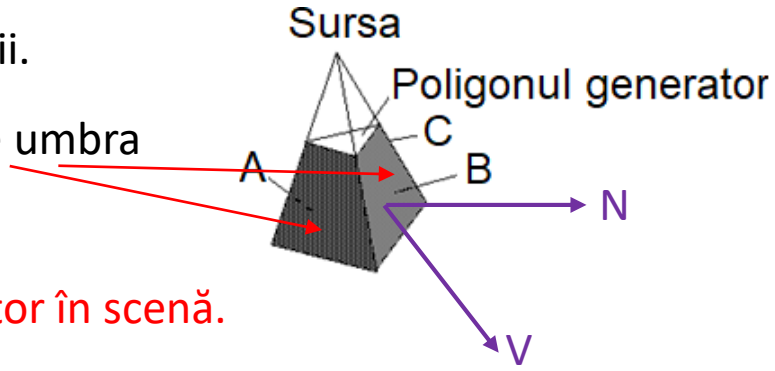




VOLUME DE UMBRA (2)

Determinarea umbririi produse in scena 3D de un volum de umbra

❑ Feţele laterale ale volumului sunt numite **poligoane de umbră**. Un poligon de umbră este determinat de o latură a poligonului generator şi cele două drepte care pleacă din sursa de lumină, fiecare trecând printr-un vârf al laturii.



❑ Poligoanele de umbră se folosesc pentru determinarea umbririi produse de poligonul generator în scenă.

❑ Normalele la poligoanele de umbra punctează înspre exteriorul volumului.

Notăm cu :

– PUV poligoanele de umbră care sunt vizibile din poziţia observatorului (A şi B în figura) şi cu

– PUN poligoanele de umbră care nu sunt vizibile din poziţia observatorului (poligonul C).

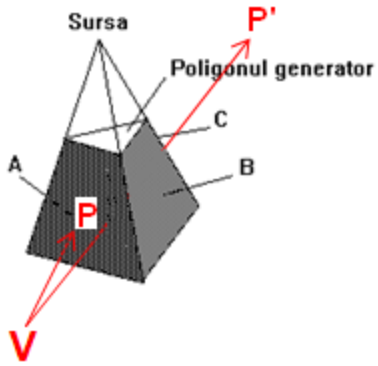
➢ Clasificarea poligoanelor de umbra in PUV si PUN se poate face pe baza normalelor la poligoane si a pozitiei observatorului (back face culling).

VOLUME DE UMBRA (3)

Fie

– un punct P al unei suprafeţe din scena 3D şi

– VP vectorul din poziţia observatorului (V) în punctul P.



Punctul P este umbrit dacă numărul de poligoane de tip PUV intersectate de vectorul VP este mai mare decât numărul de poligoane de tip PUN intersectate de vector.

Acesta este singurul caz, atunci când punctul V nu este în volumul de umbră.

❑ In general, pentru a determina dacă un punct P este în umbră, se poate folosi un contor în care iniţial se memorează numărul de volume de umbră care conţin poziţia observatorului.

– Se asociază poligoanelor de tip PUV valoarea +1 iar celor de tip PUN valoarea -1.

– Atunci când vectorul VP traversează un poligon de umbră, se adună la contor valoarea asociată poligonului.

– Punctul P este umbrit dacă valoarea contorului este pozitivă în P.

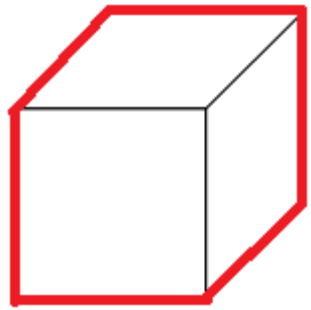
VOLUME DE UMBRA (4)

➢ Volumul de calcule presupus de acest algoritm poate fi redus dacă în loc să se calculeze volumul de umbră pentru fiecare poligon vizibil din poziţia sursei, se calculează un singur volum de umbră pentru o suprafaţă poliedrală.

In acest scop, se determină poligoanele de umbră numai pentru laturile care fac parte din silueta suprafeţei, văzută din poziţia sursei de lumina.

❑ Silueta unei suprafeţe, corespunzătoare unui punct de observare, este un set conectat de laturi care aparţin poligoanelor vizibile din punctul de observare.

❑ O latură de siluetă este fie o latură de margine a unei suprafeţe deschise, fie o latură care separă un poligon vizibil de unul nevizibil.



➢ Pentru determinarea laturilor de siluetă, este necesar să se folosească o structură de date care reflectă adiacenţa poligoanelor. Cunoscându-se poligonul adiacent pe fiecare latură a fiecărui poligon vizibil din poziţia sursei, se pot determina rapid laturile de siluetă.

VOLUME DE UMBRA (5)

Implementarea metodei folosind stencil-buffer(1)

- Buffer-ul de marcaje (stencil-buffer) este prezent pe placile grafice actuale (in memoria GPU), alaturi de buffer-ul imagine (frame-buffer) si buffer-ul de adancime (z-buffer).

– este o memorie de numere intregi asociata pixelilor imaginii, de regula un byte/pixel;

– poate fi utilizat pentru a implementa diferite operatii grafice, de regula in combinatie cu z-buffer, cum este si cazul implementarii volumelor de umbra:

– valorile din stencil-buffer pot fi incrementate/ decrementate automat pentru fiecare fragment care trece/ nu trece testul de adancime.

- Exista mai multe metode de a utiliza stencil-buffer pentru implementarea metodei volumelor de umbre.

VOLUME DE UMBRA (6)

Implementarea metodei folosind stencil-buffer(2)

- In metoda prezentata in continuare, se creaza in stencil-buffer o masca a umbrelor din imagine: orice pixel de umbra, (x,y), va fi reprezentat in stencil-buffer printr-o valoare ≠ 0.

- **Pixelii de umbra sunt afisati in culorile fragmentelor vizibile, calculate cu formula.**

Cfragment = Cemisiva + Cambianta = ke + ka*globalAmbient

ke, ka – proprietati de material

– Se presupune ca s-au calculat poligoanele de umbra.

– Conturul fiecarui poligon de umbra trebuie sa fie orientat in sens trigonometric atunci cand este privit din exteriorul volumului de umbra. In acest fel, poligoanele PUV sunt cele “din faţa” observatorului, iar poligoanele PUN sunt cele “din spate”.

VOLUME DE UMBRA (7)

Implementarea metodei folosind stencil-buffer(3)

Consideram cazul in care observatorul nu este in umbra (cazul uzual).

Pasul 1

Initializeaza z-buffer si stencil-buffer; // nu trebuie programate, acestea sunt
// initializate in mod automat pentru fiecare cadru imagine

Activeaza scrierea in z-buffer si dezactiveaza scrierea in stencil-buffer ; //implicite

Rasterizeaza scena (folosindu-se algoritmul z-buffer) fara lumini

Efectul:

– in frame-buffer se obtine imaginea scenei in umbra (fara lumini). Culoarea fiecarui pixel este:

Cpixel = culoarea emisiva + culoarea ambianta;

La sfarsitul acestui pas, z-buffer contine coordonatele z ale fragmentelor vizibile in imagine.

VOLUME DE UMBRA (8)

Implementarea metodei folosind stencil-buffer(4)

Pasul 2

glDepthMask(GL_FALSE); // Dezactiveaza scrierea in z-buffer

glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE); // Dezactiveaza scrierea in frame-buffer

glFrontFace(GL_CCW);// feţele “din faţă” sunt orientate trigonometric

glCullFace(GL_BACK);//se vor elimina feţele din spate (nu sunt rasterizate poligoanele PUN)

glStencilFunc(GL_ALWAYS, 0x0, 0xff);// se activeaza scrierea in stencil buffer

glStencilOp(GL_KEEP, GL_KEEP, GL_INCR); //se incrementeaza stencil buffer pt toate
// fragmentele care trec testul de adancime

Rasterizeaza poligoanele de umbra

Efectul:

Sunt rasterizate poligoanele de umbra de tip PUV.

Pentru fiecare fragment rezultat, f, se face testul:

daca f.z < z-buffer[f.y][f.x] // fragmentul f in fața fragmentului vizibil in pixelul (x,y)

atunci stencil-buffer[f.y][f.x]++;

// se numara poligoanele PUV aflate intre observator si obiectele scenei

VOLUME DE UMBRA (9)

Implementarea metodei folosind stencil-buffer(5)

Pasul 3

glCullFace(GL_FRONT);//se vor elimina feţele din faţă (nu sunt rasterizate poligoanele PUV)

glStencilOp(GL_KEEP, GL_KEEP, GL_DECR); //se decrementeaza stencil buffer pt toate
// fragmentele care trec testul de adancime

Rasterizeaza poligoanele de umbra

Efectul:

Sunt rasterizate poligoanele de umbra de tip PUN.

Pentru fiecare fragment rezultat, f, se face testul:

daca f.z < z-buffer[f.y, f.x] // f este in faţa fragmentului vizibil in pixelul (x,y)

atunci stencil-buffer[f.y, f.x]--; // se numara poligoanele PUN aflate intre observator
// si obiectele scenei

In stencil-buffer s-a obtinut o masca cu “gauri” in punctele care primesc lumina:

valorile din stencil-buffer sunt diferite de zero pentru pixelii care trebuie afisati in umbra.

VOLUME DE UMBRA (10)

Implementarea metodei folosind stencil-buffer(6)

Pasul 4

```
glEnable(GL_STENCIL_TEST);// Activeaza testul stencil
glStencilOp(GL_KEEP, GL_KEEP, KEEP); // Dezactiveaza scrierea in stencil-buffer
glStencilFunc(GL_EQUAL, 0x0, 0xff); // testul stencil: pixel(x,y) se moifica daca stencil(x,y)=0
glDepthMask(GL_TRUE); // Activeaza scrierea in z-buffer
glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE); // Activeaza scrierea in frame
                                                 // buffer
```

Rasterizeaza scena (folosindu-se algoritmul zbuffer) cu lumini

Efectul:

Se rasterizeaza scena modificandu-se in frame-buffer numai culorile pixelilor (x,y) pentru care stencil-buffer [y][x] =0 (fragmentul nu este in umbra)

VOLUME DE UMBRA (11)

Implementarea metodei folosind stencil-buffer(7)

Cazul general:

Se initializeaza stencil-buffer cu o valoare reprezentand numarul de volume de umbra care contin pozitia observatorului.

Aprecieri asupra metodei volumelor de umbra

Pozitive:

- Este generală şi poate fi aplicată pentru orice scena, pentru oricate surse de lumina.
- Produce forme de umbre exacte.

Negative:

- Complexa din punct de vedere al timpului de calcul:

  se rasterizeaza scena de 2 ori si poligoanele de umbra o singura data dar in 2 pasi!

- Sursele de lumina sunt considerate punctiforme, de aceea nu pot fi obtinute penumbre.

Redarea umbrelor în imagini - cont

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Shadow mapping (1)

Williams [WiLL78] a propus o metodă de generare a umbrelor bazată pe rasterizarea de 2 ori a scenei, cu execuţia algoritmului Z-Buffer:

- In prima etapă se construieşte buffer-ul Z al scenei văzută din poziţia sursei de lumina (în transformarea de vizualizare a scenei – funcția lookAt – pozitia observatorului este pozitia sursei).

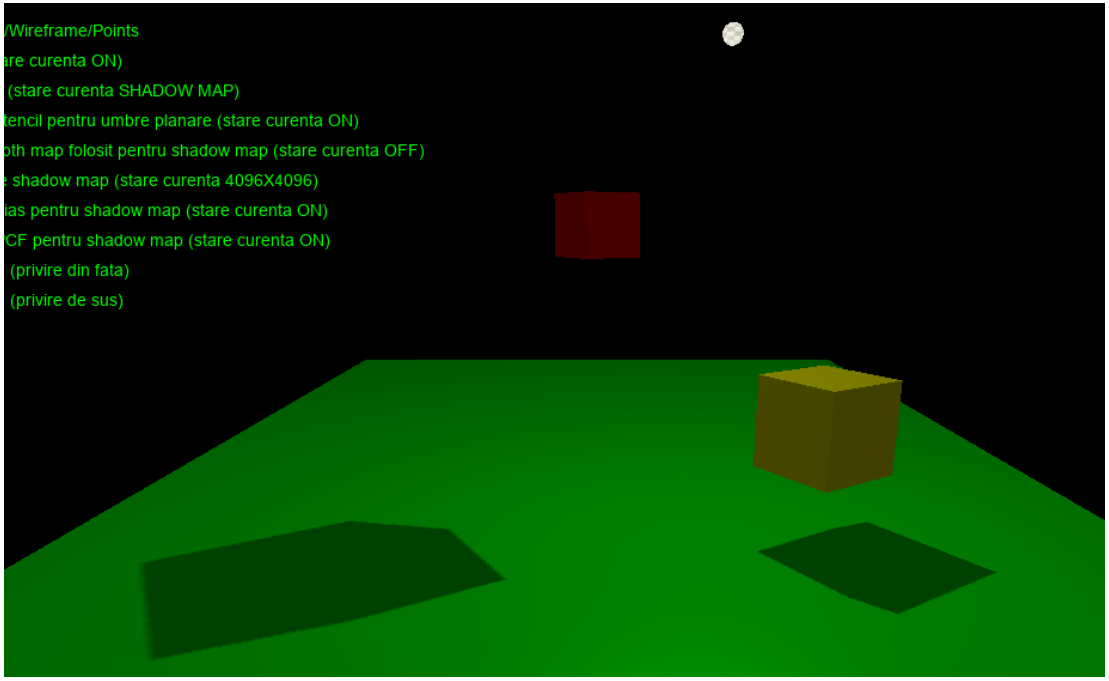
  Vom nota acest buffer cu ZS. El contine coordonatele z ale fragmentelor care primesc lumina de la sursa. Mai este numit si « **shadow map** », de unde si denumirea sub care este cunoscuta metoda in prezent.

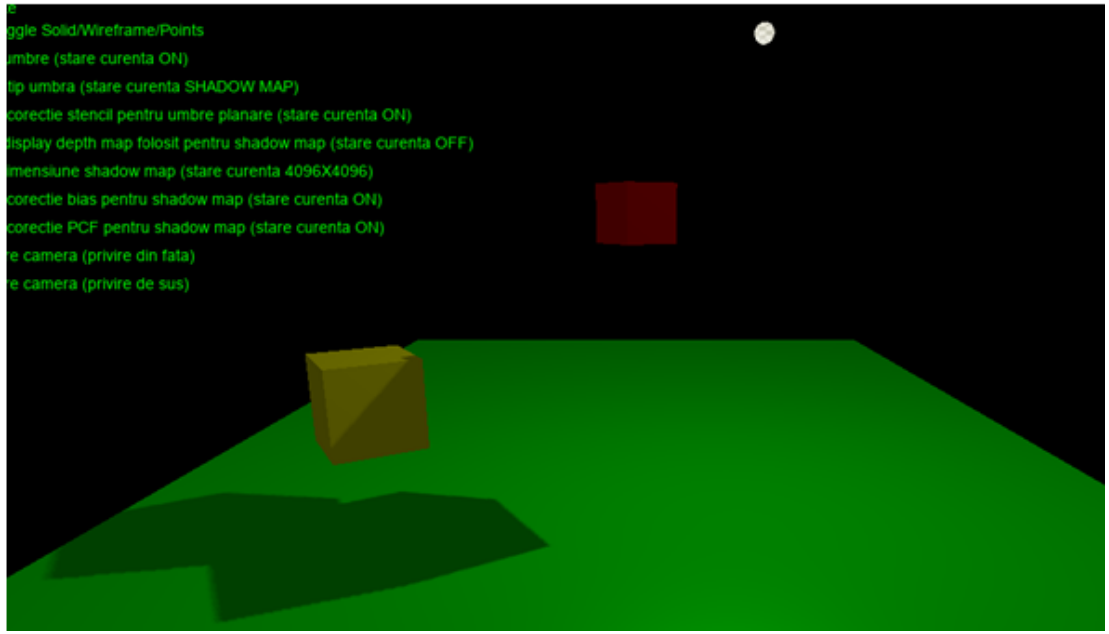
- In etapa a doua se construieşte imaginea văzută din poziţia observatorului, utilizand buffer-ul ZS pentru determinarea punctelor aflate in umbra.

In cazul mai multor surse de lumină, se utilizează câte un buffer ZS pentru fiecare sursă.

Implementarea moderna a metodei este numita “shadow mapping”.

Shadow mapping (2)





Shadow mapping (3)

Etapa I.

Se calculeaza matricea de transformare a varfurilor, MVPS, cu observatorul in pozitia sursei de lumina

Se transmite MVPS la vertex shader (ca variabila uniforma).

Se cere folosirea unui buffer ZS ca buffer de adancime, in locul buffer-ului Z implicit
(In OpenGL, ZS este implementat ca un buffer textura)

Se cere rasterizarea scenei fara scriere in frame-buffer:

1. In vertex shader se transforma varfurile primitivelor folosind MVPS
2. Se rasterizeaza fiecare primitiva a scenei, cu testul de adancime:

pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive

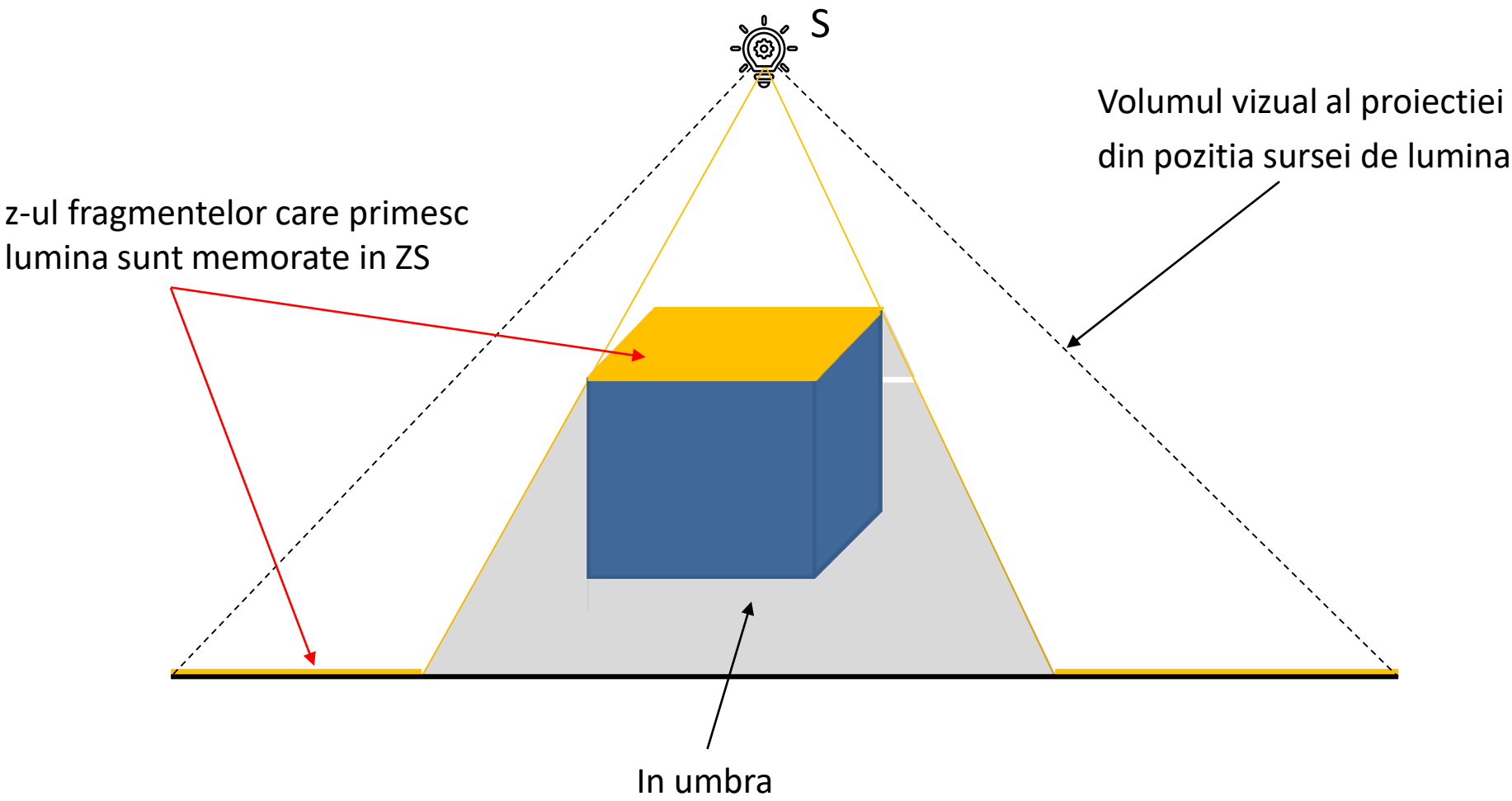
dacă z < ZS[y][x] atunci

ZS[y][x] = z //fragmentul este vizibil în pixelul (x,y) din poziţia sursei

La sfarsitul acestei etape, in ZS sunt memorate coordonatele z ale fragmentelor care primesc lumina de la sursa.

Shadow mapping (4)

Etapa I



Shadow mapping (5)

Etapa II.

Se calculeaza matricea de transformare a varfurilor MVPO, cu observatorul in pozitia sa.

Se transmit la vertex shader (ca variabile uniforme) MVPO si MVPS.

Se cere rasterizarea scenei:

1. In vertex shader se calculeaza varful transformat cu MVPO (gl_Position) si MVPS (poz_sursa)

poz_sursa - pozitia varfului transformata cu MVPS este variabila de iesire din vertex shader

2. Se rasterizeaza fiecare primitiva a scenei, cu testul de adancime:

pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive

dacă z < Z-Buffer[y][x] atunci //fragment vizibil în pixelul (x,y), din poziţia observatorului

{Z-Buffer[y][x] = z; frame-Buffer[y][x] = Culoare fragment (întoarsa de fragment shader);}

In fragment shader

poz_sursa - variabilă de intrare pt fragm shader (pozitie fragment in vederea din pozitia sursei)

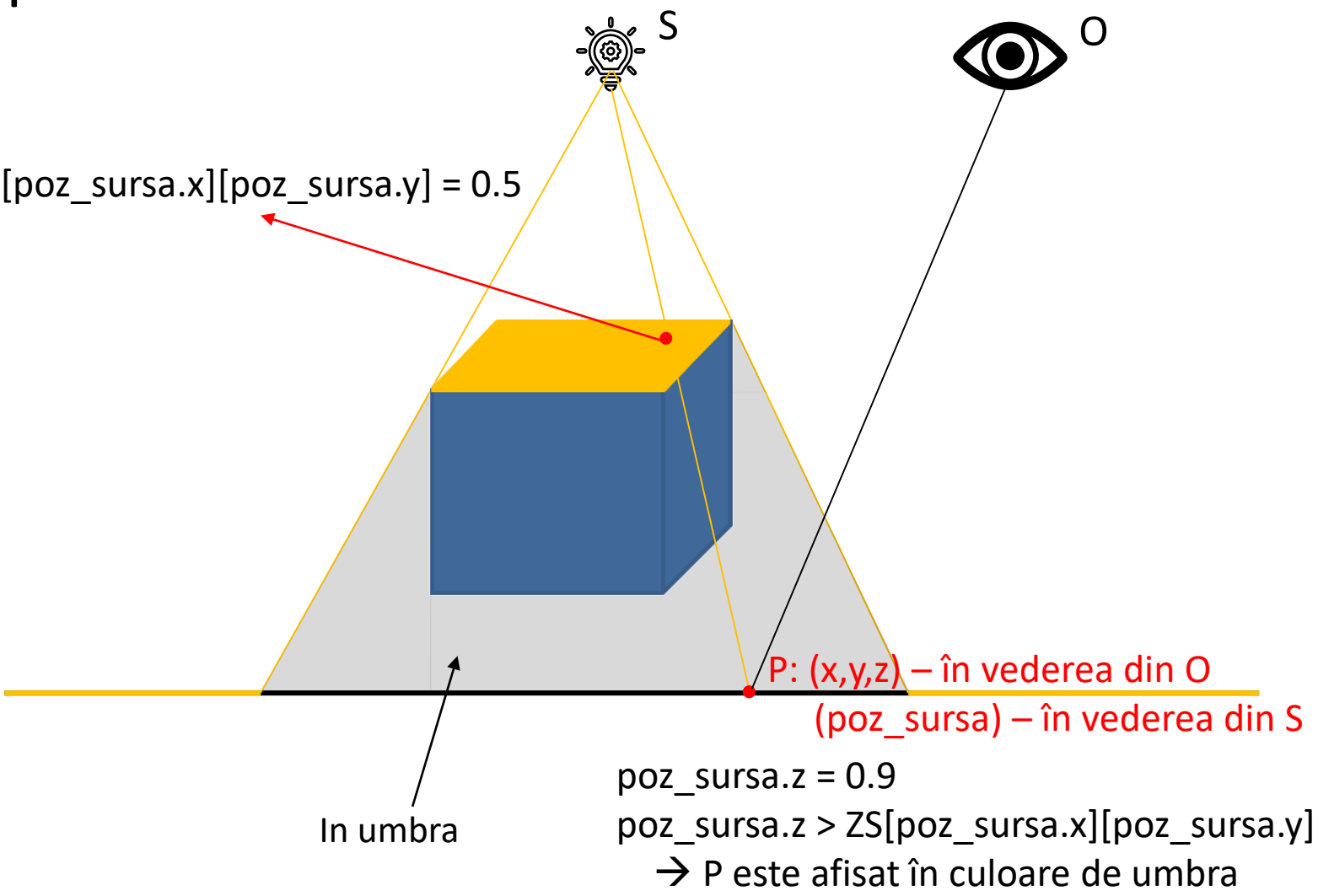
Culoare_fragment = culoare calculata folosind modelul de iluminare locală;

dacă poz_sursa.z > ZS[poz_sursa.y][poz_sursa.x] atunci //fragmentul f(x,y,z) nu primeste lumina

Culoare_fragment = factor_umbra* Culoare_fragment; (0< factor_umbra<1)

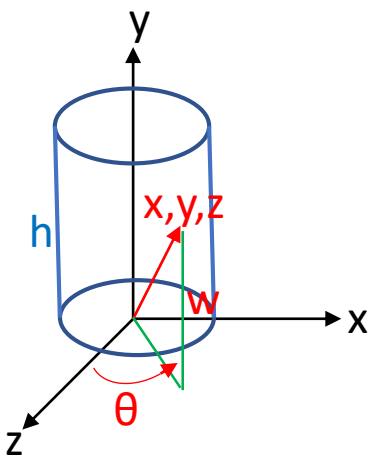
Shadow mapping (6)

Etapa II.



Implementarea maparii texturilor intr-o aplicatie OpenGL

Functiile de mapare standard în sistemul de coordonate OpenGL (1)



Ecuatiile parametrice ale suprafetei cilindrice

x=r*sin(θ)
y= w*h 0<= θ< 2π, 0 < = w <= 1
z= r*cos(θ)

Coordonatele cilindrice (ϑ, w) ale punctului (x, y, z)

θ = arctg(x/z)
w = y/h

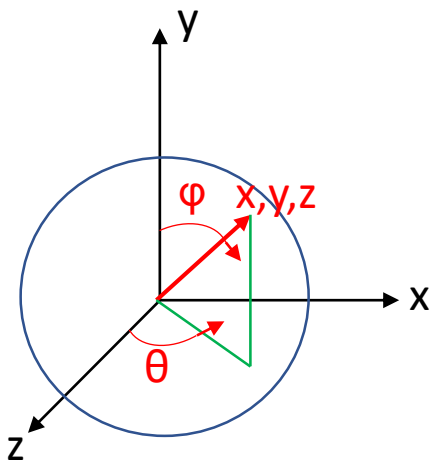
Functiile de mapare cilindrica

u = (1/2π)arctg(x/z)
v = y/h

Daca cilindrul este centrat in originea sistemului de coordonate:

-1/2 < = w <= 1/2
v = w + 1/2
v = y/h + 1/2

Functiile de mapare standard în sistemul de coordonate OpenGL (2)



Ecuatiile parametrice ale sferei:

$x = r*\sin(\theta)*\sin(\varphi)$ $\quad 0 <= \theta < 2\pi$

$y = r*\cos(\varphi)$ $\quad 0 <= \varphi <= \pi$

$z = r*\cos(\theta)*\sin(\varphi)$

- Coordonatele sferice ale punctului (x,y,z):

$\theta = \text{arctg}(x/z)$

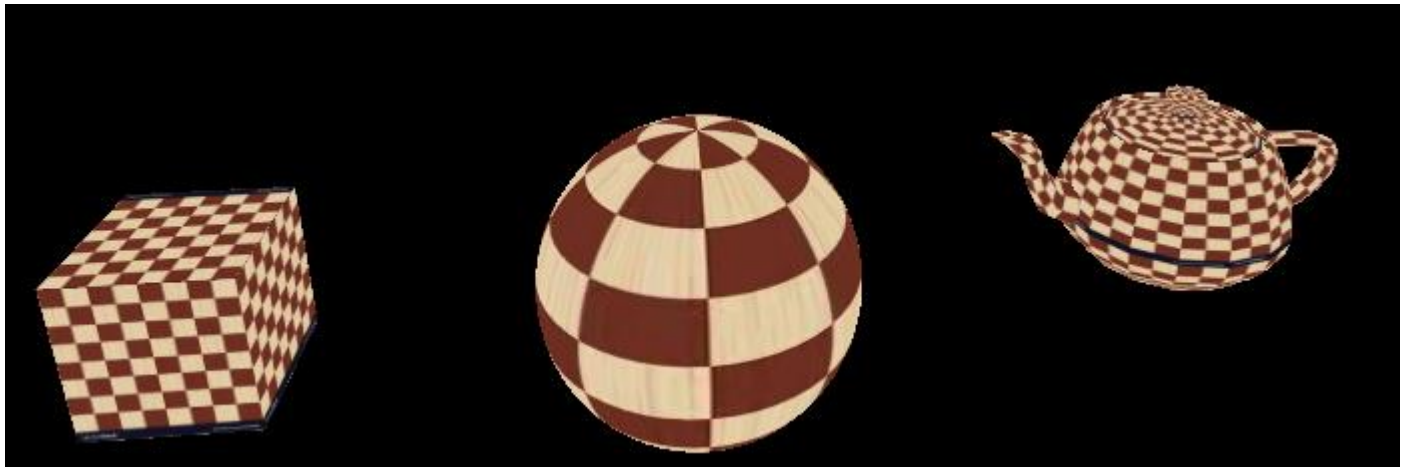
$\varphi = \arccos(y/r)$

Functiile de mapare sferica

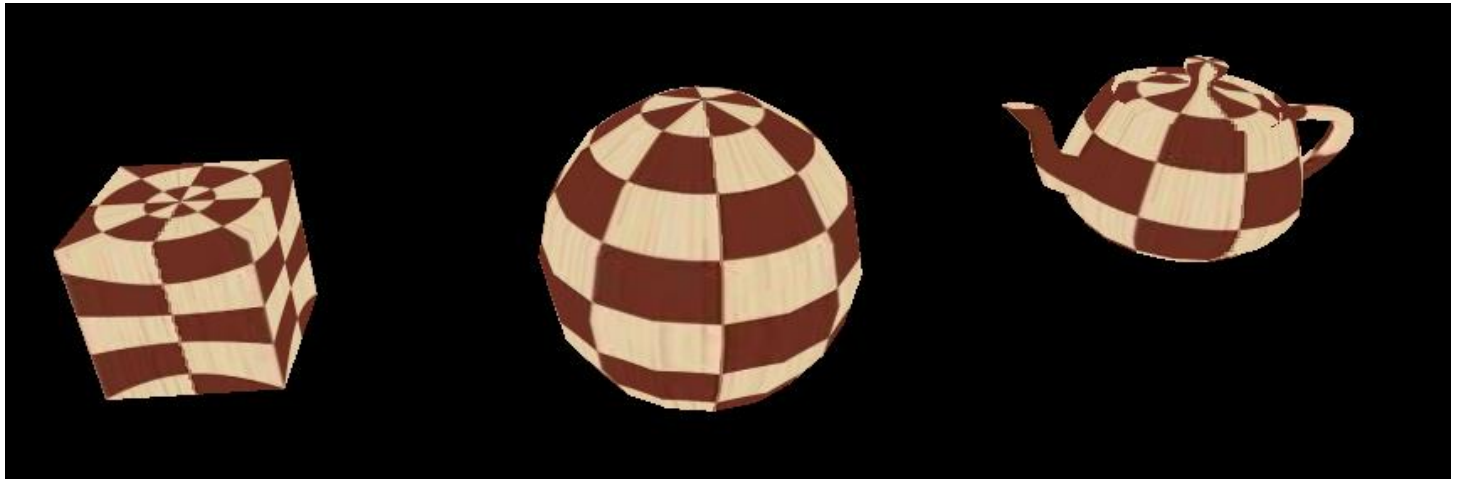
$u = (1/2\pi)*\text{arctg}(x/z)$

$v = (1/\pi)*\arccos(y/r)$

Obiecte importate cu coordonate (u,v) atasate varfurilor si transmise la Vertex shader
Fragment shader utilizeaza coordonate textura interpolate



Mapare sferica pe toate obiectele, calcul (uf,vf) in fragment shader



Mapare sferica, calcul (uf,vf) in fragment shader

Mapare cilindrica, calcul (uf,vf) in fragment shader

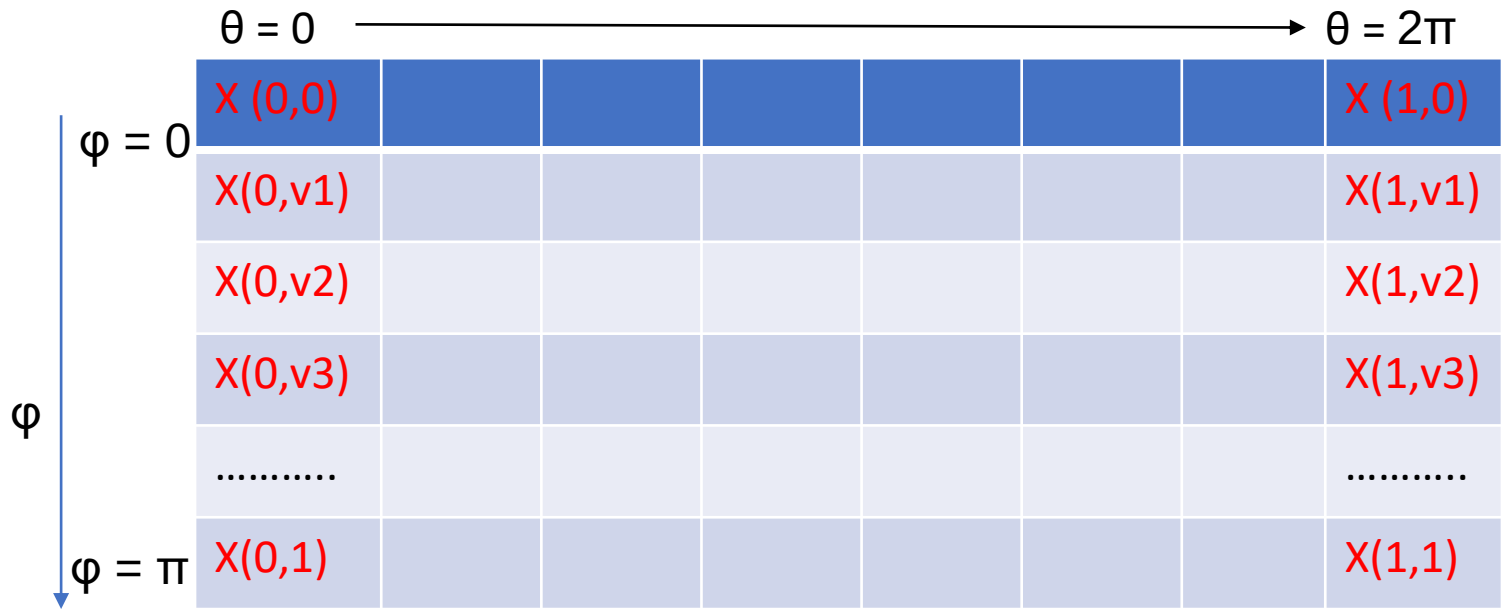
Vertex shader

```
layout(location = 0) in vec3 v_position; //pozitia varfului in spatiul obiect
layout(location = 2) in vec2 v_texture_coord; // coordonate textura varf
uniform mat4 Model;
uniform mat4 View;
uniform mat4 Projection;
uniform int mapType;
out vec3 position; //pozitia varfului in spatiul obiect – pt calcul (u,v) in fragment shader
out vec2 texcoord2;//pentru interpolare → in frag. shader se folosesc coordonate interpolate
void main()
{
   position = v_position;

   if (mapType == 5) {
        texcoord2 = v_texture_coord;
    }
    gl_Position = Projection * View * Model * vec4(v_position, 1.0);
}
```

Fragment shader

```
uniform sampler2D texture_1;
uniform int mapType;
in vec3 position; //pozitia fragmentului in spatiul obiect, pt calcul (uf,vf)
in vec2 texcoord2; // coordonate textura interpolate de rasterizator
layout(location = 0) out vec4 out_color;

void main()
{  vec2 texcoord;
   if (mapType == 1)  //mapare sferica
   {  texcoord.x = (1.0 / (2 * 3.14)) * atan (position.x, position.z);
      texcoord.y = (1.0 / 3.14) * acos (position.y / length (position));
    }
   if (mapType == 2) // mapare cilindrica
   { texcoord.x = (1.0 / (2 * 3.14)) * atan (position.x, position.z);
     texcoord.y = position.y / 2 + 0.5; //cilindru cu inaltimea 2, centrat in origine
    }
    // coordonate textura interpolate
    if ( mapType == 5) {
      texcoord = texcoord2;
    }
    vec3 color = texture2D (texture_1, texcoord).xyz;
    out_color = vec4(color, 1);
}
```

Calculul coordonatelor varfurilor de pe o sfera şi a coordonatelor textura asociate



Coordonatele varfurilor de pe ultima coloana sunt identice cu cele de pe prima coloana.

Varfurile de pe prima coloana au coordonatele textura (0,vi), vi = i*dv

Varfurile de pe ultima coloana au coordonatele textura (1,vi).

Redarea suprafetelor 3D folosind texturi - 1

Prof. univ. dr. ing. Florica Moldoveanu

Texturi (1)

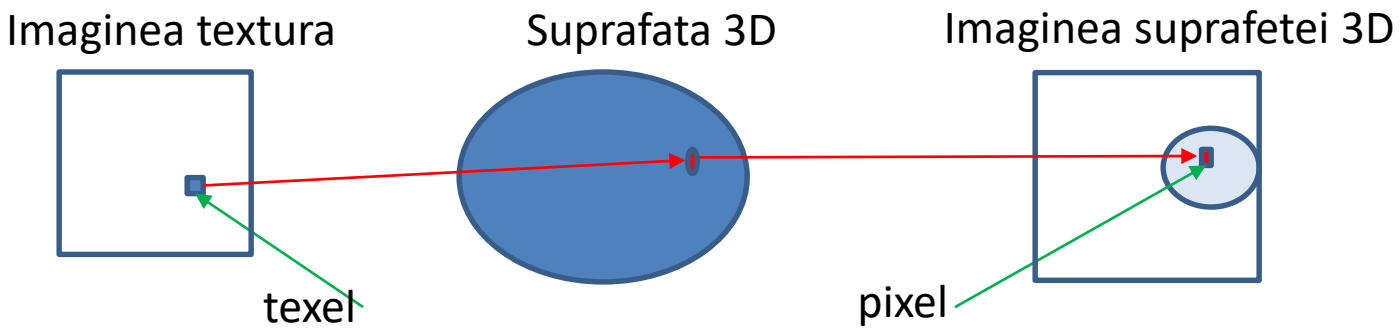
- Se folosesc in diferite scopuri:
  - Redarea unei imagini pe o suprafata

  - Simularea reflexiei/refractiei mediului inconjurator de catre o suprafata
  - Simularea detaliilor geometrice ale unei suprafete
  - Simularea suprafetelor aflate la distanta mare de observator

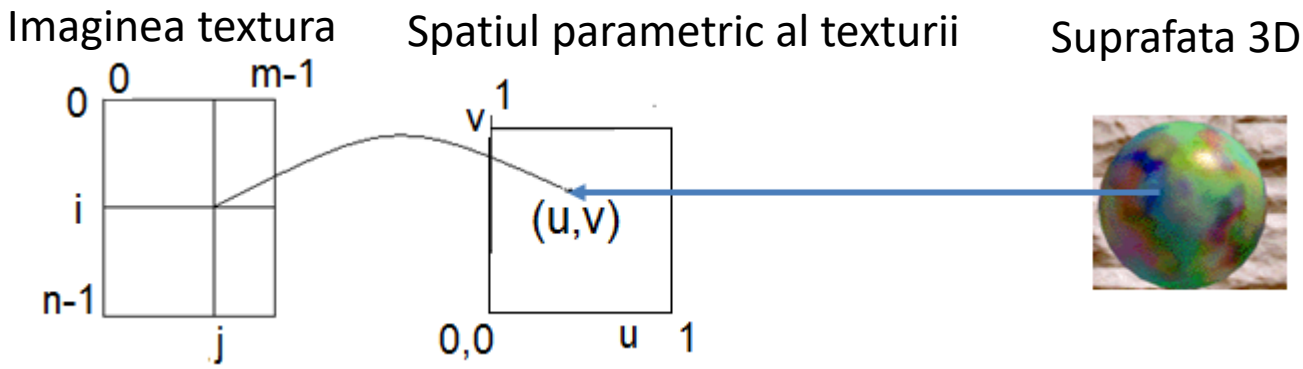
    s.a.

Aplicarea unei imagini textura pe o suprafata 3D

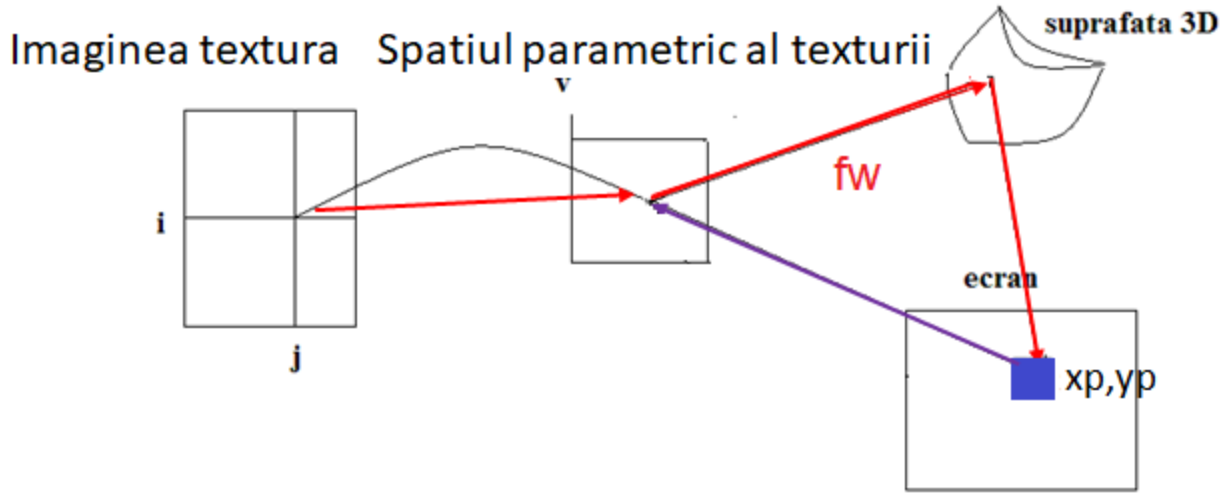
- Elementele imaginii textura sunt culori si se numesc « texeli ».



- Corespondenta dintre punctele suprafetei 3D si spatiul discret al texturii este realizata prin intermediul unui spatiu continuu, (u,v), numit *spatiul parametric al texturii,* 0 <= u,v =< 1



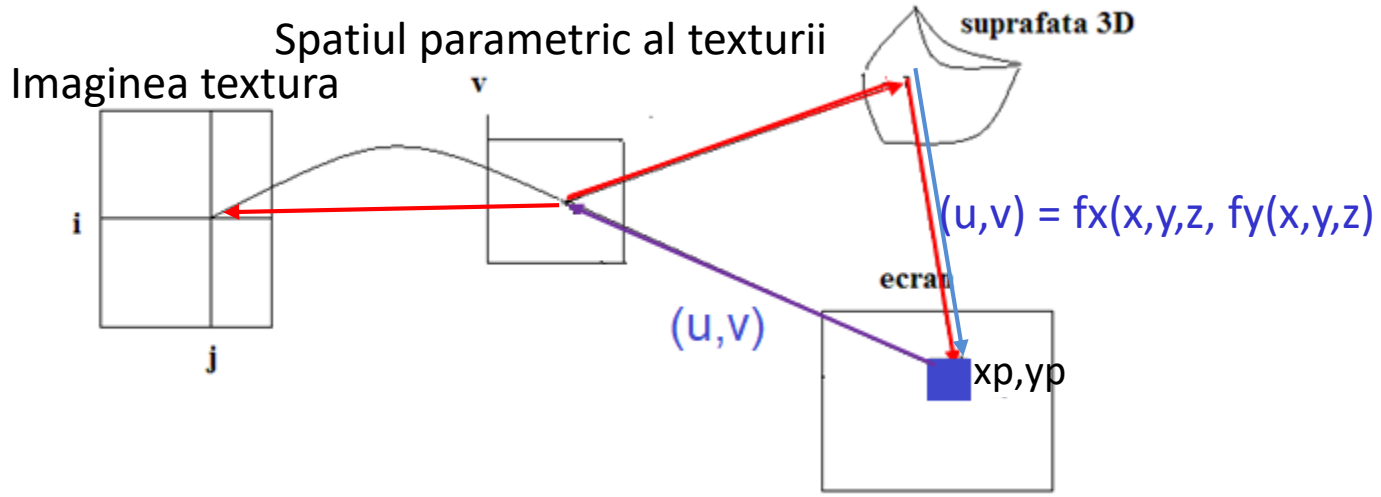
Maparea directa (forward mapping)



Se parcurge imaginea textura linie cu linie si se calculeaza pentru fiecare texel pixelul in care se afiseaza:

- adresa texel, (i,j) → adresa (u,v) → punct de pe suprafaţa 3D (x,y,z) → adresa pixel, (xp, yp)
- Se afiseaza pixelul (xp, yp) in culoarea texelului (i, j)
- Metoda nu este adecvata hardware-ului grafic actual (in plus, poate produce defecte)

Maparea inversa (inverse mapping)

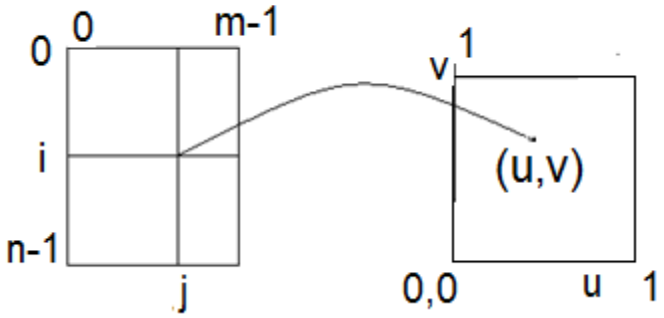


Se calculeaza pentru fiecare fragment rezultat din rasterizarea suprafetei 3D texelul/texelii din care se obtine culoarea textură.

- Pentru fiecare fragment (xp,yp) se calculeaza adresa (u,v) corespunzatoare punctului de pe suprafaţa 3D (x,y,z) care se afisaza în (xp,yp); adresa (u,v) → adresa (i,j) in spatiul discret al texturii de unde se va calcula culoarea fragmentului.
- Afiseaza fragmentul folosind culoarea obtinuta de la adresa (i, j) prin filtrarea texturii.

Obtinerea unei culori din imaginea textura in maparea inversa

Imaginea textura



Accesarea matricei textura din spatiul (u,v)

jr = u * (m-1)

ir = (1 - v) * (n-1)

⇐ (u, v)

(ir, jr) – numere reale

- Accesarea imaginii textura – prin numere intregi

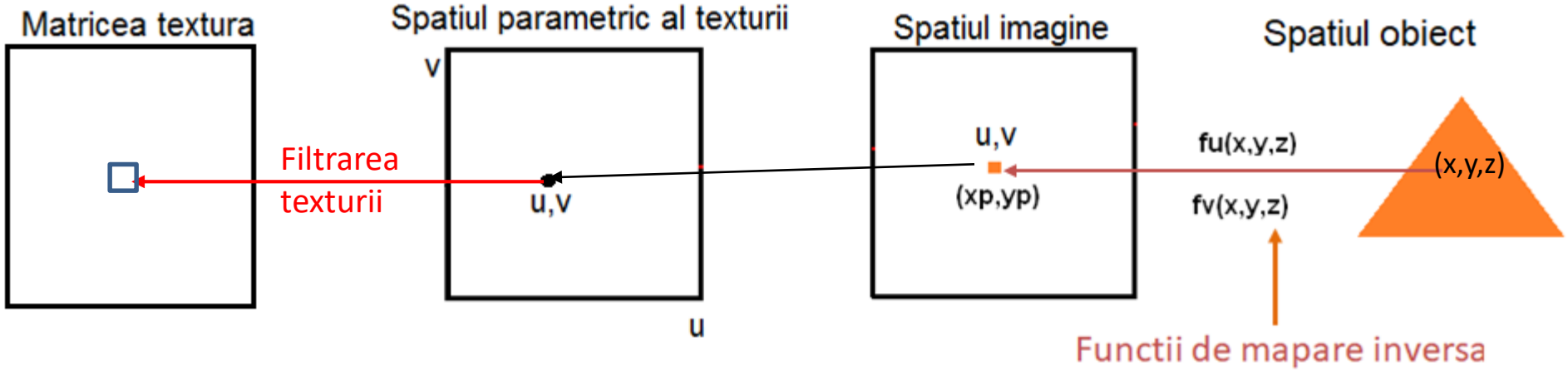
⇓ Filtrarea texturii

2 posibilitati:

1) Culoare = textura (i,j), unde (i,j) adresa cea mai apropiata de (ir,jr) in spatiul discret al texturii; în OpenGL: glTexParameteri(..,.., GL_NEAREST)
2) Culoare = interpolare_biliniara (textura, ir, jr); în OpenGL: glTexParameteri(..,.., GL_LINEAR)

Maparea inversa

- Este metoda uzuala de mapare a texturilor pe suprafetele 3D in banda grafica actuala.
- Se pleaca de la spatiul imagine, aplicarea texturii fiind inglobata în calculul culorii fragmentelor.
- Fiecare fragment rezultat din rasterizarea suprafatei 3D pe care se aplica o textură are asociata o adresa (u,v), obtinuta cu ajutorul unor functii de mapare inversa.
- Pe baza adresei (u,v) a fragmentului se extrage din imaginea textura o culoare, prin operatia de filtrare a texturii.



Functii de mapare inversa

Definesc o corespondenta intre punctele unei suprafete 3D si punctele spatiului (u,v) :

$u = f_u(x,y,z)$; $v = f_v(x,y,z)$

In general, aceasta corespondenta se stabileste pentru varfurile primitivelor rasterizate, adresele (uf,vf) pentru fragmentele rezultate din rasterizare obtinandu-se prin interpolare intre adresele (u,v) asociate varfurilor.

$f_u(x,y,z)$, $f_v(x,y,z)$ Se numesc **functii de mapare inversa**

- Exista functii de mapare inversa numai pentru suprafeţe definite intr-un spatiu parametric, (s,t) a.î. se poate stabili o corespondenta intre spatiul parametric al suprafetei si spatiul parametric al texturii: **(s,t) → (u,v)**
- Ele se numesc **funcţii de mapare standard.**

Functii de mapare standard(1)

1. Maparea plana (maparea pe un plan)

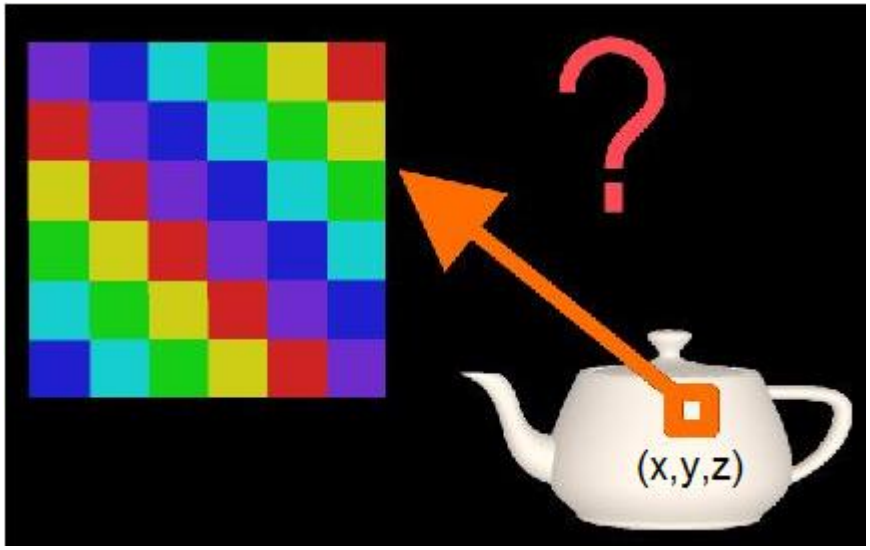
Ecuatiile care descriu aceasta mapare sunt:

$u = su*x - ou$ $\quad x \in X,\ y \in Y$

$v = sv*y - ov$ $\quad 0 <= u, v <= 1$

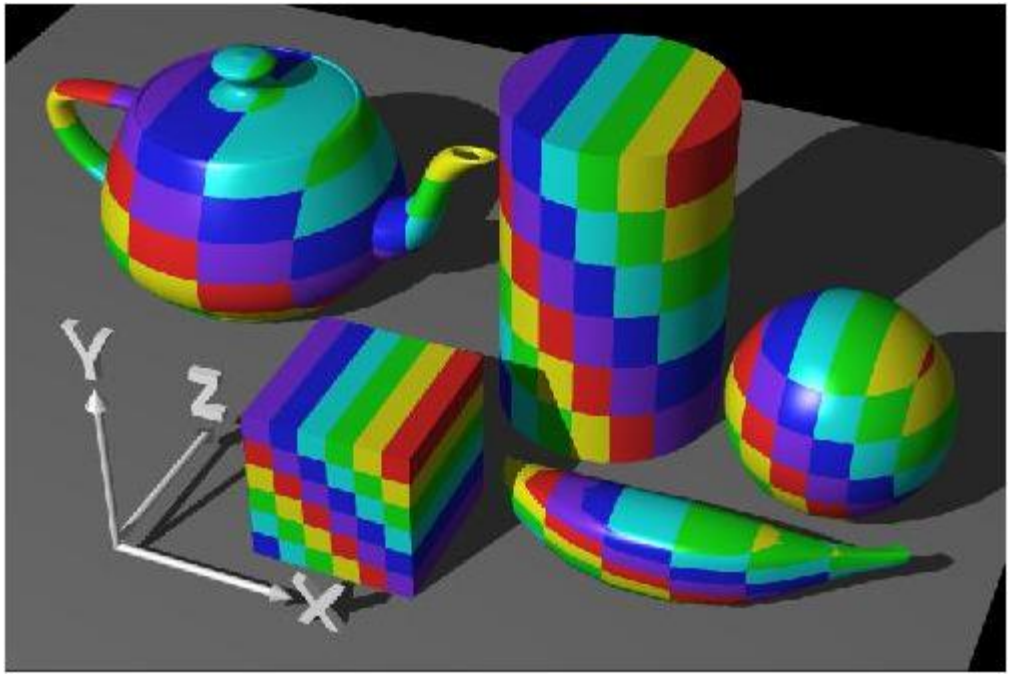
unde : su=1/X, sv=1/Y sunt factorii de scalare pentru transformarea spatiului dreptunghiular [X, Y] in [0,1] iar (ou,ov) este originea texturii: determina modul în care se aşază textura pe suprafaţă.

(u, v) ← (x, y) ← (x, y, z) - se renunta la coordonata z: suprafata 3D devine o suprafata plana



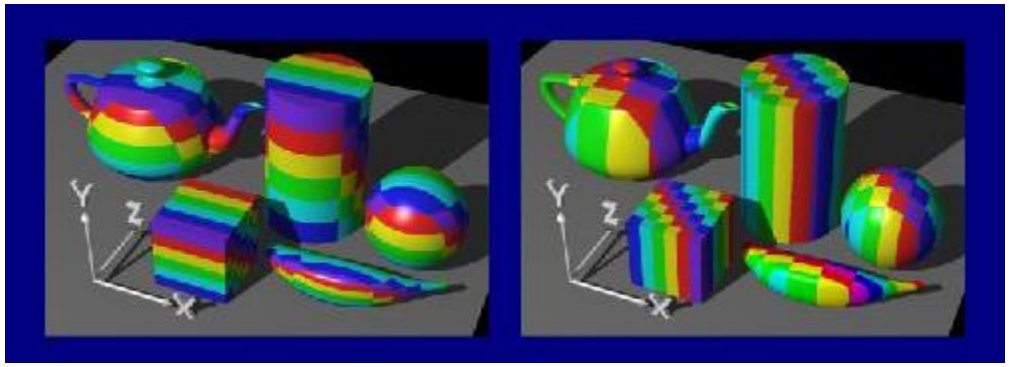
http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

Exemple de mapari plane



Maparea plana prin renuntarea la coordonata z: toate punctele suprafetei cu aceleasi coordonate (x,y) au aceeasi culoare.

http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

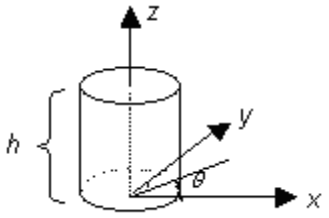


Maparea plana prin renuntarea la coordonata x, respectiv y.

Functii de mapare standard(2)

2. Maparea cilindrica

- Stabileste corespondenta dintre coordonatele (x,y,z) ale unui punct de pe o suprafata cilindrica si spatiul parametric al texturii, (u, v)



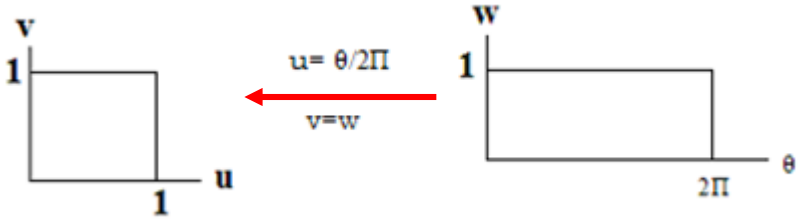
Ecuatiile parametrice ale unei suprafete cilindrice:

x=r*cos(θ)
y=r*sin(θ) 0<= θ< 2π, 0 < = w <= 1
z=w*h

- Coordonatele cilindrice (*θ, w*) ale punctului (x, y, z)

θ = arctg(y/x)
w = z/h

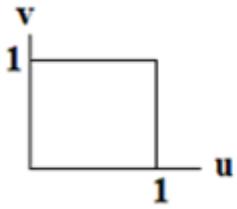
-Se pune in corespondenţă spatiul parametric al texturii, (u,v), spatiului parametric al suprafetei cilindrice:



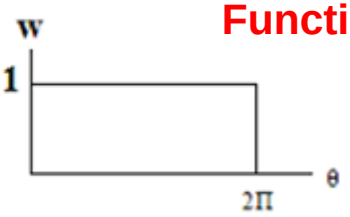
Functii de mapare standard (3)

Functiile de mapare cilindrica

$\theta = arctg(y/x)$
$w = z/h$



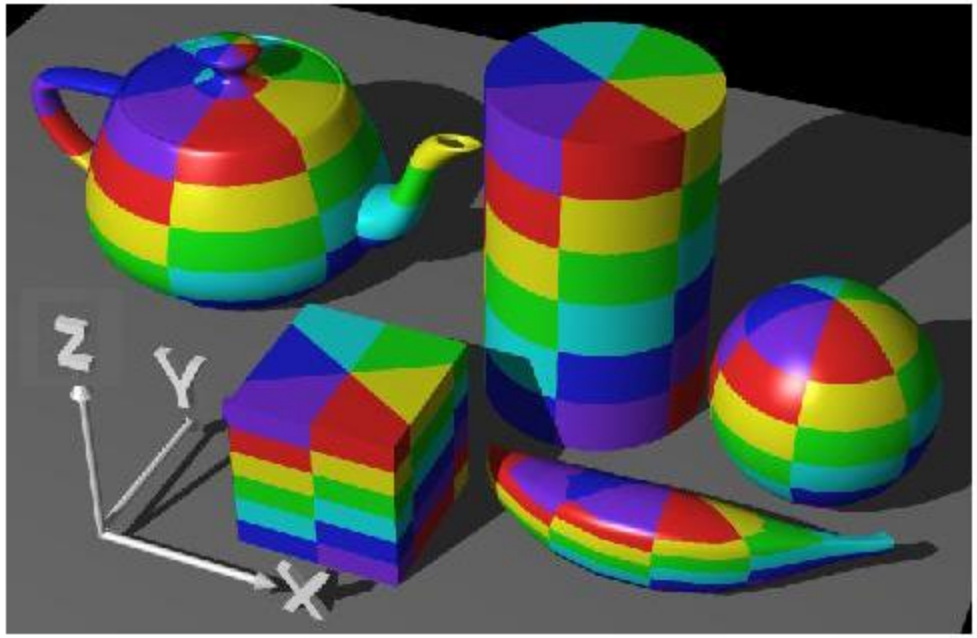
$u = \theta/2\Pi$; $v=w$



$u = (1/2\pi)arctg(y/x)$
$v = z/h$

Textura este “înfaşurată” în jurul obiectului

Transformarea patratelor texturii in punctele suprafetei de z= zmin si z= zmax



Functii de mapare standard(4)

3. Maparea sferica

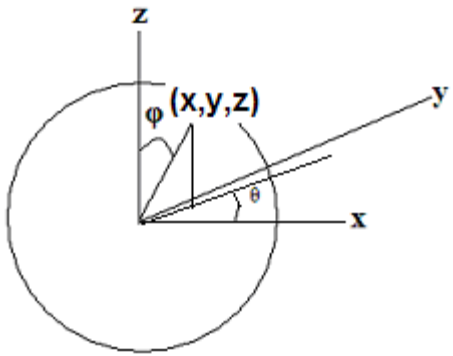
❖ Stabileste corespondenta dintre coordonatele (x,y,z) ale unui punct de pe o suprafata sferica si spatiul parametric al texturii, (u, v)

- Ecuatiile parametrice ale sferei:

$x = r*\cos(\theta)*\sin(\varphi) \quad 0 <= \theta < 2\pi$

$y = r*\sin(\theta)*\sin(\varphi) \quad 0 <= \varphi <= \pi$

$z = r*\cos(\varphi)$



- Coordonatele sferice ale punctului (x,y,z):

$\theta = \text{arctg}(y/x)$

$\varphi = \arccos(z/r)$

Punand in corespondenta spatiul (u,v) spatiului parametric al sferei rezulta:

$u = \theta/2\pi$

$v = \varphi/\pi$

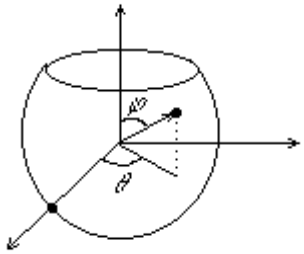
sau

$$\begin{cases} u = \dfrac{1}{2\pi} arctg\left(\dfrac{y}{x}\right) \\ v = \dfrac{1}{\pi}\arccos\left(\dfrac{z}{r}\right) = \dfrac{1}{\pi}\arccos\dfrac{z}{\sqrt{x^2+y^2+z^2}} \end{cases}$$

Functiile de mapare sferica

Functii de mapare standard (5)

Maparea pe un sector de sfera



$0 <= \theta <= \pi/2$

$\pi /4 <= \varphi <= \pi/2$

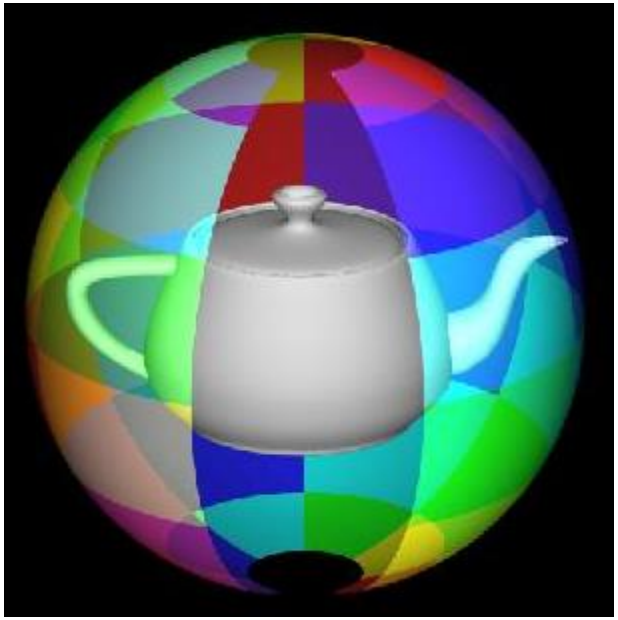
Functiile de mapare

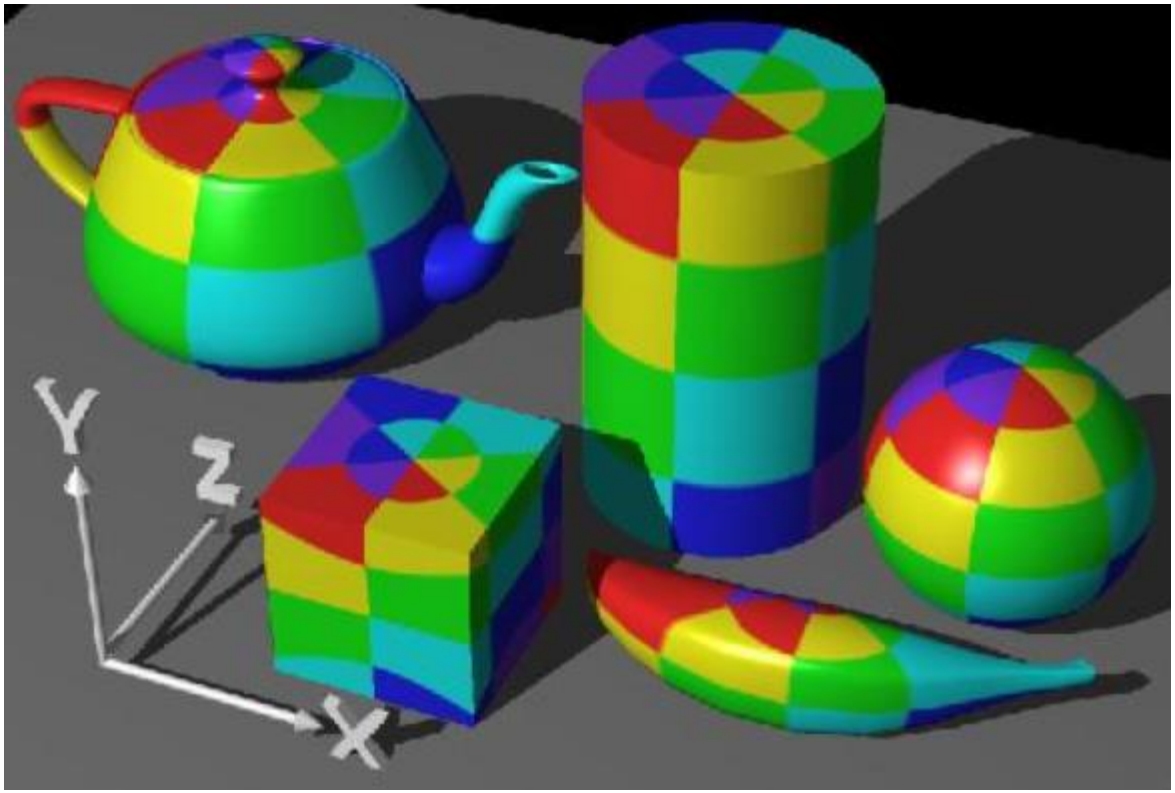
$u = \theta/(\pi/2)$

$v = ((\pi /2) – \varphi) / (\pi /4)$

Exemple de mapari sferice

Maparea sferica “întinde” patratele texturii la nivelul ecuatorului si le “comprimă” proportional cu aproprierea de poli.

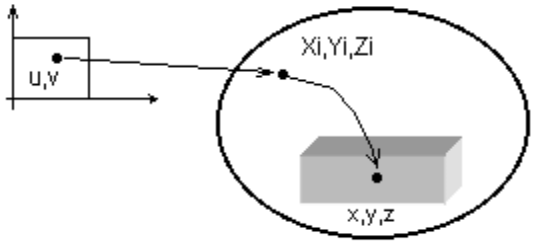




http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

Maparea în 2 pasi (1)

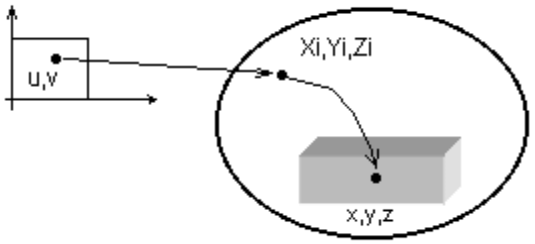
- Este o tehnica de mapare independenta de forma obiectului pe care se aplica textura. Metoda a fost definita de Bier si Sloan (1986).
- Daca pentru suprafata texturata nu exista o functie de mapare inversa, atunci, se stabileste o corespondenta între punctele suprafetei texturate si punctele unei suprafete 3D pentru care exista o functie de mapare inversa, numita in continuare « suprafata intermediara ».
- Aplicarea texturii este descompusa in doua operatii,



al caror efect este :

  - **mularea texturii pe suprafata intermediara;**
  - **mularea suprafetei intermediare pe suprafata de texturat.**

Maparea în 2 pași(2)



a) **maparea S (mapare inversa):**
u = fu(xi, yi, zi)
v = fv(xi, yi, zi)

b) (xi, yi, zi) ← **maparea O** ← (x, y, z)

a) Mularea texturii pe suprafata intermediara se realizeaza prin functia de mapare inversa. Aceasta este numita « **mapare S** », adica mapare pe *suprafata* intermediara.

b) Mularea suprafetei intermediare pe suprafata de texturat este numita « **mapare O** ». Ea trebuie sa puna in corespondenta fiecarui punct (x,y,z) al suprafetei (*obiectului*) de texturat, un punct (xi, yi, zi) al suprafetei intermediare

Maparea în 2 pași(3)

Maparea S

Se folosesc 4 tipuri de suprafete intermediare :

1. un plan cu o orientare oarecare; →maparea S: mapare plana
2. sfera; →maparea S: mapare sferica
3. cilindrul; →maparea S: mapare cilndrica
4. fețele unui cub; →maparea S: mapare plana

Alegerea tipului suprafetei intermediare este dependenta de forma obiectului pe care trebuie sa fie aplicata textura.

Maparea O

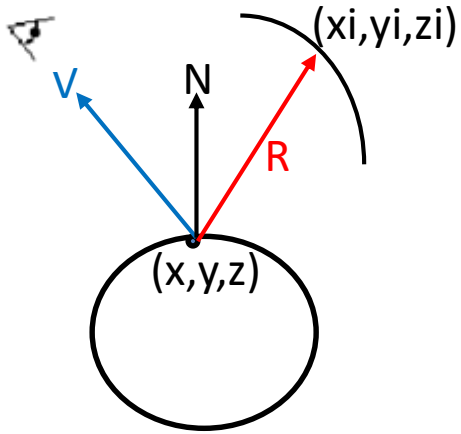
Autorii metodei au definit 4 tipuri de mapari O:

Maparea în 2 pasi(4)

Mapari O

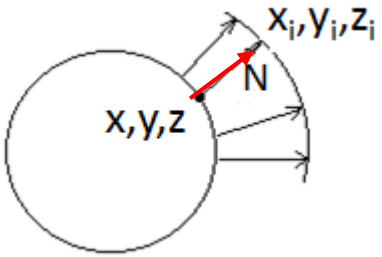
(x,y,z) – un punct al suprafetei , pentru care se doreste calculul coordonatelor (u,v)

(xi, yi, zi) – punctul corespunzator lui (x,y,z) pe suprafata intermediara



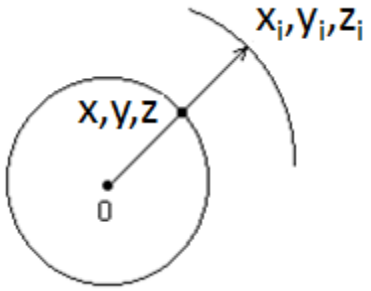
1. (xi, yi, zi), se obtine intersectand suprafata intermediara cu vectorul reflectat al vederii in punctul (x,y,z). R este simetric cu V fata de N.

Metoda este dependenta de pozitia observatorului (se foloseste la « maparea mediului inconjurator »)

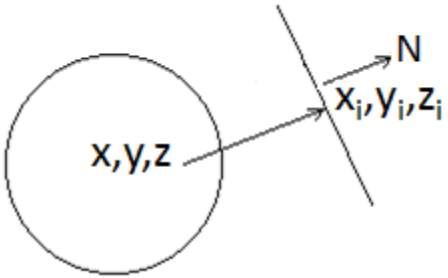


2. (xi,yi,zi) este intersectia suprafetei intermediare cu normala la obiect în (x,y,z).

Maparea în 2 pași(5)



3. (xi,yi,zi) este intersectia suprafetei intermediare cu dreapta care trece prin (x,y,z) si centroidul obiectului.



4. (xi,yi,zi) este intersectia suprafetei intermediare cu dreapta care trece prin (x,y,z) si are directia normalei la suprafata intermediara.

Acest tip de mapare O este corelat cu o mapare S pe un plan sau pe fetele unui cub.

Combinatia « mapare S pe cilindru » - « mapare O de tip 4 » este numita « shrinkwrap ».

Aplicarea texturilor pe fetele unei retele poligonale 3D (1)

In sistemele grafice actuale, orice suprafata 3D este descompusa, in vederea redarii sale, într-o rețea de fațete triunghiulare: textura se aplica pe suprafetele triunghiulare la momentul rasterizarii lor.

Pentru fiecare fragment rezultat din rasterizare se poate obtine o culoare din textură in 2 moduri:

1. In fragment shader, se calculeaza coordonatele (u,v) corespunzatoare adresei fragmentului (xf,yf), folosind functii de mapare standard sau maparea in 2 pasi:

   (u,v) ← functii de mapare (xf,yf)

2. In fragment shader, se utilizeaza coordonatele (uf,vf) calculate de rasterizator prin interpolarea coordonatelor (ui,vi) atasate varfurilor primitivei din care face parte fragmentul.

   - Pentru rezultate corecte este necesara efectuarea unei interpolari perspectiva. Interpolarea liniara este mai rapida dar poate produce defecte.
   - Metode mai exacte (ex. Mip-mapping) calculeaza coordonatele textura ale unui fragment tinand cont de aria suprafetei textură comprimată in fragment.

Aplicarea texturilor pe fetele unei retele poligonale 3D(2)

- In cazul utilizarii mai multor texturi la redarea unei primitive, in fragment shader se extrage cate o culoare din fiecare textura, fie calculand adresele (ui,vi) in fragment shader, fie folosind coordonatele textura interpolate. Culorile extrase din diferite texturi pot fi combinate in diferite moduri pentru a se obtine culoarea fragmentului.

- Daca o imagine textura trebuie mulata pe intreaga retea de fatete care aproximeaza o suprafata, atunci varfurile retelei trebuie sa se mapeze pe intreg spatiul 0 <= (u,v) <= 1.
- Coordonatele (ui,vi) atasate varfurilor unei primitive se pot obtine intr-o aplicatie OpenGL printr-o mapare in 2 pasi.
- Aplicatiile de modelare a suprafetelor 3D permit atasarea de coordonate textura varfurilor suprafetei. Obiectele importate contin coordonatele (u,v) atasate varfurilor obiectului.
- Coordonatele textura ale varfurilor se transmit ca intrari la Vertex shader.

Redarea suprafetelor 3D folosind texturi - 2

Prof. univ. dr. ing. Florica Moldoveanu

Transparenta texturii (1)

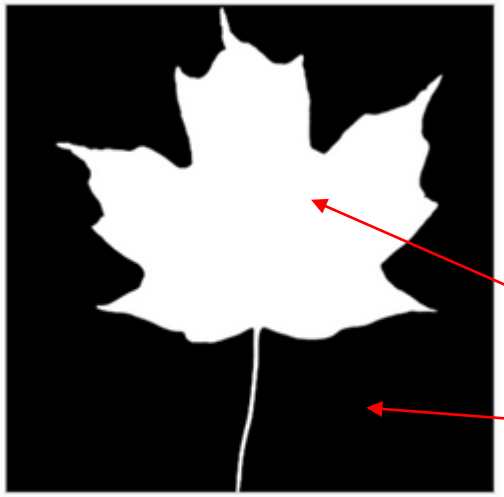
- Imaginea textura poate avea o componenta aditionala, numita “canalul alfa”, prin care se poate modela transparenta texturii.
- Fiecare texel este in acest caz reprezentat prin 4 componente: (R,G,B,A), unde A reprezinta opacitatea.
  - daca A = 1, atunci texelul este complet opac
  - daca A=0, texelul este complet transparent
- Culoarea unui fragment se poate obtine: dintr-o textura, combinand culori din mai multe texturi, combinand culoarea fragmentului calculata pe baza geometriei (modelul Gouraud sau Phong) cu o culoare extrasa dintr-o textura (sau mai multe).
- Atunci cand culoarea fragmentului are opacitatea $0< A < 1$, se poate combina culoarea sa cu aceea a pixelului destinatie, printr-o operatie de “amestec” între cele 2 culori, specificata prin functia glBlendFunc(..) (vezi curs “Transparenta în modelul de iluminare locala”). Detalii la: https://open.gl/textures.

Transparenta texturii (2)

- Atunci cand culoarea fragmentului are opacitatea A=0 sau sub o anumita valoare, se poate cere eliminarea sa (suprascrierea unui pixel cu ceva transparent nu are efect).

```
uniform sampler2D texture1;
in vec2 TexCoords;
out vec4 FragColor;
//Fragment shader
void main() {
    vec4 texColor = texture(texture1, TexCoords);
    if(texColor.a < 0.1) discard; // fragmentul este eliminat – nu va fi procesat in continuare
    FragColor = texColor;
}
```

Transparenta texturii (3)



Imaginea textura

Desenul frunzei este memorat pe canalul alfa al texturii.

Toti texelii texturii au culoarea albastru.

(0,0,1,1) – texeli opaci

(0,0,1,0) – texeli transparenti

Imaginea texturii aplicată pe o suprafață albă.

```
void main() {
  vec4 texColor = texture(texture1, TexCoords);
 // pentru eficiență:
   if(texColor.a == 0) discard;
   FragColor = texColor;
 }
```

Filtrarea texturii (1)

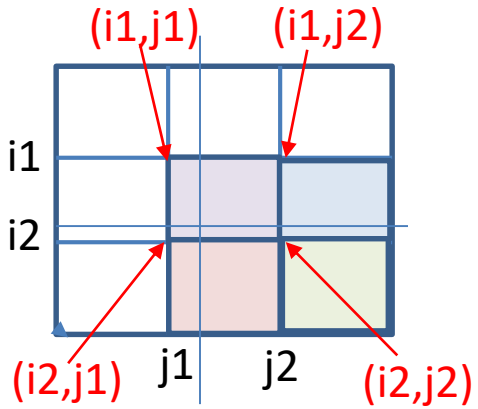
- O textura este definita pe un spatiu discret in timp ce coordonatele textura sunt valori reale. In general, o pereche (uf,vf) nu corespunde unui texel al texturii.
- Calculul culorii dintr-o textura folosind coordonatele (uf,vf), se numeste “filtrarea texturii”.

Cazul 0<= uf, vf <= 1

Fie (iv,ju) coordonatele reale textura corespunzatoare unei adrese (uf,uv).

Sunt doua moduri standard de filtrare:

1. Se considera culoarea texelului cu adresa cea mai apropiata de (iv,ju). In OpenGL: GL_NEAREST.

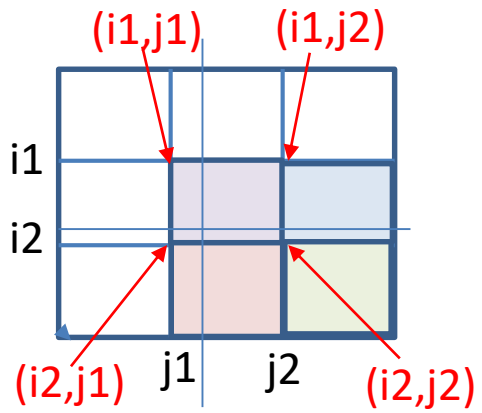


i1 = 12, i2= 13,
j1 = 4, j2 = 5
iv = 12.8, ju = 4.3
Culoare_textura ← textura[i2][j1]

   - rapida dar poate produce defecte in imagine
   - este metoda de filtrare implicita

Filtrarea texturii (2)

2. Se efectueaza o **interpolare biliniara** intre culorile celor mai apropiati texeli din spatiul texturii (în OpenGL: GL_LINEAR):



Culoare_textura ← interpolare intre culorile texelilor de adrese [i1][j1], [i1][j2], [i2][j1], [i2][j2]

- scade viteza de calcul a culorii fragmentului
- poate produce un efect de înceţoşare

- Cele 2 moduri de filtrare se pot specifica separat pentru cele 2 cazuri:
  - Micsorarea imaginii – suprafata pixelului pentru care se calculeaza culoarea se mapeaza pe o zona mai mare decat cea corespunzatoare unui texel
  - Marirea imaginii – suprafata pixelului pentru care se calculeaza culoarea se mapeaza pe o zona mai mica sau egala cu cea corespunzatoare unui texel

glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER/GL_TEXTURE_MIN_FILTER, GL_NEAREST/GL_LINEAR).

Filtrarea texturii (3)

Interpolarea biliniara intre culorile din matricea textura

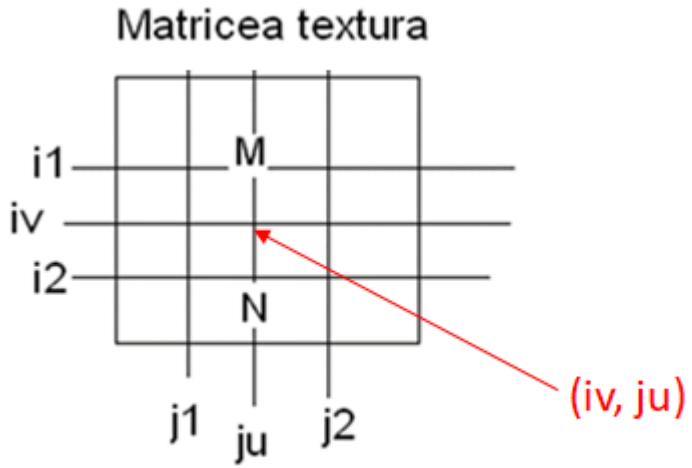
- Daca (ju, iv) sunt adresele reale corespunzatoare valorilor (uf,vf) in spatiul textura,

  iv = (1-vf)*(n-1); ju = uf*(m-1)

  n- numarul de linii ale matricei textura; m – numarul de coloane

  i1< iv < i2, i2 = i1 + 1 j1< ju < j2, j2 = j1 + 1

- Culoarea extrasa din matricea textura se calculeaza astfel:



M(i1,ju); N(i2,ju);

C(i,j) – culoarea texelului de indici (i,j)

Cul(M) = (j2-ju)*C(i1,j1) + (ju-j1)*C(i1,j2)

Cul(N) = (j2-ju)*C(i2,j1) + (ju-j1)*C(i2,j2)

Culoare_textura = (i2-iv)*Cul(M) + (iv-i1)*Cul(N)

i1, i2, j1, j2 sunt indici de linie/coloana in matricea textura

Filtrarea texturii (4)

Cazul uf, vf in afara domeniului [0,1]

- Coordonatele (ui, vi) asociate varfurilor pot fi in afara domeniului [0,1]. Rezulta:

  coordonatele (uf, vf) ale unui fragment pot fi in afara domeniului [0,1].

- Coordonatele (uf, vf) rezultate in procesul de rasterizare sunt transformate în coordonate 0<= u, v <=1.

- Modul de transformare a coordonatelor (u,v) din afara domeniului [0,1] se specifica separat pentru fiecare axa: u/v.

- Exista cateva moduri standard de transformare a coordonatelor textura in domeniul [0,1]. Doua dintre acestea sunt: “clamping” si “wrapping (sau “repeating).

Clamping (Comprimare)

Coordonatele uf/vf sunt transformate in u/ v astfel:

u = min(max(0,uf),1) v = min(max(0,vf),1)

Deci: daca uf / vf <0, atunci u / v =0; daca uf / vf >1, atunci u / v=1

daca 0<= uf, vf <=1, atunci u, v = uf, vf.

Filtrarea texturii (5)

Clamping (Comprimare)

Aplicarea acestei transformari are ca efect propagarea culorii de la marginea texturii de-a lungul directiei in care coordonata textura este < 0 sau > 1.

Filtrarea texturii (6)

Wrapping (Repetare)

- Pentru accesarea texturii se calculeaza (u,v) cu formula:

  u = uf- [uf] v= vf-[vf]

  unde [w] reprezinta cel mai mare intreg mai mic sau egal cu w.

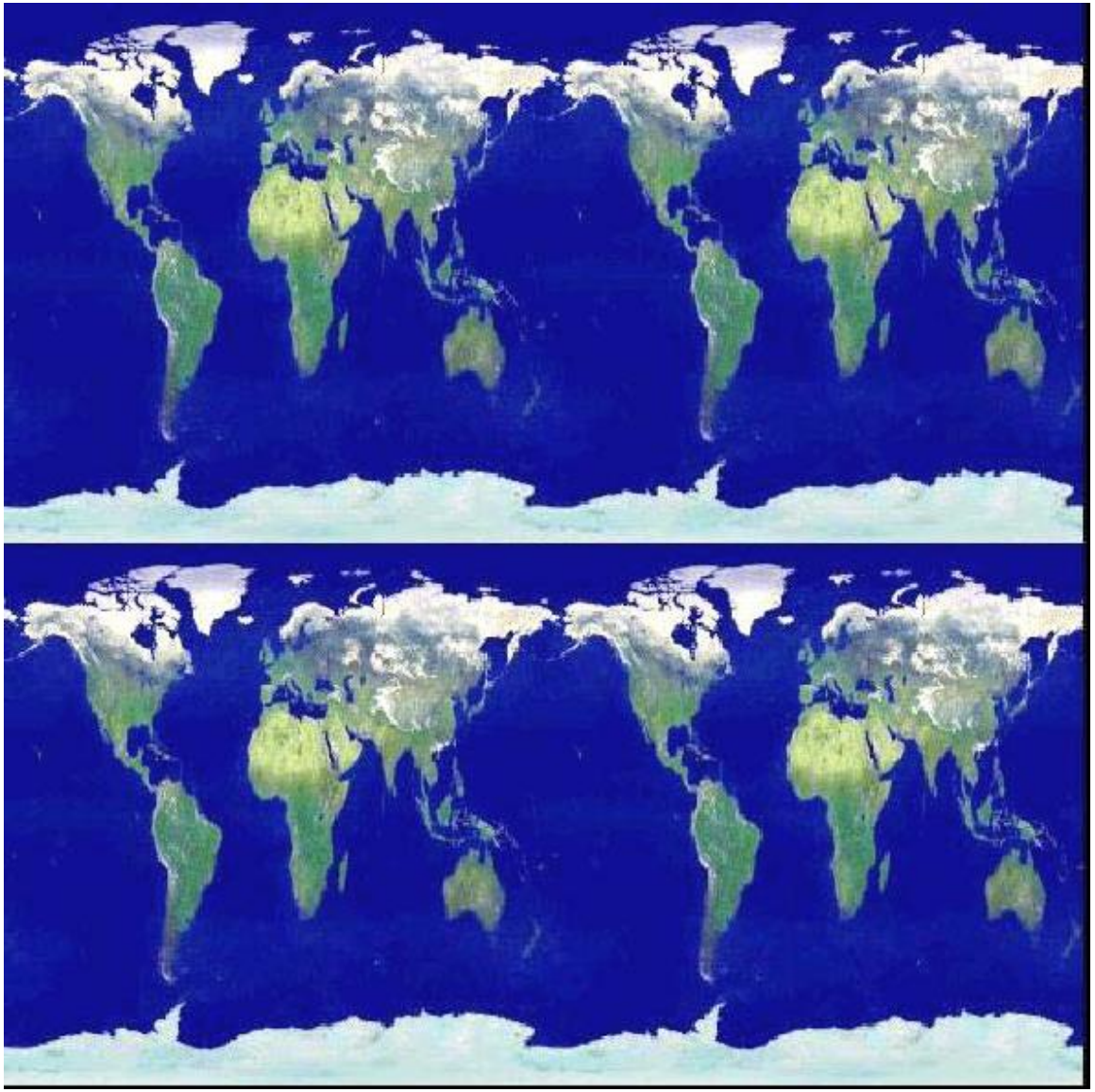
  De exemplu, (-0.1, 1.5) se transforma in (-0.1 – (-1), 1.5 – (1)) = (0.9, 0.5).

- Acest mod de transformare permite repetarea unei texturi, producand un efect periodic.
- La utilizarea acestui mod este necesar ca laturile stanga/dreapta, respectiv sus/jos ale imaginii textura se se potriveasca, astfel incat sa nu se observe marginile texturii.

- Daca pentru una dintre coordonate se utilizeaza transformarea “comprimare” iar pentru cealalta “repetare”, textura se numeste “cilindrica”.
- Daca pentru ambele coordonate se utilizeaza modul “repetare”, textura se numeste “toroidala”.

Filtrarea texturii (7)

Wrapping (Repetare)



Metoda "mip-mapping" (1)

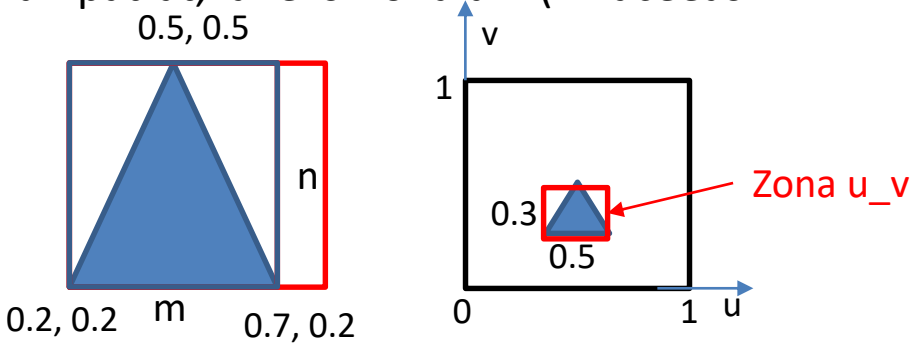
- Atunci cand suprafata texturata este la distanta mare de observator, ea este reprezentata pe un numar mic de pixeli, mult mai mic decat nr de texeli ai imaginii textura.
- De ex. daca textura este de 256x256 texeli iar imaginea suprafetei se afiseaza pe 8x8 pixeli: ar trebui ca 1024 texeli sa fie „comprimati" în culoarea unui pixel.
- In banda grafica nu se face acest calcul, in mod implicit, obtinandu-se pentru fiecare pixel o culoare din imaginea textura pe baza coordonatelor (uf,vf) ale fragmentului. Rezulta ca numai 64 de texeli vor fi folositi la afisare!
- Ideea mip-mapping: **utilizarea de imagini textura de rezolutii diferite in functie de marimea în imagine a suprafetei texturate**.
- Pornind de la o imagine textura, se pot obtine texturi de rezolutie mai joasa prin medierea culorilor din textura initiala.
- "MIP": acronim al frazei din limba latina: *multum in parvo*, insemnand "mult in puțin"

Metoda "mip-mapping" (2)

- Textura redata intr-un pixel trebuie sa fie comprimata in functie de distanța de la observator la suprafața texturata.
- Factorul de compresie a texturii se calculeaza pe GPU pentru fiecare primitiva grafica, dupa transformarea varfurilor in viewport, tinand cont de suprafata primitivei in spatiul ecran si zona u-v folosita la texturarea sa.
- Suprafata primitivei se aproximeaza printr-un patrat, la fel si zona u-v (→ aceeasi compresie pe axele u si v)

Exemplu: m=50, n= 100

Aria suprafetei: 100x100 pixeli



Zona u_v se aproximeaza prin patratul incadrator cu latura de 0.5

Factorul de compresie a texturii: f=0.5/100 = 1/200

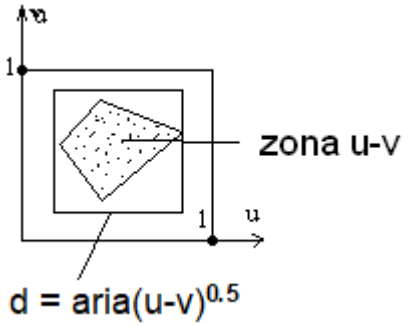
Daca rezolutia texturii este de 512x512 texeli → 512/200 = 2.56

→ 2.56x2.56 = 6.55 texeli se comprima într-un pixel.

Metoda "mip-mapping" (3)

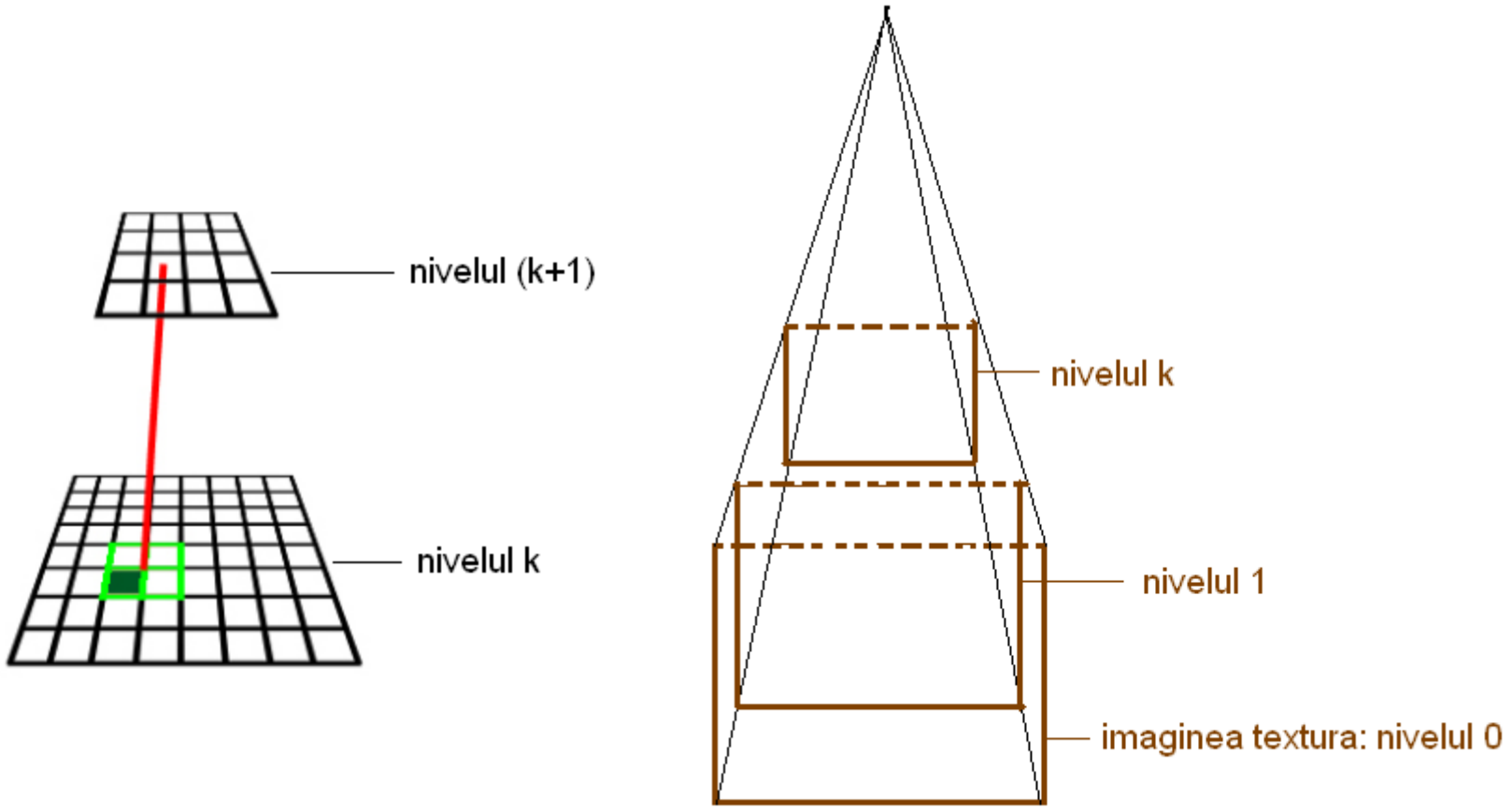
- Se pot folosi mai multe imagini textura obtinute prin compresia imaginii textura date.
- aria(u-v): aria suprafetei din spatiul (u-v) corespunzatoare unui texel
- Pentru o textura de nxn texeli, aria u-v = (1/n)x(1/n)
- n=1 → aria (u-v) = 1 → d=1: compresie maxima

  (toata textura este comprimata intr-un texel)
- d reprezinta factorul de compresie al texturii



- ➢ In calculul culorii unui fragment ar trebui sa se foloseasca o textura comprimata cu factorul de compresie corespunzator primitivei din care face parte.
- ➢ Se calculeaza o piramida de imagini textură comprimate, fiecare nivel corespunzand unui factor de compresie al texturii → nr de factori de compresie este limitat la nr. de niveluri ale piramidei!
- De exemplu, fiecare imagine textura comprimata se obtine prin aplicarea unui filtru trece-jos, într-o fereastra de 2x2 pixeli, pe imaginea textura de pe nivelul anterior.
- ➢ Culoarea unui fragment se obtine din imaginile textura ai caror factori de compresie sunt cei mai apropiati de cel calculat pentru primitiva din care face parte fragmentul.

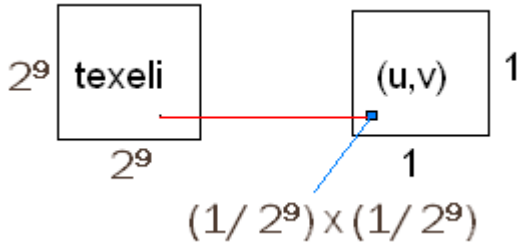
Metoda "mip-mapping" (4)
- Calculul piramidei mip-map (1)-



Metoda "mip-mapping" (5)
- Calculul piramidei mip-map (2)-

- Pentru o imagine textura de 512x512 texeli rezulta o piramida compusa din 10 niveluri:
- Nivelul 0: intreaga imagine textura (512x512)= $\mathbf{2^9 \times 2^9}$

  aria (u-v) coresp unui texel = $\mathbf{(1/\ 2^9) \times (1/\ 2^9)} = 1/2^{18} = \mathbf{2^{2(0-9)}}$



- Nivelul 1: 256x256 texeli = $2^8 \times 2^8$ →aria (u-v) coresp unui texel

  $=(1/2^8)x(1/2^8) = (2/\ 2^9) \times (2/\ 2^9) = \mathbf{2^{2(1-9)}}$
- Nivelul 2: 128x128 texeli = $2^7 \times 2^7$ →aria (u-v) coresp unui texel = $(4/\ 2^9) \times (4/\ 2^9) = \mathbf{2^{2(2-9)}}$
- Nivelul k: $2^k \times 2^k$ →aria (u-v) coresp unui texel = $(2^k / 2^9) \times (2^k / 2^9) = \mathbf{2^{2(k-9)}}$
- Nivelul 9: 1 texel →aria (u-v) coresp unui texel = $(2^9 / 2^9) \times (2^9 / 2^9) = 1$

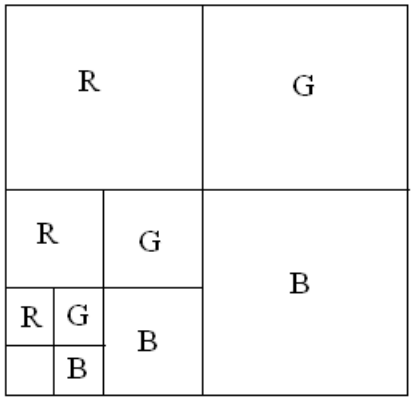
  (compresie maxima: intreaga imagine textura este comprimata intr-un singur texel)
- Factorul de compresie al imaginii textura de pe nivelul k este:

  $d_k = (\text{aria } (u\text{-}v)_k)^{0.5} = \mathbf{2^{(k-9)}}$
- Factorul de compresie al imaginii textura de pe nivelul k+1 este:

  $d_{k+1} = (\text{aria } (u\text{-}v)_{k+1})^{0.5} = \mathbf{2^{(k-8)}}$

Metoda "mip-mapping" (6)
- reprezentarea piramidei mip-map -

Pentru o imagine textura de 512x512 pixeli, piramida se poate memora intr-o matrice de (1024 x1024) octeti, cu urmatorul aspect :



Metoda "mip-mapping" (7) – filtrarea texturii -

Piramida mip-map este adresata prin 3 coordonate: (u, v, d):

(u,v) prin care se selecteaza un texel/texelii de pe un nivel al piramidei, pe baza carora se calculeaza o culoare;

d, care este o masura a compresiei texturii intr-un pixel ecran, se foloseste pentru a selecta nivelul piramidei din care se calculeaza culoarea sau nivelurile intre care se va interpola pentru calculul culorii; **d poate fi considerata ca fiind coordonata verticala a piramidei**.

1) Fie uf si vf coordonatele folosite pentru calculul culorii unui fragment
2) Fie df factorul de compresie al texturii intr-un pixel pentru primitiva din care face parte fragmentul.

Daca nu s-a cerut interpolare intre nivelurile mip-map

glTexParameteri(….GL_NEAREST_**MIPMAP_NEAREST** sau GL_LINEAR_**MIPMAP_NEAREST**),

atunci se determina nivelul piramidei pentru care factorul de compresie este cel mai apropiat de df. Se calculeaza o culoare din textura corespunzatoare nivelului folosind (uf,vf)

Metoda "mip-mapping" (8) – filtrarea texturii -

Daca s-a cerut prin OpenGL interpolare între nivelurile mip-map

glTexParameteri(….GL_NEAREST_**MIPMAP_LINEAR** sau GL_LINEAR_**MIPMAP_LINEAR**)

Fie: $d_k < df < d_{k+1}$

3) Folosind (uf, vf)

- se determina o **culoare textura de pe nivelul k**, C_k
- se determina o **culoare textura de pe nivelul k+1**, C_{k+1}

4) Se determina culoarea de afisare a fragmentului prin interpolare liniara intre C_k si C_{k+1}**:**

$C = C_k + dInterp\ (C_{k+1} - C_k)$

unde **dInterp** se determina din conditia: $df = d_k + dInterp\ (d_{k+1} - d_k) = 2^{k-9} + dInterp\ (2^{k-8} - 2^{k-9})$

Rezulta: **dInterp** $= 2^{9-k} * df - 1$

Exemplu:

Factorul de compresie: df=1/40 → d3 (1/64) < df < d4 (1/32)

Se va interpola intre culorile calculate pe nivelurile d3 si d4.

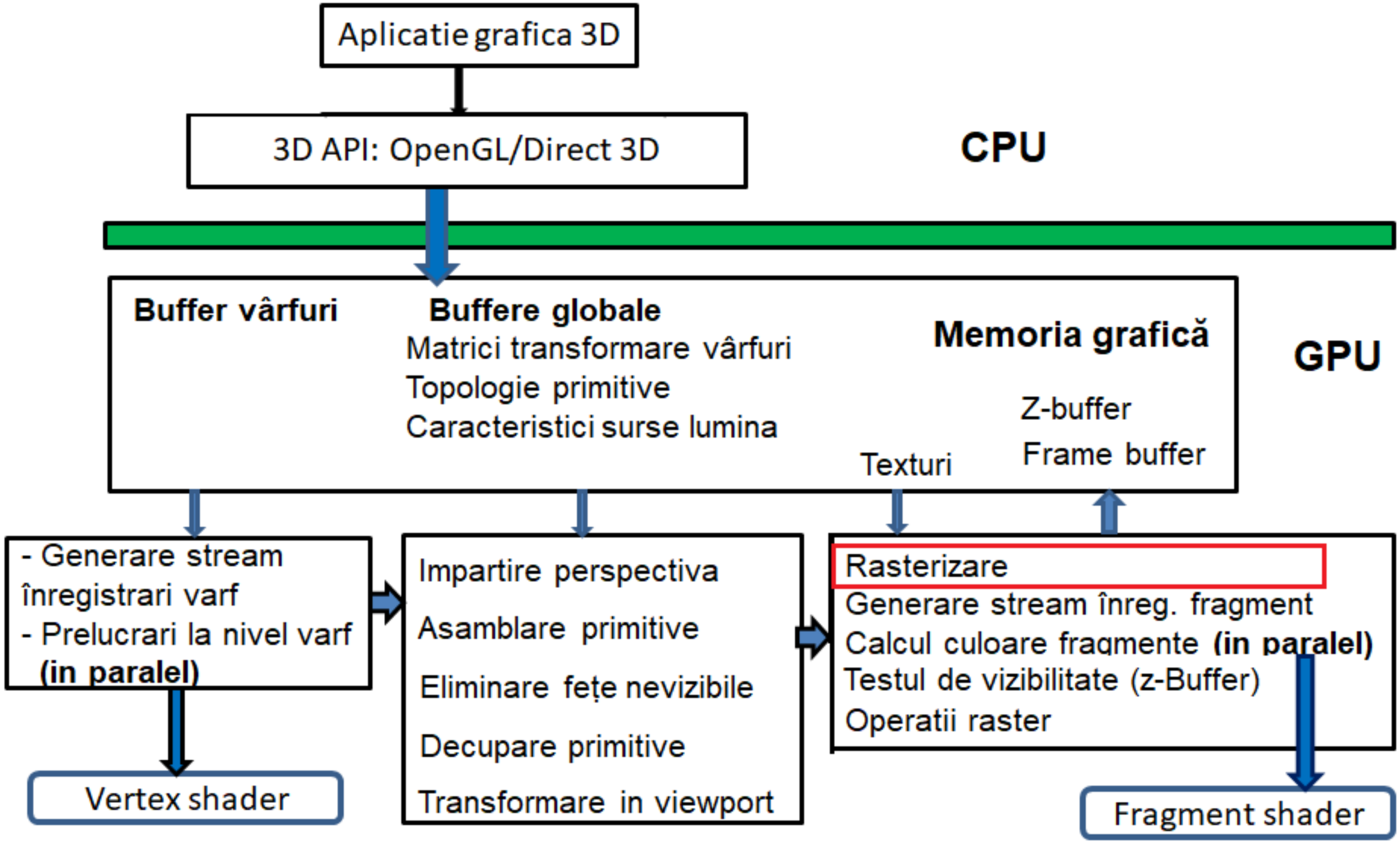
Tehnica mip-mapping:

- da rezultate bune, chiar daca se bazeaza pe o serie de aproximari
- este implementata de GPU: trebuie sa i se transmita imaginile textura ale piramidei.

Rasterizarea primitivelor grafice

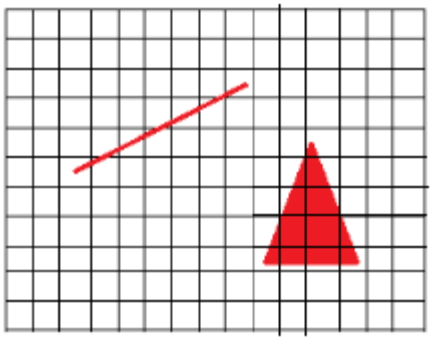
Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020


Aplicatie grafica 3D
3D API: OpenGL/Direct 3D
CPU
Buffer vârfuri
Buffere globale
Matrici transformare vârfuri
Topologie primitive
Caracteristici surse lumina
Memoria grafică
Z-buffer
Frame buffer
Texturi
GPU
- Generare stream înregistrari varf
- Prelucrari la nivel varf (in paralel)
Vertex shader
Impartire perspectiva
Asamblare primitive
Eliminare fețe nevizibile
Decupare primitive
Transformare in viewport
Rasterizare
Generare stream înreg. fragment
Calcul culoare fragmente (in paralel)
Testul de vizibilitate (z-Buffer)
Operatii raster
Fragment shader

Rasterizarea (1)

- Singurele primitive grafice acceptate de placile grafice care suporta versiunile noi ale bibliotecii OpenGL(3.x→) sunt:
  - **punctul**, definit prin coordonatele sale (x,y,z);
  - **linia**, definita prin coordonatele extremitatilor sale, (x1,y1,z1) si (x2,y2,z2); specificate in aceasta ordine, **linia este un vector cu originea in (x1,y1,z1) si varful in (x2,y2,z2);**
  - **suprafaţa triunghiulară**, definita prin coordonatele 3D ale varfurilor sale.
- Suprafata de afisare este un spatiu discret, alcatuit din celule de afisare (pixeli) adresate prin coordonate intregi, (0,0) <= (x,y) <= (xmax,ymax)
- Pentru afisare, este necesara descompunerea primitivelor in fragmente care se afiseaza in celulelele suprafetei de afisare.



- **Rasterizarea:**
  - operatia prin care o primitiva grafica este descompusa in fragmente care se afiseaza in pixelii suprafetei de afisare;
  - are ca rezultat o aproximare in spatiul discret a primitivelor definite analitic.

Rasterizarea (2)

Algoritmii de rasterizare integreaza:

1. Calculul coordonatelor (x,y) ale pixelilor in care se afiseaza fragmentele.
2. Calculul coordonatei z a fiecarui fragment.
3. Interpolarea atributelor asociate varfurilor primitivei: culoare, coordonate textura, ş.a.

   Valorile obtinute prin interpolare la nivel de fragment sunt transmise la fragment shader.
4. Calculul culorii fiecarui fragment, în fragment shader.
5. Testul de vizibilitate (adancime) a fiecarui fragment si actualizarea culorii pixelului:

   daca z< Z-buffer[y][x] atunci
   { Z-buffer[y][x] = z
   *actualizeaza culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea fragmentului
   }

Deoarece in algoritmii de rasterizare este important modul de calcul al coordonatelor (x,y), vom include pasii 2, 3, 4 şi 5 în functia **putpixel (int x, int y);**

Rasterizarea vectorilor (1)

La rasterizarea unui vector:

1. **Pixelii (x,y) in care se afiseaza fragmentele rezultate din rasterizare sunt determinati astfel incat sa fie cei mai apropiati de adresele punctelor de pe vectorul analitic.**

2. Coordonata z a fiecarui fragment se obtine prin interpolarea liniara a coordonatelor z ale varfurilor vectorului (prin calcul incremental – vezi “algoritmul z-buffer”).

3. Culoarea fiecarui fragment se obtine prin interpolarea liniara a culorilor varfurilor vectorului (prin calcul incremental) – culoarea interpolata este intrare pentru fragment shader; nu este uzual sa se calculeze culoarea unui fragment de vector in “fragment shader”

- Algoritmii de rasterizare a vectorilor se deosebesc prin pasul 1.

Rasterizarea vectorilor (2)

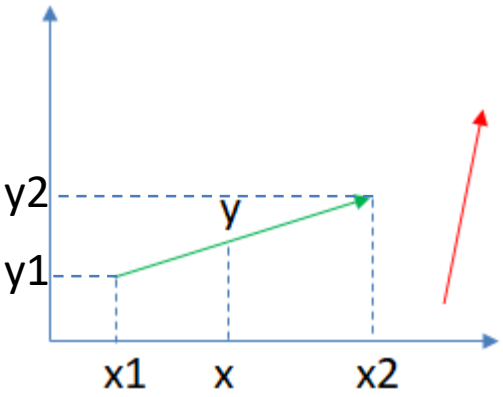
Ecuatia unui vector: y = m*x + b

m = (y2-y1)/(x2-x1) şi b=y1 - m*x1

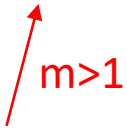
(x1,y1), (x2,y2) sunt adrese de pixeli

Consideram urmatorul algoritm de rasterizare a unui vector:

```
putpixel(x1,y1);
for(x=x1+1; x<x2; x++)
   { y=m*x+b ;
     putpixel(x, (int)(y+0.5));//adr. de pixel cea mai apropiata de (x,y)
    }
putpixel(x2,y2);
```



x++



y++

Dezavantaje algoritm:

- Nu se tine cont de panta dreptei: vectorii cu panta mare (ex. cel rosu) sunt aproximati prin câțiva pixeli!
- Calculul fiecarei adrese de pixel de pe traseul vectorului contine operatii cu numere reale

Algoritmul Digital Differential Analyser (DDA)

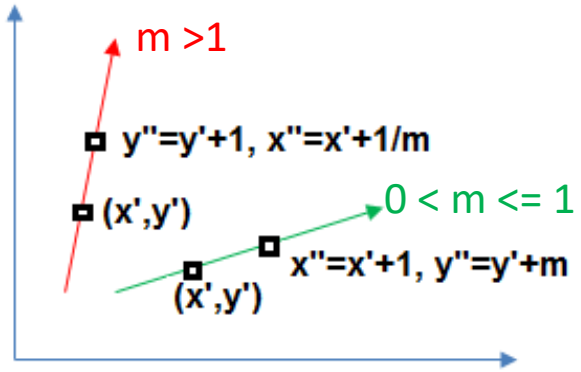
- Ține cont de panta vectorului
- Coordonatele pixelilor de pe traseul vectorului se obtin printr-un calcul incremental (eficient)

Fie (x', y') si (x'',y'') - 2 puncte succesive de pe vector

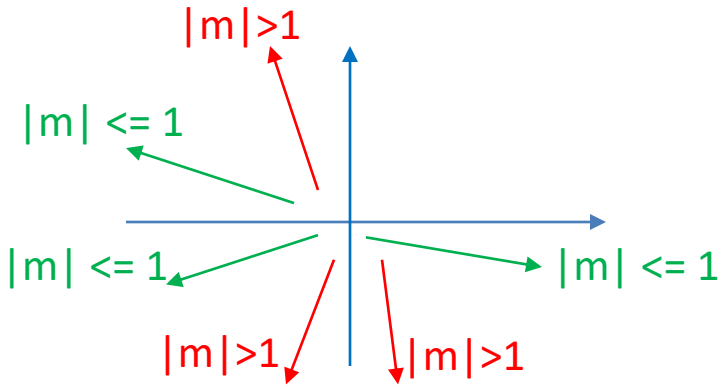
$(y'' - y') / (x'' - x') = (y2 - y1) / (x2 - x1) = m$

|m <1| : se incrementeaza x : x'' = x' +1 → y'' = y' + m

| m >1| : se incrementeaza y: y'' = y' + 1 → x'' = x' +1/m




```
void DDA(int x1, int y1, int x2, int y2)
{double m, absm, rx, ry; int x, y;
//vectorul se genereaza de la (x1,y1) la (x2,y2)
 if(x1==x2) //vector vertical
  { if(y1>y2) {y=y1; y1=y2; y2=y;}
    for(y=y1; y<=y2; y++)  putpixel(x1,y);
   return;
 }
```



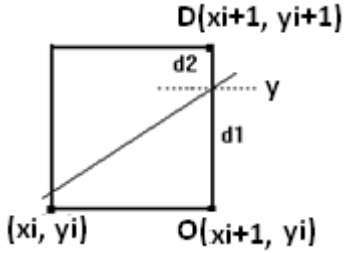
Algoritmul DDA(2)

```
if(y1==y2) //vector orizontal
  {if(x1>x2)
     {x=x1; x1=x2; x2=x;}
   for(x=x1; x<= x2; x++)  putpixel(x,y1);
   return;
  }
 m=(double)(y2-y1)/(x2-x1); absm=abs(m);
 if(absm<=1 && x1>x2  ||  absm>1 &&  y1>y2)
     {x=x1; x1=x2; x2=x; y=y1; y1=y2; y2=y;}
 putpixel(x1, y1);
 if( absm<=1)
     for(x=x1+1, ry=y1; x<x2; x++)
       {ry+=m;  putpixel(x, (int)(ry+0.5)); } // y = y + m
else
     {m=1/m;
      for(y=y1+1, rx=x1; y<y2; y++)
       {rx+=m;  putpixel((int)(rx+0.5), y); } // x = x +1/m
     }
 putpixel(x2, y2);
}
```

Dezavantaj: calcule cu numere reale

Algoritmul Bresenham (1)

- Contine numai operatii cu numere intregi. **Este definit pentru vectori din primul octant**
- Un vector face parte din"primul octant" daca panta sa, m, satisface conditia 0<m<=1.
- Coordonatele pixelilor de pe traseul vectorului se obtin prin calcul incremental.
- Pentru fiecare valoare a lui x, de la xmin_vector la xmax_vector, se alege acel punct al spaţiului discret care este mai apropiat de punctul de pe vectorul teoretic.



- Fie m=(y2-y1)/(x2-x1) panta vectorului,
- (x_i, y_i) ultimul punct de pe traseul vectorului în spaţiul discret
- d1 distanţa de la punctul de pe vectorul teoretic, (x_i+1, y), la punctul $O(x_i+1, y_i)$
- d2 distanţa de la punctul de pe vectorul teoretic la punctul $D(x_i+1, y_i+1)$.
- O si D sunt adrese de pixeli (puncte ale spatiului discret)

➢ Următorul punct al spatiului discret ales pentru aproximarea vectorului va fi:

- O dacă d1<d2, D în caz contrar.
- Dacă d1=d2 se poate alege oricare dintre cele două puncte.

Algoritmul Bresenham(2)

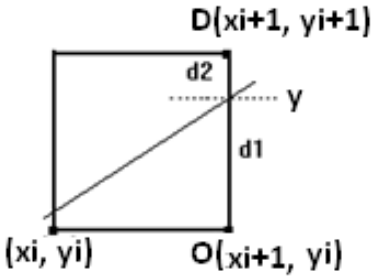
- Exprimăm difereţa d1-d2:

  $y=m*(x_i+1)+b$ este ordonata punctului de pe vectorul teoretic

  $d1= y - y_i= m*(x_i+1)+b-y_i$

  $d2= y_i + 1-y= y_i+1-m*(x_i+1)-b$

  $d1- d2= 2*m*(x_i+1)-2*y_i+2*b -1$



- Se înlocuieşte m cu dy/dx apoi se înmulţeşte în ambele părţi cu dx. Rezultă:

  $t_i= (d1-d2)*dx= 2*dy*(x_i+1)-2*dx*y_i+2*b*dx-dx = 2*dy*x_i - 2*dx*y_i + 2*b*dx - dx + 2*dy$

- t_i reprezintă eroarea de aproximare în pasul i: pe baza ei se alege urmatorul punct al spatiului discret

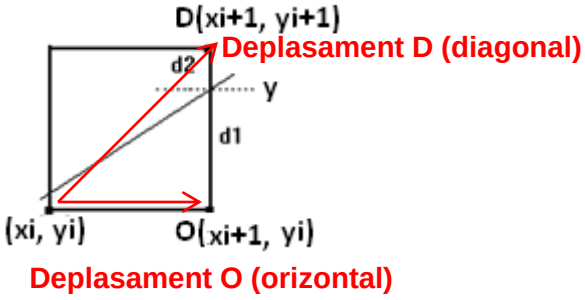
  Notăm cu (x_{i+1},y_{i+1}) punctul care se va alege în pasul i.

- Expresia erorii de aproximare în pasul i+1 este:

  $t_{i+1} = 2*dy*x_{i+1} - 2*dx*y_{i+1} + 2*b*dx - dx +2*dy$

Algoritmul Bresenham(3)

$t_i=(d1-d2)*dx = 2*dy*x_i - 2*dx*y_i + 2*b*dx-dx + 2*dy$, (dx>0)



(1) Dacă $t_i <= 0$, se alege punctul O, deci $x_{i+1} = x_i+1$ şi $y_{i+1} = y_i$

Rezultă:

$t_{i+1} = 2*dy*(x_i+1) - 2*dx*y_i + 2*b*dx - dx +2*dy$

sau **$t_{i+1} = t_i + 2*dy = t_i + c1$**

(2) Dacă $t_i > 0$, se alege punctul D, deci $x_{i+1} = x_i + 1$ şi $y_{i+1} = y_i +1$

Rezultă:

$t_{i+1} = 2*dy*(x_i+1) - 2*dx*(y_i+1) + 2*b*dx - dx +2*dy$

sau **$t_{i+1} = t_i + 2*dy - 2*dx = t_i + c2$**

- Valoarea variabilei de test se obtine prin calcul incremental: adunarea unei constante intregi

Eroarea de aproximare pentru primul pas. Se inlocuiesc in t_i : $x_i = x1$ şi $y_i = y1$:

$t_1 = 2*dy*x1 - 2*dx*y1 + 2*dy – dx + 2*dx(y1 - (dy/dx)*x1)$

sau **$t_1 = 2*dy-dx$**

Algoritmul Bresenham(4)

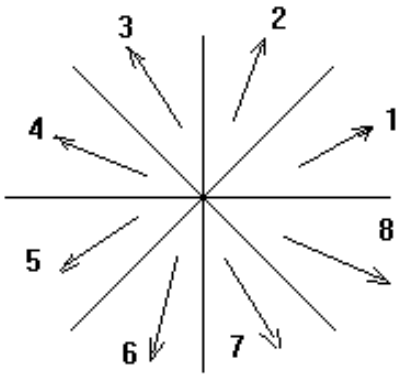
Implementare in C:

```
void Bres_vect(int x1, int y1, int x2, int y2)
{ //pentru vectori cu panta cuprinsă între 0 şi 1
  int dx, c1, c2, x, y, t;
  dx=x2-x1;
  c1=(y2-y1)<<1;  // 2*dy
  c2=c1-(dx<<1);  // 2*dy – 2*dx
  t=c1-dx;  // 2*dy - dx
  putpixel(x1, y1);
  for(x=x1+1, y=y1; x<x2; x++)
   { if(t<0) t+= c1; //deplasament  O
      else  { t+= c2; y++;} // deplasament D
    putpixel(x,y);
   }
  putpixel(x2,y2);
}
```

Algoritmul Bresenham- generalizare(1)

Generalizarea algoritmului Bresenham pentru vectori de orice panta

- Vectorii definiţi în planul XOY pot fi clasificaţi, pe baza pantei, în opt clase geometrice, numite **“octanţi“**
- Un vector care apartine unui octant O are 7 vectori simetrici in ceilalti 7 octanti



dx=x2 – x1 şi dy=y2 – y1

octantul 1: dx>0 şi dy>0 şi dx>=dy;

octantul 2: dx>0 şi dy>0 şi dx<dy;

octantul 3: dx<0 şi dy>0 şi abs(dx)<dy;

octantul 4: dx<0 şi dy>0 şi abs(dx)>=dy;

octantul 5: dx<0 şi dy<0 şi abs(dx)>=abs(dy);

octantul 6: dx<0 şi dy<0 şi abs(dx)<abs(dy);

octantul 7: dx>0 şi dy<0 şi dx<abs(dy);

octantul 8: dx>0 şi dy<0 şi dx>=abs(dy);

Algoritmul Bresenham –generalizare(2)

Intr-un pas al algoritmului generalizat sunt posibile urmatoarele deplasamente:

+h, deplasamentul orizontal spre dreapta (în sensul crescător al axei x): x++

-h, deplasamentul orizontal spre stânga (în sensul descrescător al axei x): x--

+v, deplasamentul vertical în sus (în sensul crescător al axei y): y++

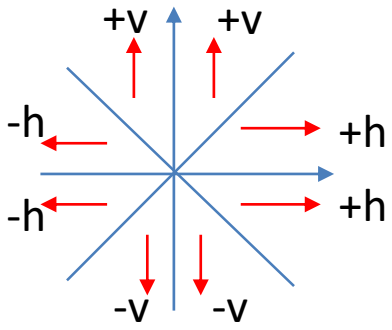
-v, deplasamentul vertical în jos (în sensul descrescător al axei y): y--

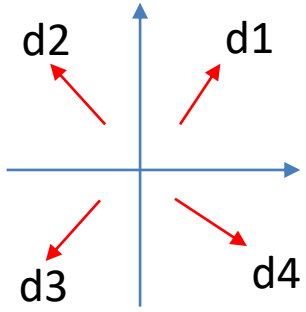
d1, deplasamentul diagonal dreapta-sus: x++; y++

d2, deplasamentul diagonal stânga-sus: x--; y++

d3, deplasamentul diagonal stânga-jos: x--; y--

d4, deplasamentul diagonal dreapta-jos: x++; y--





Octant	1	2	3	4	5	6	7	8
Deplas O	+h	+v	+v	-h	-h	-v	-v	+h
Deplas D	d1	d1	d2	d2	d3	d3	d4	d4

Corespondenţa între deplasamentul ales într-un pas al algoritmului Bresenham (punctul O sau punctul D) şi deplasamentul echivalent în fiecare octant:

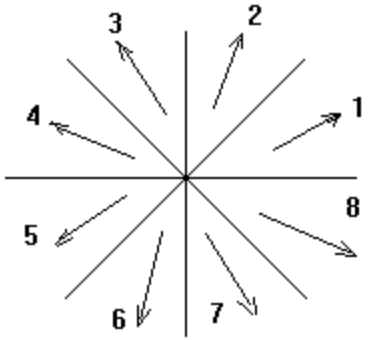
Algoritmul Bresenham – generalizare(3)

Algoritmul Bresenham generalizat - implementare in C

```
void Bres_general(int x1, int y1, int x2, int y2)
{ int x, y, i, oct, dx, dy, absdx, absdy, c1, c2, t;
  if(x1==x2) //vertical
  { if(y1>y2){y=y1; y1=y2; y2=y;}
     for(y=y1; y<=y2;y++)
       putpixel(x1,y);
     return;
  }
 if(y1==y2) //orizontal
   {if(x1>x2) {x=x1; x1=x2; x2=x;}
     for(x=x1; x<= x2; x++)
       putpixel(x,y1);
    return;
   }
```

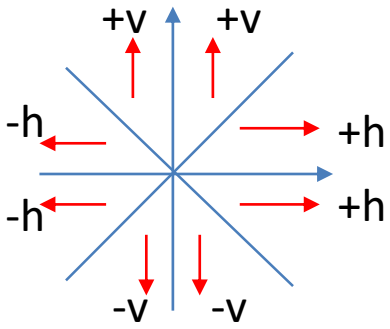
Algoritmul Bresenham – generalizare (4)

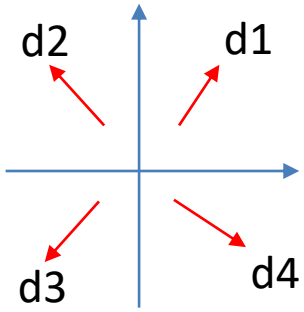
```
dx= x2-x1; dy= y2-y1; absdx= abs(dx); absdy= abs(dy);
if(dx>0)//oct=1,2,7,8
   { if(dy>0)// oct=1,2
       if(dx>=dy) oct=1; else oct=2;
     else
       if(dx>=absdy) oct=8; else oct=7;
   }
else//3,4,5,6
   { if(dy>0)// oct=3,4
       if(absdx>=dy) oct=4; else oct=3;
     else
       if(absdx>=absdy) oct=5; else oct=6;
   }
// Numărul de paşi la execuţia algoritmului generalizat este maxim(abs(dx), abs(dy))
// Dacă abs(dy) > abs(dx), se inversează rolul variabilelor dx şi dy în calculul constantelor c1 şi c2
if(absdy>absdx) //  adresele de pe traseul vectorului se obtin prin incrementarea lui y
    {x=absdx; absdx=absdy;absdy=x;}
 c1= absdy<<1;  c2= c1-(absdx<<1); t= c1-absdx;
```

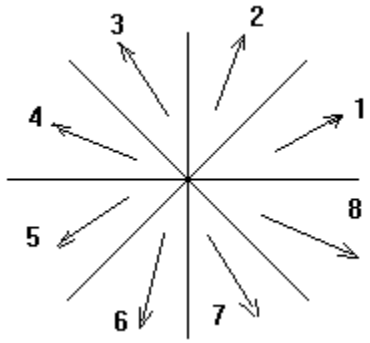


Algoritmul Bresenham – generalizare (5)

```
putpixel(x1,y1);
for(i=1, x=x1, y=y1; i<absdx; i++)
{ if(t<0) // deplasament O
   {t+=c1;
    switch(oct)
    {case 1:  case 8: x++; break; // +h
     case 4:  case 5: x--;break; // -h
     case 2:  case 3: y++;break; //+v
     case 6:  case 7: y--;break; // -v
    }
   }
  else
   {t+=c2 // deplasament D
    switch(oct)
    {case 1: case 2: x++; y++; break; // d1
     case 3: case 4: x--; y++; break; // d2
     case 5: case 6: x--; y--; break; // d3
     case 7: case 8: x++; y--; break; // d4
    }
  }
  putpixel(x,y);
 } putpixel(x2,y2);
}
```







Generarea liniilor întrerupte

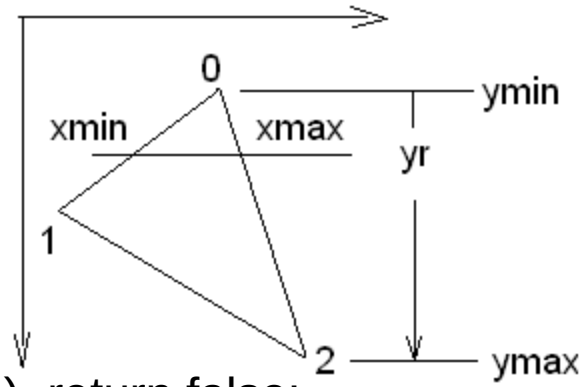
```
void Bres_gen(int x1, int y1, int x2, int y2, int şablon)
{  // şablon: intreg pe 16 biti cu valori 0 si 1
   // se suprapune şablonul  pe traseul vectorului: se afiseaza numai acei pixeli ai vectorului pentru care
      // bitul sablonului = 1

    int val, bit, biţi[16] ;
   // se extrag biții şablonului în vectorul biţi
    for(int i=0, val=1; i<16; i++)
    { if (şablon & val) biţi[i] = 1; else biţi[i] = 0;
      val = val << 1;
    }
    .....................................
   if(biţi[0]]) putpixel(x1,y1);
   for(i=1, x=x1, y=y1, bit=1; i<absdx; i++)
  { .....................................
     if(biţi[(bit++) % 16]) putpixel(x,y);
  }
  if(biţi[bit % 16) putpixel(x2,y2);
}
```

Rasterizarea suprafetelor triunghiulare

La rasterizarea unei suprafete triunghiulare

1. Pixelii (x,y) in care se afiseaza fragmentele rezultate din rasterizare sunt determinati intersectand suprafata triunghiului cu liniile raster (imagine) de la ymin_triunghi la ymax_triunghi;
2. Fiecare segment dintr-o linie raster, care este interior suprafetei triunghiulare, este descompus in fragmente.
3. Coordonatele z ale fragmentelor se obtin prin calcul incremental (vezi curs “algoritmul z-buffer”).
4. Calculul culorii fiecarui fragment este efectuat in programul “fragment shader”.
5. Sunt afisate fragmentele care trec testul de vizibilitate.

Rasterizarea suprafetelor triunghiulare - algoritmul -



Bool SupTri (int x0, int y0, int x1, int y1, int x2, int y2)

{ 1. Se sorteaza varfurile triunghiului crescator dupa coordonata y, a.î. varful cu ymin sa fie (x0,y0);

2. Daca triunghiul este degenerat (varfurile sunt coliniare), return false;
3. Se orienteaza conturul in sens trigonometric (v0-v1-v2: sens trigonometric);
4. pentru (yr = ymin-triunghi; yr<= ymax-triunghi; yr++)
   - se calculeaza segmentul din linia yr, care este interior suprafetei triunghiului
   - se memoreaza extremitatile sale in 2 tablouri, XMIN, XMAX:
     XMIN[yr] = extr_stanga; XMAX[yr] = extr_dreapta;
5. Se afiseaza pixelii segmentelor interioare (de la ymin_triunghi la ymax_triunghi);

return true;

}

Pasii 1 si 2

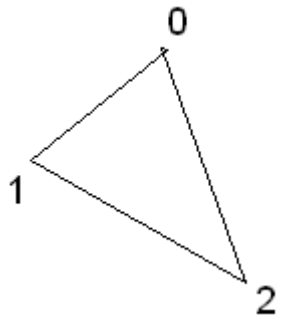
1. Se sorteaza varfurile triunghiului crescator dupa coordonata y, a.i. y0 = ymin

```
if( y1 < y0) { t= y1; y1 = y0; y0 =t; t= x1; x1 = x0; x0 =t;}
if( y2 < y0) { t= y2; y2 = y0; y0 =t; t= x2; x2 = x0; x0 =t;}
```

2. Daca triunghiul este degenerat (varfurile sunt coliniare), return false

$$\det = \begin{vmatrix} x1 - x0 & y1 - y0 \\ x2 - x0 & y2 - y0 \end{vmatrix}$$

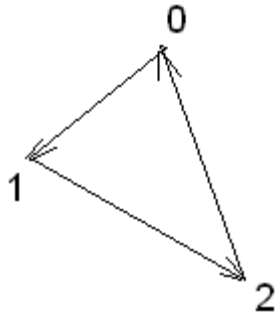
$$= (x1 - x0)(y2 - y0) - (x2 - x0)(y1 - y0)$$



if(det==0) → varfurile sunt coliniare

Pasul 3: orientarea conturului

3. Se orienteaza conturul in sens trigonometric (v0-v1-v2: sens trigonometric)



Sens trigonometric,
Cazul: ymax = y2

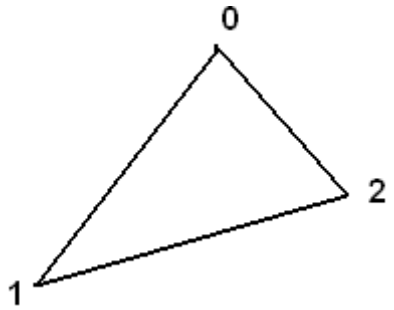
dx0 = x1 – x0; dx1 = x2 – x0;

dy0 = y1 – y0; dy1 = y2 – y0;

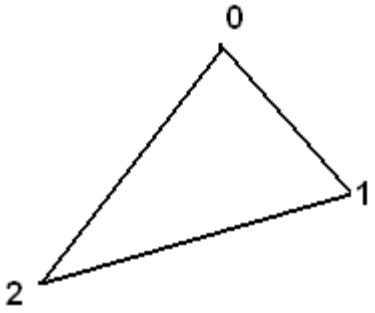
x1 < x0 , x2 > x0

dx0 < 0, dx1 > 0, dy0 > 0, dy1 > 0

det = dx0*dy1- dx1*dy0 **< 0**



Sens trigonometric,
Cazul: ymax = y1



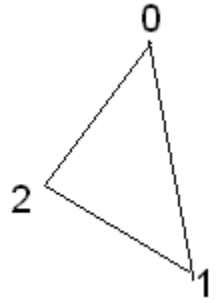
Sens anti-trigonometric,
Cazul: ymax = y2

x1> x0, x2 < x0

dx0 > 0, dx1 < 0, dy0 > 0, dy1 > 0

det = dx0*dy1- dx1*dy0 **> 0**

Se inverseaza varful 1 cu varful 2



Sens anti-trigonometric
Cazul: ymax = y1

Pasul 4: calculul extremitatilor segmentelor interioare(1)

4. Se calculeaza extremitatile segmentelor interioare suprafetei triunghiului

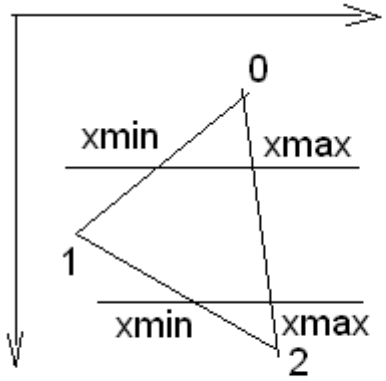
- Coordonatele xmin, xmax ale extremitatilor segmentelor interioare suprafetei triunghiului se memoreaza in 2 tablouri:

  int XMIN[H], XMAX[H]; // H este numarul de linii imagine

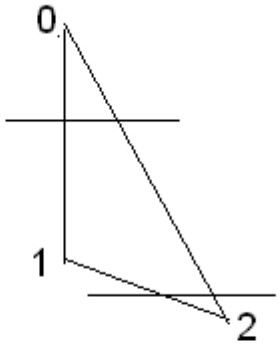
- Calculul extremitatilor este efectuat in functiile **CalculXmin()** si **CalculXmax()**;

Cazuri:

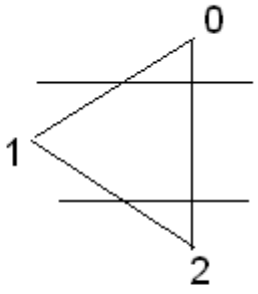
cazul I.



y0 < y1 < y2



x0 = x1



x0 = x2

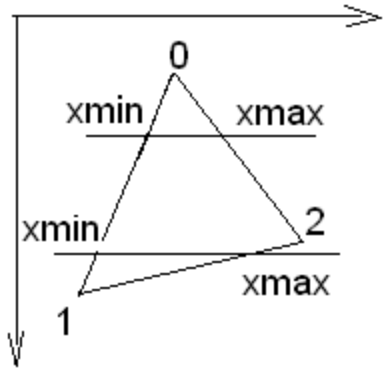
CalculXmin(x0, y0, x1, y1);

CalculXmin(x1, y1, x2, y2);

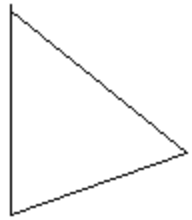
CalculXmax(x0, y0, x2, y2);

Pasul 4: calculul extremitatilor segmentelor interioare (2)

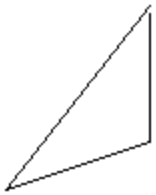
cazul II.



y0 < y2 < y1



x0 = x1



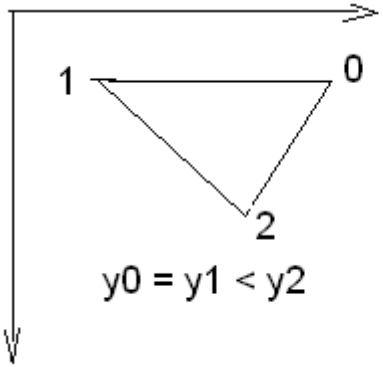
x0 = x2

CalculXmin(x0, y0, x1, y1);

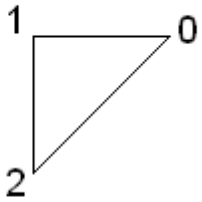
CalculXmax(x0, y0, x2, y2);

CalculXmax(x2, y2, x1, y1);

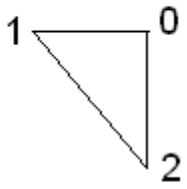
cazul III.



y0 = y1 < y2



x1 = x2

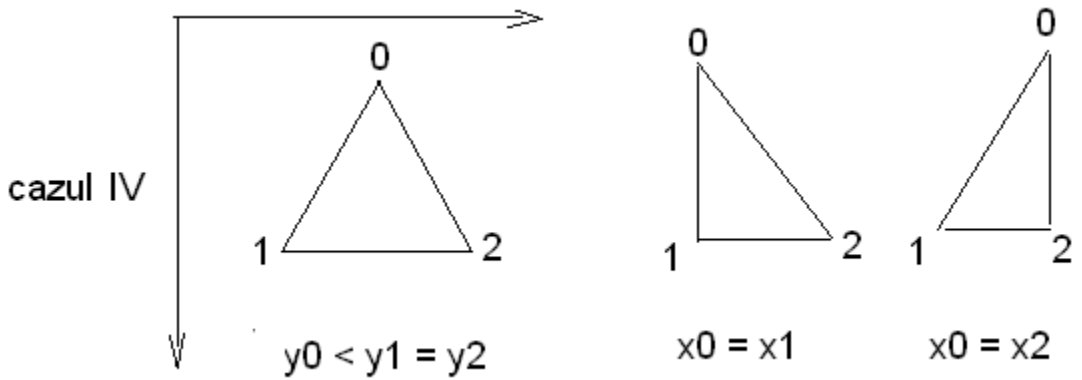


x0 = x2

CalculXmin(x1, y1, x2, y2);

CalculXmax(x0, y0, x2, y2);

Pasul 4: calculul extremitatilor segmentelor interioare (3)



CalculXmin(x0, y0, x1, y1);

CalculXmax(x0, y0, x2, y2);

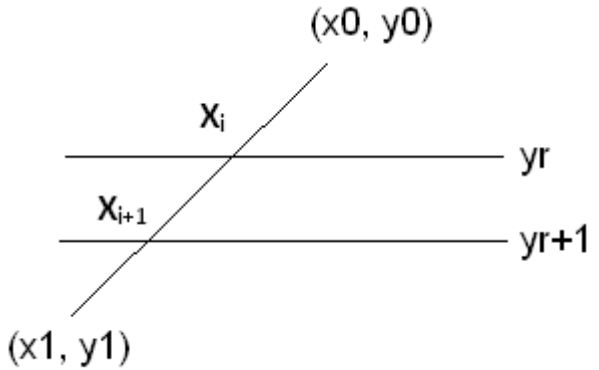
Calculul coordonatelor xmin, xmax

m = (y1 - y0)/ (x1 – x0) - panta laturii

$(yr+1 - yr)/ (x_{i+1} - x_i) = m \rightarrow x_{i+1} = x_i + (1/m)$

dx = 1.0/m - constanta a laturii (x0,y0) - (x1,y1)

$x_{i+1} = x_i + dx \rightarrow$ calcul incremental



Pasul 4: calculul extremitatilor segmentelor interioare (4)

```
int  XMIN[H], XMAX[H];

void Calcul Xmin(int x0, int y0, int x1, int y1)

{ // y0 < y1

 // calculeaza toate intersectiile dintre

//latura (x0,y0) – (x1,y1) si liniile raster

  int y;

  if( x0==x1)// latura verticala

   {for(y =y0; y<=y1;y++)

              XMIN[y] = x0;

    return;

   }

float dx = (float)(x1 – x0) / (y1 – y0);

XMIN[y0] = x0; XMIN[y1] = x1;

 for(y = y0 +1; y<y1; y++)

      XMIN[y] = XMIN[y-1] + dx;

}
```

```
void CalculXmax(int x0, int y0, int x1, int y1)

{ - similara cu functia CalculXmin

  - memoreaza punctele succesive de pe latură

    in tabloul XMAX

}
```

Pasul 5: afisarea pixelilor suprafetei triunghiulare

5. Se afiseaza pixelii interiori triunghiului, intre xmin si xmax pe fiecare linie raster

```
ymax = y1; if (y2>y1 )ymax = y2;
for(int y= y0; y<= ymax; y++) // pt fiecare linie raster care intersecteaza suprafata triunghiului
    for( int x = XMIN[y]; x <= XMAX[y]; x++)
        putpixel(x, y); // afisare fragment interior suprafetei triunghiului
```

Generarea cercurilor în spatiul discret

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Aproximarea cercurilor in spatiul discret(1)

- Cercul nu este primitiva grafica in OpenGL, deci nu poate fi transmis in banda grafica.
- Cercul este o primitiva grafica 2D, definita, de regula, în planul XOY.
- Pentru afisare, cercul este descompus in primitive de tip vector sau primitive de tip punct.
- Daca vrem sa includem cercul intr-o imagine descrisa intr-o aplicatie OpenGL:

  - asociem punctelor de pe cerc coordonata z si transmitem in banda grafica aproximarea cercului printr-o secventa de vectori (x1,y1,0) – (x2,y2,0) sau puncte 3D (x,y,0);
  - mai eficient: aproximarea cercurilor prin vectori sau puncte se poate face in “Geometry shader”.

- In continuare consideram cercurile definite in planul XOY.
- Fie (xc,yc) coordonatele centrului unui cerc. Pentru afisarea sa vom folosi functiile:

  punct (int x, int y, RGB culoare); //afiseaza punctul (x,y) in culoarea specificata

  linie (int x1, int y1, int x2, int y2, RGB culoare); //afiseaza linia in culoarea specificata

✓ Cercul poate fi generat in coordonate ecran sau in coordonate logice

Aproximarea cercurilor in spatiul discret(1)

1) Folosind ecuatia cercului in coordonate carteziene

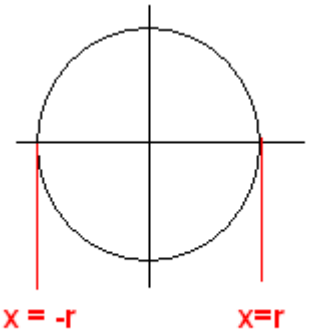
$(x - xc)^2 + (y - yc)^2 = r^2$

Generarea cercului cu centrul in (xc,yc) se reduce la generarea cercului centrat in origine, plus o translatie cu (xc, yc) a coordonatelor.

$x^2 + y^2 = r^2$

Se incrementeaza x de la –r la r si se calculeaza y din ecuatia cercului:

```
punct (xc - r, yc, culoare);
punct (xc + r, yc, culoare);
for (x = -r +1; x< r; x++)
{   y = sqrt(r^2 – x^2);
    punct (xc + x, yc + int(0.5+y), culoare);
    punct (xc + x, yc -int(0.5+y), culoare);
}
```

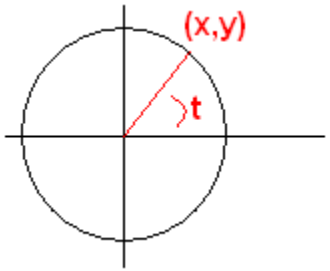


Dezavantaje:

- Spatierea inegala a punctelor de pe cerc afişate
- Calcule complexe cu numere reale

Aproximarea cercurilor in spatiul discret(2)

2) Folosind ecuatia cercului in coordonate polare



$x = xc + r^* \cos(t)$

$y = yc + r^* \sin t(t)$ $\quad 0 <= t <= 2\pi$

Se genereaza cercul centrat in origine si se aplica punctelor rezultate o translatie cu (xc, yc):

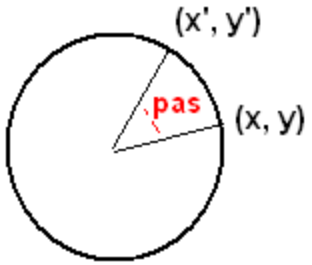
$x = r^* \cos(t)$

$y = r^* \sin t(t)$ $\quad 0 <= t <= 2\pi$

Cercul poate fi aproximat pintr-o polilinie:

- daca se calculeaza N puncte de pe cerc, egal distantate, t creste cu pasul $2\pi/N$

Calculul punctelor succesive de pe cerc, pe baza unei relatii de recurenta:



- (x, y): punctul curent

- (x', y'): punctul urmator

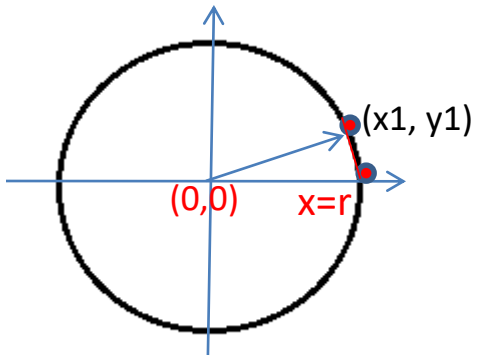
$x' = r^* \cos(t + pas) = r(\cos(t) * \cos(pas) - \sin(t)* \sin(pas)) = x*\cos(pas) - y*\sin(pas)$

$y' = r^* \sin(t + pas) = r(\cos(t) * \sin(pas) + \sin(t)* \cos(pas)) = x*\sin(pas) + y*\cos(pas)$

Aproximarea cercurilor in spatiul discret(3)

Avantajul calculului prin recurenta: nu este necesar sa se calculeze sin si cos in fiecare iteratie!

```
// Aproximarea unui cerc printr-o linie poligonala
void Cerc( int xc, int yc, int r, int N, RGB culoare)
{ double pas = 2*M_PI /N ; //pasul unghiular
  double u, x, y, x1,y1, s, c;
  c=cos(pas); s=sin(pas); // sin si cos se calculeaza o singura data
  for( int  i = 1, x=r, y=0; i<N; i++)
  { x1 =x*c - y*s; // x'
    y1 =x*s + y*c;  // y'
    linie(xc+(int)(x+0.5), yc+(int)(y+0.5), xc+(int)(x1+0.5), yc+(int)(y1+0.5), culoare);
    x=x1 ; y= y1 ;
}
linie(xc+(int)(x+0.5), yc+(int)(y+0.5),xc+r,yc, culoare);
}
```



Aproximarea cercurilor in spatiul discret(4)

Simetria punctelor de pe cerc

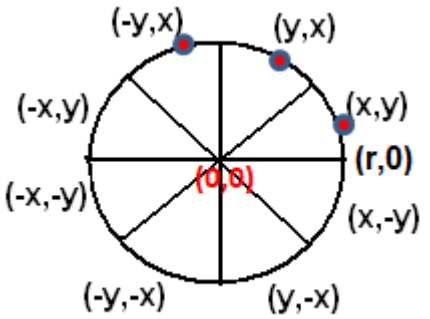
- **Fiecare punct de pe cerc are 7 puncte simetrice pe cerc:**

  - Este suficient sa se calculeze adresele punctelor dintr-un singur octant al cercului, celelalte puncte de pe cerc fiind calculate prin simetrie

// Afiseaza punctele simetrice de pe cerc

void punct_simetric(int xc, int yc, int x, int y, RGB culoare)

```
{ punct(xc+x,yc+y,culoare);
  punct(xc+x,yc-y,culoare);
  punct(xc-x,yc-y,culoare);
  punct(xc-x,yc+y,culoare);
  punct(xc+y,yc+x,culoare);
  punct(xc+y,yc-x,culoare);
  punct(xc-y,yc-x,culoare);
  punct(xc-y,yc+x,culoare);
}
```



Obs:

Simetricele punctului (r,0): (r, 0), (-r, 0), (-r,0), (0,r), (0,-r), (0,-r), (0,r)

→ pixelii (xc+r,yc), (xc-r,yc), (xc,yc+r), (xc,yc-r) vor fi suprascrisi

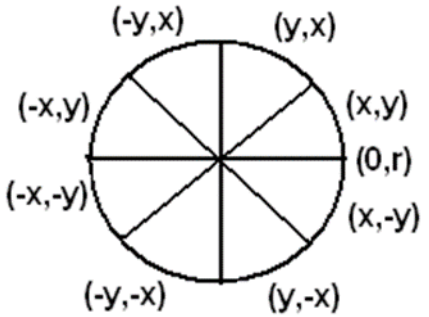
→ nu va fi apelata functia pentru punctul (r,0)

Aproximarea cercurilor in spatiul discret(5)

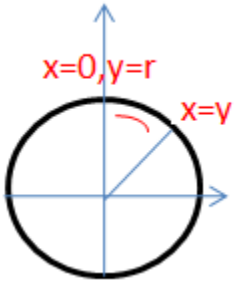
- Pentru afisarea punctelor succesive de pe circumferintă, pasul unghiular trebuie sa fie aprox. (1/ raza).

// Calculeaza numai punctele din primul octant si afiseaza toate punctele simetrice

```
void Cerc_sim( int xc, int yc, int r, RGB culoare)
{ double pas, u, x, y, xx, s, c;
  pas=1.0/r; // pas unghiular pentru obtinerea de puncte succesive de pe cerc
  c=cos(pas); s=sin(pas);
  punct(xc+r,yc,culoare);
  punct(xc-r,yc,culoare);
  punct(xc,yc+r,culoare);
  punct(xc,yc-r,culoare);
  for(u=pas, x=r, y=0; u<M_PI_4; u+=pas)
  { xx=xx;
    x =xx*c - y*s;
    y =xx*s + y*c;
    punct_simetric(xc, yc,(int)(x+0.5), (int)(y+0.5), culoare);
  }
}
```

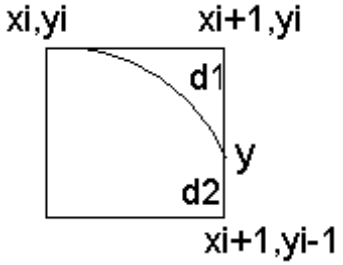


Algoritmul Bresenham pentru aproximarea cercurilor in spatial discret(1)



- Consideram cercul cu centrul în originea sistemului de coordonate si raza r.
- In cadrul algoritmului **se calculeaza numai punctele din octantul al 2-lea,** celelalte obtinandu-se prin simetrie.
- Se calculeaza punctele de pe cerc incepand cu (x=0,y=r) pana la (x=y), prin incrementarea lui x.
- Fie (x_i,y_i) ultimul punct al spatiului discret, ales pentru aproximarea cercului.

  Urmatorul punct va fi unul dintre (x_i+1,y_i) si (x_i+1,y_i-1):



Fie y ordonata punctului de pe cercul teoretic, cu abscisa x_i+1.

$$y^2 = r^2-(x_i+1)^2$$

Fie d1 si d2 distantele de la cele 2 puncte ale spatiului discret la punctul de pe cercul teoretic.

Intereseaza semnul diferentei (d1-d2):

- daca $(y_i - y) > (y- (y_i-1))$ atunci

$$(y_i^2 - y^2) > (y^2 - (y_i-1)^2)$$

$$d1' = y_i^2 - y^2 = y_i^2 - r^2 + (x_i+1)^2$$

$$d2' = y^2 - (y_i-1)^2 = r^2 - (x_i+1)^2 - (y_i-1)^2$$

Algoritmul Bresenham (2)

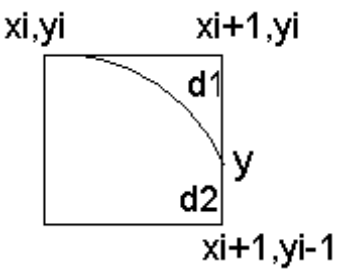
- Notam cu t_i eroarea de aproximare în pasul curent:

$t_i = d1'-d2' = y_i^2 + 2*(x_i+1)^2 + (y_i-1)^2 -2*r^2$

- Se obtine o relatie de recurenţă pentru calculul erorii de aproximare in pasul urmator:

$t_{i+1}= y_{i+1}^2 + 2*(x_{i+1}+1)^2 + (y_{i+1}-1)^2 -2*r^2$

(1) Daca **$t_i<0$** ($d1' < d2'$) atunci $x_{i+1}=x_i+1$ si $y_{i+1}=y_i$. Deci,



$t_{i+1}= y_i^2 + 2*((x_i+1)+1)^2 + (y_i-1)^2 -2*r^2$

sau **$t_{i+1} = t_i + 4*x_i + 6$**

(2) Daca **$t_i >=0$** atunci $x_{i+1} = x_i + 1$ si $y_{i+1} = y_i - 1$. Deci,

$t_{i+1} = (y_i-1)^2 + 2*((x_i+1)+1)^2 + ((y_i-1)-1)^2 -2*r^2$

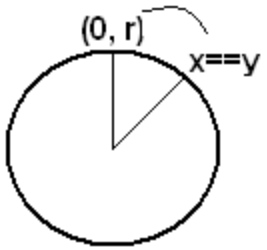
sau **$t_{i+1} = t_i + 4*(x_i - y_i) + 10$**

Algoritmul Bresenham (3)

- Valoarea erorii de aproximare în primul pas:

$t_i = y_i^2 + 2*(x_i+1)^2 + (y_i-1)^2 - 2*r^2$: $x_1=0$, $y_1 = r$. Rezultă, **t1 = 3-2*r**

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{ int x,y,t;
  t=3-(r<<1);
  punct(xc+r,yc,culoare); punct(xc-r,yc,culoare);
  punct(xc,yc+r,culoare); punct(xc,yc-r,culoare);
  for(x=1,y=r; x<y; x++)
  {
    if(t<0)
      t+=6+(x<<2);
    else
     { t+=10+((x-y)<<2); y--;}
    punct_simetric(xc,yc,x,y,culoare);
  }
// pentru x==y se afiseaza 4 puncte simetrice
  punct(xc+x,yc+y,culoare); punct(xc+x,yc-y,culoare);
  punct(xc-x,yc+y,culoare); punct(xc-x,yc-y,culoare);
}
```

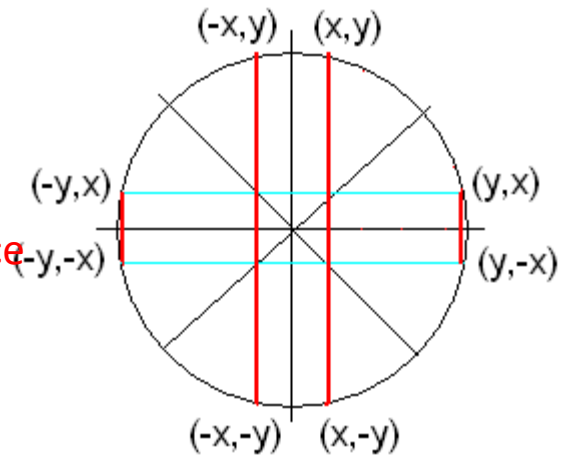


Generarea unei suprafete circulare

- Se modifică algoritmul Bresenham pentru generarea cercurilor astfel incat sa se obtina o suprafata circulara.

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{ int x,y,t;
  t=3-(r<<1);
  linie(xc, yc+r, xc, yc-r, culoare); punct(xc-r, yc, culoare); punct(xc+r, yc, culoare);
  for(x=1, y=r; x<y; x++)
  {
   if(t<0)
     t+=6+(x<<2);
   else
    { t+=10+((x-y)<<2); y--;}
   linie_simetric(xc,yc,x,y,culoare);//traseaza 4 linii simetrice
  }
 punct(xc+x,yc+y,culoare); punct(xc+x,yc-y,culoare);
 punct(xc-x,yc+y,culoare); punct(xc-x,yc-y,culoare);
}
```

De ce linii verticale si nu orizontale?



Decuparea vectorilor

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Decuparea primitivelor grafice

Decuparea: necesara pentru obiectele partial vizibile in imagine

Intr-un sistem grafic 2D:

— Decupare la frontiera unui dreptunghi din spatiul de afisare, avand laturile paralele cu cele ale sistemului de coordonate carteziene 2D (viewport)

— Decupare la frontiera unui poligon oarecare din spatiul de afisare

— Decuparea se poate efectua:

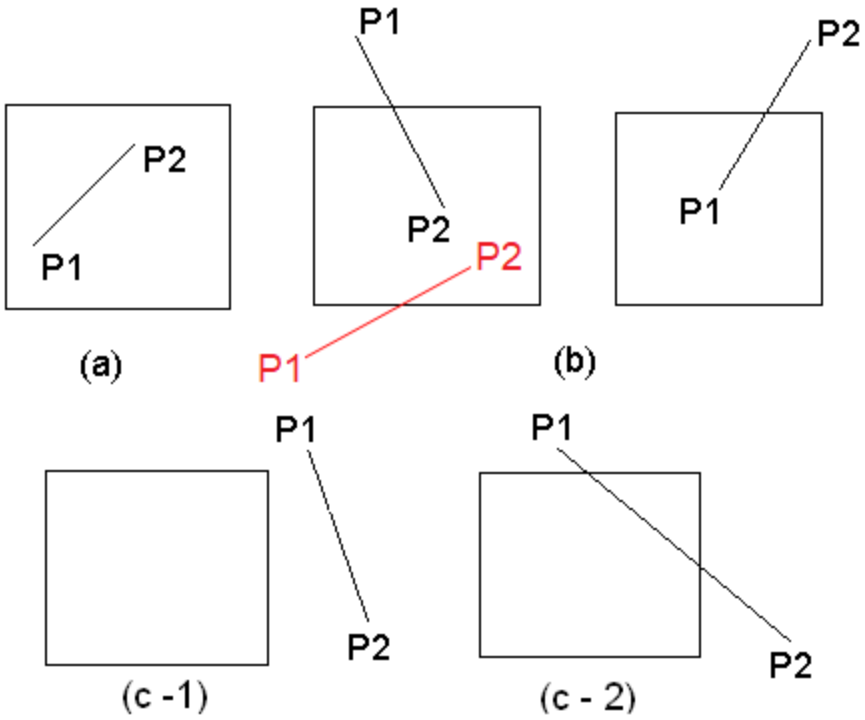
- Analitic, prin intersectia primitivelor grafice cu frontiera zonei de decupare (dreptunghi/poligon oarecare), inainte de rasterizare
- La momentul rasterizarii primitivelor, prin testul de apartenenta a fiecarui fragment la dreptunghiul/poligonul de decupare – de ex. pentru cercuri si elipse

— Pentru linii si poligoane, decuparea se efectueaza analitic (mai eficient).

Intr-un sistem grafic 3D bazat pe OpenGL:

— Decuparea se efectueaza la frontiera volumului vizual canonic, in coordonate de decupare (dupa transformarea de proiectie, inainte de transformarea in poarta de afisare).

Decuparea vectorilor 2D(1)



Pozitia vectorului fata de dreptunghiul de decupare

Presupunem urmatoarea ordine a calculelor de intersectie a unui vector P1-P2 cu dreptunghiul:

a. Calcul intersectie cu latura din stanga
b. Calcul intersectie cu latura de sus
c. Calcul intersectie cu latura din dreapta
d. Calcul intersectie cu latura de jos

In cazul vectorului rosu, calculele din pasii a, b, c sunt inutile.

Optimizari: evitarea calculelor de intersectie inutile

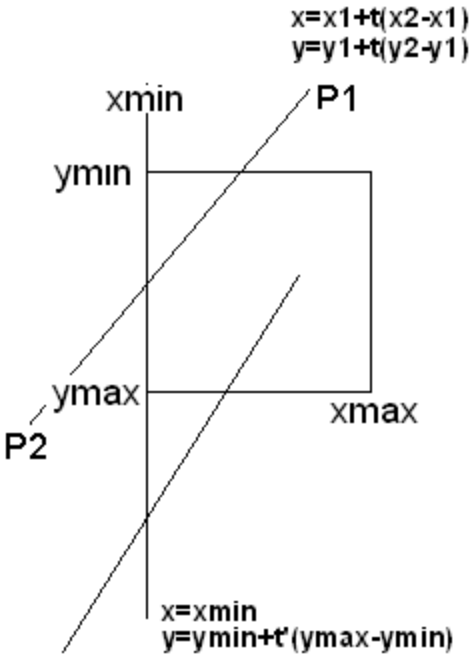
- Evitarea calculelor de intersectie in cazurile de tip (c-1):

  - cum se diferentiaza cazurile (c-1) de cazurile (c-2) fara a efectua calcule de intersectie?

- Efectuarea calculelor de intersectie **numai** pentru laturile intersectate in cazurile de tipul (b) si (c-2).

Decuparea vectorilor 2D(2)

Calculul intersectiilor cu laturile dreptunghiului de decupare



Intersectia segmentului P1-P2 cu latura x = xmin a dreptunghiului

Ec. parametrica a vectorului P1-P2:

P(t) = P1 + (P2-P1)*t 0<= t <=1 intre P1-P2

Punctul de intersectie cu dreapta x=xmin are abscisa xmin:

1) xmin=x1+t(x2-x1) → t=(xmin-x1)/(x2-x1)

2) este 0<=t<=1 ???

da: punctul de intersectie se afla pe segmentul P1-P2

- se calculeaza yi=y1+t*(y2-y1) = y1 + (xmin-x1)*(y2-y1)/(x2-x1)
- este ymin<= yi <= ymax ??

da: exista intersectie intre vectorul P1-P2 si latura x=xmin

nu: punctul de intersectie se afla pe prelungirea laturii

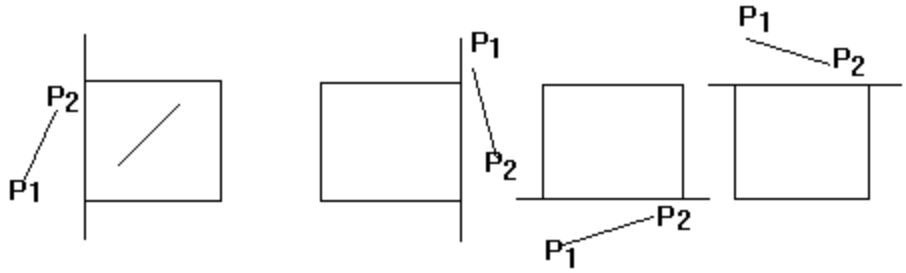
→tot calculul este inutil

Algoritmul Cohen-Sutherland (1)

1001	1000	1010
0001	0000	0010
0101	0100	0110

- Punctele din plan sunt codificate prin 4 biti, in functie de pozitia lor fata de dreptunghiul de decupare. De exemplu: b3b2b1b0
- b0=1: pentru punctele din stanga dreptunghiului,
- b1=1: dreapta, b2=1: sub dreptunghi, b3=1: deasupra dreptunghiului

Fie **cod1** codul binar al punctului P1 si **cod2** codul binar al punctului P2.



1) cod1 ==0 && cod2==0 : segment acceptat

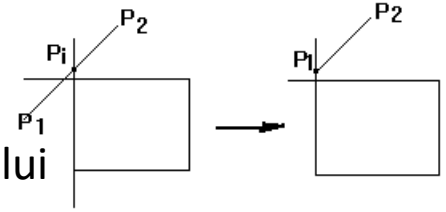
2) cod1 & cod2 != 0: segment rejectat (cele 2 coduri au acelasi bit =1; P1 si P2 de aceeasi parte a dreptunghiului) segmentul nu intersecteaza dreptunghiul

3) altfel (un varf in afara drept. celalalt in interior): se intersecteaza segmentul P1-P2 cu dreptunghiul de decupare

Algoritmul Cohen-Sutherland (2)

- Alegerea laturii cu care se intersecteaza segmentul P1-P2: pe baza codului varfului P1, care trebuie sa fie situat in afara dreptunghiului de decupare.
- Daca cod1 = 0 → se interschimba P1 cu P2

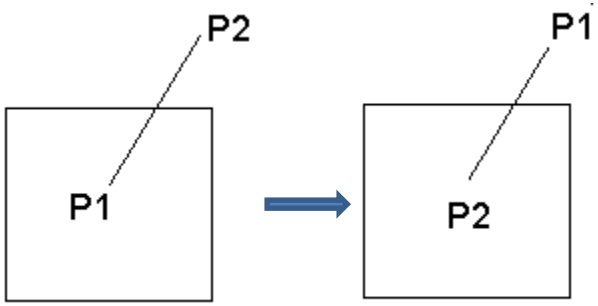
cod1 & 1 != 0
P1 in stanga dreptunghiului



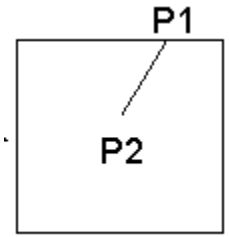
- Se muta P1 in punctul de intersectie
- In iteratia urmatoare cod1&cod2 != 0

→ segmentul este rejectat

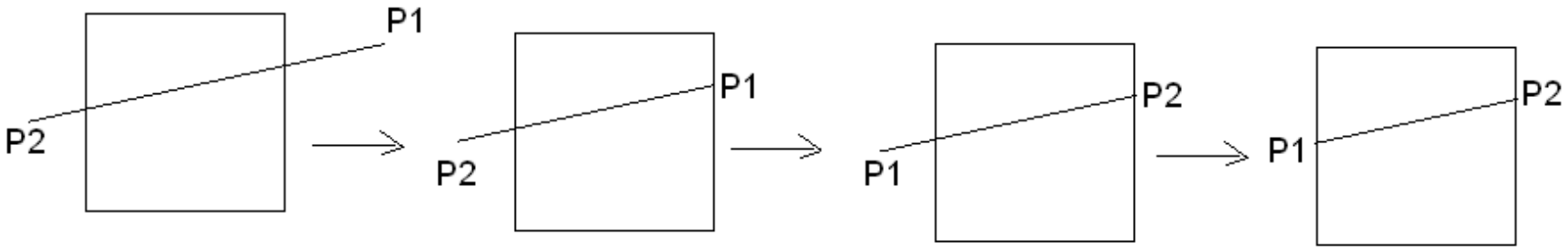
cod1 = 0



cod1 & 8 != 0
P1 deasupra dreptunghiului



Algoritmul Cohen-Sutherland (3)



Calculul punctelor de intersectie

$P(t) = P1 + (P2-P1)*t \rightarrow x(t) = x1 + t(x2-x1);\ y(t) = y1 + t(y2-y1)$

Intersectia cu latura situata pe dreapta y = ymax

ymax=y1+t(y2-y1) de unde, t=(ymax-y1)/(y2-y1)

x=x1+(ymax-y1)*(x2-x1)/(y2-y1) → **xi=x1+(ymax-y1)/m** m: panta segmentului P1-P2

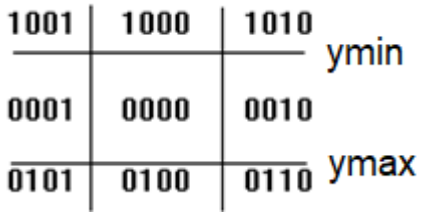
Intersectia cu latura situata pe dreapta x = xmin

xmin=x1+t(x2-x1), deci t=(xmin-x1)/(x2-x1)

y=y1+(xmin-x1)*(y2-y1)/(x2-x1) → **yi=y1+(xmin-x1)*m**

Algoritmul Cohen-Sutherland (4)

```
int xmin, ymin, xmax, ymax;  // colturile dreptunghiului de decupare

// functia calculeaza codul binar al unui punct din plan
int codif(int x, int y)
{
  int cod =0;
  if(x < xmin)  cod =1;
  if(x > xmax) cod=2;
  if(y > ymax)  cod |=4;  //b2=1
  if(y < ymin) cod |=8;  //b3=1
  return  cod;
}
```



Algoritmul Cohen-Sutherland (5)

```
int Cohen_Suth(int x1i, int y1i, int x2i, int y2i, int* x1d, int* y1d, int* x2d, int* y2d)
{ // primeste coord. capetelor vectorului de decupat si intoarce coord. partii din vector
  // inclusa in dreptunghi, daca exista
  int acceptat,rejectat,vertical;
  float m, x1=x1i, y1=y1i, x2=x2i, y2=y2i;
  int cod1, cod2, t;
  acceptat=rejectat=vertical=0;
  if (x1i != x2i)  //vectorul nu este vertical
      m=(y2-y1)/(x2-x1);
  else
      vertical=1;
 do
   { cod1=codif(x1,y1);  cod2=codif(x2,y2);
     if(cod1==0 && cod2==0) { acceptat=1; break;}
     if((cod1 & cod2) !=0 )  { rejectat=1; break;}
```

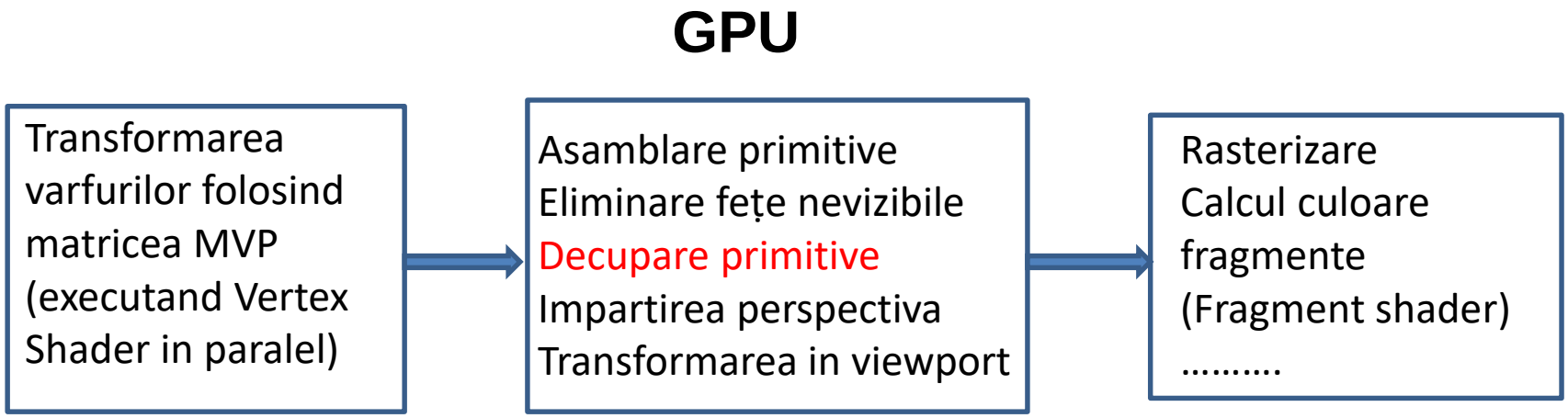
Algoritmul Cohen-Sutherland (6)

```
if(cod1==0) // P1 este in interiorul dreptunghiului de decupare
    {t=x1;x1=x2;x2=t; t=y1;y1=y2;y2=t; cod1=cod2;} //interschimbare capete
if(cod1 & 1)  //P1 in stanga dreptunghiului
     { y1+=(xmin-x1)*m;  x1=xmin;}  // punctul de intersectie devine P1 pentru iteratia urmatoare
  else
   if(cod1 & 2) //P1 in dreapta dreptunghiului
     { y1+=(xmax-x1)*m;  x1=xmax; }
      else
     if(cod1 & 4) // P1 sub dreptunghi
       { if (!vertical)   x1+=(ymax-y1)/m; //intersectia cu dreapta y = ymax
         y1=ymax;
       }
     else
       {if (!vertical) x1+=(ymin-y1)/m;
        y1=ymin;
       }
  }   while ( acceptat==rejectat); // cat timp vectorul nu este acceptat sau rejectat
```

Algoritmul Cohen-Sutherland (7)

```
 if(acceptat)
  {*x1d=x1;*y1d=y1;*x2d=x2;*y2d=y2;
   return 1;
  }
  return 0;
 }
}
```

Decuparea în banda grafica (1)



- Varfurile primitivelor pot avea asociate atribute: culoare, normala, coordonate textura
- Din decupare rezulta noi varfuri
- Pentru varfurile rezultate din decupare se calculeaza atribute prin interpolarea atributelor asociate varfurilor, folosind valoarea variabilei parametrice (t) în punctul de decupare.

Decuparea în banda grafica (2)

Transformarea de proiectie:

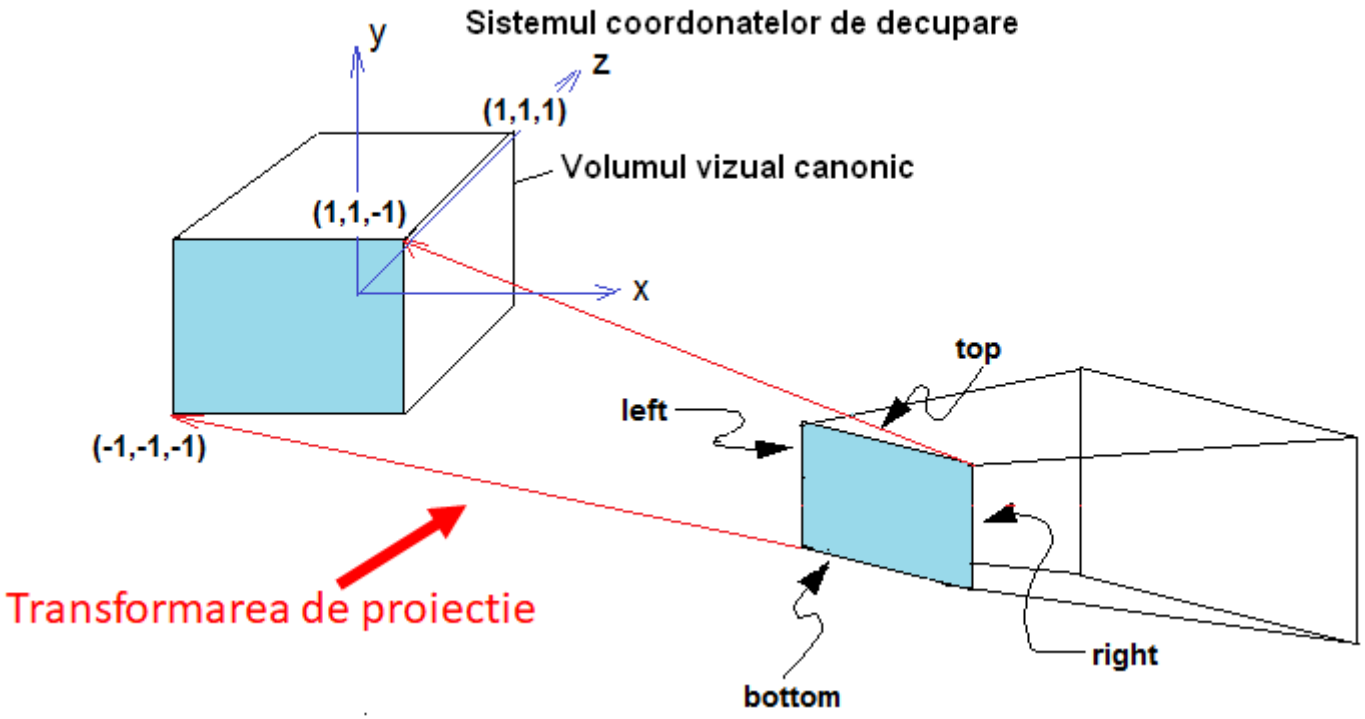
$[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ 1]^T$

(clip coordinates) ← (eye coordinates)

Coordonate omogene → împartirea perspectiva → coordonate carteziene:

xd = xc/w , yd = yc/w , zd= zc/w , w = -ze

(xd, yd, zd): coordonate dispozitiv normalizate (**N**ormalized **D**evice **C**oordinates) raportate la sistemul coordonatelor de decupare



Decuparea în banda grafica (3)

Decuparea: înainte sau dupa împarţirea perspectivă?

1. Inainte: decupare in coordonate omogene (mai complicata decat in coord. carteziene)
2. Dupa: decupare in coordonate carteziene (NDC) la marginile volumului vizual canonic

- Problema: cazul w=0 - impartirea prin zero

  $[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ 1]^T$

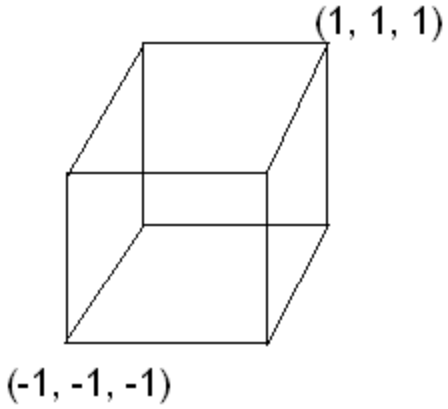
  w=-ze → w=0 daca ze =0: punctul transformat se afla in planul camerei

Problema nu apare in cazul 1, caci varfurile din planul camerei sunt eliminate la decupare, ele nefiind incluse in volumul vizual (near>0).

In cazul 2, la conversia in NDC trebuie tratat cazul special w=0.

Cel mai simplu: w←0.000001 pt a evita impartirea prin zero, caci varful este eliminat prin decupare (fața z=near → fața z=-1) , iar eroarea introdusa este neglijabila.

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(1) (decuparea in coordonate carteziene)



Faţă de volumul vizual canonic, un punct din spatiu se poate afla: in interior, in stanga, in dreapta, sub volum, deasupra volumului, in fata, in spate:

→pentru codificarea pozitiei unui punct din spatiu fata de volumul vizual canonic sunt necesari 6 biti.

De ex. se poate face conventia:

b0 = 1 pentru puncte (x,y,z) cu x < -1

b1 = 1 pentru puncte (x,y,z) cu x > 1

b2 = 1 pentru puncte (x,y,z) cu y < -1

b3 = 1 pentru puncte (x,y,z) cu y > 1

b4 = 1 pentru puncte (x,y,z) cu z < -1

b5 = 1 pentru puncte (x,y,z) cu z > 1

daca: cod(P1) ==0 && cod(P2)==0
    segment acceptat

daca (cod(P1) & cod(P2)) != 0
    segment rejectat
altfel
    se intersecteaza segmentul cu volumul

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(2)

Intersectiile segmentului cu volumul vizual

Ec. parametrice ale segmentului

$x = x1 + t(x2 – x1)$

$y = y1 + t(y2 –y1)$

$z = z1 + t(z2 – z1)$

P1-P2 se intersecteaza cu o față a volumului, aleasa pe baza codului varfului P1

daca cod(P1) & 8 !=0 //P1 deasupra volumului

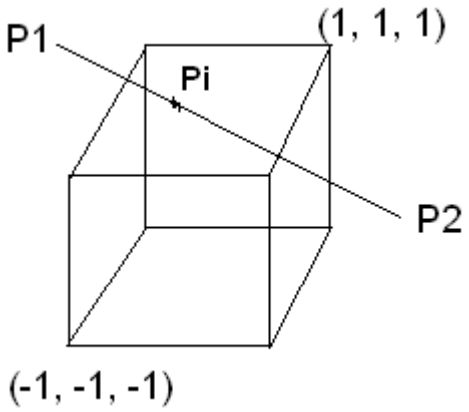
se efectueaza intersectia cu planul y= 1:

$1 = y1 + t(y2 – y1) \rightarrow tinters = (1-y1)/(y2-y1)$

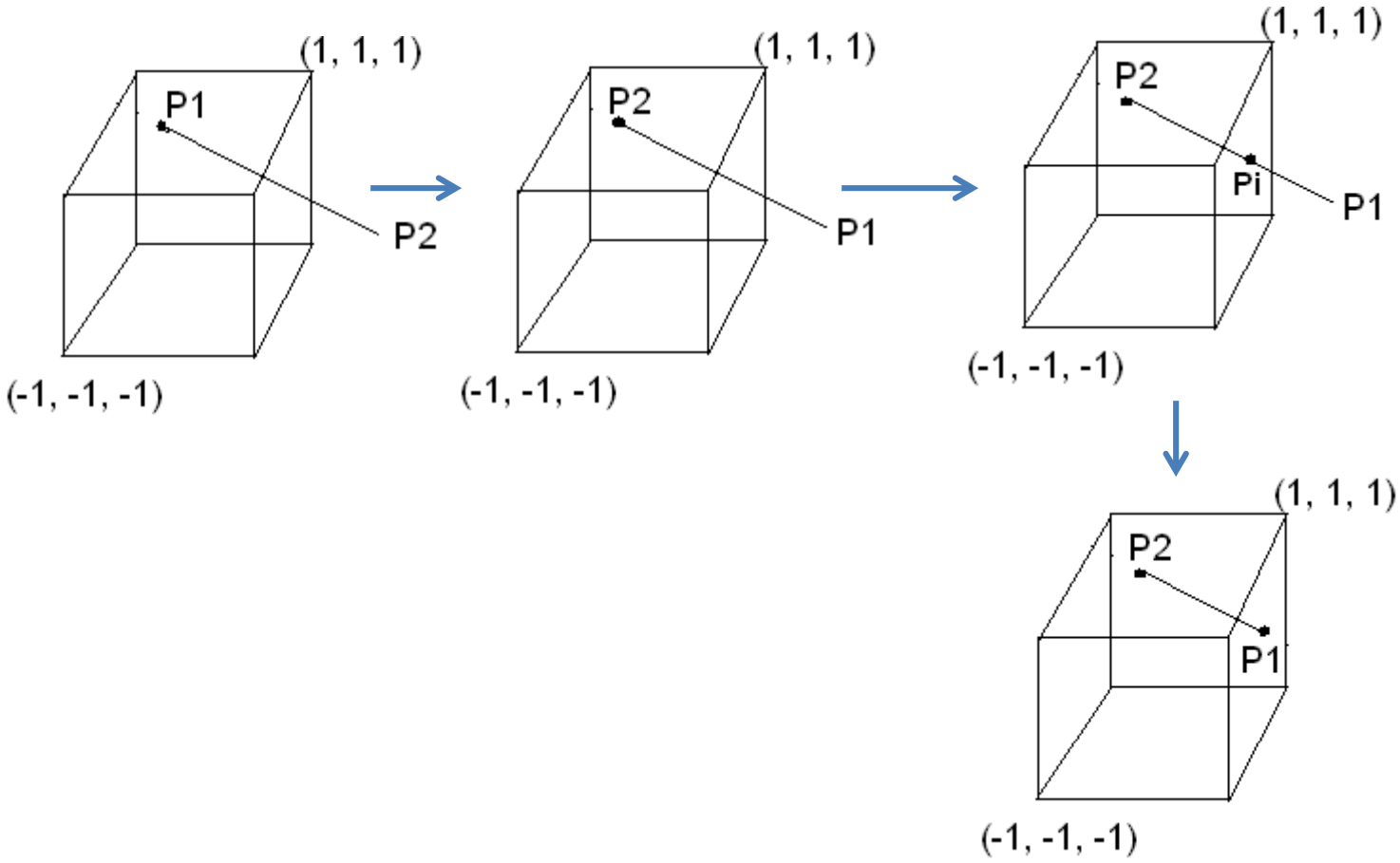
$x1 = x1 + tinters(x2 – x1)$

$y1 = 1;$

$z1 = z1 + tinters(z2 – z1)$



Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(3)

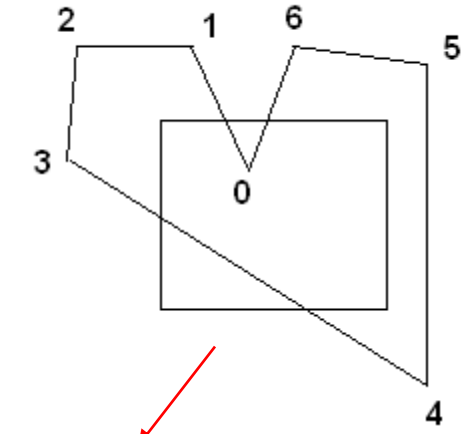


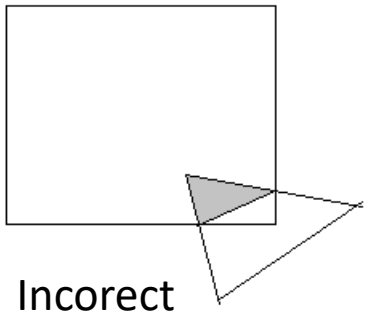
Decuparea poligoanelor

Prof. univ. dr. ing. Florica Moldoveanu

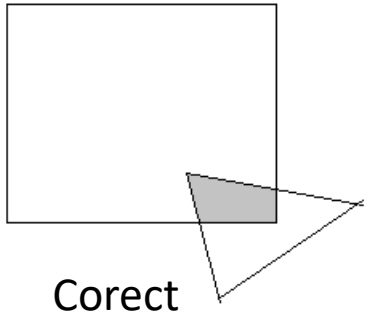
Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

Decuparea poligoanelor 2D

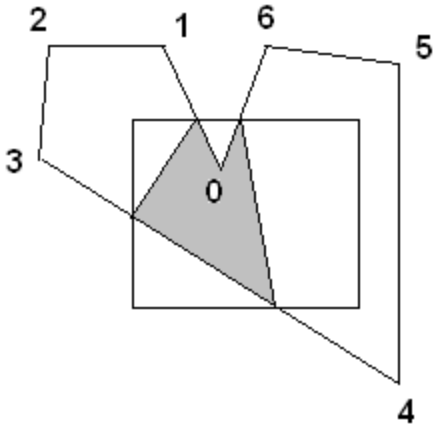


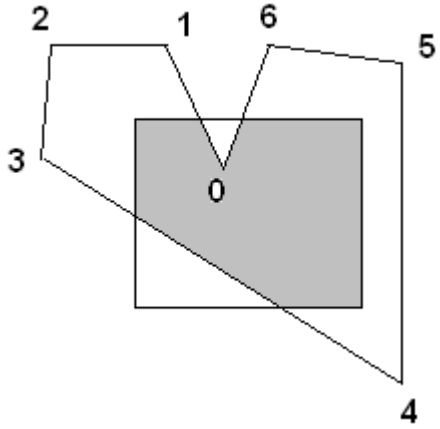


Incorect



Corect



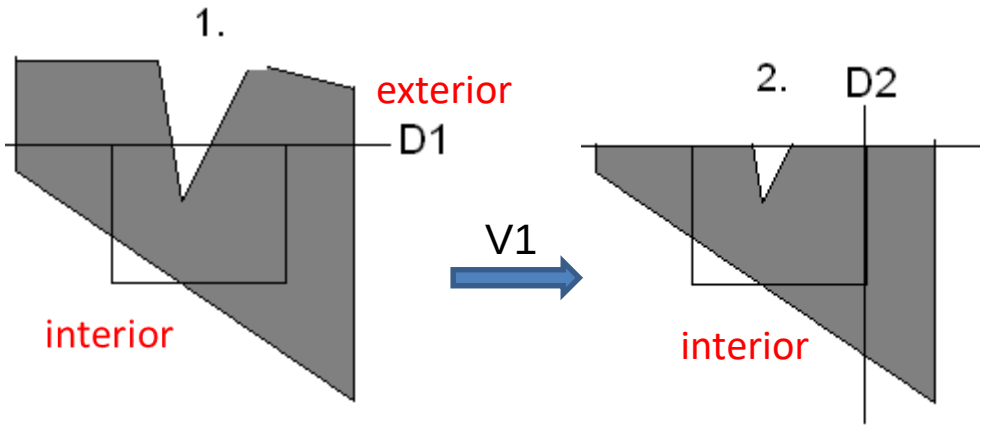


Rezultatul obtinut daca se decupeaza pe rand laturile poligonului folosind algoritmul de decupare vectori.

Rezultatul corect.

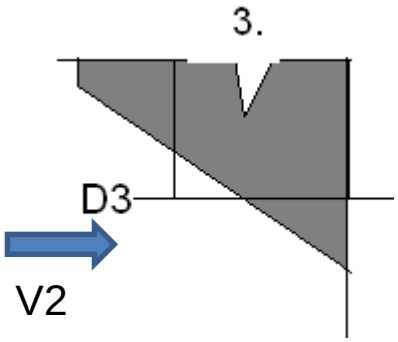
Algoritmul Sutherland-Hodgman(1)

Decuparea poligoanelor convexe față de o zona dreptunghiulară

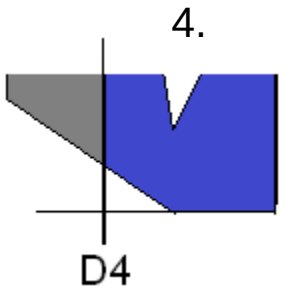


2. Se decupeaza poligonul cu varfurile din lista V1 fata de dreapta D2
   - Se creaza lista varfurilor poligonului rezultat ,V2

1. Se decupeaza laturile poligonului fata de dreapta pe care se afla o latura a zonei de decupare, D1
   - Se creaza lista varfurilor poligonului rezultat (partea din poligonul initial cuprinsa in semiplanul interior), V1



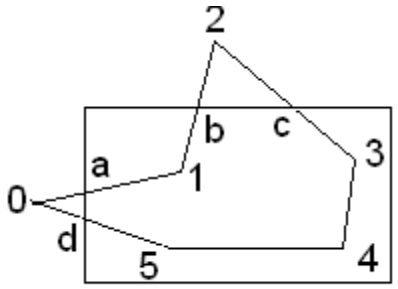
3. Se decupeaza poligonul cu varfurile din lista V2 fata de dreapta D3
   - Se creaza lista varfurilor poligonului rezultat, V3



4. Se decupeaza poligonul cu varfurile din lista V3 fata de dreapta D4
   - Se creaza lista varfurilor poligonului rezultat ,V4

Algoritmul Sutherland-Hodgman(2)

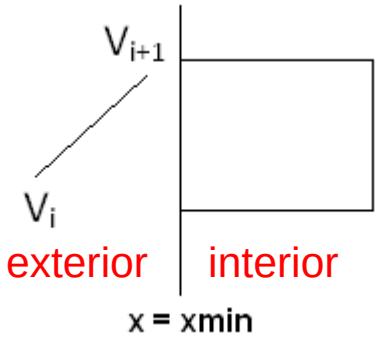
Calculul varfurilor poligonului rezultat intr-o etapa a decuparii



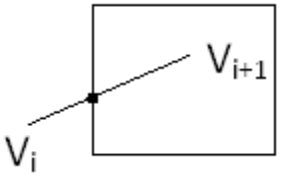
0-1: a, 1 --> a, 1
1-2: 1,b --> a, 1, 1, b
2-3: c,3 --> a, 1, 1, b, c, 3
3-4: 3,4 --> a,1, 1, b, c, 3, 3, 4
4-5: 4,5 --> a,1, 1, b, c, 3, 3, 4, 4, 5
5-0: 5,d --> a,1, 1, b, c, 3, 3, 4, 4, 5, 5, d

Lista corecta: a, 1, b, c, 3, 4, 5, d

Convenție:
- Fiecare latura a poligonului se considera un interval: [V_i -> V_{i+1})
- Numai varful V_i apartine laturii

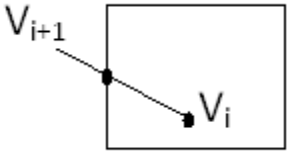


0 varfuri



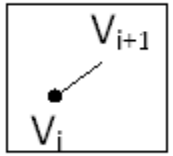
Latura intra in semiplanul interior

un varf



Latura iese din semiplanul interior

2 varfuri

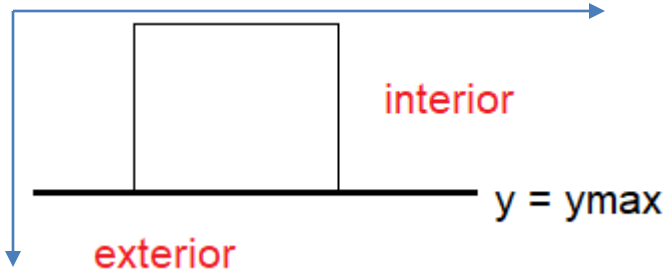


un varf

Algoritmul Sutherland-Hodgman(3)

int DecupLatura(P2D v1, P2D v2, P2d * v1d, P2D *v2d, int latura)

// decupeaza latura v1→v2, fata de dreapta pe care se afla o latura a zonei de decupare




```
{ // functia intoarce numarul de varfuri rezultate din decupare
  switch(latura)
  { case 1: // dreapta y = ymax
      if(v1.y > ymax && v2.y > ymax) return 0; // latura in semiplanul exterior al zonei de decupare
      if(v1.y <= ymax && v2.y <= ymax) { *v1d = v1; return1;} // latura in semiplanul interior
      if(v1.y <= ymax && v2.y > ymax) // latura iese din semiplanul interior
          { * v1d = v1; x_intersectie = …; v2d->x = x_intersectie; v2d->y = ymax; return 2;}
      else// v1.y > ymax && v2.y <= ymax  // latura intra in semiplanul interior
         { x_intersectie = ….; v1d->x = x_intersectie; v1d->y = ymax; return 1;}
    case 2:  ……..
  }
}
```

Algoritmul Sutherland-Hodgman(4)

void DecupLaturi(int nv, P2D *vrf, int *nd, P2D *vrfd, int latura)

```
{ // decupeaza toate laturile poligonului fata de dreapta pe care se afla o latura a zonei de
      decupare
 // vrf: lista varfurilor poligonului de decupat
//  nv: numarul de varfuri ale poligonului de decupat
 // vrfd: lista varfurilor poligonului rezultat din decupare
//  nd: numarul de varfuri ale poligonului rezultat din decupare


   *nd = 0;

   for(int i =0; i< nv; i++)

       *nd+=  DecupLatura(vrf[i], vrf[i+1], &vrfd[*nd], &vrfd[*nd+1], latura);

  *nd+= DecupLatura(vrf[nv-1], vrf[0], &vrfd[*nd], &vrfd[*nd + 1], latura);

 }
```

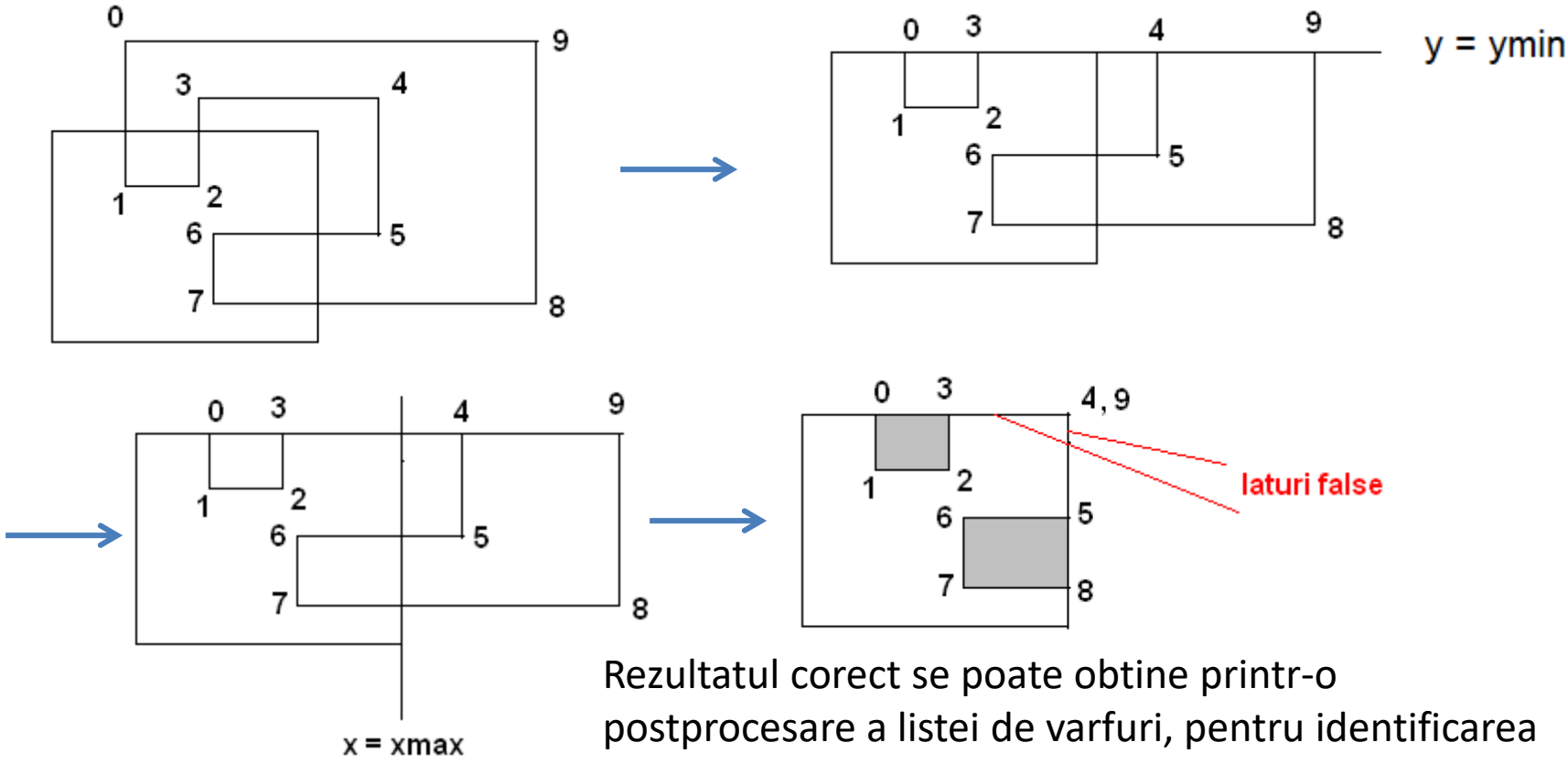
Algoritmul Sutherland-Hodgman(5)

```
void DecupPoligon(int nv, P2D * vrf, int *nd, P2D **vrfd)
{ // vrf: lista varfurilor poligonului de decupat
 // vrfd: lista varfurilor poligonului rezultat din decupare
   P2D * v1 = new P2D[2*nv]; P2D * v2 = new P2D[2*nv]; // se folosesc 2 liste de varfuri
   memcpy(v1, vrf, nv*sizeof(P2D));
  for(int latura =1 ; latura <= 4; latura+ =2) // zona de decupare un dreptunghi
  { DecupLaturi(nv, v1, nd, v2, latura);
    DecupLaturi(*nd, v2, &nv, v1, latura+1);
  }
*nd = nv;
*vrfd = new P2D[nv];
  memcpy(*vrfd, v1, nv*sizeof(P2D));
  delete v1; delete v2;
}
```

In cazul unei zone de decupare poligon oarecare, decuparea se efectueaza faţă de fiecare latura a poligonului.

Algoritmul Sutherland-Hodgman(6)

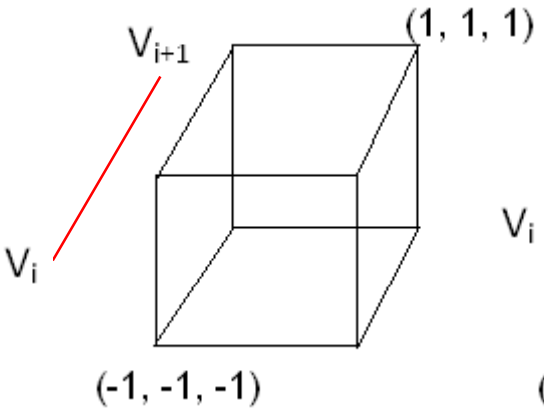
Limitarea algoritmului Sutherland-Hodgman: la decuparea poligoanelor concave, atunci cand din decupare pot rezulta mai multe poligoane. Algoritmul produce o singura lista de varfuri: un singur poligon, cu posibile laturi false.



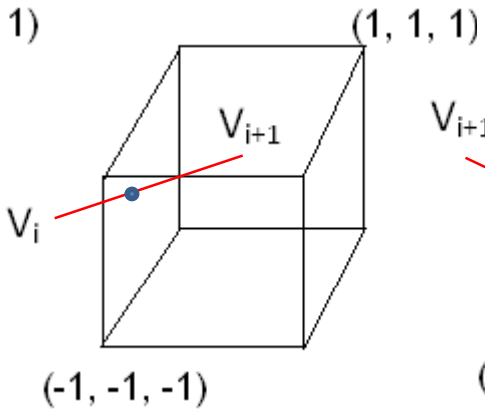
Rezultatul corect se poate obtine printr-o postprocesare a listei de varfuri, pentru identificarea ciclurilor de varfuri.

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(1)

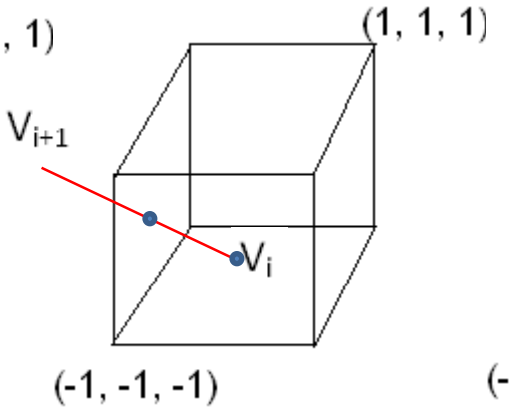
- Decuparea unui poligon 3D se efectueaza la frontiera volumului canonic.
- Se intersecteaza poligonul, pe rand, **cu cele 6 planuri** care delimiteaza volumul.
- Fiecare plan de intersectie imparte spatiul in 2 semispatii:
  - semispatiul interior – care conține volumul
  - semispatiul exterior
- Din fiecare intersectie pot rezulta noi varfuri. Exemplu: intersectia cu planul x=-1



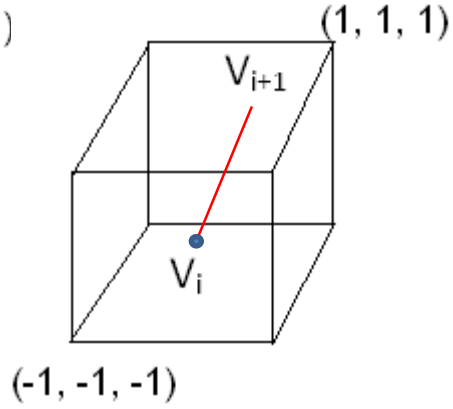
0 varfuri



1 varf: intersectia vectorului cu planul x=-1



2 varfuri: Vi si intersectia vectorului cu planul x=-1



1 varf: varful Vi

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(2)

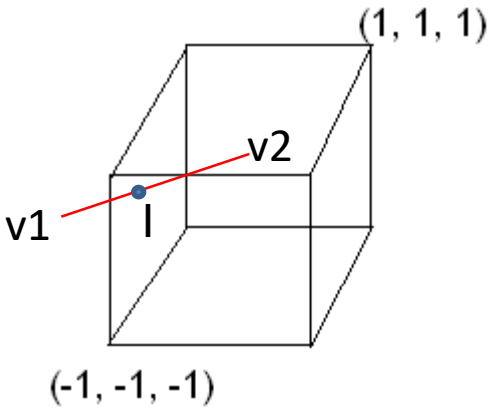
Intersectia unei laturi V1-V2 cu planul x=-1

Ec. parametrice ale laturii

$x = x1 + t(x2 - x1)$

$y = y1 + t(y2 - y1)$

$z = z1 + t(z2 - z1)$



Intersectia cu planul x=-1

$-1 = x1 + t(x2-x1)$ → $tinters = -(x1+1)/(x2-x1)$

$xi = -1$

$yi = y1 + tinters *(y2-y1)$

$zi = z1 + tinters *(z2-z1)$

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(3)

```
int DecupLatura(P3D v1, P3D v2, P3d * v1d, P3D *v2d, int plan)
// decupeaza latura v1->v2 fata de planul in care se afla o față a volumului de decupare
{ // functia intoarce numarul de varfuri rezultate din decupare
  float tint;
  switch(plan)
  { case 1: // planul x = -1
    if(v1.x < -1 && v2.x < -1) return 0; // latura in semispatiul exterior

    if(v1.x >= -1 && v2.x >= -1 ) { *v1d = v1; return1;} // latura in semispatiul interior

    if(v1.x <-1 && v2.x >= -1) // latura intra in semispatiul interior
    {tint = -(v1.x+1)/(v2.x-v1.x);  v1d->x = -1; v1d->y = v1.y + tint*(v2.y-v1.y); v1d->z= v1.z + tint*(v2.z-v1.z);
    return 1;}
    else // v1.x >= -1 && v2.x  < -1) // latura iese din semispatiul interior
    { * v1d = v1; tint = -(v1.x+1)/(v2.x-v1.x);  v2d->x = -1; v2d->y = v1.y + tint*(v2.y-v1.y);
        v2d->z= v1.z + tint*(v2.z-v1.z) ; return 2;}
   case 2: …….
…… } …….}
```

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(4)

void DecupLaturi(int nv, P3D *vrf, int *nd, P3D *vrfd, int plan)

```
{ // decupeaza toate laturile poligonului fata de planul in care se afla o față a volumului de
     decupare
 // vrf: lista varfurilor poligonului de decupat
//  nv: numarul de varfuri ale poligonului de decupat
 // vrfd: lista varfurilor poligonului rezultat din decupare
//  nd: numarul de varfuri ale poligonului rezultat din decupare


   *nd = 0;

   for(int i =0; i< nv; i++)

       *nd+=  DecupLatura(vrf[i], vrf[i+1], &vrfd[*nd], &vrfd[*nd+1], plan);

  *nd+= DecupLatura(vrf[nv-1], vrf[0], &vrfd[*nd], &vrfd[*nd + 1], plan);

 }
```

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(5)

```
void DecupPoligon(int nv, P3D * vrf, int *nd, P3D **vrfd)
{ // vrf: lista varfurilor poligonului de decupat
 // vrfd: lista varfurilor poligonului rezultat din decupare
   P3D * v1 = new P3D[2*nv]; P3D * v2 = new P3D[2*nv]; // se folosesc 2 liste de varfuri

   memcpy(v1, vrf, nv*sizeof(P3D));

  for(int plan =1 ; plan <= 6; plan+ =2) // se decupeaza laturile poligonului fata de cele 6

      //planuri in care se afla fețele volumului de decupare

  { DecupLaturi(nv, v1, nd, v2, plan);

    DecupLaturi(*nd, v2, &nv, v1, plan+1);

  }

 *nd =nv; *vrfd = new P3D[nv];

  memcpy(*vrfd, v1, nv*sizeof(P3D));
  delete v1; delete v2;
}
```

Iluminarea globala
-Ray Tracing-

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2019-2020

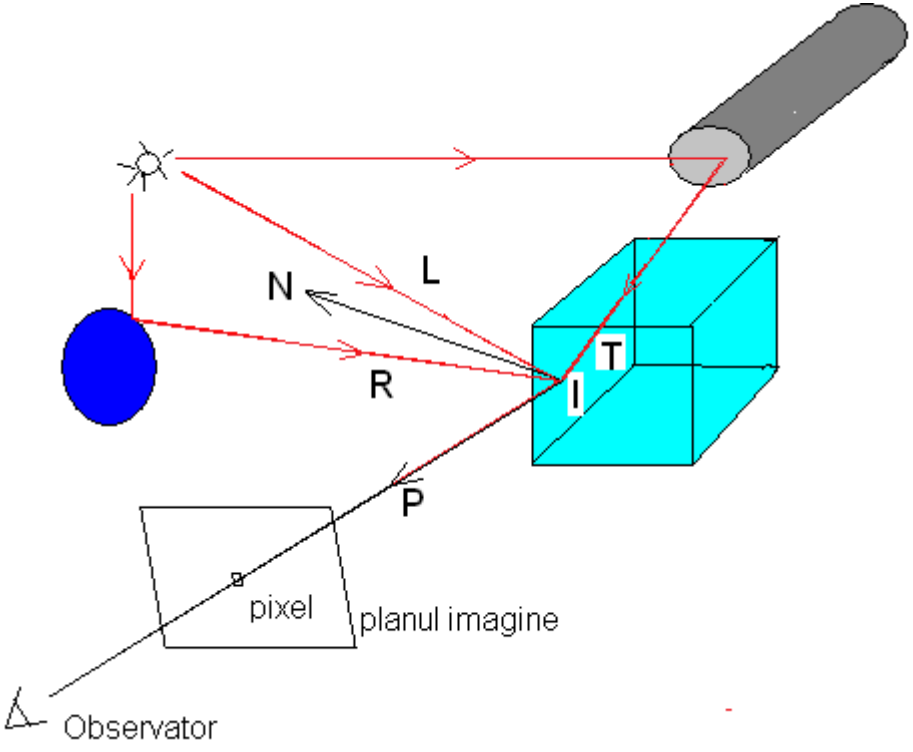
Iluminarea globala in Ray tracing(1)

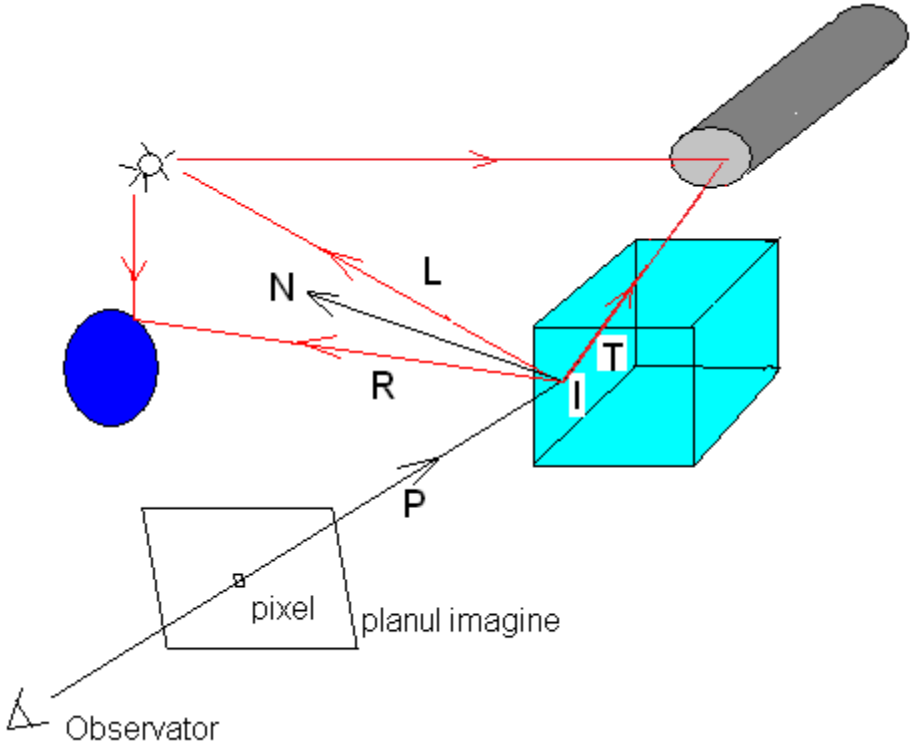
Imbina:

- Eliminarea partilor nevizibile ale scenelor 3D
- Calculul reflexiei luminii
- Calculul refractiei luminii (transparenta)
- Calculul umbrelor
- Interactiunea globala a reflexiei si refractiei luminii la nivelul intregii scene 3D

- In algoritmul Ray Tracing:
  - Imaginea unei scene 3D NU se obtine prin rasterizarea primitivelor grafice in care a fost descompusa scena
  - Imaginea este calculata intersectand scena 3D cu raze care pleaca din pozitia observatorului, fiecare trecand printr-un pixel din planul imaginii
- Culoarea unui pixel este calculata in punctul de intersectie al razei cu scena, care este cel mai apropiat de observator.

Iluminarea globala in Ray tracing(2)





Directia razelor de lumina **in realitate**.

Pixelul este colorat in culoarea punctului I.
P este directia pe care lumina din punctul I ajunge la observator.

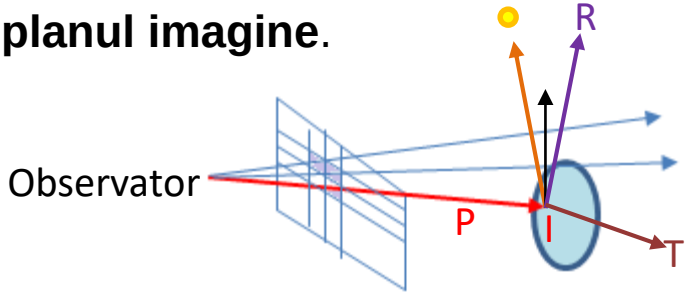
In algoritmul Ray-tracing,

pentru calculul culorii pixelului se considera raze cu directia inversa celor din realitate

Algoritmul Ray tracing

Imaginea se calculeaza pixel cu pixel, pornind de la planul imagine.

Considerand o singura sursa de lumina:



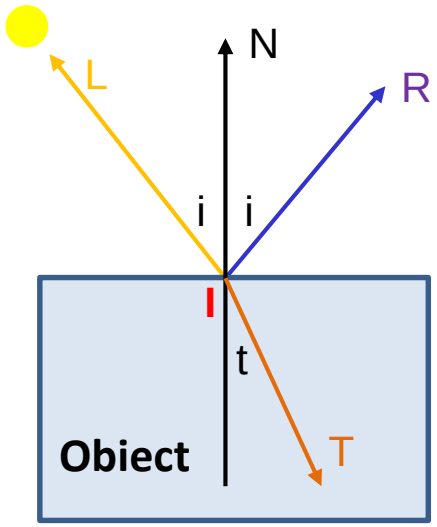
Pentru fiecare pixel al imaginii:

- Se calculeaza raza primara, P, care porneste din pozitia observatorului si trece prin centrul pixelului, in planul imaginii
- Se determina intersectiile razei primare cu scena si punctul de intersectie**, I**, cel mai apropiat de observator
- **Daca** raza nu intersecteaza nici un obiect al scenei,
  - Se afiseaza pixelul in culoarea fondului

  altfel
  - Se calculeaza culoarea in punctul I, tinand cont de:
    - Reflexia luminii provenita direct de la sursa de lumina, raza L
    - Lumina provenita in I de la alte obiecte pe directia reflexiei speculare, raza R
    - Lumina provenita in I de la alte obiecte pe directia razei transmise, raza T
  - Se afiseaza pixelul in culoarea obtinuta prin combinarea contributiei celor trei raze

Ray tracing – calculul razelor

Calculul razelor care contribuie la culoarea (lumina) in punctul de intersectie cu raza primara



L: raza din I catre sursa de lumina

R: raza reflexiei speculare, simetrica cu L, fata de N,

$$R = 2(N \cdot L) \cdot N - L$$

Pe directia R ajunge lumina in I de la celelalte obiecte ale scenei daca **obiectul** produce reflexie speculara.

T: raza refractiei luminii in I, calculata pe baza legii lui Snell:

$n1/n2 = \sin(t)/\sin(i)$, n1,n2: indicii de refractie

$T = L*(n1/n2) - (\cos(t) + (n1/n2)*(L \cdot N))*N$

Pe directia T ajunge lumina in I de la celelalte obiecte ale scenei, daca **obiectul** este transparent.

✓*Reflexia speculara si refractia au loc fara împrastiere (sunt perfect focalizate):* **reflexia speculara este considerata strict pe directia R şi nu într-un con de raze, ca in modelul Phong.**

➢**Efect**: obiectele din imaginea produsa sunt de regula stralucitoare, producand reflexii multiple focalizate.

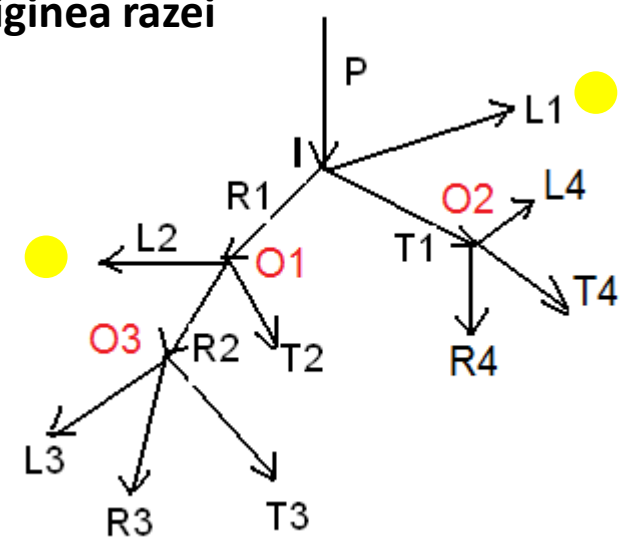
Ray tracing – arborele de raze

Algoritmul Ray tracing este recursiv:

Lumina provenita in punctul I prin reflexie speculara de la un obiect O1 (de-a lungul razei R1) sau prin transmisie de la un obiect O2 (de-a lungul razei T1), poate fi compusa din:

- Reflexia luminii provenita direct de la o sursa, de catre O1/O2 – raza L2/L4
- Reflexia speculara a altor obiecte de catre O1/O2 – raza R2/R4
- Transmisia luminii provenita de la alte obiecte ale scenei prin O1/O2 – raza T2/T4

➢ **Pentru fiecare raza se considera punctul de intersectie cu scena, care este cel mai apropiat de originea razei**



Arborele de raze

➢*Pentru obtinerea culorii in punctul I, se evalueaza arborele de raze de la frunze catre radacina*

Ray tracing- calculul reflexiei luminii(1)

Calculul culorii în punctul I:

$$I_\lambda(I) = I_{local\lambda}(I) + Ks*R_\lambda(I) + Kt*T_\lambda(I)$$

unde:

λ – reprezinta lungimea de unda: expresia se evalueaza pentru R,G,B

$I_{local\lambda}(I)$ – reprezinta reflexia luminii provenite direct de la sursele de lumina din scena 3D (calculata folosind modelul de iluminare locala)

Ks – este coeficientul de reflexie speculara al materialului obiectului

$R_\lambda(I)$ – reprezinta reflexia speculara a luminii provenite de la alte obiecte ale scenei in punctul I: **se obtine prin evaluarea arborelui de raze**

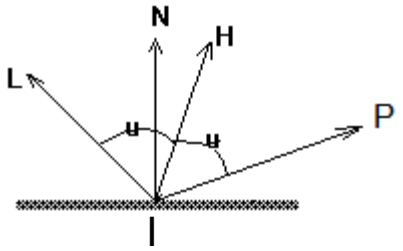
Kt – este coeficientul de transmisie, specific materialului obiectului

$T_\lambda(I)$ - reprezinta lumina provenita prin transmisie (refractie) in punctul I de la alte obiecte ale scenei: **se obtine prin evaluarea arborelui de raze**

0<= ks, kt <= 1

Ray tracing- calculul reflexiei luminii(2)

Modelul de iluminare locala (reflexia lumii in punctul I) :



$I_{local\lambda}(I) = Ia_{\lambda} * Ka + Isursa_{\lambda} * fat * s * lum * [kd * (N \cdot L) + Ks * (N \cdot H)^{n}]$

Iaλ – reprezinta intensitatea luminii ambiante

Ka – este coeficientul de difuzie a luminii ambiante, specific materialului obiectului, 0<= ka <= 1

$Isursa_{\lambda}$ – reprezinta intensitatea luminii provenite de la sursa

L – este versorul directiei din punctul I catre pozitia sursei de lumina

N – este normala in punctul I (versor)

H – este versorul directiei bisectoare a unghiului dintre L si vectorul din I catre observator (raza P)

fat – este factorul de atenuare a luminii de la sursa, proportional cu distanta de la sursa la punctul I

0<= s <= 1,

s = 1, daca vectorul L nu intersecteaza un alt obiect al scenei

s = 0, daca lumina de la sursa nu ajunge in punctul I: L intersecteaza un obiect opac al scenei

0 < s <1, daca vectorul L intersecteaza un obiect transparent (sau mai multe)

Punctul I primeste lumina de la sursa daca:

- produsul scalar (N·L)>0 (lum =1, altfel =0)
- raza L nu intersecteaza un obiect opac al scenei

➢ **Daca I nu primeste lumina de la sursa, iluminarea locala se reduce la componenta ambianta.**

Ray tracing- calculul reflexiei luminii(3)

- Daca in scena 3D exista mai multe surse de lumina, fiecare poate contribui in mod diferit la $I_{local\lambda}(I)$:

$$I_{local\lambda}(I) = Ia_{\lambda} * Ka + \sum_{i=1,n} Isursa_{i,\lambda} * fat_i * s_i * lum_i * [kd * (N \cdot L_i) + Ks * (N \cdot H_i)^n]$$

- Pentru evaluarea componentelor $R_{\lambda}(I)$ si $T_{\lambda}(I)$ din calculul culorii in I,

$$I_{\lambda}(I) = I_{local\lambda}(I) + Ks * R_{\lambda}(I) + Kt * T_{\lambda}(I)$$

se coboara in arborele de raze pana la un numar pre-specificat de nivele (energia luminoasa scade destul de repede!).

Algoritmul Ray tracing

Algoritmul Ray tracing

Pentru fiecare pixel al imaginii

```
{    *calculeaza raza primara, P;//din pozitia observatorului prin centrul suprafetei pixelului
     *culoare_pixel = TraseuRaza(P, 1); // culoarea in punctul de intersectie al razei P cu scena
     *afiseaza pixelul in culoare_pixel;
}
```

Culoare TraseuRaza**(Raza R, int n)** //apelata pt. toate razele care contribuie la cul. unui pixel

```
{    // n este nivelul in arborele de raze
     // intoarce culoarea in punctul de intersectie al razei R cu scena
     *calculeaza intersectiile razei R cu obiectele scenei;
     daca (nu exista intersectii), atunci
               return  (culoare_fond);
     altfel
               *fie I punctul de intersectie cel mai apropiat de observator si O obiectul intersectat;
               *calculeaza normala, N, in punctul I;
               return  CuloarePunct(O, R, I, N, n); // culoarea in I
}
```

Algoritmul Ray tracing

```
Culoare CuloarePunct(Obiect O, Raza R, Punct I, Normala N, adancime_arbore n)
{    Culoare  culoare;
     culoare = culoare_ambianta;
     pentru fiecare sursa de lumina S executa //calculeaza iluminarea locala
          *calculeaza vectorul L, din I catre S
           daca ((Nu·Lu) > 0 si vectorul L nu intersecteaza un obiect opac al scenei) atunci
             *calculeaza contributia sursei S la culoare, CS, folosind modelul de iluminare locala
              culoare = culoare + CS
     daca (n<=nivel_max) atunci // n>nivel_max: iesirea din recursivitate
          daca (obiectul O produce reflexii speculare) atunci
               *calculeaza raza reflectata in punctul I, RS;
                culoare = culoare + Ks* TraseuRaza(RS, n+1); // se coboara in arbore
          daca (obiectul O este transparent) atunci
               *calculeaza raza transmisa (refractata) in punctul I, RT;
                culoare = culoare + Kt* TraseuRaza(RT, n+1); //se coboara in arborele de raze
     return (culoare)
}
```

Algoritmul Ray tracing

Deficienţele algoritmului

1. **Efecte de aliasing**: razele sunt convergente si infinit subtiri, esantionarea spatiului 3D este neuniforma
   - Se produc defecte la marginile suprafetelor (aliasing: marginile au aspect zimţat)
   - Obiectele mici si subtiri pot sa apara si sa dispara din imagine, la schimbarea pozitiei observatorului

   Imbunatatire:

   – marirea rezolutiei esantionarii spatiului 3D; ex. 4 raze primare/pixel → creste complexitatea

2. **Complexitatea computationala**
   - Buna din punct de vedere teoretic:

     - pentru o raza/pixel este de O(p*n), unde p este nr. de pixeli - constant (independent de scena) = O(n), unde n este nr. de obiecte din scena
   - Creste liniar cu numarul de raze/pixel
   - Complexitatea calculelor de intersectie depinde de geometria obiectelor intersectate

   Imbunatatire:

   – Reducerea calculelor de intersectie, prin folosirea de volume incadratoare ale obiectelor si gruparea obiectelor din scena 3D

   – Complexitatea se poate reduce la O(log (n))

Imbunatatirea calitatii imaginii(1)

Marirea numarului de raze primare

1) Uniforma

4 raze primare/pixel, care trec prin colturile suprafetei pixelului

- Culoarea pixelului se determina ca medie a culorilor obtinute cu cele 4 raze primare
- Razele primare utilizate pentru un pixel contribuie si la culoarea pixelilor adiacenti
  - Pentru o imagine de mxn pixeli: (m+1)x(n+1) raze→ numarul de raze creste cu (m+n+1)

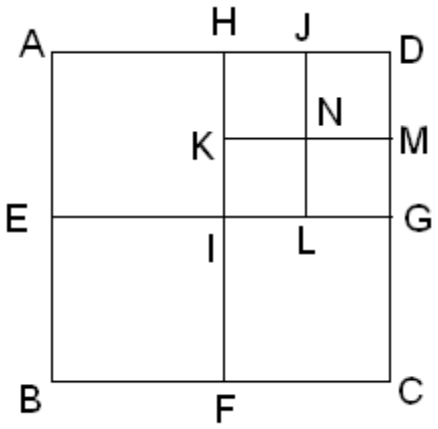
2) **Adaptiva** (Whitted) – subdivizarea adaptiva a suprafetei pixelului

- Se creste rezolutia eşantionarii spatiale numai in zonele in care este necesara operatia de anti-aliasing:
- Daca diferenta dintre culorile celor 4 raze primare pentru un pixel este mare, se subdivizeaza suprafata pixelului in 4 subzone si se duc raze prin colturile subzonelor
- Se aplica acelasi criteriu de comparatie intre culorile celor 4 raze primare corespunzatoare unei subzone
- Subdivizarea se continua recursiv pana la un nivel maxim prestabilit sau pana cand diferenta dintre cele 4 culori scade sub un prag dat
- Culoarea pixelului se obtine ca o medie ponderata a culorilor subzonelor in care a fost divizata suprafata pixelului

Imbunatatirea calitatii imaginii(2)

Imbunatatirea calitatii imaginii(2)

Exemplu de subdivizare adaptiva



CA, CB, CC, CD, CE, CF, CG, CH, CI, CJ, CK, CL, CM, CN

- sunt culorile obtinute cu razele primare care trec prin punctele respective

Culoare pixel (A-B-C-D) =

¼((CA+CE+CI+CH)/4 + (CE+CB+CF+CI)/4 + (CI+CF+CC+CG)/4 +

¼((CH+CK+CN+CJ)/4 + (CK+CI+CL+CN)/4 + (CJ+CN+CM+CD)/4 +

(CN+CL+CG+CM)/4))

Reducerea complexitatii computationale

Reducerea calculelor de intersectie

1. **Utilizarea de volume incadratoare la nivel de obiect**

   Se testeaza intersectia raza-volum incadrator in loc de raza-obiect:

   - Daca raza nu intersecteaza volumul incadrator → nu se va calcula intersectia cu obiectul

2. **Divizarea scenei in volume incadratoare**

   Se divizeaza scena in volume egale pana la o anumita rezolutie spatiala sau se divizeaza scena adaptiv.

   Se testeaza intersectia razei cu volumele in care a fost divizata scena.

3. **Reprezentarea scenei printr-o ierarhie de volume incadratoare**

   Scena este reprezentata printr-un arbore in care frunzele sunt obiectele scenei iar nodurile interne sunt grupari logice de obiecte ale scenei.

   Fiecare nod are un volum incadrator.

Utilizarea de volume incadratoare (1)

Utilizarea de volume incadratoare la nivel de obiect/ grup de obiecte

- Permite evitarea calculelor de intersectie cu obiecte ale scenei care pot avea geometrie complexa
- Permite eliminarea din calculele de intersectie a unui intreg grup de obiecte care nu este intersectat de raza

Volume incadratoare:

- Sfera, paralelipipedul cu fetele paralele cu planele principale, elipsoidul, cilindrul.
- Calculul intersectiei raza-volum incadrator trebuie sa fie mai simplu decat calculul intersectiei cu obiectul (de ex. o retea poligonala)
- Volumul incadrator al unui obiect se alege in functie de forma obiectului

Utilizarea de volume incadratoare (2)

Calcule de intersectie raza-volum incadrator

Ecuatia razei: r(t) = P0 + t*D ,

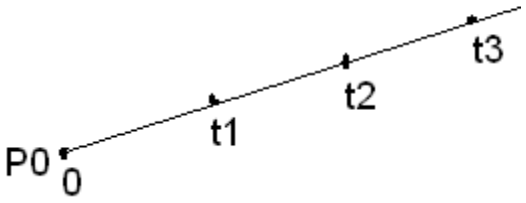
P0 – este originea razei

D – este directia razei

$x = x0 + t*dx$

$y = y0 + t*dy$

$z = zo + t*dz$



Punctul cel mai apropiat de originea razei este r(tmin)

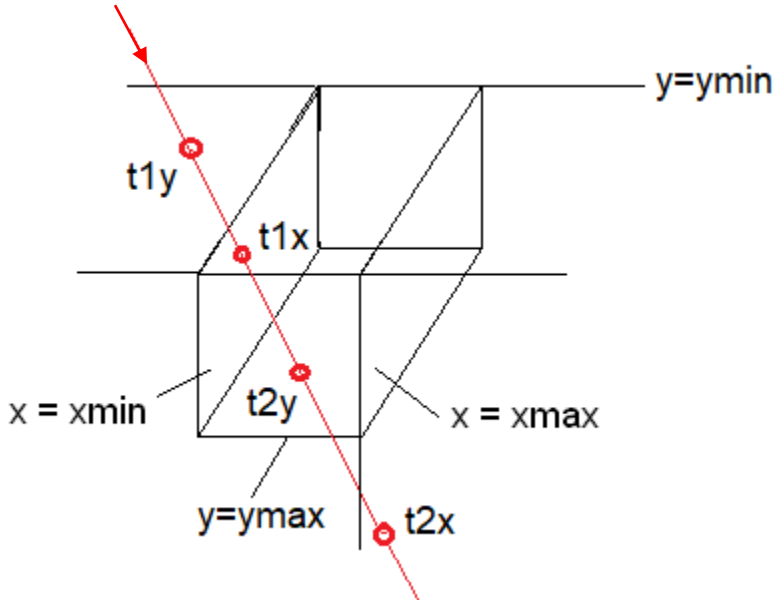
1) Intersectia cu sfera

$(x-a)^2 + (y-b)^2 + (z-c)^2 = r^2$

- Se inlocuiesc in ec sferei x, y, z, cu cele din ecuatia razei
- Rezulta o ecuatie de grad 2 in t
- Se calculeaza discriminantul D, al ecuatiei:
  - D<0 – raza nu intersecteaza sfera
  - Daca sfera este obiect al scenei (intereseaza punctul de intersectie):
    - D=0 – raza este tangenta la sfera
    - D>0 – se calculeaza radacinile, t1, t2
      - punctul de intersectie mai apropiat de observator este r(tmin(t1,t2))

Utilizarea de volume incadratoare (3)

2) Intersectia cu un paralelipiped cu fetele paralele cu planele principale



Planele care delimiteaza volumul:

x= xmin, x = xmax, y=ymin, y = ymax, z= zmin, z= zmax

Intersectia cu x=xmin si x=xmax

xmin = x0 + t*dx → t1x = (xmin – x0)/dx

xmax = x0 + t*dx →t2x = (xmax – x0)/dx

tix = min(t1x, t2x), tex = max(t1x, t2x)

Analog pentru intersectia cu planele y=ymin si y=ymax:

tiy = min(t1y, t2y), tey = max(t1y, t2y)

Raza intersecteaza volumul daca: max(tix, tiy) < min (tex, tey)

si

coordonatele punctelor de intersectie sunt intre zmin – zmax

- Se testeaza intersectiile cu planele zmin, zmax

tix=min(t1x,t2x) = t1x

tex=max(t1x,t2x) = t2x

tiy=min(t1y,t2y) = t1y

tey=max(t1y,t2y) = t2y

max(tix,tiy) = max(t1x,t1y) = t1x

min(tex,tey) = min(t2x,t2y) = t2y

t1x<t2y

Utilizarea de volume incadratoare (4)

2) Intersectia cu un paralelipiped cu fetele paralele cu planele principale - cont

Raza albastra intersecteaza prelungirile fețelor x=xmin si x=xmax

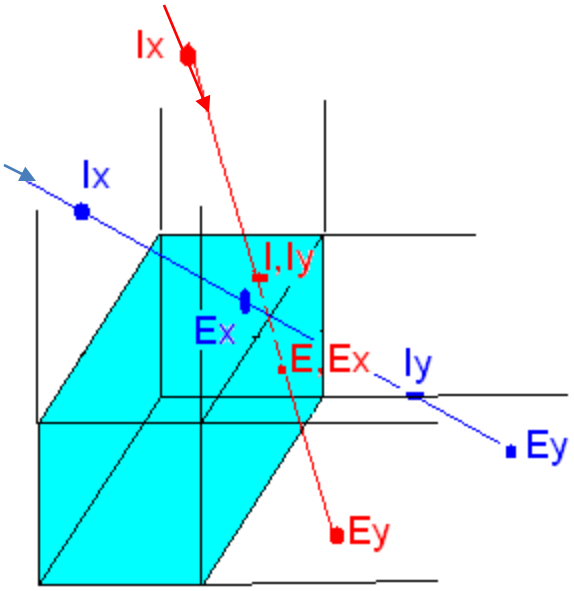
Raza rosie intra in volum prin fața y=ymin si iese prin fața x=xmax

Ix, Ex – intersectiile razei albastre cu x=xmin, x=xmax

Iy, Ey – intersectiile razei albastre cu y=ymin, y=ymax

Ix, Ex – intersectiile razei rosii cu x=xmin, x=xmax

Iy, Ey – intersectiile razei rosii cu y=ymin, y=ymax



daca max(tix,tiy) > min(tex,tey) → raza nu intersecteaza volumul

altfel, se calculeaza intersectia cu z=zmin si z= zmax

daca max(tix, tiy, tiz) < min(tex, tey, tez)

atunci raza intersecteaza volumul

max(tix, tiy) = tiy , min(tex,tey)=tex

max(tix,tiy) > min(tex,tey)

- Raza albastra nu intersecteaza volumul

max(tix, tiy) = tiy, min(tex,tey) = tex

max(tix,tiy) < min(tex,tey)

- Se calculeaza tiz, tez

Punctele de intrare si iesire din volum sunt:

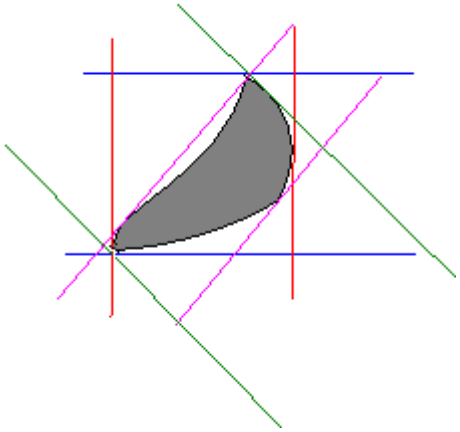
I = r(max(tix, tiy, tiz))

E = r(min(tex,tey,tez)

Utilizarea de volume incadratoare (5)

3) Intersectia cu un volum incadrator poliedru convex

- Volumul incadrator (Kay, Kajiya[1986]) este un poliedru convex format din intersectiile a 4 perechi de plane paralele, inclinate la 0, 45, 90, 135 grade fata de planul orizontal.
- Planele incadreaza minimal obiectul



Ec. unui plan: A*x + B*y + C*z + D =0

- Fie t11 si t12 - valorile lui t pentru intersectiile cu o pereche de plane paralele

  t1min = min(t11,t12) corespunde punctului de intersectie mai apropiat de observator
- Fie t21, t22 - valorile lui t pentru intersectiile cu urmatoarea pereche de plane paralele
- daca max (t1min, t2min) > min (t1max, t2max) raza nu intersecteaza volumul

  altfel, se continua cu intersectia urmatoarei perechi de plane

➢ **Punctele de intersectie cu volumul sunt**: I = r(max($t_i$min)), E = r(min($t_i$max))

Scena reprezentata printr-o ierarhie de volume incadratoare

Ierarhie de volume incadratoare de grupuri de obiecte (Bounding Volume Hierarchy – BVH)

▪ Scena 3D este reprezentata printr-un arbore (arborele scenei) in care fiecare nod are atasat un volum incadrator pentru un grup de obiecte: vezi cursul 8 (BVH).

▪ Radacina contine volumul incadrator al intregii scene

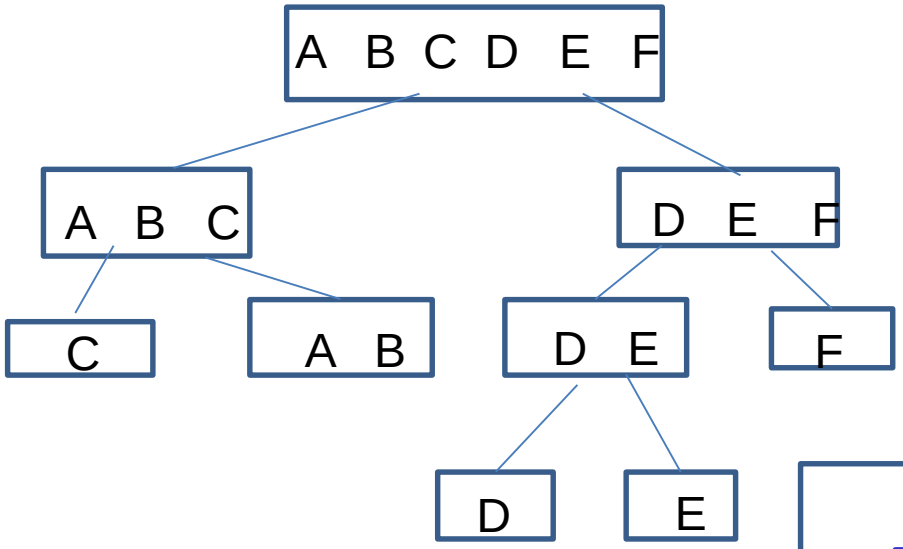
▪ Fiecare frunza contine un obiect al scenei (sau mai multe)

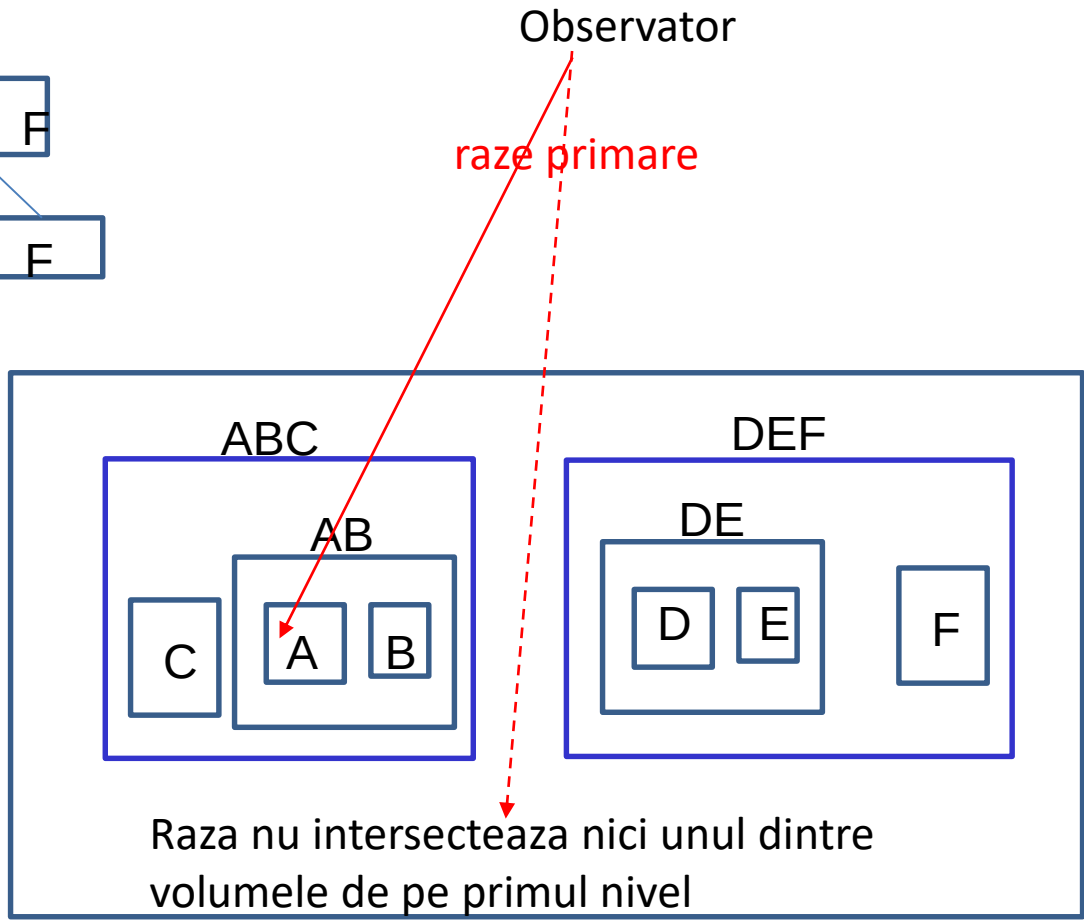
▪ Nodurile interne contin grupuri de obiecte.

▪ Pentru intersectia razei cu scena este parcurs arborele de la radacina spre frunze

- daca raza nu intersecteaza volumul incadrator al unui nod atunci:
  - nici unul dintre obiectele din volumul nodului nu este intersectat
  - nici unul dintre obiectele din subarborele nodului nu este intersectat
- daca raza intersecteaza volumul incadrator al unui nod atunci:
  - se testeaza intersectia razei cu volumele incadratoare din subarborele nodului

Scena reprezentata printr-o ierarhie de volume incadratoare





- Raza intersecteaza volumul incadrator al nodului (ABC)

↓

Se coboara in subarborele nodului
- Raza nu intersecteaza volumul C
- Raza intersecteaza volumul (AB)

↓

Se coboara in subarborele nodului
- Raza intersecteaza volumul A

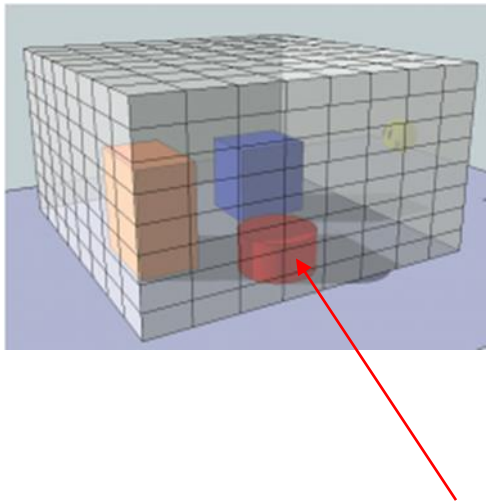
↓

Se intersecteaza raza cu obiectul din A

Scena divizata in volume incadratoare (1)

Divizarea regulata a scenei

- Se porneste de la paralelipipedul incadrator al scenei, care se divizeaza recursiv in opt subvolume egale, pana la o anumita rezolutie.
- Subvolumele finale sunt numite **voxeli**.
- La divizare nu se tine cont de structura scenei
- Fiecarui voxel ii este asociata o lista de obiecte pe care le contine.
- Un obiect se poate afla in mai multi voxeli (liste)



Intersectia unei raze cu scena:

- Raza este intersectata numai cu obiectele din voxelii traversati de raza
- Traseul razei prin volumul de voxeli poate fi calculat eficient printr-un algoritm DDA 3D (extensie pentru 3D a algoritmului DDA pentru rasterizarea vectorilor).

Scena divizata in volume incadratoare (2)

Voxelul de start: prin care raza primara intra in volumul scenei/ din care pleaca o raza secundara

Voxel_curent = voxel de start; gata = false; in_volum = true;

cât timp (!gata && in_volum)

{ **daca** voxelul curent nu este gol (lista de obiecte nu este vida) **atunci**

{*se intersecteaza raza cu obiectele continute in voxelul current;

daca exista intersectii **atunci**

{ *se retine punctul de intersectie cel mai apropiat de originea razei;

gata = true;

continue;

}

}

// voxelul curent este gol sau nu exista intersectii cu obiectele voxelului curent

*determina urmatorul voxel de pe traseul razei;

daca raza iese din volum **atunci**

in_volum = false;

altfel voxel_curent = urmatorul voxel de pe traseul razei;

}

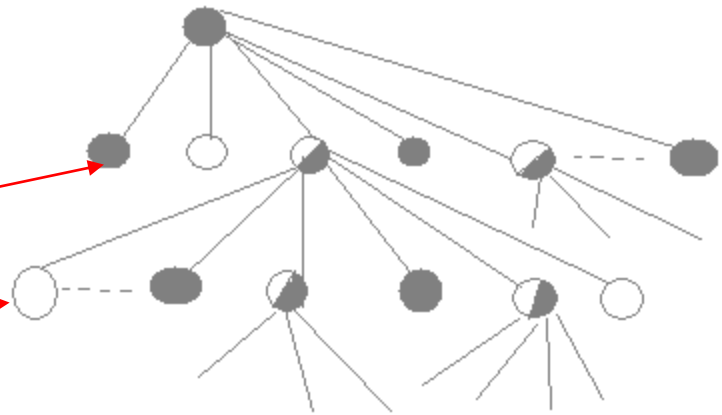
Scena divizata in volume incadratoare (3)

Divizarea adaptiva a scenei (scena reprezentata prin arbore octal)

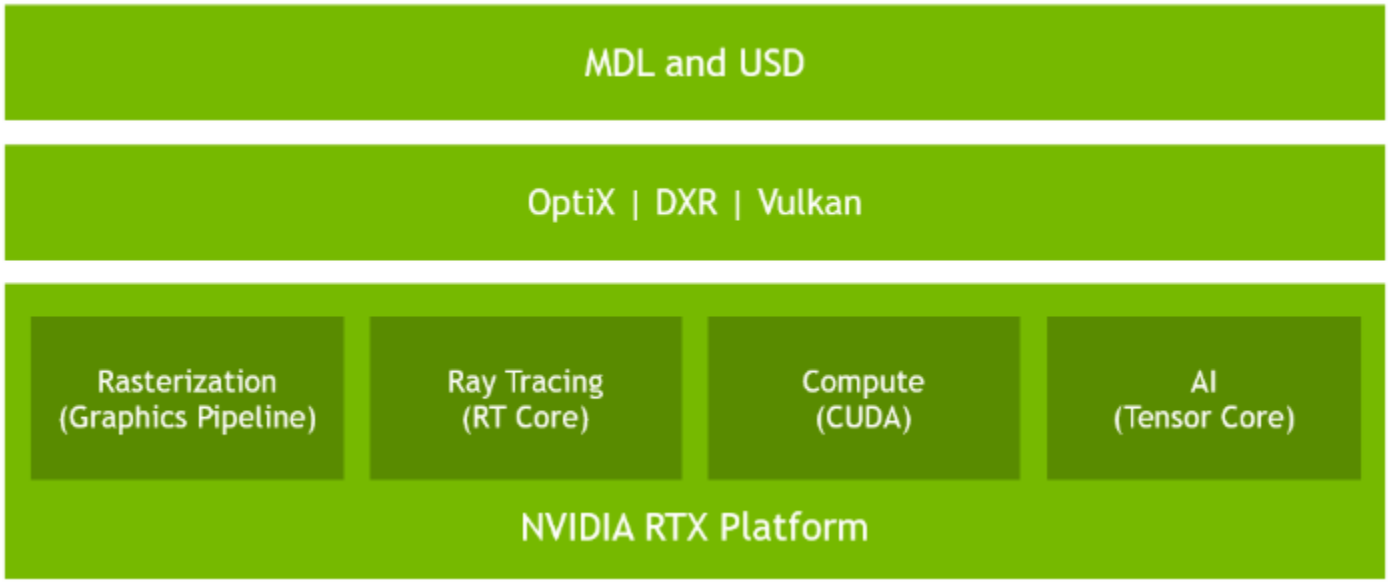
- Se porneste de la paralelipipedidul incadrator al scenei, care se divide in 8 subvolume egale, in mod recursiv, divizarea unui subvolum terminandu-se daca el nu contine nici un obiect al scenei sau contine un singur obiect.
- Scena se reprezinta printr-un arbore octal, fiecare nod fiind asociat unui subvolum.
- Pentru intersectia razei cu scena se fac teste de intersectie pornind din nodul de start (radacina pentru raza primara) spre frunze.
- Daca subvolumul unui nod nu este intersectat de raza, atunci niciunul dintre subvolumele (obiectele) din subarborele nodului nu va fi intersectat.

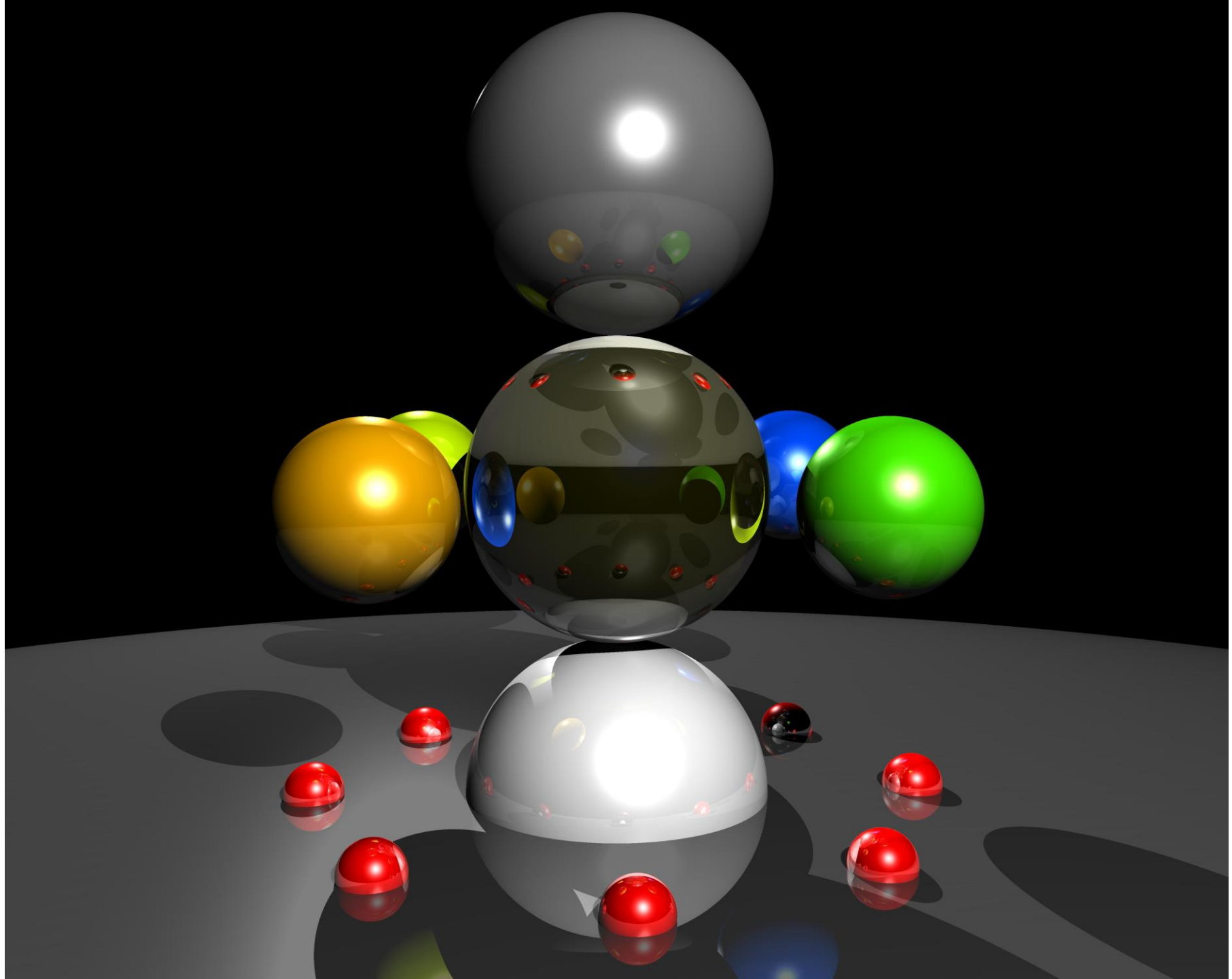
Arborele octal al scenei

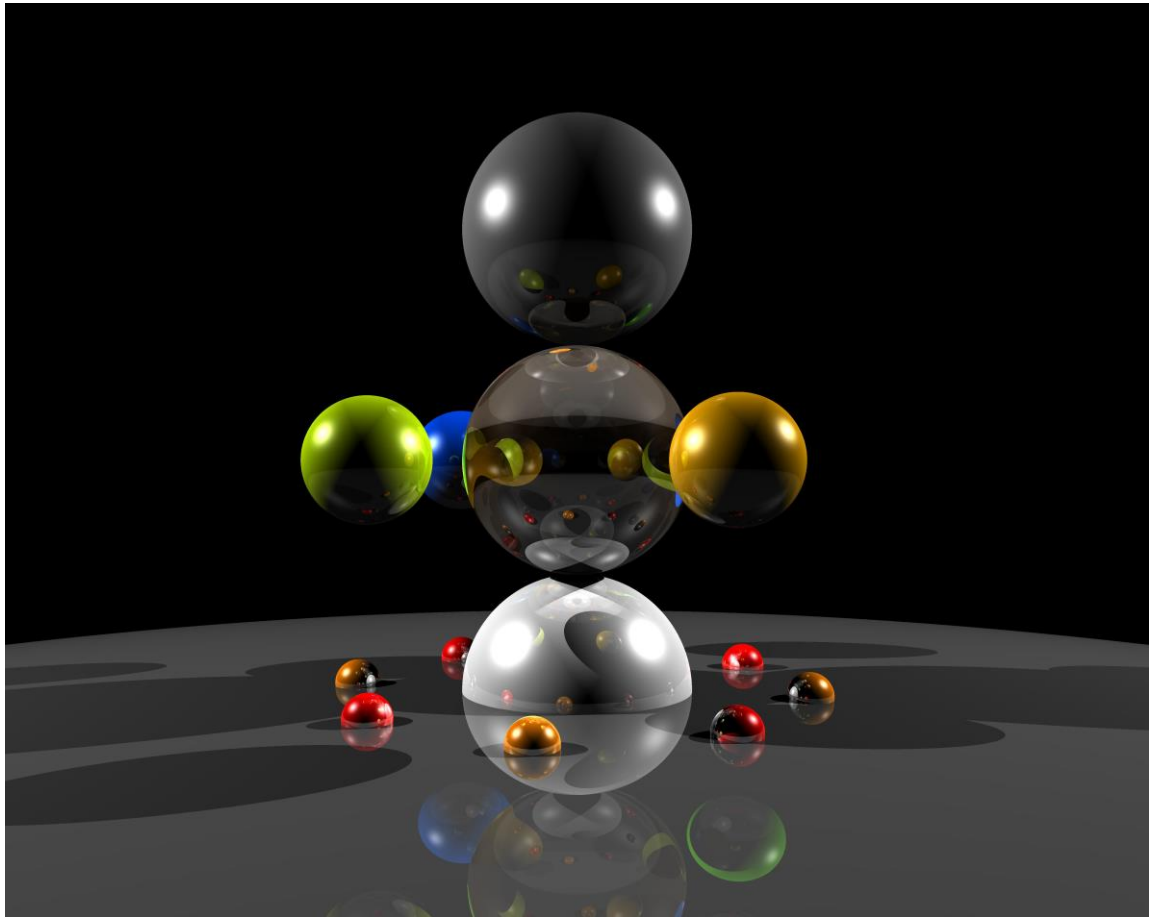
- subvolumul contine un singur obiect
- subvolum gol

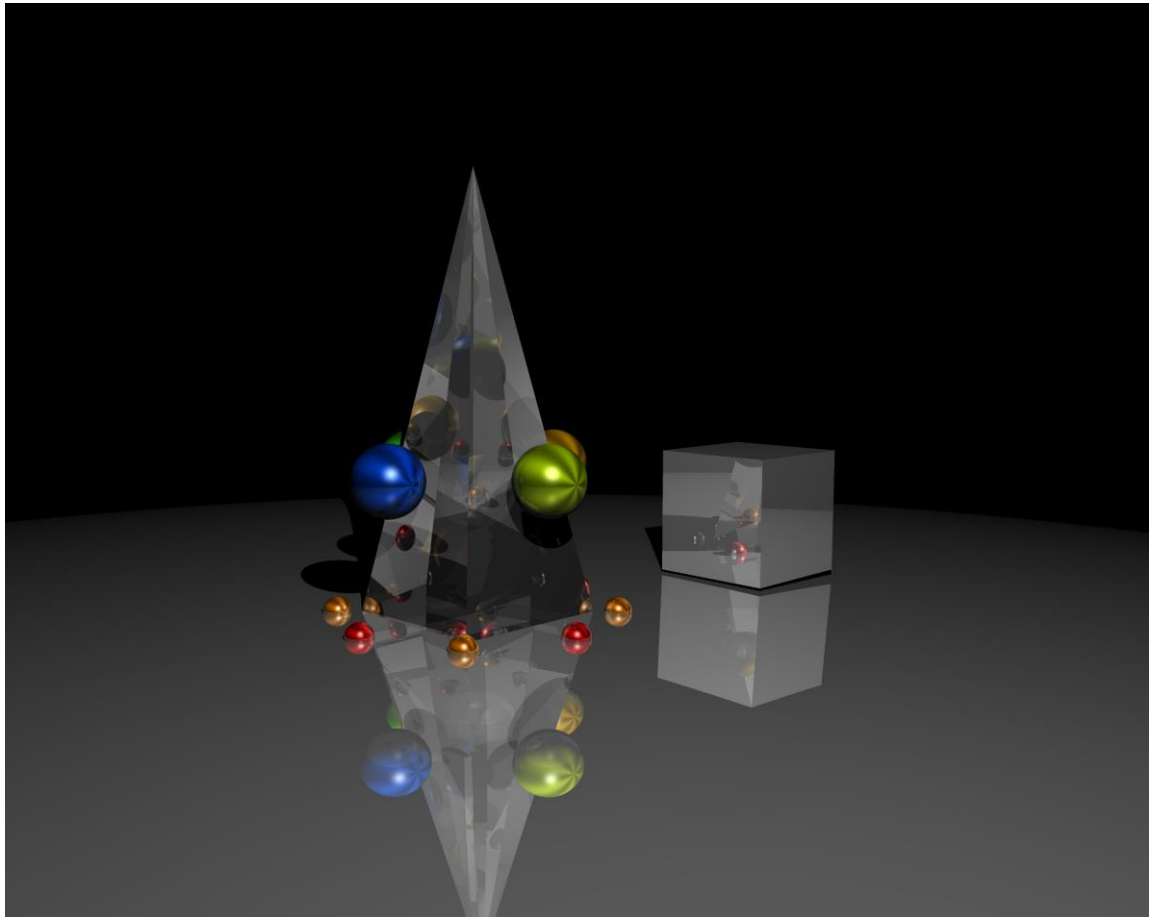


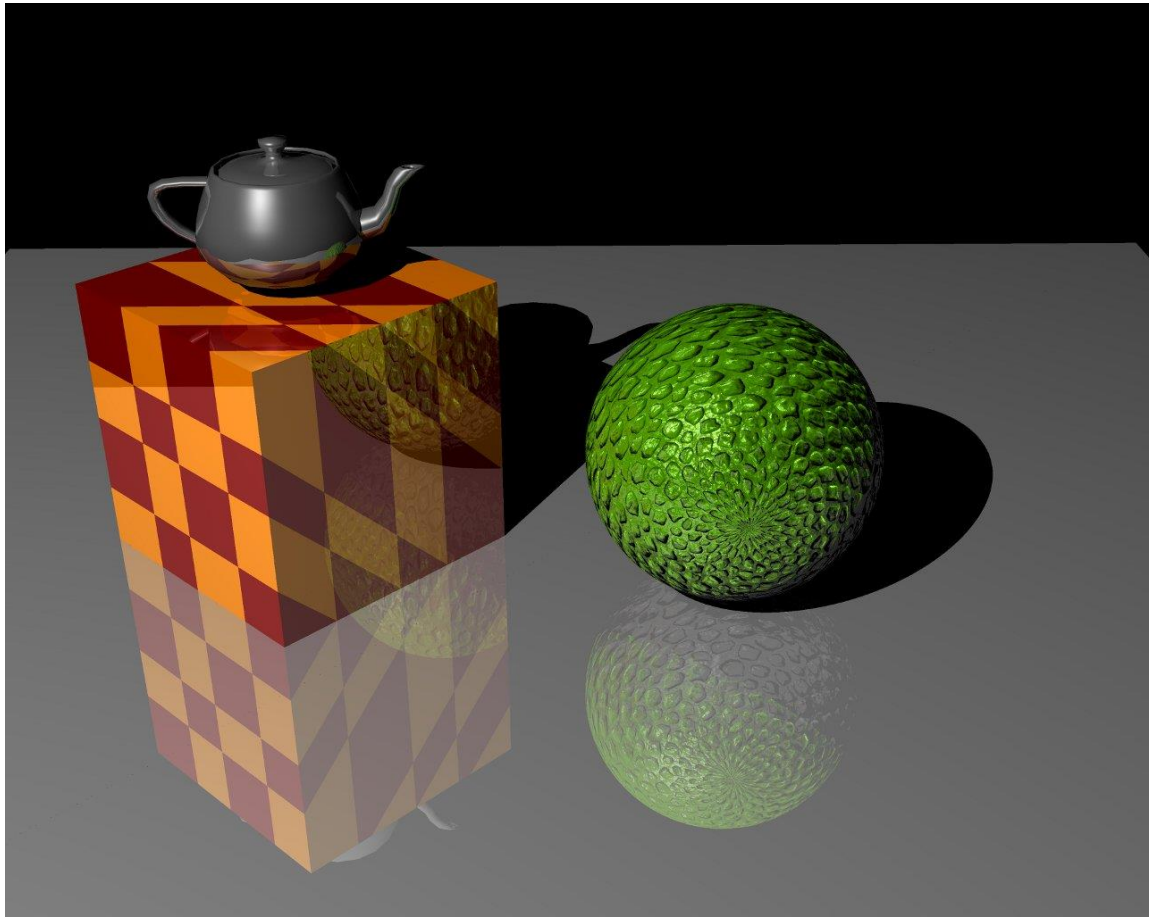
Platforma NVIDIA RTX permite implementarea algoritmului Ray-tracing folosind API-uri si kituri software de dezvoltare dedicate: OptiX, Microsoft si DXR Vulkan











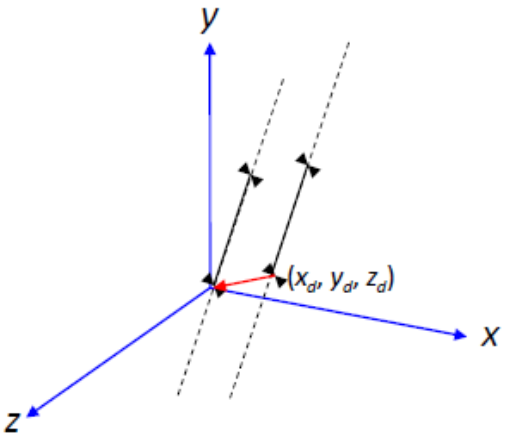
I. TRANSFORMARI SI PROIECTII:

- **1** (0.25p) Sa se scrie produsul matriceal prin care se poate exprima rotatia unui punct din spatiul 3D, in jurul unei drepte oarecare. Se va preciza efectul fiecarei matrice. (septembrie 2011)

Se considera dreapta data printr-un punct (xd, yd, zd) si directia sa, D[a, b, c].

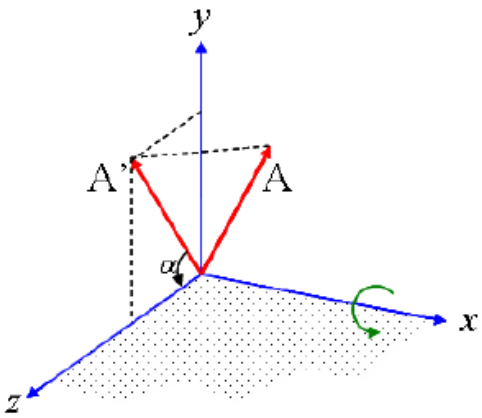
1. Translatie care face ca dreapta sa treaca prin origine: T(-xd, -yd, -zd)
2. Alinierea dreptei cu una dintre axele principale, de ex. cu axa OZ:
   2.1. Rotatie in jurul axei OX, cu un unghi ux, prin care dreapta ajunge in planul XOZ: Rox(ux)
   2.2. Rotatie in jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: Roy(uy)
3. Rotatia cu unghiul dat, u, in jurul axei pe care s-a aliniat dreapta: rotatie in jurul axei OZ : Roz(u)
4. Transformarea inversa celei din pasul 2:
   4.1. Rotatie in jurul axei OY, cu unghiul –uy: Roy(-uy)
   4.2. Rotatie in jurul axei OX, cu unghiul –ux: Rox(-ux)
5. Transformarea inversa celei de la pasul 1: T(xd, yd, zd)

1. Translaţie

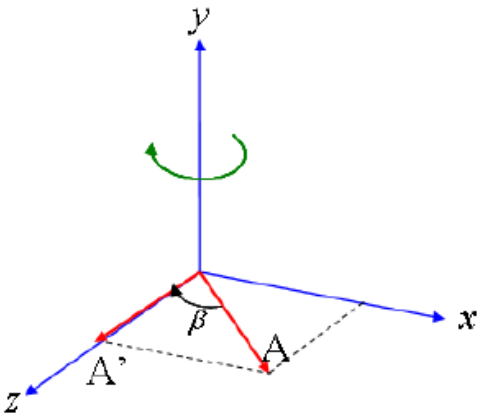


$$T = \begin{bmatrix} 1 & 0 & 0 & -xd \\ 0 & 1 & 0 & -yd \\ 0 & 0 & 1 & -zd \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. Aliniere *D* cu axa OZ

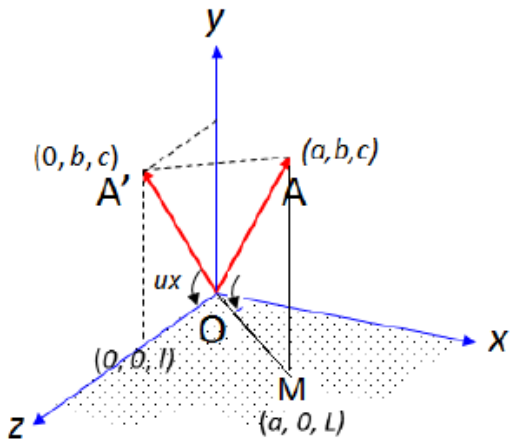


Rotatie in jurul axei OX



Rotatie in jurul axei OY

2. 1. Rotatia in jurul axei OX



$R_x(ux)$

Lungime (OA) = L; lungime (OA') = l;

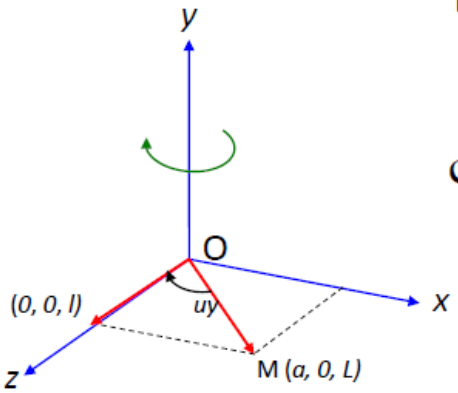
$$L = \sqrt{a^2 + b^2 + c^2}$$

$$l = \sqrt{b^2 + c^2}$$

$$\sin(ux) = \frac{b}{l}$$

$$\cos(ux) = \frac{c}{l}$$

2. 2. Rotatia in jurul axei OY



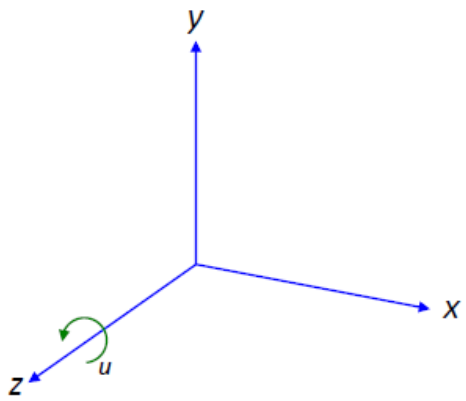
$R_y(uy)$

Lungime (OM) = Lungime (OA) = L

$$\sin(-uy) = a / L -> \sin(uy) = -a / L$$

$$\cos(-uy) = \cos(uy) = l / L$$

3. Rotatia in jurul axei OZ cu unghiul dat



$$R_z(u) = \begin{bmatrix} \cos(u) & -\sin(u) & 0 & 0 \\ \sin(u) & \cos(u) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

[x' y' z' 1] = [x y z 1] * T(-xd,-yd, -zd) * Rx(uyz) * Ry(uxz) * Rz(u) * Ry(-uxz) * Rx(-uyz) * T(xd,yd, zd)

Daca dreapta e data print-run punct (xd, yd, zd) si o directie D[a, b, c].
uyz = arctg(b / c)
uxz = arctg(a / c)

Rx(u) = [1 0 0 0
0 cosu sinu 0
0 -sinu cosu 0
0 0 0 1]

Ry(u) = [cosu 0 sinu 0
0 1 0 0
-sinu 0 cosu 0
0 0 0 1]

Analog Rz(u).

T(x, y, z) = [1 0 0 0
0 1 0 0
0 0 1 0
x y z 1]

^^ Le-am scris sub forma de vectori linie. Acum am vazut ca in curs transformarile 3D sunt facute cu vectori coloana, asa ca transpunem matricele si la compunerea matricelor intoarcem pe dos formula (inversam ordinea).

Proiectii perspectiva / axonometrice / oblica:

- **1** (0.75) Precizati pe scurt care sunt asemanarile si deosebirile dintre efectele proiectiilor perspectiva si cele ale proiectiilor axonometrice (7 februarie 2015)

Asemanari:

Ambele proiectii **schimba unghiurile dreptelor** care nu sunt paralele cu planul de proiectie. Și ambele pot reda mai multe fețe ale obiectului proiectat

- Deosebiri:

Dimensiunea obiectelor la proiectiile perspectiva depinde de distanta fata de punctul de convergenta, in schimb la proiectiile axonometrice dimensiunile obiectelor nu depind de distanta obiectelor fata de planul de proiectie.

La proiectiile perspectiva nu se conserva paralelismul (nu sunt **transformari afine**), pe cand proiectiile axonometrice sunt transformari afine (conserva paralelismul). La proiecțiile perspectivă distanta dintre centrul de proiecție și planul de proiecție este finita, în celălalt caz infinita? deosebirea e buna, doar ca cere efecte, deci nu cred ca ar puncta asta :-?

- **4** (0.25p) Prin ce se deosebesc proiectiile oblice de proiectiile axonometrice? (1 februarie 2015) (9 septembrie 2015 Restanta) (3 februarie 2014 - a doua zi) (31 ianuarie 2013)

La proiectiile oblice, **proiectorii** nu sunt **perpendiculari** pe **planul de proiectie**, pe cand la proiectiile axonometrice proiectorii sunt perpendiculari pe planul de proiectie.

La proiectiile axonometrice, **planul de proiectie** nu este **perpendicular** pe o **axa principala**, iar la proiectiile oblice planul de proiectie este perpendicular pe o axa principala.

Scalarea laturilor la axonometrice se face schimband unghiul facut de planul de proiectie cu una din axele sistemului (izometrice, dimetrice, ...), la oblice laturile perpendiculare pe proiectori pot aparea mai mici sau mai mari in functie de unghiul facut de proiectori cu planul de proiectie;

- **2** (0.25p) Care dintre urmatoare tipuri de proiectii **NU** este o transformare afina? (29 ianuarie 2014) (30 ianuarie 2013)
  a) proiectia oblica
  b) proiectia perspectiva **da да, ca e singura care nu e ortogonala da da**
  c) proiectia axonometrica
  d) proiectia ortogonala

  Justificati.

Proiectiile paralele sunt proiectii afine (pastreaza paralelismul liniilor). Proiectiile sunt de 2 tipuri ortogonale(d) si oblice(a). Proiectiile ortogonale(d) sunt ortografice si axonometrice(c). Toate aceste tipuri de proiectii sunt afine.

Proiectia perspectiva(b) nu pastreaza paralelismul liniilor. Liniile paralele intre ele, care nu sunt paralele cu planul de proiectie converg catre un punct numit Punct de Convergenta.

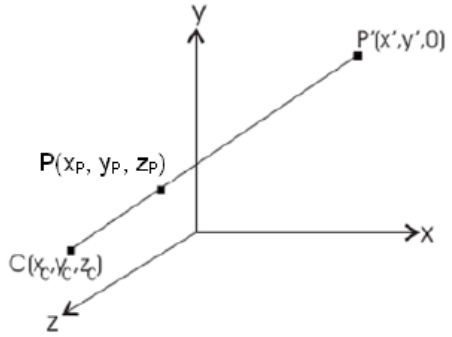
- **1** Proiectia perspectiva cu centru oarecare in xOy (19 sep 2011 nr 2)

Oare vrea si toate matricele alea? buna intrebare

Cred ca este asemănătoare cu ceva de mai jos unde spune “exprimarea matematica a proiecțiilor…. Matricea proiectiei”

Da, doar ca la matrice se mai fac niste schimbari incat sa fie matrice independente de punctul P (daca le scrii asa direct sunt dependente pentru ca au zp in ele).

Proiectia perspectiva in planul XOY cu un centru de proiectie oarecare(1)



C – centrul de proiectie
P – punctul proiectat
P' – proiectia lui P in XOY

Ecuatiile parametrice ale proiectorului care trece prin P:

x = xc + (xp – xc)*t
y = yc + (yp – yc)*t 0<= t <= 1, pentru punctele aflate pe segmentul C-P
z = zc + (zp –zc) *t

0 = zc + (zp –zc) *t' → t' = – zc / (zp – zc) → **x' = (xc * zp – zc*xp)/(zp – zc)**

y' = (yc*zp – zc*yp)/(zp – zc)

7

Volumuri vizuale:

- **4** (0.25) Care sunt parametrii care definesc volumul vizual al proiectiei perspectiva in OpenGL? Care este planul in care se efectueaza proiectia? Care este forma geometrica a volumului vizual al proiectiei perspectiva? (7 februarie 2015)(29 ianuarie 2014) (29 ianuarie 2014) (30 ianuarie 2013)

Parametrii: (cred ca acestia sunt)
Eu cred ca doar near și far

- trunchi de piramida delimitat de urmatoarele:
- planul din fata (near)
- planul din spate (far)

Varful piramidei este in pozitia observatorului.
Planul in care se efectueaza proiectia perspectiva a obiectelor cuprinse in volumul vizual este cel din fata (z = zNear), cu centrul de proiectie in pozitia observatorului. Obiectele aflate in afara volumului vizual sunt eliminate sau decupate la frontiera volumului vizual.
Proiecția se efectuează în planul din față z = - near.

Eu aici as zice zNear, zFar, FieldOfView, AspectRatio.

Top (resp. bottom) sunt luati din tan(FoV/2) = top / zNear
Right, left sunt luati din aspect ratio.

eu as zice ca si la Alina este bine deoarece este functia aia frostrum (sau ceva de genul), care ia aprametrii scrisi de ea cel mai bine le scriem pe toate

Forma geometrica este trunchi de piramida. **(piramide, pira-pira-mide ta na na na)**

- **4** (0.5p) Care sunt parametrii care definesc sistemul de coordonate observator. (1 februarie 2015) (9 septembrie 2015 Restanta) (3 februarie 2014 - a doua zi) (31 ianuarie 2013)
  Parametrii care definesc sistemul de coordonare observator.
  - **pozitia observatorului (xo, yo, zo)**
  - **vectorul forward/look, directia de privire (N)**
  - **vectorul Up (rotatia in jurul axei Oz)**

  Care este functia OpenGL care permite definirea acestor parametrii?
  glm::lookAt? (desi nu e chiar OpenGL, e glm)
  glm::lookAt(glm::vec3(xo, yo, zo), glm::vec3(Px, Py, Pz), glm::vec3(Upx, Upy, Upz))

  Unde (Px, Py, Pz) e un punct spre care priveste observatorul.

  sau gluLookAt(..)?
Aici este glm::lookAt(...), nu? Da, ca parca zicea ca glLookAt() era folosita inainte de 3.x si noi la laborator am folosit glm::lookAt().
  Care sunt valorile implicite ale acestor parametrii in OpenGL?
  (0, 0, 0, 0, 0, -1, 0, 1, 0)
  aici oare putem da si explicatii? gen ca vectorul Up este oy? sau vectorul Look este -oz? cred ca da

- **1** (0.5p) Exprimarea matematica a proiectiilor perspectiva in planul XOY cu un centru de proiectie oarecare. Matricea proiectiei. (septembrie 2011)

Cred ca asta este asemănător cu cel de mai sus, nu?
S-ar putea sa fie chiar aceeasi ca vad ca e tot septembrie 2011, doar ca aia e mai specifica

Transformari:

- **1** (0.5p) Transformarea de vizualizare in OpenGL.

Transformarea de vizualizare are ca scop transformarea modelului unei scene 3D intr-o imagine redata pe o suprafata de afisare. (asemanator procesului realizat de o camera foto).
Imaginea obtinuta depinde de:
- pozitia camerei (adica observator)
- orientarea camerei
- ceea ce vede ochiul prin vizorul camerei foto (adica volum vizual)

Este o transformare compusa din mai multe transformari, printre care si o proiectie din 3D in 2D. Se efectueaza asupra varfurilor obiectelor din scena 3D. Include operatia de decupare (eliminare a obiectelor care nu sunt in volumul vizual).
Matricile transformarii de vizualizare:
- M = matrice de modelare (difera de la obiect la obiect)

- V = matrice de vizualizare (aceeasi pentru intreaga scena)
- P = matrice de proiectie (aceeasi pentru intreaga scena)

Putem avea:

- MV = V * M (matricea model-view)
- MVP = P * V * M (matriea model-view-projection)

O fi suficient sa scriem atat? Ce am mai putea scrie (ceva ce putem si retine)?
Haha bine zis ceva ce putem si retine.. pai ma gandeam ca e un subiect care poate fi dezvoltat mult, dar asa e, pe cat de general e subiectul pe atat de general ar fi sa si scriem.. mai are si doar 0.5p :P stiu ca m-am oprit din a scrie la subiectul asta pt ca ma gandeam ca nu reusim sa retinem absolut toate detaliile si am scris ce mi s-a parut esential :)

II. ELIMINAREA PARTILOR NEVIZIBILE:

Z-buffer:

- **1** [0.75p]3. Algoritmul z-buffer. Descriere. Cand se executa? Ceva legat de depth complexity… (1 februarie 2015)

Este un algoritm de eliminare a partilor nevizibile ale primitivelor grafice.
Se executa pe GPU, fiind integrat in procesul de rasterizare a primitivelor, dupa transformarea in poarta de afisare a varfurilor.
Observatorul se considera plasat la infinit, pe axa Z negativa, iar asupra primitivelor se efectueaza proiectie ortografica in planul XOY.

Algoritmul z-buffer (integrat in procesul de rasterizare):
*Initializeaza buffer-ul imagine la culoarea de fond
*Initializeaza Z-buffer la coordonata z maxima (z=1)
Pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive
* calculeaza culoarea fragmentului
//testul de vizibilitate (/adancime)
daca z< Z-buffer[y][x] atunci
{ Z-buffer[y][x] = z
*actualizeaza culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea fragmentului f
}

Depth complexity reprezinta numarul de suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine.

- **2** [0.25p] Care dintre urmatoarele afirmatii sunt corecte: (3 februarie 2014 - a doua zi) (31 ianuarie 2013)
  a. Algoritmul z-buffer necesita rasterizarea poligoanelor scenei 3D in ordinea descrescatoare a distantei lor fata de observator
  nu, primitivele grafice in care a fost descompusa scena 3D sunt rasterizate in ordinea transmiterii lor din programul aplicatie (nu sunt sortate) de acord

b. Bufferul z memoreaza coordonatele z ale fragmentelor de primitive care sunt vizibile intr-o imagine
da (curs 5, slide 10) - atunci cand se rasterizeaza o noua primitiva, se compara cu vechea valoare z pentru fragmentul respectiv, deci este necesara memorarea coordonatelor da

c. Bufferul z se actualizeaza pentru fiecare fragment rezultat din rasterizarea primitivelor dintr-un cadru imagine da, cand se rasterizeaza o primitiva cu coordonata z mai mica pentru fragmentul curent decat cea deja memorata in z-buffer, se actualizeaza valorile (aici sunt putin in dubiu pentru ca zice pentru fiecare fragment, daca intrebarea se refera la faptul ca buffer-ul z se actualizeaza indiferent de ce coordonata z are fragmentul curent, atunci raspunsul ar fi nu) si mie mi se pare ambigua, eu as fi raspuns cu nu din motivul pe care l-ai zis in paranteza. **eu as zice tot nu**

Eu aici as înțelege a doua variantă pe care ai precizat o și as spune nu :? da, si eu inteleg tot a doua varianta (dar avand in vedere ca nu e grila, cred ca putem si explica saaau sa intrebam la fata locului mai bine :))

d. Algoritmul z-buffer este exectutat de procesoarele placii grafice
da, este executat pe GPU da

BSP:

- **5** (1p) Raspunde la urmatoarele intrebari despre alg BSP: (7 februarie 2015) (9 septembrie 2015 Restanta) (29 ianuarie 2014) (30 ianuarie 2013) (19 sep 2011 nr1)
  - (0.25) Cum se construieste arborele BSP al unei scene 3D
  - GENERAREA arborelui BSP

Se împarte planul de partitionare in două semi spatii. Se începe cu un poligon oarecare (de regula primul) pentru care se creează nodul rădăcină. Primitivele din semi spațiul “față “ formează lista față și generează subarborele față al nodului. Primitivele “spate” formează lista spate și generează subarborele spate al nodului.

  - (0.25p) Ce se memoreaza in nodurile arborelui BSP al unei scene 3D?

Fiecare nod al arborelui corespunde unui plan de partiționare => ... Semispatiul fata și Semispatiul spate? cred ca e suficient sa zicem plan de partitionare :-? **Conform algoritmului de creare al arborelui, se pastreaza o lista de primitive care apartin planului de partitionare.**

  - (0.15) Cand trebuie sa fie reconstruit arborele BSP al unei scene?

Arborele nu trebui sa fie reconstruit dacă scena nu s-a modificat (avantaj pentru scenele statice), în acest doar se apelează funcția afișare.

  - (0.2) Cum se afiseaza o scena reprezentata printr-un arbore BSP?
  - AFISAREA spatiului 3D pe baza arborelui BSP. Explicatie + pseudocod
    Afisarea tine cont de pozitia observatorului, chiar daca crearea arborelui nu. Afisarea este back-to-front (incepand cu cea mai indepartata primitiva fata de observator). Se porneste din radacina arborelui si se avanseaza spre frunze, in functie de pozitia observatorului fata de planul din fiecare nod astfel: se avanseaza in subarborele opus observatorului fata de plan.

  - (0.25) Scrieti functia de afisare a arborelui BSP?

Pseudocod:

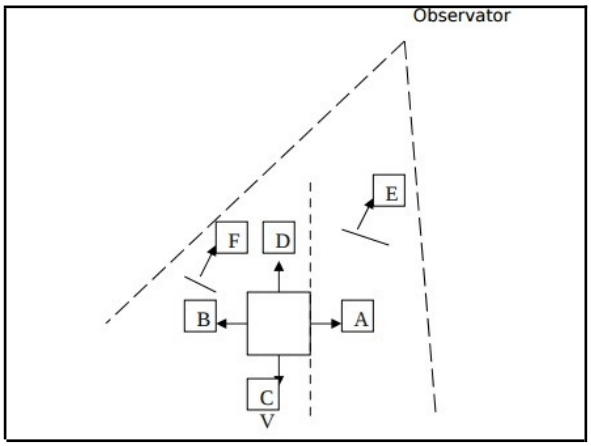
```
void afisareBSP(arbore *A, Pozitie Observator) {
        daca (!A) return;
        daca (Observator este in semispatiul fata al planului radacinii) atunci {
                afisareBSP(A->spate);
                afisare primitive nod radacina;
                afisareBSP(A->fata);
        } altfel {
                afisareBSP(A->fata);
                afisare primitive nod radacina;
                afisareBSP(A->spate);
        }
}
```

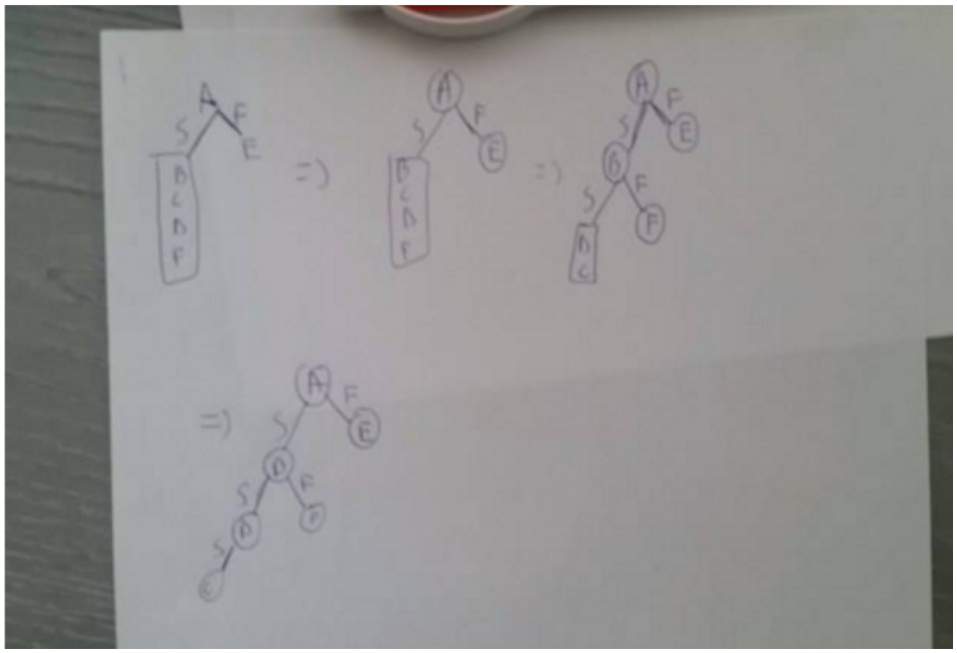
- (0.15) Cum poate fi utilizat algoritmul BSP pentru eliminarea poligoanelor nevizibile din banda grafica?

Dacă planul de partiționare al unui nod nu intersectează volumul vizual, atunci numai subarborele aflat de aceeași parte cu volumul vizual va fi afișat, celelalt subarbore fiind eliminat din banda grafică

- **1** (0.5p) Figura urmatoare reprezinta o proiectie a unei scene 3D, compusa din 6 poligoane, notate prin litere de la A la F. Sagetile reprezinta normalele la planele poligoanelor.
    - a) Sa se reprezinte grafic arborele BSP al scenei, atunci cand se incepe construirea cu poligonul A.
    - b) Sa se specifice ordinea in care vor fi afisate poligoanele, tinand cont de pozitia observatorului si directia in care priveste. Volumul vizual include intreaga scena 3D.
    - Extra: (0.25p) Cum pot fi eliminate din banda grafica obiectele nevizibile, in cadrul algoritmului BSP?

      Dacă planul de partiționare al unui nod nu intersectează volumul vizual, atunci numai subarborele aflat de aceeași parte cu volumul vizual va fi afișat, celelalt subarbore fiind eliminat din banda grafică





Afisare: F,B,C,D,A,E

Tot de pe la eliminari:

- **3** [0.5p] Determinarea fetelor auto-obturate ale poliedrelor convexe. (3 februarie 2014 - a doua zi) (31 ianuarie 2013) (septembrie 2011)
    - Conditia de vizibilitate atunci cand determinarea este efectuata:
        - a) in coordonate globale;
            - **Nu • Ou > 0 (normala la suprafata, directia spre observator) - se poate dezvolta putin cum se ajunge la formula si de ce merge (cos de unghi > 0 etc.)**
        - b) in coordonate de decupare.
            - **-zN > 0 (caz particular pentru Nu • Ou > 0, unde Ou = (0, 0, -1) si Nu = (xN, yN, zN)**

- **1** Warnock pentru eliminarea partilor nevizibile. (19 sep 2011 nr2)

Se bazeaza pe divizarea spatiului de afisare.

Fata de zona dreptunghiulara a spatiului de afisare , poligoanele din lista curenta pot fi intr-una dintre categoriile:

- poligon acoperitor = acopera intreaga zona
- poligon inclus in zona
- poligon disjunct in zona
- poligon intersectat cu zona : se divide in poligon inclus + poligon disjunct

In zona curenta sunt vizibile poligoanele incluse si unul dintre cele acoperitoare daca exista.

Lista poligoanelor vizibile se sorteaza crescator dupa coordonata zmin a fiecarui poligon: primul din lista este cel mai apropiat de observator -> toate poligoanele care urmeaza in lista dupa unul acoperitor pot fi eliminate.

Algoritmul este recursiv : la un apel se transmit functiei de afisare: zona de afisare (viewportul) si lista de poligoane de afisat. Iesirea din recursivitate:

- lista de poligoane este vida
- lista are un singur element, care este poligon inclus in zona

- primul in lista este un poligon acoperitor al zonei
- zona are un singur pixel

III. LUMINA. EFECTE. MODELE. ILUMINARE

- **1** Spatiul culorilor in modelul RGB. Culorile complementare in RGB. Unde se foloseste modelul RGB si unde cel CMY? (19 sep 2011 nr 2)

Culorile rosu, verde, albastru se numesc culori primare

Conform teoriei tricromaticitatii, orice culoare care poate fi perceputa de ochiul liber reprezinta un amestec aditiv al celor 3 culori primare, iar prin amestecul lor in proportii egale se obtine albul.

Doua culori care prin amestec produc lumina alba se numesc complementare.

Culorile complementare culorilor primare sunt:

Red - Cyan
Green - Magenta
Blue - Yellow

Amestecand doua culori primare in proportii egale se obtine culoarea complementara celei de-a treia. (ex. albastru + verde = cian)

Fiecare culoare se reprezinta printr-un punct in subspatiul culorilor:

1. Model RGB:

Red - (1, 0, 0) => Cian - (0, 1, 1)
Green - (0, 1, 0)
Blue - (0, 0, 1)
Alb - (1, 1, 1)
Negru - (0, 0, 0)

2. Model CMY:

Cyan - (1, 0, 0) => Red - (0, 1, 1)
Magenta - (0, 1, 0)
Yellow - (0, 0, 1)
Alb - (0, 0, 0)
Negru - (1, 1, 1)

Modelul RGB corespunde echipamentelor emitatoare de lumina (dispozitive de vizualizare cu ecran).

Modelul CMY se foloseste pentru a defini culorile reflectate de un document imprimat. (pigmentii din cernelurile tipografice absorb culorile complementare lor)

Phong-Gouraud:

- **1** [1p] Comparatie intre modelul Gourraud si modelul Phong. (1 februarie 2015)

Asemanari:

- amandoua sunt modele de shading
- se foloseste interpolarea pentru calculul culorilor
- culoarea e calculata pe baza unui model de iluminare local
- modul de calcul al normalei la modelul Phong si cel al culorii la modelul Gouraud sunt asemanatoare

- Gouraud prezinta calcul incremental al culorilor, iar Phong al normalelor

Deosebiri:

- la rasterizare, la modelul Gouraud fiecarui varf ii este asociata o culoare, in timp ce la modelul Phong fiecarui varf ii este asociata o normala
- la modelul Gouraud se calculeaza culoarea unui fragment prin interpolarea culorilor varfurilor, in timp ce la modelul Phong culoarea fragmentului se calculeaza prin aplicarea unui model de iluminare local asupra normalei interpolate din normalele varfurilor.
- Gouraud are calcule mai simple, Phong are calcule mai complexe
- Goruaud e mai rapid, Phong mai lent
- Gouraud nu elimina complet efectul de banda Mach, in timp ce Phong il reduce considerabil

- **4** (0.5p) Calculul culorilor fragmentelor la afisarea suprafetelor 3D folosind modelul Gourand. Calculul incremental al culorilor. Avantaje si dezavantaje ale modelului Garound in comparatie cu alte modele. (29 ianuarie 2014) (septembrie 2011, comparatie doar cu Phong) (septembrie 2011 nr1, cu Phong) (30 ianuarie 2013)

Modelul Gouraud se integrează foarte ușor în algoritmii de rasterizare, realizează calcule simple pentru culoarea unui fragment (calcul incremental, rapid).
La momentul rasterizarii, fiecărui vârf al unui poligon ii este asociata o culoare. Culoarea unui vârf este calculată pe baza unui model de iluminare locală în programul vertex shader sau poate fi asociată vârfului în programul aplicație. Culorile fragmentelor interioare poligoanelor sunt obținute la momentul rasterizarii prin interpolare liniară a culorilor vârfurilor.

1. Culoarea într un vârf este obținută folosind normala la vârf (calculată de la ecuația analitică de la care provine poligonul sau aproximata ca media aritmetică a normalelor la fețele adiacente)
2. La rasterizarea unui poligon, GPU calculează culoarea fiecărui fragment rezultat (prin interpolarea liniară a culorilor vârfurilor pentru fragmentele de pe laturi sau prin interpolarea liniară a culorilor capetelor fiecărui segment interior)

Avantaje : 1. este mai simplu și mai rapid, calcule mai puțin complexe
2. Se integrează ușor în algoritmi de rasterizare
Dezavantaje : 1. nu reduce efectul de bandă mach
2. Nu tine cont de tangentele la fețele adiacente pe o latură (culorile de pe laturi se calculează ținând cont de culorile din vârfuri)

- **1** [0.5p] Calculul culorilor fragmentelor la afisarea suprafetelor 3D folosind modelul Phong. Avantajele si dezavantajele modelului Phong in comparatie cu alte modele. (3 februarie 2014 - a doua zi)

La momentul rasterizarii, fiecarui varf al unui poligon ii este asociata normala in varf care este calculata la fel ca la modelul Gouraud, iar normalele sunt calculate si asociate varfurilor in programul aplicatie. Pentru fiecare fragment rezultat din rasterizarea unui poligon , GPU calculeaza o normala prin interpolare liniara intre normalele varfurilor. Culoarea fiecarui fragment se obtine pe baza unei normale interpolate folosind un model de iluminare local.

La rasterizarea unui poligon se procedeaza asemanator modelului Gouraud si se calculeaza o normala pentru fiecare fragment prin interpolare liniara a normalelor varfurilor

pentru fragmentele de pe laturi si prin interpolarea liniara intre normalele capetelor fiecarui segment interior
Avantaje:

- reduce efectul de banda Mach
- permire redarea reflexiei speculare in orice fragment al unui poligon

Dezavantaje:

- componentele Nx, Ny, Nz ale normalei unui fragment se obtine printr-un calcul incremental(asemanator cu cel folosit pentru calculul culorii in modelul Gouraud), dar pentru acesta normala trebuie sa fie normalizata.
- Calcule mai complexe la nivel de fragment
- De asemenea, vectorii L (catre sursa de lumina) si V(catre observator) folositi la nivel de fragment sunt calculati si normalizati in programul fragment shader.

Cu reflexii:

- **5** [0.75p] (Aproximarea) Reflexia difuza a luminii intr-un punct al unei suprafete 3D; luminata de o sursa punctiforma (modelul empiric) (1 februarie 2015) (9 septembrie 2015 Restanta) (29 ianuarie 2014) (30 ianuarie 2013) (19 septembrie 2011 - nr1)

  Caracteristicile luminii reflectate de suprafata unui obiect sunt:
  - lungimile de unda din raza incidenta
  - directia si geometria sursei de lumina
  - orientarea suprafetei
  - caracteristicile materialului din care e facuta suprafata

  Expresia care modeleaza intensitatea luminii reflectate e definita pentru o suprafata necolorata si o lumina incidenta monocromatica, dar cum lumina nu e monocromatica de obicei, expresia se evalueaza separat pentru cele 3 culori primare: R, G, B.

  Lumina reflectata difuz pe suprafata unui obiect este dispersata regulat in toate directiile.
  Legea lui Lambert care defineste reflexia luminii provenite de la o sursa punctiforma de catre un difuzor perfect este:
  Id = Isursa * kd * cos i
  Id = intensitatea luminii reflectate difuz
  Isursa = intensitatea luminii provenite de la sursa
  kd = coeficient de difuzie al luminii incidente, dependent de materialul suprafatei, 0 <= kd <= 1
  Daca i > pi / 2, suprafata nu primeste lumina de la sursa => sursa este in spatele suprafetei.

Suplimentar: (cateva informatii despre culoarea luminii reflectate difuz)

- lumina incidenta poate contine mai multe lungimi de unda
- acestea sunt reflectate diferit de o suprafata (adica reflecta diferit R, G, B)
- deci coeficientii de difuzie a luminii depinde de componentele luminii incidente

Cel mai smecher exemplu:

o suprafata de culoare verde absoarbe din lumina incidenta alba componentele de rosu si albastru, reflectand doar culoarea verde => suprafata este vazuta ca verde (!vezi plante)

- **3** [0.5p] Aproximarea reflexiei speculare a luminii intr-un punct al unei suprafete 3D, luminata de o sursa punctiforma (modelul empiric) (3 februarie 2014 - a doua zi) (31 ianuarie 2013) (septembrie 2011)

Un reflector perfect (ex. o oglinda) reflecta lumina intr-o singura directie, R, care e simetrica cu L fata de N si numai un observator care se afla pe aceeasi directie poate vedea lumina reflectata.

Reflexia speculara e determinata de suprafata materialului. Pentru materialele imperfect reflectante, cantitatea de lumina care ajunge la observator depinde de distributia spatiala a luminii reflectate specular:

- pentru suprafate netede (ex. metale), distributia este dreapta si focalizata
- pentru suprafete cu rugozitati (ex. hartia), distributia este dispersata.
- se aproximeaza prin cos(u)^n, unde n = exponent de stralucire al materialului.

Modelul practic de reflexie speculara:

lsA = lsursaA * ksA * cos(u)^n, unde A = lambda = lungime de unda

!! Ati ghicit: ls se calculeaza pentru fiecare din culorile primare: R,G ,B

ksA = coeficient de reflexie speculara (specific fiecarei lungimi de unda)

Observatie: Reflexia speculara nu poate avea loc daca in punctul respectiv nu se primeste deloc lumina de la sursa.

- **1** Modele de culoare pentru aproximarea reflexiei difuze si speculare (19 sep 2011 nr2)

Pentru lumina reflectata difuz.

- lumina incidenta poate contine mai multe lungimi de unda
- acestea sunt reflectate diferit de o suprafata (adica reflecta diferit R, G, B)
- deci coeficientii de difuzie a luminii depinde de componentele luminii incidente

Cel mai smecher exemplu:

o suprafata de culoare verde absoarbe din lumina incidenta alba componentele de rosu si albastru, reflectand doar culoarea verde => suprafata este vazuta ca verde (!vezi plante)

Nu pricep subiectul, dar poate merge sa zici ca in mod normal ar trebui pentru fiecare culoare din spectrul continuu, dar in practica se face doar pentru R, G si B (cred ca de-aia zice aproximare).

Sau e posibil sa se refere la aproximarile de genul ca se considera ca observatorul / sursa de lumina sunt la infinit etc.

Sau mai poate fi pusa in discutie si aproximarea functiei de distributie a luminii reflectate specular cu cos(alfa)^n, dupa modelul Bui-Tuong Phong, unde n e coeficientul de stralucire al materialului.

IV. TRANSPARENTA, CEATA, UMBRE

- **4** (0.75) Introducerea umbrelor in imagini prin metoda volumelor de umbra(fara implementare) (7 februarie 2015) (3 februarie 2014 - a doua zi) (31 ianuarie 2013) (septembrie 2011)

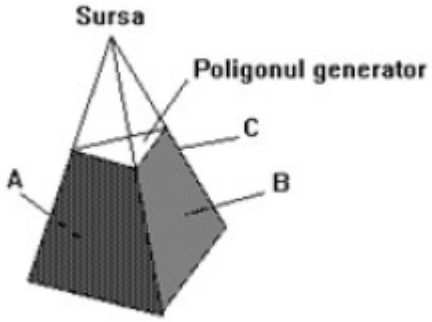
Sursa de lumina este considerata punctiforma iar scena 3d este alcatuita din poligoane.

Un volum de umbra este definit de o sursa de lumina si un poligon luminat(vazut din pozitia sursei de lumina) numit poligon generator.

Fiecare fata laterala a volumului este numita poligon de umbra. Ea este determinata de o latura a poligonului generator si de cele doua drepte care pleaca din sursa de lumina si care trec printr-un varf al laturii. Normalele la fetele laterale puncteaza spre exteriorul volumului.

Volumul infinit determinat de o sursa si un poligon generator este delimitat de o fata care reprezinta poligonul generator scalat. Aceasta fata este situata la o distanta fata de sursa, dincolo de care orice punct este umbrit.

Notatie:
PUV = poligoanele de umbra care sunt vizibile din pozitia observatorului (A,B)
PUN = poligoanele de umbra care nu sunt vizibile din pozitia observatorului ©
Clasificarea in PUV si PUN se poate face pe baza normalelor la poligoane(back face culling).



Punctul P este umbrit daca numarul de poligoane de tip PUV intersectate de vectorul VP e mai mare decat numarul de poligoane de tip PUN intersectate de vector.
(P e un punct al unei suprafete, VP e vectorul din pozitia observatorului V in punctul P)

In general pentru a determina daca un punct este in umbra, se poate folosi un contor care initial memoreaza numarul de volume de umbra care contin pozitia observatorului.

- **se asocieaza poligoanelor de tip PUV +1 iar celor PUN -1**
- **atunci cand vectorul VP traverseaza un poligon de umbra, se aduna la contor valoarea asociata poligonului**

- **Punctul P este umbrit daca valoarea contorului este pozitiva in P**

Avantaje:
- e generala si poate fi aplicata pentru orice scena, pentru oricate surse de lumina
- produce forme de umbra exacte

Dezavantaje:
- **complexa din punct de vedere al timpului de calcul(se rasterizeaza scena de 2 ori si poligoanele de umbra tot de 2 ori)**
- **sursele de lumina sunt considerate punctiforme => nu se pot obtine penumbre**

- **1** (0.5p) Introducerea cetei in imagini: (septembrie 2011)
  a) cum se modifica culoarea unui fragment tinand cont de factorul de ceata si culoarea de ceata
  Cfog–culoarea de ceaţă
  Cfragment–culoarea calculată pentru un fragment
  0<= Φ <= 1 –factorul de ceaţă, proporţional cu distanţa de la observator la punctul din spaţiul 3D vizibil in pixel (Φ=1: opacitate aici nu pricep la ce se refera)

Atunci, Cpixel, culoarea de afişarea pixelului corespunzator fragmentului este:
Cpixel= (1-Φ)*Cfragment+ Φ*Cfog

  b) cum se calculeaza factorului de ceata de liniar?

Factorul de ceata liniar(obtinut prin interpolare liniara) actioneaza intre 2 distante definite in sistemul coordonatelor observator, z0 si z1, masurate pe directia de observare.

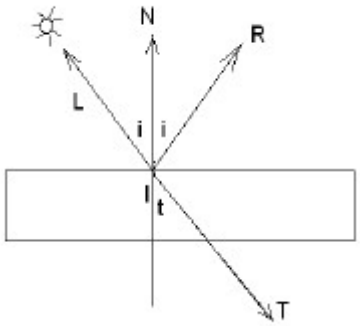
Factorul de ceata este calculat in timpul rasterizarii, pentru fiecare fragment, folosind coordonata z a fragmentului si distantele z0 si z1 transformate in coordonate ale spatiului de afisare.
z - coordonata z a fragmentului
Φ = 0 pentru z < z0
Φ = (z-z0) / (z1-z0) pentru z0 <= z <= z1
Φ = 1 pentru z > z1
Valorile implicite pentru z0 si z1 sunt 0 si 1 (coordonatele z ale planului din fata si planului din space al vederii dupa transformarea in poarta de afisare)

V. ILUMINARE GLOBALA - RAY TRACING

5 La Ray Tracing: (7 februarie 2015) (29 ianuarie 2014) (30 ianuarie 2013) (19 septembrie 2011) (19 sep 2011 nr2)

- Care sunt razele folosite in calculul culorii unui pixel in algoritmul Ray Tracing?

RASPUNSUL E SCRIS CU ALBASTRU:



Desenul asta induce in eroare. R trebuie sa fie reflexia fata de observator, nu fata de lumina.

Enunt intai modul de calcul al culorii unui pixel:

Pentru fiecare pixel al imaginii:

- **se calculeaza raza primara, P, care porneste din pozitia observatorului si trece prin centrul pixelului in planul imaginii**
- **se determina punctul de intersectie, I, al razei primare cu cel mai apropiat obiect de observator**
- **Daca raza nu intersecteaza niciun obiect al scenei, se afiseaza pixelul in culoarea fondului**
- **Altfel : se calculeaza culoarea obiectului in punctul I, tinand cont de :**
  1 reflexia luminii provenita direct de la sursa de lumina, raza L;
  2 lumina provenita in I prin reflexie speculara de la alte obiecte ale scenei, raza R
  3 Lumina provenita in I prin transmisie de la alte obiecte ale scenei, raza T

La final se afiseaza pixelul in culoarea obtinuta prin combinarea celor 3 raze.

- Cum se calculeaza directia lor?

L: raza din I catre sursa de lumina

R: raza reflectata specular, simetrica, fata de N, cu L

$$R = 2(N \cdot L) \cdot N - L$$

T: raza transmisa, calculata pe baza legii lui Snell:

$n1/n2 = \sin(t)/\sin(i)$, n1,n2: indicii de refractie

$$T = L^*(n1/n2) - (\cos(t) + (n1/n2)^*(L \cdot N))^*N$$

Formula asta de aici cu R = 2(N•L)•N - L induce in eroare, R trebuie sa depinda de pozitia privitorului (P), nu de sursa de lumina (L).
La L care e formula?? L = I - S(pozitie sursa) (de unde sunt aceste formule?!)

- Cum se determina razele(fara exprimare matematica)?

Care e diferenta intre intrebarea asta si cea de mai sus?

- Reprezentarea matematica a unei raze si determinarea punctului de intersectie cu scena 3D, care este cel mai apropiat de originea razei.

??? (nici eu nu inteleg aici)

Presupun ca razele sunt reprezentate ca niste vectori tridimensionali.

Punctul de intersectie e calculat ca intersectia intre o dreapta (determinata de vectorul razei si punctul de pornire) si un plan (suprafata obiectului cu care se intersecteaza). Pentru a fi mai usor se folosesc volume incadratoare.

Pentru sfera: se scrie ecuatia dreptei (x = x0 + t*dx, analog y si z), se scrie ecuatia sferei (x - a)^2 + (y - b)^2 + (z - c)^2 = r^2, se inlocuiesc x, y, z, ajungem la o ecuatie de gradul 2, calculam determinantul, daca e < 0 nu se intersecteaza, daca e 0 e tangent la sfera, daca e > 0 se calculeaza radacinile si se ia cel mai mic.

La paralelipiped avem 2h30min, suficient cat sa deducem formula :)).

- Ce este arborele de raze?

Algoritmul Ray-Tracing este recursiv : lumina proveninta in punctul I prin reflexie speculara de la un obiect O1 sau prin transmisie de la un obiect O2 poate fi compusa din:

*reflexia luminii direct de la sursa

*reflexia speculara a altor obiecte

* transmisia luminii provenita de la alte obiecte

Pentru a obtine culoarea pixelului, se evalueaza arborele de raze de la fruze spre radacina.

Arborele de raze se obtine aplicand in mod recursiv algoritmul ray tracing si se formeaza in momentul in care obiectele primesc lumina pe care in mod normal ar fi primit-o de la sursa de lumina de la alte obiecte. (cel putin eu asta am inteles)

- ***Formula de calcul a culorii unui pixel?***

Calculul culorii luminii reflectate în punctul I:

$$I_\lambda(I) = I_{local\lambda}(I) + Ks*R_\lambda(I) + Kt*T_\lambda(I)$$

unde:

λ – reprezinta lungimea de unda: expresia se evalueaza pentru R,G,B

$I_{local\lambda}(I)$ – reprezinta componenta rezultata prin reflexia luminii provenite direct de la sursele de lumina din scena 3D (calculata folosind modelul de iluminare locala)

Ks – este coeficientul de reflexie speculara al materialului obiectului

$R_\lambda(I)$ – reprezinta lumina provenita prin reflexie speculara in punctul I:

se obtine prin evaluarea arborelui de raze

Kt – este coeficientul de transmisie, specific materialului obiectului

$T_\lambda(I)$ - reprezinta lumina provenita prin transmisie (refractie)in punctul I:

se obtine prin evaluarea arborelui de raze

- $0 <= ks, kt <= 1$

VI. RASTERIZAREA PRIMITIVELOR GRAFICE

La Bresenham:

- **4** [1p] Algoritmul Bresenham pentru rasterizarea vectorilor. (1 februarie 2015) (9 septembrie 2015 Restanta) (3 februarie 2014 - a doua zi) (30 ianuarie 2013)

Alegerea punctelor spatiului discret.

- Fie m=(y2-y1)/(x2-x1) panta vectorului,
- (x_i, y_i) ultimul punct al spaţiului discret ales în procesul de generare a vectorului.
- d1 distanţa de la punctul de pe vectorul teoretic, (x_i+1, y), la punctul $O(x_i+1,y_i)$
- d2 distanţa de la punctul de pe vectorul teoretic la punctul $D(x_i+1,y_i+1)$.
- O si D sunt adrese de pixeli (puncte ale spatiului discret)

➢ Următorul punct al spatiului discret ales pentru aproximarea vectorului va fi:

- O dacă d1<d2, D în caz contrar.
- Dacă d1=d2 se poate alege oricare dintre cele două puncte.

Deducerea valorii expresiei de test.
Implementare in C

Implementare in C:

```
void Bres_vect(int x1, int y1, int x2, int y2)
{ //pentru vectori cu panta cuprinsă între 0 şi 1
 int dx, c1, c2, x, y, t;
 dx=x2-x1;
 c1=(y2-y1)<<1;  // 2*dy
 c2=c1-(dx<<1);  // 2*dy – 2*dx
 t=c1-dx;  // 2*dy - dx
 putpixel(x1, y1);
 for(x=x1+1, y=y1; x<x2; x++)
  {if(t<0) t+=c1; //deplasament  O
    else  { t+=c2; y++;} // deplasament D
   putpixel(x,y);
  }
 putpixel(x2,y2);
}
```

generalizarea algoritmului pentru vectori de orice panta

Calculul incremental al expresiei variabilei test folosita in atingerea punctelor spatiului discret.

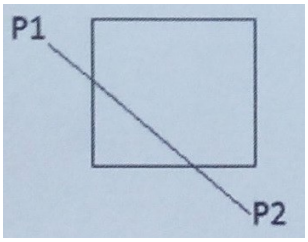
- **3** (1p) Algoritmul Bresenham pentru rasterizarea cercurilor; (29 ianuarie 2014) (31 ianuarie 2013) (septembrie 2011 nr1)
    - Cum se aproximeaza un cerc in spatiul discret la executia algoritmului Bresenham?
      Se aproximeaza mereu fiecare punct de pe circumferinta in functie de eroarea fata de cercul teoretic.
    - Calculul incremental al expresiei variabilei de test folosita in alegerea punctelor spatiului discret.
      t(i+1) = ti + 4x(i) + 6, daca t(i) < 0
      t(i+1) = ti + 4(x(i) - y(i)) + 10, daca t(i) >= 0
    - Implementare in C (fara functia punct_simetric()).
      Ce-ar mai fi sa ne dea asa ceva :)
    - alegerea punctelor spatiul discret,
      daca t(i) < 0, x(i+1) = x(i) + 1, y(i+1) = y(i)
      daca t(i) >= 0, x(i+1) = x(i) + 1, y(i+1) = y(i) - 1
    - calcul si functia de afisare a punctelor pe cerc.
      cat timp x < y se tot aproximeaza si se pun puncte

1 (0.75p) Forma generala (in pseudocod) a algoritmului de rasterizare a suprafetelor poligonale. (30 ianuarie 2013)

- Care este definitia punctului interior unui poligon, pe care se bazeaza algoritmul?
  Un punct este interior daca orice semidreapta dusa din el intersecteaza poligonul de numar impar de ori.
- Cate puncte de intersectie trebuie sa se obtina atunci cand o linie raster traverseaza un varf al poligonului?
  0 sau 2 puncte de intersectie daca e varf de minim / maxim
  1 singur punct altfel

VII. DECUPARE

La Cohen-Sutherland:



- **3** (0.5) Algoritmul Cohen-Sutherland pentru decuparea vectorilor 2D: (7 februarie 2015) (septembrie 2011) (31 ianuarie 2013)
    a) descriere
    fiecare punct este codificat in functie de pozitia sa fata de poligon (<sus><jos><dreapta><stanga>) pe biti.
    Daca !(P1 | P2), segmentul este acceptat
    Daca P1 & P2, segmentul este respins

    Apoi intr-o bucla se repozitioneaza punctele pe intersectiile dintre ele si laturile din directia in care se afla, pana cand segmentul este taiat si acceptat sau respins.

b) explicatie
c) descriere in C (pseudocod)
d) pasii algoritmului pentru decuparea vectorului din figura
e) EXTRA (Comun NR1 si NR2): Nu sunt sigura ca suna chiar asa: Ce este volumul standard de afisare in OpenGL(parca) si la ce foloseste transformarea de proiectie?

Volumul standard este un cub cu latura de 2 (intre -1 si 1 pe fiecare axa). Transformarea de proiectie proiecteaza pe fata cu z = -1 obiectele din volumul de proiectie pentru a fi afisate pe ecran.

********** **PANA AICI** **********

Exemplul dat de profa :) -> ultimul curs (incomplet ….)

1. (0.25p) Prin ce se deosebesc proiectiile oblice de proiectiile axonometrice?
2. (0.5p) Care sunt parametrii prin care este definit sistemul de coordonate observator in OpenGL? Care sunt valorile implicite ale acestor parametrii?
3. (0.75p) Algoritmul z-buffer; descriere; cand se executa? Care este legatura intre z-buffer si fragment shader; ce se intelege prin depth complexity si care sunt calculele de care depinde.
4. (0.75p) Aproximarea reflexiei difuze intr-un punct unei suprafete 3D
5. (1p) Comparatie intre modele Gourand si Phong folosite a calcului culorii fragmentelor la redarea suprafetelor 3D.
6. (1p) Algoritmul Bresenham pentru rasterizarea vectorilor prin primul octant: deducerea expresiei variabilei de test pentru alegerea punctelor spatiului discret. Implementare in C.

FILS - 2016 !!!

1. Sa deducem matricea de rotatie in juruluj unui punct oarecare din planul xoy
2. Despre proiectiile axonometrice. Cum le-ai implementa in opengl
3. Despre proiectiile perspwctiva si ortogonale. Cum e volumul vizual, functiile in opengl
4. Arborele bsp cu tot cu exemplu dat de noi(6 polig in scena). Cum il construim, cum il afisam, ce e in noduri, cum putem reduce obiectele obturate (culling). Plus functia de afisare scrisa
5. Phong iluminare locala. Sa scriem cine formeaza lumina reflectata, formula

1. Proiectii AXONOMETRICE
    a. Caracteristici
        i. Planul de proiectie este oarecare.
        ii. Planul de proiectie nu este paralel cu niciuna dintre axele sistemului de coordonate.
        iii. Exista 3 tipuri:
            1. Izometrice: planul face unghiurile egale cu toate cele 3 axe
            2. Dimetrice: planul face unghiuri egale cu 2 dintre axe
            3. Trimetrice: unghiurile dintre cele 3 axe si plan sunt diferite
    b. Produsul de matrici pentru obtinerea unei proiectii axonometrice
        i. Se aplica transofrmarea PA versorilor axelor principale:

$$\begin{bmatrix} Xux & Xuy & Xuz \\ Yux & Yuy & Yuz \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} = PA \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

    c. Deosebiri fata de proiectiile oblice x3
        i. Proiecţia este ortografică, deci proiectorii sunt în continuare perpendiculari pe planul de proiecţie.
        ii. La proiecţiile oblice tipic considerăm ca planul în sine să fie paralel cu unul din planele paralele (cel mai des, XOY), adică cu două dintre axele sistemului de coordonate. Aici planul nu este paralel cu nici una dintre axele sistemului de coordonate. Putem desigur folosi şi alte plane.
    d. Asemanari si deosebiri cu proiectiile perspectiva

	Proiectii perspectiva	Proiectii oblice	Proiectii axonometrice
Tip proiectii	perspectiva	paralele	paralele
proiectorii	Drepte convergente in centrul de proiectie	Drepte paralele. Nu sunt perpendiculari pe planul de proiectie	Drepte paralele. Sunt perpendiculari pe planul de proiectie
Plan de proiectie	oarecare	oarecare, tipic XOY	Oarecare, neparalel cu niciuna dintre axele sistemului de coordonate

2. Proiectii PERSPECTIVA
    a. Parametrii care definesc volumul vizual in OpenGL x2

i. Se consideră centrul de proiecție și planul „near”. Pentru acest plan, avem o distanță Znear și coordonate top, bottom, left, right. Se consideră și planul „far” pt care cunoaștem Zfar. Marginile acestui plan sunt determinate complet de cei 6 parametri: top, bottom, left, right, Znear, Zfar. glm::frustum preia acești parametri și întoarce un mat4, matricea de proiecție.
ii. Există proiecții perspective speciale mai simple, simetrice, în c are top=-bottom și left=-right. Pt asta vom folosi glm::perspective, care ia fov, ratio, Znear și Zfar. Znear și Zfar au același înțeles, FOV este unghiul de vizualizare vertical și se respectă tan(FOV/2) = top/Znear; ratio = (right-left)/(top-bottom) (poate fi luat și ca right/top).
iii. Znear trebuie să fie diferit de 0 (mai mare), altfel avem împărțiri la 0. Cu cât mai mare, cu atât mai bine. Zfar trebuie să fie cât mai mic (dacă e prea mare putem avea erori de precizie). Znear este transformat până la urmă în planul (*, *, -1) și Zfar în planul (*, *, 1). Pentru a ascunde decuparea obiectelor la Zfar, putem folosi tehnici precum ceața. [nu pun explicație aici, ceața poate fi considerat un subiect independent

b. Planul in care se5 efectueaza proiectia: Proiecția se efectuează în planul „near”.

3. Sistemul de coordonate OBSERVATOR
   a. Parametri x4
      i. Poziția camerei, orientarea ei (direcția „față” -- forward -- și direcția „sus” -- up).
      ii. forward indică o direcție care s-ar duce prin „centrul” ecranului în spate.
      iii. up indică orientarea camerei ca o rotație în jurul vectorului forward. Schimbarea lui efectiv rotește imaginea de pe ecran.
   b. Valori implicite x3
      i. Poziția va fi în originea coordonatelor globale.
      ii. Vectorul forward va indica spre (0, 0, -1).
      iii. Vectorul up va indica spre (0, 1, 0).
   c. Produsul de matrici pentru a obtine transformarea din sistem de coordonate global in sistem de coordonate observator
      i. $V = (T * R * S)^T$ (mai întâi scalare [tipic scalăm cu 1 adică punem matricea identitate], apoi rotație [orientarea camerei, adică vectorii forward și up, generează această matrice], apoi translație [poziția camerei], totul transpus).

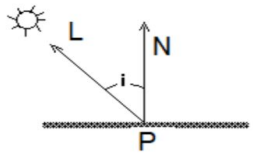
4. Transformarea din sistem de coordonate OBIECT in sistem de coordonate de DECUPARE
   a. Volumul de decupare
      i. Volumul canonic este între (-1;1) pe cele 3 axe de coordonate.
   b. Cand se efectueaza decuparea primitivelor grafice pe banda grafica
      i. Decuparea se efectuează după ce a terminat vertex shaderul (și geometry shaderul, dacă există unul) și s-au determinat coordonatele din spațiul canonic de decupare (s-a făcut și împărțirea perspectivă). Este ultimul pas înainte de a

începe rasterizarea efectivă, înainte de a împărți primitivele rămase în fragmente.

5. Modelul Phong
    a. Reflexia speculara intr-un punct al unei suprafete 3D x2
        i. Se folosește modelul de iluminare locală, în care se calculează pentru fiecare fragment normala, direcția sursei de lumină și se determină culoarea.

        1. *Cantitatea de lumină care ajunge la observator pe suprafețele ce nu sunt reflectori perfecți (o oglindă e refl. perfect) depinde de **distribuția spațială a luminii reflectate specular. Aceasta se aproximează cu cos(alf)^n, unde n este exponentul de de strălucire al materialului.** Is = intensitatea luminii reflectate specular, Ks = coeficientul de reflexie specular(specific material)*
        Is = Isursă * Ks * cos(alf)^n

    b. Comparatie cu modelul Gouraud
        i. În modelul Gouraud, calculele de lumină se efectuează doar în fiecare vârf, iar culorile obținute se interpolează în mijlocul primitivelor. În modelul Phong în schimb se interpolează normalele și se fac calculele de lumină pe toate fragmentele.
    c. Aproximarea reflexiei difuze a luminii intr-un punct al unei suprafete 3D, luminata de o sursa punctiforma (modelul empiric) *(v. curs 7, modele de reflexie și modele de shading)*



Reflexia difuza in punctul P

*Fie **Id = intensitatea luminii reflectate difuz***
Isursa = intensitatea luminii din sursa punctiforma
i = unghiul de incidență al vectorului de lumină de la sursă cu normala la suprafață
Kd - coeficientul de difuzie al suprafeței pe care se reflectă lumina
*Caz simplu: **Id = Isursă * Kd * cos(i)***
*De obicei într-o scenă avem și lumină ambientală (lumina de la celelalte obiecte din scenă). Reflexia luminii ambientale = Ia * ka (Intensitatea luminii ambientale * coef. de difuzie al lum. ambientale).*
*Caz avansat: **Id = Reflexia luminii ambientale + reflecția luminii de la sursă =>***
Id = Ia * Ka + Isursă * Kd * cos(i)
Dacă luăm în calcul și atenuarea luminii (lumina scade în intensitate invers proporțional cu inversul pătratului distanței față de sursă: 1 / d^2, adică lumina scade în intensitate cu cât sursa e mai departe) =>
Id = Ia * Ka + Isursă * (1/d^2) * Kd * cos(i)

d. Calculul culorilor fragmentelor la afisarea suprafetelor 3D
    i. Se determină normala locală. Interpolăm normalele vârfurilor, apoi normalizăm (interpolarea liniară va schimba valoarea absolută a normalei; noi avem nevoie de ceva care are lungimea 1 deci facem normalizare).
    ii. Se determină direcţia sursei de lumină. Vom folosi poziţia interpolată din coordonate globale (primită şi ea de la VS) şi, of course, poziţia sursei de lumină în sine.
    iii. Folosind vectorii N, L şi, pentru reflexia speculară, V, facem calculele de iluminare locală corespunzătoare.
e. Avantaje si dezavantaje
    i. Se elimină efectul de bandă Mach aproape în totalitate. (avantaj)
        1. Dacă avem o sferă aproximată cu prea puţine triunghiuri, problema e mult redusă faţă de Gouraud şi va părea rotundă oricum
    ii. Reflexiile speculare pot fi redate corespunzător chiar şi pt sursa de lumină apropiată de centrul unui poligon „prea mare” (avantaj)
        1. În iluminarea Gouraud, normalele se determină doar în vârfuri şi, pt sursele din situaţia asta iluminarea va fi aproape zero. Deci în loc să se ilumineze, în Gouraud obiectele se ÎNTUNECĂ pe măsură ce sursa de lumină se apropie de centrul poligonului. Phong rezolvă problema asta
    iii. Calculele de lumină se fac pentru fiecare fragment, deci sunt în număr mai mare. (dezavantaj). La Gouraud, se calculau doar per vârf.

6. Ray Tracing
    a. Utilizarea pentru eliminarea partilor nevizibile
        i. Razele de lumină care vin spre cameră vor fi „reflectate” automat către direcţia care influenţează fragmentul afişat. Astfel, părţile nevizibile vor fi automat ascunse pentru că nu vor ajunge raze efectiv pe calea directă către ele.
    b. Calculul culorilor pixelilor
        i. Razele folosite x2
            1. Se foloseşte raza reflectată specular.
            2. Se foloseşte raza refractată (obiecte transparente).
            3. Se foloseşte evident raza primară în sine.
        ii. Formula de calcul x2
            1. Fiecare obiect are constante Ks şi Kt (Ks pt reflexie speculară, Kt pt transmisie/refracţie).
            2. Se preia I + Ks*R() + Kt * T()
                a. R() se determină recursiv din raza reflectată (până la adâncimea maximă; altfel se ia 0)
                b. T() se determină recursiv din raza transmisă/refractată (din nou, până la adâncimea maximă).

c. I este iluminarea locală; avem lumină ambiantă, lumină difuză şi specular. În calculul luminii difuze şi a celei speculare se adaugă un factor s, prin care se determină atenuarea luminii sursei (s=0 dacă există un obiect opac între sursă şi punctul I, s=1 dacă nu există absolut nimic, şi o valoare intermediară dacă avem un obiect transparent).

c. Utilizarea pentru afisare in umbra
    i. Factorul s calculează automat umbrele.
d. Cum se calculeaza directia razelor x2
    i. Raza reflectată specular se determină prin simpla reflexie. R = 2 * dot(N, L) * N - L
    ii. Raza transmisă se determină folosind indicii de refracţie (spaţiul liber este considerat ca având indicele 1). T = L*(n1/n2) – (cos(t) + (n1/n2)*(L·N))*N (din curs)

7. Algoritmul Bresenham pentru rasterizarea vectorilor *(Curs 10, rasterizarea primitivelor grafice; avem toate răspunsurile în curs; pentru cercuri răspunsul este în cursul 11)*
    a. Alegerea punctelor spatiului discret x3
    b. Deducerea valorii expresiei de test x2
    c. Implementare C x3
    d. Cum se aleg pixelii
    e. Generalizare pentru orice panta x2
    f. Functia de afisare a punctelor de pe cerc

8. BSP pentru object culling x2
    a. Cum se construieste arborele BSP si ce se memoreaza in noduri x3
    BSP = Binary Space Partitioning
    Arborele memorează în noduri lista primitivelor care compun scena.
    Lista primitivelor (LP) este data la intrare. Proces creare:
        1. *Se alege un poligon P din LP în funcţie de care este creat arborele. P va root în arbore*
        2. *Dacă o primitivă este în spaţiul "faţă" al lui P, este adăugată în "lista faţă" ce va genera sub-arborele "faţă" al nodului.*
        3. *Dacă o primitivă este în spaţiul "spate" al lui P, este adăugată în "lista spate" ce va genera sub-arborele "spate" al nodului*
        4. *Se alege un poligon din lista faţă şi se creează listele "faţă" şi "spate" pentru acesta şi tot aşa*
    b. Cum se afiseaza arborele BSP x2
    Afişarea arborelui este "back-to-front".
        1. *Dacă observatorul este în spaţiul faţă al nodului rădăcină afişarea este : "spate" -> nod rădăcină -> "faţă"*

2. *Dacă observatorul este în spațiul "spate" al nodului rădăcină afișarea este : "față" -> nod rădăcină -> "spate"*

c. Functia de afisare a arborelui BSP x3

```
void afișareBSP(arbore *A, Poziție Observator) {
    if (!A) return;
    if (Observator este în subspațiul față al planului rădăcinii lui A) {
        afișareBSP(A->spate);
        *afișare rădăcină A;
        afișareBSP(A-față);
    } else { // Observator este în spațiul "spate" al root
        // exact ca mai sus dar inversezi față cu spate
    }
}
```

d. Cum poate fi utilizat algoritmul pentru eliminarea obiectelor nevizibile din banda grafica x3

e. Cand trebuie sa fie reconstruit arborele BSP al unei scene

Arborele BSP trebuie reconstruit doar când scena se schimbă. Arborele nu trebuie modificat atunci când observatorul își schimbă poziția (se apelează din nou afișareBSP cu noua poziție a observatorului)

Arborele BSP este OK și în cazul în care există obiecte dinamice. În acest caz z-buffer trebuie să fie activat. Se randează scena statică în ordinea inversă algoritmului normal, apoi se randează elementele dinamice. (ordinea normală ar avea o performanță puțin mai rea decât z-buffer pur pt că ar fi mai multe suprascrieri în medie -- depth complexity mai mare)

9. Algoritmul z-buffer x2
   a. Descriere
   Z-buffer este algoritmul de eliminare a fragmentelor nevizibile ale primitivelor grafice. Observatorul este la infinit pe axa Z negativă. Z-buffer = tablou bidimensional cu nr_elemente = nr_pixeli din imagine.
   b. Cand se executa?
   Este executat de GPU la momentul rasterizării primitivelor.
   c. Depth complexity
   Depth complexity este numărul de suprascrieri ale unui pixel la momentul rasterizării. Dacă fragmentul (x,y,z) are z mai mic decât fragmentul afișat pe pixelul(x,y), fragmentul curent devine cel afișat în pixelul(x,y). Depth complexity, înseamnă pe românește de câte ori a fost găsit un fragment cu z mai mic decât cel curent.
   d. Care dintre urmatoarele afirmatii sunt corecte:
      i. Algoritmul z-buffer necesita rasterizarea poligoanelor scenei 3D in ordinea descrescatoare a distantei lor fata de observator *[Fals]. Dacă dai într-o altă ordine, bufferul Z detectează asta și refuză suprascrierea fragmentului (și a*

bufferului) în această situație, și va da discard la fragmentele corespunzătoare -- care ar fi în spate.

ii. Bufferul z memoreaza coordonatele z ale fragmentelor de primitive care sunt vizibile intr-o imagine [*Adevărat]. Bufferul Z este o matrice cu valorile Z ale pixelilor. Ex: z-buffer[x][y] = valoarea z pentru pixel[x][y]*

iii. Bufferul z se actualizeaza pentru fiecare fragment rezultat din rasterizarea primitivelor dintr-un cadru imagine *[Fals] Se actualizează doar pentru fragmente cu coord z mai mică decât ce are el stocat în zona aia*

iv. Algoritmul z-buffer este exectutat de procesoarele placii grafice [*Adevărat] Fiind implicat în procesul de rasterizare, este realizat de GPU.*

10. Reflexia DIFUZA
    a. A luminii intr-un punct al unei suprafete 3D
    b. Aproximare x2

11. Modelul Gouraud x2
    a. Afirmatii cu A/F (nespecificate)
    b. Calculul culorilor fragmentelor la afisarea suprafetelor 3D
        i. Se calculează culoarea cf modelului de iluminare locală (lumină ambiantă + reflexie difuză; cea specular de obicei e omisă pt că Gouraud nu o reprezintă foarte bine) în vârfurile primitivei
        ii. Culoarea este apoi interpolată pe suprafața primitivei.
    c. Avantaje si dezavantaje fata de alte modele

        i. Calcul rapid, calculele de lumină se efectuează doar în vârfuri. Se poate interpola pe laturi, apoi o altă interpolare liniară permite să punem culorile de pe fiecare linie în progresie aritmetică. *[Avantaj]*

        ii. Față de Lambert, efectul de bandă Mach este redus; nu mai există diferențe clare între laturile poligoanelor vecine. *[Avantaj]*

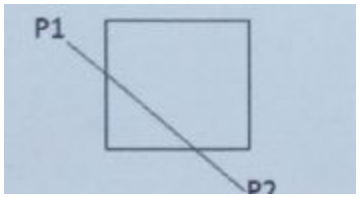
        iii. Față de Phong, încă există efect de bandă Mach (Phong reduce și mai mult). *[Dezavantaj]*

        iv. Reflexia specular nu e modelată corespunzător (v. ex 5.e, iii)

12. Determinarea fetelor auto-obturate ale poliedrelor convexe. Conditia de vizibilitate atunci cand determinarea este efectuata:
    a. In coordonate globale
        i. Normala feței este înspre observator (dacă el este în exterior), sau opus față de observator (dacă el este în interior). Pe exterior, dot(N, V) > 0 pt fețele vizibile, pe interior dot(N, V) < 0.
    b. In coordonate de decupare

i. Dacă observatorul e în exterior, vârfurile triunghiului trebuie să se proiecteze în sens trigonometric în planul de proiecţie al observatorului. La interior, sensul acelor de ceasornic.

13. Introducerea umbrelor in imagini prin metoda volumelor de umbra (fara implementare cu OpenGL). X2

14. Cohon-Sutherland pentru decuparea vectorilor 2D
    a. Descriere
        i. Pentru capetele vectorului se face o categorisire în funcţie de poziţia faţă de dreptunghiul de decupare; se determină dacă este deasupra, dedesubt, la stânga sau la dreapta, şi se păstrează aceste informaţii într-un cod binar. Marginea se consideră în interior şi va da 0 pt bitul corespunzător.
        ii. Se face „şi" între codurile binare. Dacă rezultatul nu este 0, înseamnă că întregul vector este în stânga, dreapta, deasupra sau dedesubt în funcţie de ce bit (sau biţi) au rămas 1. În acest caz putem da discard.
        iii. Dacă codurile binare sunt ambele 0000, vectorul este în întregime în interior. Putem trece la desenarea efectivă.
        iv. Altfel, dacă P1 este în interior (are codul 0000), se interschimbă punctele P1 şi P2 şi codurile lor.
        v. Decuparea la capătul cu P1
            1. Dacă P1 este în stânga, se face intersecţia cu marginea din stânga al dreptunghiului de decupare. O notăm P. Noua valoare a lui P1 va fi P, şi reluăm algoritmul de la i.
            2. Dacă P1 este în dreapta, se face intersecţia cu marginea din dreapta. Simiar cu 1.
            3. Deasupra? Analog cazurilor anterioare.
            4. Dedesubt? Analog cazurilor anterioare.
    b. Pasii algoritmului pentru un exemplu
        i. 

            1. Vom considera codul ca fiind (up, down, left, right). P1 are codul 0010, P2 are codul 0101 (este puţin mai la dreapta de marginea din dreapta). Pasul i este făcut.
            2. 0010 & 0101 = 0000. Continuăm.
            3. 0010 nu este 0000. Continuăm.

4. P1 este în stânga, deci determinăm intersecţia cu capătul din stânga. Mutăm P1 în ea. [aici punem un desen cu segmentul decupat astfel]. Luăm alg de la capăt.
5. P1 are codul 0000. P2 are codul 0101.
6. 0000 & 0101 = 0000. Continuăm.
7. 0101 nu este 0000. Continuăm.
8. P1 este în interior. Schimbăm P1 cu P2, so acuma P1 este punctul din dreapta şi are codul 0101. Vedem că P1 este în dreapta, facem intersecţia cu marginea din dreapta… Mă rog, cu prelungirea. Se face trunchierea.
9. Calculăm din nou codurile. P1 are codul 0001, P2 are codul 0000.
10. … ajungem la pasul v al algoritmului
11. P1 este sub dreptunghiul de decupare, se face inters cu marginea de jos a acestuia şi se taie vectorul acolo (mutăm P1 acolo)
12. Calculăm codurile din nou. Ambele au acuma codul 0000.
13. Ajungem la pasul iii al algoritmului, şi vedem că ambele puncte sunt în interior. Să desenăm efectiv segmentul!

15. Care dintre proiectii nu este o transformare afina (justificati)
    a. Proiectia oblica *[Fals] Este o proiecţie ortografică combinată cu alte transformări afine: rotaţii, translaţii, shear/forfecare, potenţial scalări.*
    b. Proiectia perspectiva *[Adevărat] În proiecţia asta trebuie făcută o împărţire la ceva ce depinde de poziţia vârfului, anume coordonata lui z. Doar înmulţirile şi împărţirile la constante sunt afine, împărţirea la ceva neconstant nu mai e afină. [totuşi putem folosi matrice pt că există împărţirea perspectivă care o face GPU-ul -- se face automat împărţire la coordonata w deci putem face w = z].*
    c. Proiectia axonometrica *[Fals] Este o proiecţie ortografică combinată cu alte transformări afine: rotaţii, translaţii, potenţial scalări.*
    d. Proiectia ortografica *[Fals] Proiecţiile*

<3 Subete EGC <3

1.Precizati pe scurt care este asemanarile si deosebirile dintre .efectele proiectiilor perspectiva si cele ale proiectiilor axonometrice.

Asemanari:

a) Redau mai multe fete ale obiectului proiectat.
b) Pot reprezenta in plan 2D un spatiu 3D.
c) Unghiurile se conserva pentru fetele obiectului paralele cu planul de proiectie

*

Deosebiri:

a) La proiectiile perspectiva, proiectiile liniilor paralele care nu sunt paralele cu planul de proiectie converg catre un punct din plan, numit punct de convergenta; proiectiile axonometrice sunt afine - conserva paralelismul liniilor.
b) Obiectele aflate la distanta mare se micsoreaza la perspectiva; in cazul proiectiilor axonometrice nu.

2. Prin ce se deozebejte proiectiile oblice de proiectiile axonometrice?

Proiectii oblice: plan de proiectie perpendicular pe o axa principala

Proiectii axonometrice: planul de proiectie nu e perpendicular pe niciuna din axele sistemului de coord.

Proiectii oblice:proiectorii nu sunt perpendiculari pe planul de proiectie

Proiectii axonometrice: proiectorii sunt perpendiculari pe planul de proiectie

3. Care dintre urmatoarele tipuri de proiectii nu este o transformare afina? Justificati.

b) Proiectia perspectiva nu este afina, adica nu conserva paralelismul liniilor (celelalte tipuri de proiectii erau paralele). In cazul proiectiei perspectiva, liniile care nu sunt paralele cu planul de proiectie converg catre un punct numit punct de convergenta.

4. Care sunt parametrii care definesc volumul vizual al proiectiei perspectiva in OpenGL? Care este planul in care se efectueaza proiectia?

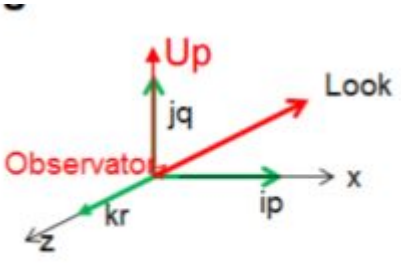
Volumul vizual al proiectiei perspectiva (View frustum): trunchi de piramida delimitat de planul din față (near) si planul din spate (far) al vederii, cu varful in pozitia observatorului.
Se efectueaza proiectia perspectiva în planul din față (in fereastra vizuala).

Proiectia perspectiva este reprezentata de un trunchi de piramida (frustum) definit prin cele 2 planuri, **cel din apropiere** si **cel din departare**, cat si de deschiderea unghiurilor de vizualizare pe cele 2 axe, OX si OY

Matricea de proiectie in acest caz poate fi calculata cu ajutorul functiei glm::perspective ce *primeste ca si parametri deschiderea unghiului de vizualizare pe orizontala (**Field of View - FoV**), raportul dintre latimea si inaltimea ferestrei de vizualizare (**aspect ratio**), cat si distanta pana la cele 2 planuri zFar si zNear.*

5. Care sunt parametrii care definesc sistemul de coordonate observator? Care sunt valorile implicite ale acestor parametrii in OpenGL?

a) poziţia observatorului (camerei): originea sistemului de coordonate globale
b) direcţia în care priveşte observatorul: direcţia negativa a axei OZ
c) direcţia sus a planului de vizualizare: direcţia pozitivã a axei OY

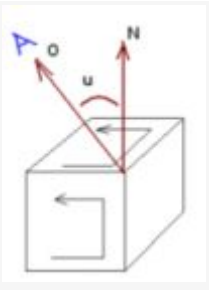


6. Determinarea fetelor auto obturate ale poliedrelor convexe. Conditia de vizibilitate atunci cand determinarea este efectuata: a) in coordonate globale; b) in coordonate de decupare.

Tip back face culling:
- Poate reduce substantial numarul de poligoane procesate in banda grafica.

Algoritmul
- Observatorul si obiectul sunt raportati la acelasi sistem de coordonate.
- Conturul fiecarei fețe a obiectului este orientat in sens trigonometric atunci cand poliedrul este vazut din exterior.
- Observatorul este situat in afara obiectului. Fața este vizibila daca : 0<= u < 90°

N: normala la fața



O: vectorul orientat catre observator

Determinarea fetelor auto-obturate poate fi efectuata:
1. In sistemul coordonatelor globale (in care este definit si observatorul)

Produsul scalar:

$N \bullet O = |N| * |O| * \cos(u) \rightarrow \cos(u) = N \bullet O / (|N| * |O|) = Nu \bullet Ou$

Conditia de vizibilitate: Nu • Ou > 0 (produsul scalar al versorilor)

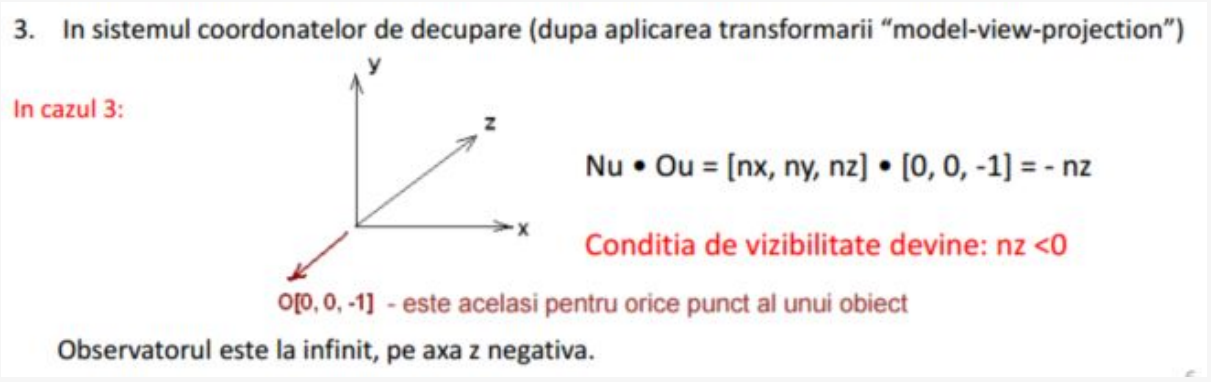
2. In sistemul de coordonate observator

L: raza din I catre sursa de lumina

R: raza reflectata specular, simetrica, fata de N, cu L

T: raza transmisa, calculata pe baza legii lui Snell:

3. In sistemul coordonatelor de decupare (dupa aplicarea transformarii "model-view-projection")



7. Algoritmul Z-buffer. Descriere. Cand se executa? Depth complexity?

- Algoritm de eliminare a partilor nevizibile ale primitivelor grafice.
- Executat de GPU, fiind integrat in procesul de rasterizare a primitivelor (dupa transformarea în poarta de afişare a vârfurilor)
- Observatorul este la infinit, pe axa z negativa

-Asupra primitivelor se efectueaza proiectie ortografica(de ce asa?) in planul XOY
Doua fragmente aflate pe acelasi proiector, rezultate din rasterizarea a doua primitive diferite, au coordonata z diferita:

Primitivele grafice (triunghiuri, linii) in care a fost descompusa scena 3D sunt rasterizate in ordinea in care au fost transmise din programul de aplicatie (nu sunt sortate). Buffer-ul imagine (frame buffer) este actualizat pe masura rasterizarii primitivelor, astfel:

daca (fragmentul curent se proiecteaza in pixelul (x,y) si z-fragment < z-fragment afisat in pixelul (x,y))

=> actualizeaza culoarea pixelului (x,y) in bufer-ul imagine, la culoarea fragmentului curent

Rezulta: este necesar sa se memoreze coordonatele z ale fragmentelor afisate in pixelii imaginii

Z-buffer: - tablou bidimensional cu numar de elemente egal cu numarul de pixeli ai suprafetei de afisare - în memoria placii grafice

Algoritm:
*Initializeaza buffer-ul imagine la culoarea de fond
*Initializeaza Z-buffer la coordonata z maxima (z=1)
Pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive *
calculeaza culoarea fragmentului

daca z< Z-buffer[y][x] atunci
{ Z-buffer[y][x] = z
*actualizeaza culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea fragmentului f
}

-Ultima transformare din lantul de transformari efctuate asupra varfurilor 3D conserva coordonata z a fiecarui varf.

Aprecieri asupra algoritmului z-Buffer
1) Numarul de comparatii de valori z pentru un pixel = numarul de fragmente care se proiecteaza in acel pixel. Timpul de calcul tinde sa devina independent de numarul de poligoane: in medie, numarul de pixeli acoperiti de un poligon este invers proportional cu numarul de poligoane.
(stiu ca e in curs formularea, dar nu inteleg, daca ai mai multe poligoane nu e mai mare posibilitatea sa se suprapuna?)
2) “Depth complexity”: numarul d

suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine .
* calcul culoare fragment: model iluminare, utilizare texturi
3) Pentru o precizie buna, este necesara o reprezentare pe 32 biţi a valorilor din zbuffer.

8. Care dintre urmatoarele afirmatii este corecte:

a. Algoritmul z-buffer necesita rasterizarea poligoanelor scenei 3D in ordinea descrescatoare a distantei lor fata de observator - **Fals**

b. Bufferul z memoreaza coordonatele z ale fragmentelor de primitive care sunt vizibile intr-o imagine - **Adevarat**

c. Bufferul z se actualizeaza pentru fiecare fragment rezultat din rasterizarea primitivelor dintr-un cadru imagine - **Fals (testul de depth)**

d. Algoritmul z-buffer este exectutat de procesoarele placii grafice - **Adevarat**

9. Algoritmul BSP //aici erau in mai multe variante exercitii cu a,b,c,d si majoritatea se repetau

a. **Cum se contruieste arborele BSP al unei scene 3D si ce se memoreaza in nodurile sale?**

Fiecare nod al arborelui corespunde unui plan de partitionare a spatiului 3D (de regula, planul unui poligon al scenei)

Fiecare plan de partitionare imparte spatiul in 2 semi-spatii:
- cel din faţa planului (de aceeasi parte cu normala la plan)
- cel din spatele planului

Se incepe cu un poligon oarecare din lista (de regula primul), pentru care se creeaza nodul radacina al arborelui

Primitivele din semi-spatiul "faţă" formeaza "lista-faţă", care va genera subarborele "faţă" al nodului. Primitivele din semi-spatiul "spate" formeaza "lista-spate", care va genera subarborele "spate" al nodului.

Se alege un poligon din lista-faţă si se creaza nodul radacina al subarborelui "faţă, etc.

b. **Cand trebuie sa fie reconstruit arborele BSP al unei scene?**

Stie cineva?
// Daca modificarea pozitiei observatorului nu determina si modificarea scenei (curs 5 - Eliminarea… slide 8).
La modificarea pozitiei obiectelor din scena. ← E bine?
Eu cred ca da.

Eu cred ca arborele trebuie modificat (nu reconstruit) atunci cand scena contine obiecte dinamice. Construirea arborelui BSP este costisitoare, si de aceea exista adaptari ale alogoritmului BSP care sa evite reconstructia arborelui.

Ex: 1. Pentru fiecare cadru imagine se insereaza in arborele BSP cu obiecte statice noduri corespunzatoare obiectelor dinamice.

2. Se transmite arborele bsp nemodificat si obiectele dinamice, iar eliminarea partilor obturate de obiectele dinamice se face la rasterizare, folosind algoritmul z-buffer.

c. **Cum se afiseaza o scena reprezentata printr-un arbore BSP?**

-Tine cont de pozitia observatorului

-Afisare "back-to-front" : se incepe cu primitiva cea mai indepartata de observator

-Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in functie de pozitia observatorului fata de planul atasat fiecarui nod.

- In fiecare nod: se avanseaza in subarborele nodului care este opus observatorului fata de plan.

e. **Scrieti functia de afisare arborelui BSP.**

```
void afisareBSPcull (arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (Observator este in semispatiul fata al planului radacinii) atunci
    daca (planul nodului radacina nu intersecteaza volumul vizual) atunci
          afisareBSPcull(A→fata)
    altfel { afisareBSPcull(A->spate); *afisare primitive din nodul radacina; afisareBSPcull(A->fata);}
  altfel
    daca (planul nodului radacina nu intersecteaza volumul vizual) atunci
          afisareBSPcull(A→spate)
    altfel { afisareBSPcull(A->fata); *afisare primitive din nodul radacina; afisareBSPcull(A->spate);}
}
```

7

```
void afisareBSP(arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (Observator este in semispatiul fata al planului radacinii) atunci
    { afisareBSP(A->spate); *afisare primitive din nodul radacina; afisareBSP(A->fata);}
  altfel
    { afisareBSP(A->fata); *afisare primitive din nodul radacina; afisareBSP(A->spate);}
}
```

f. **Cum poate fi utilizat algoritmul BSP pentr eliminarea poligoanelor nevizibile din banda grafica?**

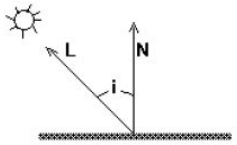
Daca planul de partitionare al unui nod nu intersecteaza volumul vizual, atunci numai subarborele aflat de aceeasi parte cu volumul vizual va fi afisat, celalalt subarbore fiind eliminat din banda grafica.

10. Reflexia difuza a luminii intr-un punct al unei suprafete 3D.

Aproximarea reflexiei difuze intr-un punct al unei suprafete 3D

- Lumina reflectată difuz de o suprafaţă este dispersată regulat în toate direcţiile.
- **Legea lui Lambert** defineşte reflexia luminii provenite de la o **sursă punctiformă**, de către un difuzor perfect:

$$Id = Isursa * kd * \cos(i) \qquad 0 <= i <= \pi/2$$



Reflexia difuza

Isursa – este intensitatea luminii incidente (provenita de la sursa de lumina)

Kd – este coeficientul de difuzie a luminii incidente, dependent de materialul suprafetei: suprafata reflecta/absoarbe/transmite lumina in anumite proportii: 0<=kd<=1

Id este intensitatea luminii reflectate difuz de suprafata

- Dacă i este mai mare ca π/2, suprafaţa nu primeşte lumină de la sursă (sursa de lumină se află în spatele suprafeţei).

11. Aproximarea reflexiei difuze.
1

- Stiind că

  $$\cos(i)=L\cdot N / (|L|\cdot|N|) = Lu\cdot Nu$$

  rezulta ecuatia care modeleaza lumina reflectata difuz:

❖ $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * (Lu \cdot Nu)$

Cd = [IdR, IdG, IdB] – culoarea luminii reflectate difuz

Pentru a include si cazul in care $i > \pi/2$ (lumina de la sursa nu ajunge in punctul considerat):

❖ $I\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \max((Lu \cdot Nu), 0)$

(In caz ca cere si lumina ambientala + l de unda diferite + factor de atenuare)

12. Aproximarea reflexiei speculare a luminii intr-un punct al unei suprafete 3D, luminata de o sursa punctiforma (modelul empiric).

Modelul Phong pentru aproximarea reflexiei speculare intr-un punct al unei suprafete 3D:

$$Is\lambda = Isursa\lambda * w(i,\lambda) * \cos(\alpha)^n$$

w(i, λ) este funcţia de reflectanţă, i - unghiul de incidenţă iar λ - lungimea de undă a luminii incidente

In practica, w(i, λ) este înlocuită cu o constantă determinată experimental, numită **coeficientul de reflexie speculară al materialului, notat** $ks\lambda$.

❖ Modelul practic al reflexiei speculare:

$$Is\lambda = Isursa\lambda * ks\lambda * \cos(\alpha)^n$$

$\cos(\alpha)=R\cdot V / (|R|\cdot|V|) = Ru\cdot Vu$

Rezulta:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * (Ru \cdot Vu)^n$$

Pentru a include si cazul in care $\alpha = 90$:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n, 0)$$

14.Calculul culorilor fragmentelor la afisarea suprafetelor 3D folosind modelul Gouraud. Calculul incremental al culorilor. Avantajele si dezavantajele modelului Gouraud in comparatie cu alte modele.(Phong)

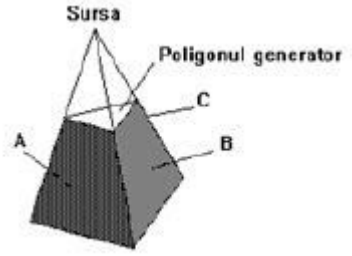
La asta cred ca scrii ca se face folosind modelul de iluminare locala Phong si ca se calculeaza culorile pentru fiecare vertex, fiind apoi interpolate pentru fragmente. Asta inseamna ca vom avea efectul de banda Mach.

La Phong, se calculeaza doar normala per vertex, iar apoi pentru fiecare fragment normalele sunt interpolate si in fragment shader se calculeaza culoarea folosind tot modelul de iluminare locala Phong.

Gouraud cred ca e mai eficient, dar Phong da rezultate mai realiste.

17. Introducerea umbrelor in imagini prin metoda volumelor de umbra (fara implementare cu OpenGL).

- sursa de lumina considerata punctiforma, scena 3D alcatuita din poligoane
- volum de umbra: definit de o sursa de lumina si un poligon luminat (vizibil din pozitia sursei)
- **poligon generator** = poligonul luminat



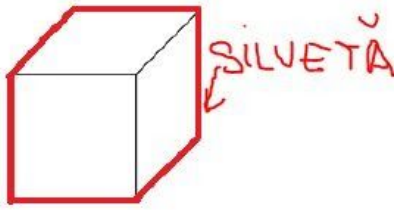
- volumul infinit este determinat de o sursa si un poligon generator; este delimitat de o fata aflata la o anumita distanta fata de sursa, distanta dincolo de care intensitatea luminii este neglijabila. Acea fata reprezinta poligonul generator,
- volumul de umbra este decupat la marginile volumului vizual
- **poligoane de umbra** = fete laterale ale volumului; normalele la suprafete - indreptate spre exterior

Fie:

- PUV - poligoane vizibile observatorului (A si B in figura)
- PUN - nevizibile obs (C in desen)

Clasificarea poligoanelor de umbra in PUV si PUN de poate face pe baza normalelor (back face culling).

Pe foarte scurt: fie P un punct al unei suprafete si VP vectorul din pozitia observatorului in P. **Daca V nu e in umbra**, P este umbrit ⇔ nr. PUV intersectate de VP > nr. PUN intersectate.
Optimizare: in loc sa calculam volumul de umbra pt fiecare poligon generator, vom calcula un singur volum de umbra pt o suprafata poliedrala => se vor determina poligoane de umbra doar pt laturile care fac parte din silueta unei suprafete (si anume doar pt cele luminate).
latura de silueta = margine a unei suprafete sau latura care separa un poligon vizibil de unul nevizibil



Se implementeaza folosind stencil buffer de pe GPU, impreuna cu z-buffer. Pixelii aflati in umbra au valori nenule in buffer.
Pixelii de umbra au culoarea:
Cfragment = Cemisiva + Cambianta = ke +ka*globalAmbient

Aprecieri asupra metodei volumelor de umbra

Pozitive:

- Este generala si poate fi aplicata pentru orice scena, pentru oricate surse de lumina.
- Produce forme de umbre exacte.

Negative:

- Complexa din punct de vedere al timpului de calcul:

  se rasterizeaza scena de 2 ori si poligoanele de umbra de 2 ori!

- Sursele de lumina sunt considerate punctiforme, de aceea nu pot fi obtinute penumbre.

18. Introducerea cetei in imagini :

a) cum se modifica culoarea unui fragment tinand cont de factorul de ceata si culoarea de ceata?

Fără ceață, pe măsură ce observatorul se îndepărtează de un obiect, obiectul se apropie de planul din spate şi deci este decupat atunci când planul din spate îl intersectează.

Cu ceață, dacă intensitatea ceții creşte proporțional cu distanța de la poziția observatorului, se crează efectul de distanță pentru obiectele îndepărtate.

Dacă densitatea ceții creşte până la opacitate la o distanta mai mica decat aceea a planului din spate, decuparea față de planul din spate nu mai este necesară căci obiectele dispar datorită opacității.

Cfog–culoarea de ceață
Cfragment–culoarea calculată pentru un fragment
0<= Φ <= 1 –factorul de ceață, proporțional cu distanța de la observator la punctul din spațiul 3D vizibil in pixel (Φ=1: opacitate)
Atunci, Cpixel, culoarea de afişare a pixelului corespunzator fragmentului este:
Cpixel= (1-Φ)*Cfragment+ Φ*Cfog

b) cum se calculeaza factorul de ceata?

Factorul de ceaţă liniar (obtinut prin interpolare liniara) actioneaza intre 2 distante definite in sistemul coordonatelor observator, zo si z1, masurate pe directia de observare.

Factorul de ceata este calculat in timpul rasterizarii, pentru fiecare fragment, folosind coordonata z a fragmentului si distantele z0 si z1 transformate in coordonate ale spatiului de afisare:
z: coordonata z a fragmentului
Φ - Factorul de ceata
Φ = 0 pentru z < z0
Φ = (z-z0) / (z1-z0) pentru z0 <= z <= z1
Φ = 1 pentru z > z1

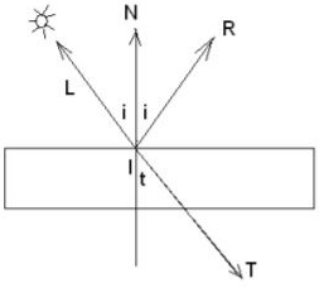
Valorile implicite pentru z0 si z1 sunt 0 si 1, in spatiul de afisare: coordonatele z ale planului din fata si planului din spate al vederii, dupa transformarea in poarta de afisare.

Factorul de ceata moduleaza culoarea fragmentului dupa texturare.

//mai taiati din ele daca vi se par prea lungi raspunsurile
/
Se mai poate calcula si exponential.

19. Ray Tracing (curs 8):

- **Care sunt razele folosite in calculul culorii unui pixel in algoritmul Ray Tracing?**



- **Cum se calculeaza directia lor?**

$R = 2(N \cdot L) \cdot N - L$

$n1/n2 = \sin(t)/\sin(i)$, n1,n2: indicii de refractie

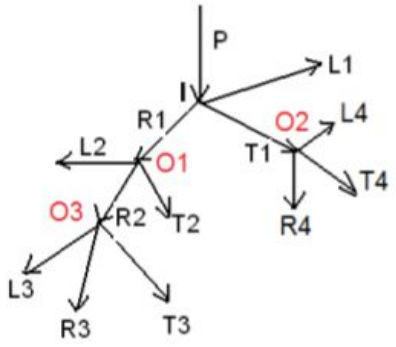
$T = L^*(n1/n2) - (\cos(t) + (n1/n2)^*(L \cdot N))^*N$

- **Formula de calcul a culorii unui pixel?**

Calculul culorii luminii în punctul I:

$$I_\lambda(I) = I_{local\lambda}(I) + Ks^*R_\lambda(I) + Kt^*T_\lambda(I)$$

- **Care este arborele de raze?**



Arborele de raze

➢*Pentru obtinerea culorii pixelului, se evalueaza arborele de raze de la frunze catre radacina*

- **Cum se determina razele(fara exprimarea matematica)?**

// TODO nu seamana cu ce s-a scris la calcularea lor? adica se urmaresc cele trei tipuri L, R, T? Si daca o raza nu intersecteaza nimic se returneaza fondul altfel se aplica iar recursiv?
Da

20. Pentru decuparea vectorilor 2D: Descriere, Pasii algoritmului pentru decuparea vectorului din figura (vezi sus figura cu un patrat taiat de o linie)

21. Decuparea vectorilor in algoritmul Cohen-Sutherland (fara imlpementare)

https://www.youtube.com/watch?v=dQw4w9WgXcQ

24. Algoritmul Bresenham pentru rasterizarea vectorilor: alegerea punctelor spatiului discret, implementare in C, generalizarea algoritmului pentru vectori de orice panta.[1p]

(curs 9)

- *Alegerea punctelor spatiului discret.*
  - O si D sunt adrese de pixeli (puncte ale spatiului discret)
  - Următorul punct al spatiului discret ales pentru aproximarea vectorului va fi:
    - O dacă d1<d2, D în caz contrar.
    - Dacă d1=d2 se poate alege oricare dintre cele două puncte

- *Deducerea valorii expresiei de test.*
  - Valoarea variabilei de test se obtine prin calcul incremental: adunarea unei constante intregi

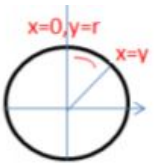
- *Implementare in C*

- ***Generalizarea algoritmului pentru vectori de orice panta.***

25. Algoritmul Bresenham pentru rasterizarea cercurilor: alegerea punctelor spatiului discret, implementare in C, functia de afisare a punctelor de pe cerc. (curs 10)

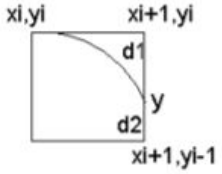
- *alegerea punctelor spatiului discret:*

- Consideram cercul cu centrul în originea sistemului de coordonate si raza r.
- In cadrul algoritmului **se calculeaza numai punctele din octantul al 2-lea,** celelalte obtinandu-se prin simetrie.



- Se calculeaza punctele de pe cerc incepand cu (x=0,y=r) pana la (x=y), prin incrementarea lui x.
- Fie (x_i,y_i) ultimul punct al spatiului discret, ales pentru aproximarea cercului.

Urmatorul punct va fi unul dintre (x_i+1,y_i) si (x_i+1,y_i-1):



Fie y ordonata punctului de pe cercul teoretic, cu abscisa x_i+1.

$$y^2 = r^2-(x_i+1)^2$$

Fie d1 si d2 distantele de la cele 2 puncte ale spatiului discret la punctul de pe cercul teoretic.

Intereseaza semnul diferentei (d1-d2):

- daca $(y_i - y) > (y- (y_i-1))$ atunci

$$(y_i^2 - y^2) > (y^2 - (y_i-1)^2)$$

$$d1' = y_i^2 - y^2 = y_i^2 - r^2 + (x_i+1)^2$$

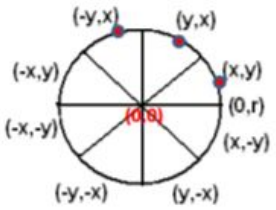
$$d2' = y^2 - (y_i-1)^2 = r^2 - (x_i+1)^2 - (y_i-1)^2$$

- *implementare in C*

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{ int x,y,t;
  t=3-(r<<1);
  point(xc+r,yc,culoare); point(xc-r,yc,culoare);
  point(xc,yc+r,culoare); point(xc,yc-r,culoare);
  for(x=1,y=r; x<y; x++)
  {
    if(t<0)
      t+=6+(x<<2);
    else
     { t+=10+((x-y)<<2); y--;}
    punct_simetric(xc,yc,x,y,culoare);
  }
// pentru x==y se afiseaza 4 puncte simetrice
  point(xc+x,yc+y,culoare); point(xc+x,yc-y,culoare);
  point(xc-x,yc+y,culoare); point(xc-x,yc-y,culoare);
}
```

- *functia de afisare a punctelor de pe cerc. (e singura chestie care parea mai aproape de intrebare - slide 6)*

```
// Afiseaza punctele simetrice de pe cerc
void punct_simetric(int xc,int yc,int x,int y, RGB culoare)
{ point(xc+x,yc+y,culoare);
  point(xc+x,yc-y,culoare);
  point(xc-x,yc-y,culoare);
  point(xc-x,yc+y,culoare);
  point(xc+y,yc+x,culoare);
  point(xc+y,yc-x,culoare);
  point(xc-y,yc-x,culoare);
  point(xc-y,yc+x,culoare);
}
```



27.Transformarea de vizualizare in OpenGL. Parametrii care definesc sistemul de coordonate observatoare. Care este functia de OpenGL care permite definirea acestor parametrii?

Sigur e vorba despre fereastra-poarta si nu despre V din MVP?

Transformarea de vizualizare in OpenGL:
Pentru setarea zonei de afisare in OpenGL se apeleaza
glViewport(ox, oy, width, height);
unde (ox,oy) sunt coord. coltului stanga jos al viewportului, in coordonatele ferestrei de afisare.
Orientarea axelor viewportului esate aceeasi cu a sistemului coordonatelor logice:
- Nu este necesara corectia coordonatei y (de ce nu?; asa scrie in curs :-??)

//pana la urma cum e aici? legat de asta “Sigur e vorba despre fereastra-poarta si nu despre V din MVP?”

28.Forma generala (in pseudocod) a algoritmului de rasterizare a suprafetelor poligonale. Care e definitia punctului interior a unui poligon, pe care se bazeaza algoritmul? Cate puncte de intersectie trebuie sa se obtina atunci cand o linie raster traverseaza un varf al poligonului?

Un punct este interior daca orice segment dus din el intersecteaza un numar impar de laturi. Pentru cele de maxim local nu se interseccteaza nicio latura, iar pentru cele de minim local se intersecteaza 2.

Parca se lua o linie raster. Se calculau toate intersectiile cu laturile, se determinau punctele de intersectie si se ordonau in functie de x. Si apoi se desenau segmentele.

(Examen 6 Februarie 2017)
29. (1pct.) Modelul de reflexie Phong(modelul de iluminare locala) pentru calculul luminii reflectate intr-un punct al unei suprafete 3D. Precizati care sunt componentele luminii reflectate si care sunt calculate.
30. (0.5 pct.) Care este matricea prin care se transforma un varf din sistemul coordonator obiect in sistemul coordonator de decupare? Care este volumul de decupare? Cand este efectuata decuparea primitivelor in banda grafica?

31. (0.5 pct.) Care sunt parametrii prin care este definit sistemul de coordonate observator(atasat camerei virtuale)? Care sunt valorile implicite ale acestor parametrii? Care este produsul de matrici prin care se obtine matricea transformarii de vizualizare(din sistemul de coordonate globale in sistemul de coordonate observator).

32. (0.5 pct.) Care sunt caracteristicile proiectiilor axonometrice? Scrieti produsul de matrici prin care se poate obtine proiectia axonometrica a unui punct.

33. (0.5 pct.) Cum se aleg pixelii de pe traseul unui vector in algoritmul Bresenham pentru rasterizarea vectorilor? Cum se generalizeaza algoritmul pentru rasterizarea vectorilor de orice panta?

34. (1 pct.) Cum este efectuate urmatoarele operatii in algoritmul Ray-tracing:

a) (0.25 pct.) Eliminarea partilor nevizibile.
b) (0.5 pct.) Calculul culorii pixelilor intr-un cadru imagine(precizati care sunt componentele luminii care contribuie la culoarea unui pixel si cum sunt evaluate)

c) (0.25 pct.) Afisarea in umbra a partilor unui obiect.

35. (0.25 pct - suplimentar) Cum poate fi utilizat algoritmul BSP pentru eliminarea obiectelor nevizibile din banda grafica (Object culling)?

Sesiune iarna 2018:

1. Rotatia unui punct (3D) in jurul unei drepte oarecare.
2. Transformarea de proiectie, mai multe intrebari. Cum se poate implementa proiectia axonometrica in OpenGl.
3. Sistemul de coordonate in care se realizeaza auto-obturarea fetelor nevizibile + ...
4. BSP (functia de afisare + …)
5. Modelul de culoare HSV
6. Grile modelul Phong
7. Reflexia speculara
8. Rulare Cohen-Sutherland si explicatii pasi

Sesiune iarnă 2019 - Varianta 1

1. Produsul de matrici prin care se obtine transformarea de scalare a unui punct (x, y, z) fata de punctul (xf, yf, zf)

2. Care sunt parametrii prin care este definit sistemul de coordonate observator in OpenGL si cum se folosesc in determinarea axelor SCO? care este SCO impicit?
3. Algoritmul z-buffer. In ce etapa a benzii grafice se executa? cum se foloseste pentru eliminarea partilor nevizibile din imaginea unei scene 3D? (explicatii + pseudocod)
4. Modelul de culoare HSV: a) subspaţiul culorilor (desen + explicaţii) b) care sunt valorile param H, S şi V pentru culorile pure
5. aproximarea reflexiei difuze intr-un punct al unei suprafete 3d, in modelul de iluminare locala
6. modelul Gourand pentru calculul culorii fragmentelor - 5 afirmatii
7. introducerea umbrelor in imagini prin metoda volumelor de umbra. cum este definit un volum de umbra? cum se determina daca un punct al unei suprafete din scena 3d este in umbra?
8. algoritmul Bresenham pentru rasterizarea vectorilor. Cum sunt alesi pixelii prin care se aproximeaza traseul unui vector (fără implementare). cum se generalizează algoritmul pentru rasterizarea vectorilor de orice pantă (fără implementare)

Sesiune iarnă 2019 - Varianta 2

- era în plus față de celelalte subiecte decuparea cu Liam Barsky